Computers count. They never measure.

CONCEPT

Every value inside a computer is stored as a finite pattern of bits. That makes computing a discrete world: things you can list, count, and reason about one step at a time.

This course gives you the mathematics of that world. Sets describe collections of data. Logic describes conditions in your code. Proofs show that an algorithm is correct. Counting and probability tell you how many possibilities exist and how likely each one is. Graphs and trees model networks, file systems, and the decision structure inside many AI models.

Try it on the right: flip the eight bits of one byte. That single byte is already a number, a set, a logical statement, and a counting problem.

ONE BYTE · CLICK A BIT TO FLIP IT

0	0	1	0	1	1	0	1
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

NUMBER Read as base 2, the byte is the integer 45 (hex 2D). Module 6 studies integers.

SET The positions that are 1 form the set $S = \{0, 2, 3, 5\}$, with $|S| = 4$. Module 1 studies sets.

LOGIC The statement "the number is even" is **false**, because bit 0 is 1. Module 2 studies logic.

COUNTING There are $2^8 = 256$ possible bytes, and $\binom{8}{4} = 70$ of them have exactly 4 ones. Module 7 studies counting.

PROBABILITY A random byte has exactly 4 ones with probability $70/256 \approx 0.273$. Module 13 studies probability.

START HERE

How this lab works

CONCEPT

Each module follows the same path from idea to skill. Move with the → and ← keys or the buttons at the bottom. The dots at the top right show where you are inside a module.

CONCEPT

A definition, the notation, a worked example, and where the idea shows up in computing and AI. Dotted words are glossary terms: hover or focus them for a definition.

INTERACTIVE LAB

A hands-on model of the idea. Change the inputs and watch the mathematics respond. Every lab reacts immediately; nothing needs a server.

CODE WALKTHROUGH

A short Python program where **every line** has its own explanation. Read it line by line, edit and run it, or trace it one step at a time while you watch the variables change.

DESIGN PRACTICE

Habits that professionals follow when they turn the mathematics into software, each shown as a side-by-side comparison of code to avoid and code to write.

PRACTICE

Endless generated problems with instant checking and a worked solution for each one. A new problem appears every time you ask.

CHECK YOURSELF

A short quiz. Every answer, right or wrong, comes with an explanation of the reasoning behind it.

WHY IT MATTERS

The Python in this course is real Python 3, compiled to WebAssembly and running inside your browser tab. The first run downloads it once; after that, runs are instant. Nothing you type leaves your computer.

HANDY KEYS

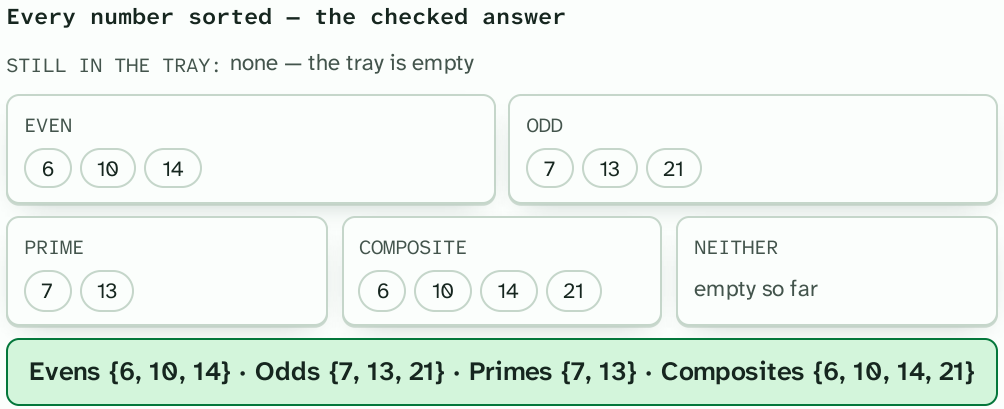
`Ctrl`/`⌘`+`K` search everything · `N` notation · `G` glossary · `T` theme · `?` all shortcuts. Your progress rings in the module list are saved in this browser only.

Worked example: sort the tray (meet the player)

WORKED EXAMPLE

PROBLEM

Sort each number in the tray $\{6, 7, 10, 13, 14, 21\}$ twice: once by parity, into the evens and the odds, and once by kind, into the primes and the composites. As you work, learn the player: **Step ▶** advances one step, **▶ Play** advances on its own but pauses at prediction checkpoints, and the **Problem** switcher reruns the same method on new numbers.



1. Meet the worked-example player on a tiny problem: sort every number in the tray twice, once into the even or odd box and once into the prime or composite box. Press Step ▶ below the picture to place the first number; every step changes the picture and explains what changed and why.
2. 6 lands in the even box because 6 = 2 × 3 with nothing left over, and in the composite box because it has a divisor besides 1 and itself, since 6 = 2 × 3. One number always sits in two boxes at once, because parity and primality are two independent questions about it.
3. 7 lands in the odd box because dividing 7 by 2 leaves a remainder of 1, and in the prime box because no whole number from 2 up to 2 divides it, so it has exactly two divisors, 1 and itself. One number always sits in two boxes at once, because parity and primality are two independent questions about it. Tip: the ▶ Play button advances the steps on its own, and it always pauses at a prediction checkpoint so you can think before the reveal.
4. 10 lands in the even box because 10 = 2 × 5 with nothing left over, and in the composite box because it has a divisor besides 1 and itself, since 10 = 2 × 5. One number always sits in two boxes at once, because parity and primality are two independent questions about it.
5. 13 lands in the odd box because dividing 13 by 2 leaves a remainder of 1, and in the prime box because no whole number from 2 up to 3 divides it, so it has exactly two divisors, 1 and itself. One number always sits in two boxes at once, because parity and primality are two independent questions about it. Tip: ↺ Restart and the scrubber under the picture let you replay any step — nothing here can be broken, so experiment freely.
6. 14 lands in the even box because 14 = 2 × 7 with nothing left over, and in the composite box because it has a divisor besides 1 and itself, since 14 = 2 × 7. One number always sits in two boxes at once, because parity and primality are two independent questions about it.
7. 21 lands in the odd box because dividing 21 by 2 leaves a remainder of 1, and in the composite box because it has a divisor besides 1 and itself, since 21 = 3 × 7. One number always sits in two boxes at once, because parity and primality are two independent questions about it.
8. **Answer: the evens are {6, 10, 14} and the odds are {7, 13, 21}; the primes are {7, 13} and the composites are {6, 10, 14, 21}. The green banner is the checked answer, recomputed independently of the steps; the Problem switcher above the picture reruns the same method on a new tray with two troublemakers in it.**

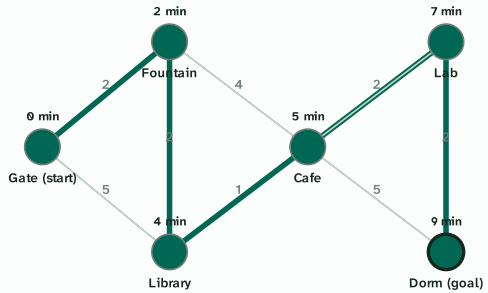
Worked example: the shortest route across campus

WORKED EXAMPLE

PROBLEM

On the campus map, every road is labeled with its walking time in minutes: Gate–Fountain 2, Gate–Library 5, Fountain–Library 2, Fountain–Cafe 4, Library–Cafe 1, Library–Lab 3, Cafe–Lab 2, Cafe–Dorm 5 and Lab–Dorm 2. Find the fastest route from the Gate to the Dorm. Predict before each reveal: autoplay pauses at the checkpoint, and the final banner is the checked answer.

Route found: Gate → Fountain → Library → Lab → Dorm = 9 minutes



Shortest route: Gate → Fountain → Library → Lab → Dorm — 9 minutes in all, and no other route ties or beats it

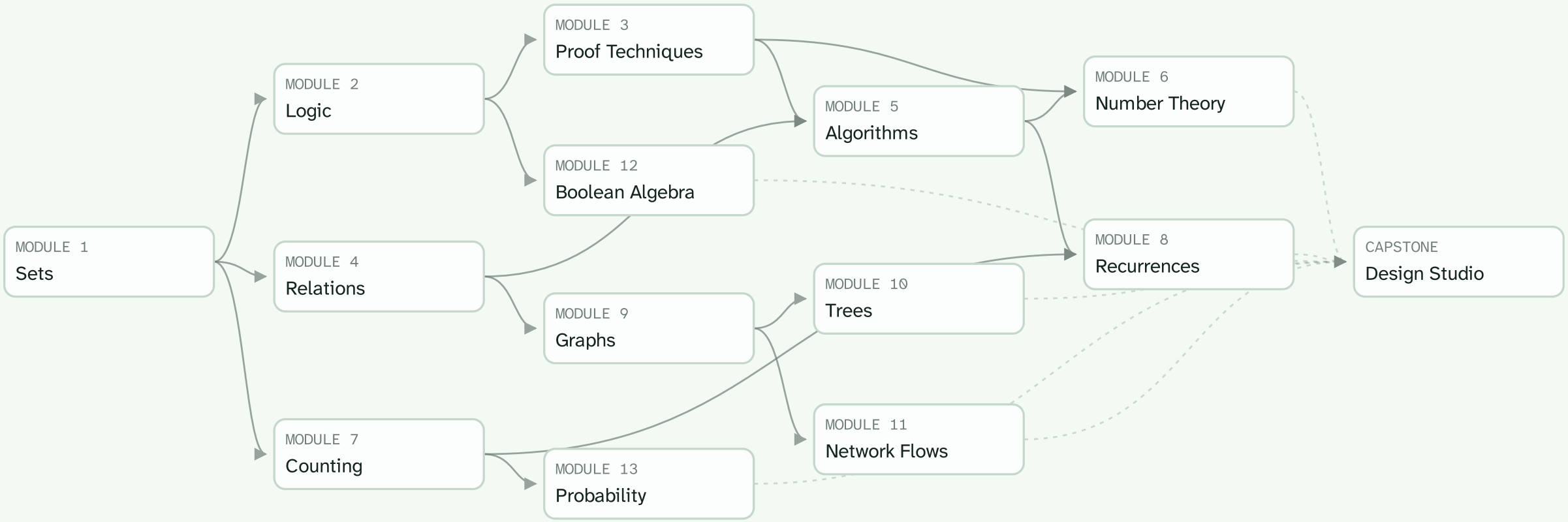
1. Find the fastest walking route from Gate to Dorm on this little map, where every road is labeled with its minutes. The method: repeatedly settle the closest place that is not settled yet — its distance can never improve later, because every road costs a positive number of minutes. Press Step ▶ to settle Gate first.
2. Settle Gate at 0 minutes: among the places not yet settled it holds the smallest tentative distance, so that distance is final and can never be beaten later. From Gate, the roads out update the neighbors they improve: Fountain drops to 2; Library drops to 5.
3. Settle Fountain at 2 minutes: among the places not yet settled it holds the smallest tentative distance, so that distance is final and can never be beaten later. From Fountain, the roads out update the neighbors they improve: Library drops to 4; Cafe drops to 6.
4. Settle Library at 4 minutes: among the places not yet settled it holds the smallest tentative distance, so that distance is final and can never be beaten later. From Library, the roads out update the neighbors they improve: Cafe drops to 5; Lab drops to 7.
5. Settle Cafe at 5 minutes: among the places not yet settled it holds the smallest tentative distance, so that distance is final and can never be beaten later. From Cafe, the roads out update the neighbors they improve: Dorm drops to 10.
6. Settle Lab at 7 minutes: among the places not yet settled it holds the smallest tentative distance, so that distance is final and can never be beaten later. From Lab, the roads out update the neighbors they improve: Dorm drops to 9.
7. **Answer: Dorm settles at 9 minutes, and following the "came from" links back to Gate gives the route Gate → Fountain → Library → Lab → Dorm, totaling 9 minutes. The green banner is the checked answer — it was verified by trying every possible route, not just the settled one. The Problem switcher above the picture reruns the same method on a second map.**

START HERE

Course map: how the ideas build on each other

INTERACTIVE LAB

This map is itself a **directed graph** (Module 9). An arrow from one module to another means the first supplies ideas the second uses. Hover or focus a module to light up its prerequisites; click it, or press Enter, to jump there. A green outline means you have opened every slide in that module.



START HERE

What you will be able to do

CONCEPT

These are the thirteen course objectives. Each row links to the module that teaches it, and the Design Studio at the end ties all of them together in realistic software decisions.

#	OBJECTIVE	WHERE YOU LEARN IT
1	Apply set terminology and notation.	Module 1: Sets · Design Studio
2	Use logic to read and develop proofs.	Module 2: Logic · Module 3: Proof Techniques · Design Studio
3	Examine proof techniques in discrete mathematics and computer science.	Module 3: Proof Techniques · Design Studio
4	Describe the use of functions, sequences and relations in computer science and engineering.	Module 4: Functions, Sequences & Relations · Design Studio
5	Explain the application and analysis of algorithms to solve problems.	Module 5: Algorithms · Module 6: Integers & Number Theory · Design Studio
6	Analyze the relationship between integers and number theory.	Module 6: Integers & Number Theory · Design Studio
7	Use counting techniques to solve the binomial theorem.	Module 7: Counting & the Binomial Theorem · Design Studio
8	Apply recurrence relations to analyze algorithms.	Module 8: Recurrence Relations · Design Studio
9	Explain the use of graphing.	Module 9: Graphs · Design Studio
10	Describe the use of trees as a data structure in computer science.	Module 10: Trees · Design Studio
11	Use network modeling to maximize data flow.	Module 11: Network Flows · Design Studio
12	Examine the relationship of Boolean algebra and logic circuits.	Module 12: Boolean Algebra & Circuits · Design Studio
13	Understand basic probability theory.	Module 13: Probability · Design Studio

What is a set?

CONCEPT

DEFINITION

A set is an unordered collection of distinct objects, called its elements. We write $x \in A$ for "x is an element of A" and $x \notin A$ for "it is not".

There are two standard ways to describe a set:

- **Roster notation** lists the elements: $V = \{a, e, i, o, u\}$.
- **Set-builder notation** gives a rule: $E = \{x \in \mathbb{Z} \mid 0 \le x \le 10,\ x \text{ even}\}$. Read the bar as "such that".

Two facts surprise beginners. **Order does not matter**: $\{1, 2, 3\} = \{3, 1, 2\}$. **Repeats do not count**: $\{1, 1, 2\} = \{1, 2\}$.

WHERE THIS SHOWS UP IN AI

A language model's vocabulary is a set of tokens: each token appears once, and the model only asks "is this token in the vocabulary?" A spam filter's list of blocked senders is also a set.

Size, emptiness, and the famous number sets

The cardinality $|A|$ counts the elements of a finite set: $|\{a, e, i, o, u\}| = 5$. The empty set $\varnothing = \{\}$ has $|\varnothing| = 0$.

Symbol	Name	Elements
$\mathbb{N}$	natural numbers	0, 1, 2, 3, … (this course includes 0)
$\mathbb{Z}$	integers	…, −2, −1, 0, 1, 2, …
$\mathbb{Q}$	rational numbers	fractions p/q with $q \neq 0$
$\mathbb{R}$	real numbers	every point on the number line

WORKED EXAMPLE

Is $\{\varnothing\}$ empty? No. It is a set with *one* element, and that element happens to be the empty set. So $|\{\varnothing\}| = 1$ while $|\varnothing| = 0$. Think of an empty box inside a box: the outer box is not empty.

Subsets, equality, and the power set

CONCEPT

DEFINITION

$A \subseteq B$ (a subset) means every element of A is also in B. $A \subset B$ (a proper subset) adds that B has something extra.

THEOREM / RULE

Two sets are equal exactly when each is a subset of the other:

$$A = B \iff (A \subseteq B \text{ and } B \subseteq A)$$

This is how you *prove* two sets are equal: show every element of the left is in the right, then the reverse.

WORKED EXAMPLE

Let $A = \{1, 2\}$ and $B = \{1, 2, 3\}$. Then $A \subseteq B$ and $A \subset B$, but $B \not\subseteq A$ because $3 \notin A$. Also $\emptyset \subseteq A$: the empty set has no element that could fail the test.

COMMON PITFALL

$\in$ and $\subseteq$ are different questions. $1 \in \{1, 2\}$ is true, but $1 \subseteq \{1, 2\}$ is meaningless because 1 is a number, not a set. The correct subset statement is $\{1\} \subseteq \{1, 2\}$.

DEFINITION

The power set $\mathcal{P}(A)$ is the set of *all* subsets of A, including $\emptyset$ and A itself.

$$\mathcal{P}(\{a, b\}) = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$$

THEOREM / RULE

If $|A| = n$, then $|\mathcal{P}(A)| = 2^n$.

Why 2^n? Building a subset means making one yes/no decision per element: "is a in? is b in? ...". That is n independent two-way choices, so $2 \cdot 2 \cdots 2 = 2^n$ subsets. Each subset matches one n-bit binary string, where bit i says whether element i is in. The next slide lets you see this happen.

WHERE THIS SHOWS UP IN AI

Feature selection in machine learning asks which subset of n input features gives the best model. There are 2^n candidate subsets, so with 40 features there are over a trillion. Exhaustive search becomes impossible, and that is why greedy and randomized search methods exist.

Lab: build a power set one bit at a time

INTERACTIVE LAB

Choose how many elements A has. Every subset is listed, grouped by size. Hover or tap a subset to see the binary string that encodes it: a 1 in position i means element i is included.

Elements in A: 1 | 2 | **3** | 4 | 5

$A = \{a, b, c\}$, so $|A| = 3$

$|\mathcal{P}(A)| = 2^3 = 8$

SUBSET UNDER YOUR POINTER

Point at a subset →

Notice the row sizes: they are $\binom{n}{k}$, the number of ways to choose k of n elements. For $n = 3$ they read 1, 3, 3, 1, which add up to 8. Module 7 explains why.

SIZE 0 · 1 SUBSET

∅

SIZE 1 · 3 SUBSETS

{a} {b} {c}

SIZE 2 · 3 SUBSETS

{a,b} {a,c} {b,c}

SIZE 3 · 1 SUBSET

{a,b,c}

Worked example: building a power set by doubling

WORKED EXAMPLE

PROBLEM

List the power set $\mathcal{P}(S)$ for $S = \{a, b, c\}$, building it one include-or-exclude decision at a time: first decide a, then b, then c. Count the subsets at each stage.

Complete: |𝒫(S)| = 2^3 = 8

∅ {a} {b} {a, b} **{c}** **{a, c}** **{b, c}** **{a, b, c}**

Subsets after 3 of 3 decisions: 1 → 2 → 4 → 8 — each decision doubled the count.

𝒫(S) = {∅, {a}, {b}, {a, b}, {c}, {a, c}, {b, c}, {a, b, c}} — |𝒫(S)| = 2^3 = 8 subsets

1. Build 𝒫(S) by deciding, for each element of S = {a, b, c} in turn, whether it goes into the subset or stays out. Every finished sequence of decisions is one subset, and because each decision has exactly 2 outcomes, the count should double at every element until it reaches 2^3 = 8.
2. Start before any element is decided: there is exactly one partial subset, the empty set ∅, because with no decisions made there is only one way things can stand. This single subset is the seed that every later doubling grows from, so the count starts at 1, not at 0.
3. Decide a: every existing subset appears twice in the new list, once leaving a out and once with a added, so the list doubles from 1 to 2 subsets. The last new subset is {a}, formed by adding a to ∅, the largest subset built so far.
4. Decide b: every existing subset appears twice in the new list, once leaving b out and once with b added, so the list doubles from 2 to 4 subsets. The last new subset is {a, b}, formed by adding b to {a}, the largest subset built so far.
5. Decide c: every existing subset appears twice in the new list, once leaving c out and once with c added, so the list doubles from 4 to 8 subsets. The last new subset is {a, b, c}, formed by adding c to {a, b}, the largest subset built so far.
6. **Answer: 𝒫(S) = {∅, {a}, {b}, {a, b}, {c}, {a, c}, {b, c}, {a, b, c}}, which is 8 subsets — the doubling lands exactly on 2^3 = 8, as promised. The full set {a, b, c} itself appears as the very last subset, the one where every decision said "in", and ∅ is there too, from the sequence where every decision said "out".**

Lab: Venn diagram calculator

INTERACTIVE LAB

The universal set is $U = \{1, \ldots, 12\}$. Edit *A*, *B*, *C*, then type any expression or pick a preset. The shaded regions and the result update as you type. Operators: `∪` `∩` `−` `Δ` and a trailing `′` for complement (you can also type `|` `&` `-` `^` `'`).

A `1, 2, 3, 4, 5, 6`

B `4, 5, 6, 7, 8`

C `2, 5, 8, 9, 10`

Expression

`A ∩ (B ∪ C)`

`∪` `∩` `−` `Δ` `′` `(` `)`

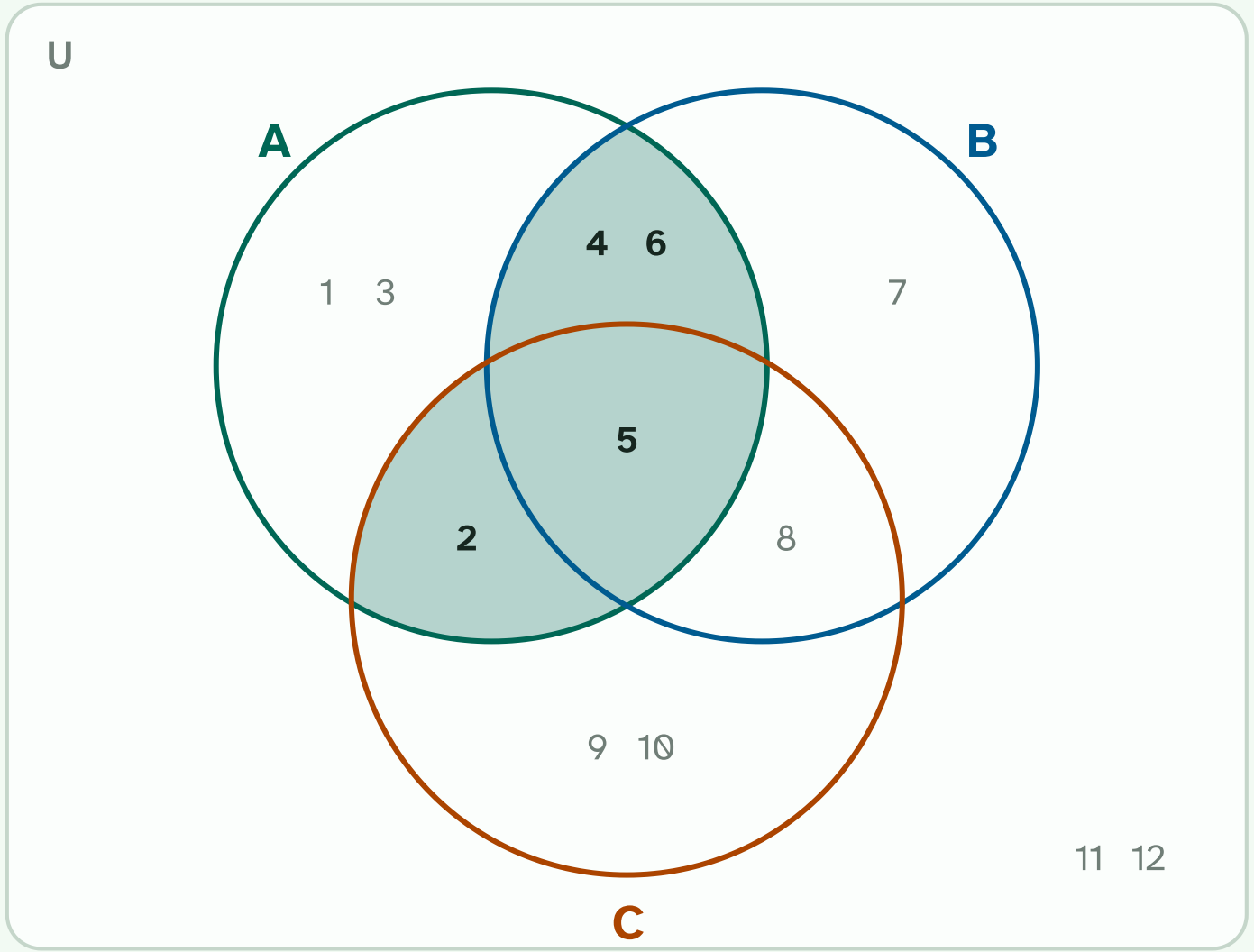
`A ∪ B` `A ∩ B` `A − B` `A Δ B` `A′` `(A ∪ B)′` `A′ ∩ B′` `A ∩ (B ∪ C)` `(A ∩ B) ∪ (A ∩ C)` `A ∩ B ∩ C`

`A ∩ (B ∪ C)` = **{2, 4, 5, 6}**

Cardinality: |result| = 4. Shaded regions are exactly where the expression is true.

ORDER OF EVALUATION (INSIDE OUT)

1. `B ∪ C` = {2, 4, 5, 6, 7, 8, 9, 10}
2. `A ∩ (B ∪ C)` = {2, 4, 5, 6}



The laws of set algebra

CONCEPT

Set operations obey algebraic laws, just like $+$ and $\times$ do for numbers. Each law below holds for *every* choice of sets. Check any of them in the Venn lab: type both sides and confirm the shading is identical.

Law	Union form	Intersection form
Commutative	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associative	$(A \cup B) \cup C = A \cup (B \cup C)$	$(A \cap B) \cap C = A \cap (B \cap C)$
Distributive	$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
De Morgan	$(A \cup B)' = A' \cap B'$	$(A \cap B)' = A' \cup B'$
Identity	$A \cup \varnothing = A$	$A \cap U = A$
Complement	$A \cup A' = U$	$A \cap A' = \varnothing$
Idempotent	$A \cup A = A$	$A \cap A = A$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$

Notice the pattern: swap $\cup \leftrightarrow \cap$ and $\varnothing \leftrightarrow U$ in any law and you get another law. This is called **duality**. The same laws reappear for logic (Module 2) and for circuits (Module 12), because all three are the same algebra in different clothes.

THEOREM / RULE

Inclusion–exclusion for two sets:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Adding $|A|$ and $|B|$ counts every shared element twice, once in each set, so subtract the overlap once.

WORKED EXAMPLE

A survey of 40 students: 25 take Python, 18 take Java, 9 take both. How many take at least one? $25 + 18 - 9 = 34$. How many take neither? $40 - 34 = 6$.

DESIGN TIP

When you need the size of an "or" condition, count each piece and subtract the overlap. Forgetting the overlap is the classic double-counting bug in database reports: a customer who bought two products gets counted twice.

Lab: ordered pairs and the Cartesian product

INTERACTIVE LAB

DEFINITION

An ordered pair (a, b) remembers order, so $(1, 2) \neq (2, 1)$. The Cartesian product is $A \times B = \{(a, b) \mid a \in A,\ b \in B\}$, and $|A \times B| = |A| \cdot |B|$.

A (for example, shirt sizes)

S, M, L

B (for example, colors)

red, blue, black, white

Show B × A instead

$$|A \times B| = 3 \cdot 4 = 12$$

A \ B	red	blue	black	white
S	(S, red)	(S, blue)	(S, black)	(S, white)
M	(M, red)	(M, blue)	(M, black)	(M, white)
L	(L, red)	(L, blue)	(L, black)	(L, white)

Each row picks one element of A; each column picks one of B. Every cell is one ordered pair, so the grid has rows × columns cells. This is the **product rule** of counting (Module 7).

Press the button to swap the order and see that A × B ≠ B × A.

WHERE THIS SHOWS UP IN AI

A grid search for hyperparameters tries every pair in *learning rates* × *batch sizes*. With 5 rates and 4 sizes that is 20 training runs, and each extra parameter multiplies the cost again.

Worked example: building A × B on a grid

WORKED EXAMPLE

PROBLEM

A shirt comes in sizes $A = \{S, M, L\}$ and colors $B = \{\text{red}, \text{blue}\}$. Build the Cartesian product $A \times B$ — every (size, color) ordered pair — by filling the grid one cell at a time, row by row, and state how many pairs there are.

Complete: |A × B| = 3 · 2 = 6

A \ B	red	blue
S	(S, red)	(S, blue)
M	(M, red)	(M, blue)
L	(L, red)	(L, blue)

Pairs placed: 6 of 6 — one row of 2 pairs for each element of A.

A × B = {(S, red), (S, blue), (M, red), (M, blue), (L, red), (L, blue)} — |A × B| = 3 · 2 = 6

1. Set up the grid for A × B with A = {S, M, L} down the side and B = {red, blue} across the top. Each cell will hold exactly one ordered pair (a, b), so filling the grid cell by cell, row by row, builds the whole product with nothing missed and nothing repeated.
2. Fill the cell in row S, column red with the ordered pair (S, red): the first coordinate must come from A and the second from B, in that order. The grid now holds 1 of the 6 pairs, and the sweep moves one cell to the right, staying in the same row.
3. Fill the cell in row S, column blue with the ordered pair (S, blue): the first coordinate must come from A and the second from B, in that order. The grid now holds 2 of the 6 pairs, and the sweep moves down to the next row, because this row has one cell per element of B.
4. Fill the cell in row M, column red with the ordered pair (M, red): the first coordinate must come from A and the second from B, in that order. The grid now holds 3 of the 6 pairs, and the sweep moves one cell to the right, staying in the same row.
5. Fill the cell in row M, column blue with the ordered pair (M, blue): the first coordinate must come from A and the second from B, in that order. The grid now holds 4 of the 6 pairs, and the sweep moves down to the next row, because this row has one cell per element of B.
6. Fill the cell in row L, column red with the ordered pair (L, red): the first coordinate must come from A and the second from B, in that order. The grid now holds 5 of the 6 pairs, and the sweep moves one cell to the right, staying in the same row.
7. Fill the cell in row L, column blue with the ordered pair (L, blue): the first coordinate must come from A and the second from B, in that order. That was the last empty cell of the grid, so the sweep is finished and every combination now appears exactly once.
8. **Answer: A × B = {(S, red), (S, blue), (M, red), (M, blue), (L, red), (L, blue)}, which is 6 pairs, because each of the 3 rows contributes 2 pairs — the count is 3 · 2 = 6, the product rule. Swapping the factors would give B × A, whose pairs are reversed, a different set unless the two sets happen to be equal.**

Code: set operations in Python

CODE WALKTHROUGH

Python has a built-in `set` type whose operators mirror the mathematical symbols. Click any line (or use ↓) to read its explanation.

set_operations.py

Read | Edit & run | Trace

```
1  A = {1, 2, 3, 4}
2  B = {3, 4, 5}
3  U = set(range(1, 8))
4  print("A ∪ B =", A | B)
5  print("A ∩ B =", A & B)
6  print("A − B =", A - B)
7  print("A Δ B =", A ^ B)
8  print("A´    =", U - A)
9  print(3 in A, 9 in A)
10 print({1, 2} <= A, len(A))
11 evens = {x for x in U if x % 2 == 0}
12 print("evens =", evens)
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
A ∪ B = {1, 2, 3, 4, 5}
A ∩ B = {3, 4}
A − B = {1, 2}
A Δ B = {1, 2, 5}
```

LINE 1 OF 12

Show all notes in code

```
A = {1, 2, 3, 4}
```

Creates a set named `A` using braces, which is roster notation written in Python. The variable `A` now refers to a set object holding four integers. This mirrors $A = \{1, 2, 3, 4\}$ on paper.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

A tour of every basic set operation (union, intersection, difference, symmetric difference, complement, membership, subset and size) on two small sets of integers.

Code: set operations in Python: every line explained

1	`A = {1, 2, 3, 4}`	Creates a set named `A` using braces, which is roster notation written in Python. The variable `A` now refers to a set object holding four integers. This mirrors $A = \{1, 2, 3, 4\}$ on paper.
2	`B = {3, 4, 5}`	Creates a second set, `B`. It shares the elements 3 and 4 with `A`, which makes the intersection and difference results below interesting.
3	`U = set(range(1, 8))`	`range(1, 8)` produces 1, 2, ..., 7 (the stop value 8 is excluded). `set(...)` turns that sequence into a set. This is our universal set U. We define it explicitly because a complement only makes sense relative to some universe.
4	`print("A ∪ B =", A | B)`	The `|` operator is union: every element in `A`, in `B`, or in both. `print` shows the label text and then the result, separated by a space. Output: `{1, 2, 3, 4, 5}`. Note that 3 and 4 appear once, not twice.
5	`print("A ∩ B =", A & B)`	The `&` operator is intersection: only elements found in *both* sets. The symbol was chosen because intersection matches the logical "and". Output: `{3, 4}`.
6	`print("A − B =", A - B)`	The `-` operator is set difference: elements of `A` that are not in `B`. Order matters here, because `B - A` would give `{5}` instead. Output: `{1, 2}`.
7	`print("A Δ B =", A ^ B)`	The `^` operator is symmetric difference: elements in exactly one of the two sets. It equals $(A - B) \cup (B - A)$ and matches the logical "exclusive or". Output: `{1, 2, 5}`.
8	`print("A′    =", U - A)`	Python has no complement operator because a set does not know its universe. We compute $A' = U - A$ ourselves. Output: `{5, 6, 7}`. If you forgot to define `U`, there would be no correct answer, which is the point of design principle 3 later.
9	`print(3 in A, 9 in A)`	The `in` keyword tests membership, the Python spelling of $\in$. It returns a Boolean. `3 in A` is `True` and `9 in A` is `False`. For sets this check is fast no matter how large the set is, because sets are stored as hash tables.
10	`print({1, 2} <= A, len(A))`	`<=` between sets means "is a subset of" ($\subseteq$); `<` would mean proper subset. `{1, 2} <= A` is `True`. `len(A)` is the cardinality $\|A\| = 4$.
11	`evens = {x for x in U if x % 2 == 0}`	A **set comprehension**, which is Python's version of set-builder notation: $\{x \in U \mid x \bmod 2 = 0\}$. Read it left to right: "the set of `x`, for each `x` in `U`, if `x` is even". `%` gives the remainder, so `x % 2 == 0` means even.
12	`print("evens =", evens)`	Prints the comprehension's result, `{2, 4, 6}`. Small integer sets usually print in ascending order, but that is an implementation detail. A set has no order, so never write code that depends on the order in which a set prints.

Code: Jaccard similarity, sets in AI

CODE WALKTHROUGH

Search engines and recommendation systems use set overlap to decide how similar two documents are. The Jaccard similarity is $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$.

jaccard.py

Read | Edit & run | Trace

```
1  doc1 = "the cat sat on the mat"
2  doc2 = "the dog sat on the log"
3  words1 = set(doc1.split())
4  words2 = set(doc2.split())
5  shared = words1 & words2
6  together = words1 | words2
7  similarity = len(shared) / len(together)
8  print("shared words:", sorted(shared))
9  print("distinct words in total:", len(together))
10 print(f"Jaccard similarity = {similarity:.2f}")
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
shared words: ['on', 'sat', 'the']
distinct words in total: 7
Jaccard similarity = 0.43
```

LINE 1 OF 10

Show all notes in code

```
doc1 = "the cat sat on the mat"
```

A string holding the first document. In a real system this text would come from a file or a web page.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Measures how similar two short documents are by comparing their sets of distinct words.

Code: Jaccard similarity, sets in AI: every line explained

1	`doc1 = "the cat sat on the mat"`	A string holding the first document. In a real system this text would come from a file or a web page.
2	`doc2 = "the dog sat on the log"`	The second document. Two words changed: *dog* replaces *cat* and *log* replaces *mat*, so the two word sets differ in four words in total.
3	`words1 = set(doc1.split())`	`doc1.split()` cuts the string at spaces into a list of words: `["the", "cat", "sat", "on", "the", "mat"]`. `set(...)` keeps each word once, so the repeated "the" collapses. `words1` has 5 elements.
4	`words2 = set(doc2.split())`	Same conversion for the second document: `{"the", "dog", "sat", "on", "log"}`, also 5 elements.
5	`shared = words1 & words2`	Intersection: the words both documents use, $A \cap B = \{\text{on}, \text{sat}, \text{the}\}$. This is the numerator of the Jaccard formula.
6	`together = words1 | words2`	Union: every distinct word used anywhere, $A \cup B$, with 7 elements. This is the denominator. By inclusion–exclusion, $5 + 5 - 3 = 7$.
7	`similarity = len(shared) / len(together)`	`len` gives each cardinality, and `/` performs true division, so the result is the float $3/7 \approx 0.4286$. If both documents were empty, this would divide by zero, so production code must guard that case.
8	`print("shared words:", sorted(shared))`	`sorted` turns the set into an alphabetical list. We sort before printing because the order in which a set of strings prints can change between runs, and sorted output is predictable and easy to test.
9	`print("distinct words in total:", len(together))`	Prints $\lvert A \cup B \rvert = 7$, so you can check the denominator by hand.
10	`print(f"Jaccard similarity = {similarity:.2f}")`	An **f-string**: the `f` before the quote lets `{...}` insert a value. `:.2f` formats the float with two decimal places, so the output reads `0.43`.

Code: generating a power set with bit masks

CODE WALKTHROUGH

This program turns the "one bit per element" idea from the power-set lab into an algorithm. Switch to **Trace** to watch `mask` count from 0 to 7 while the subsets appear.

power_set.py

Read | Edit & run | Trace

```
1  def power_set(items):
2      items = list(items)
3      n = len(items)
4      result = []
5      for mask in range(2 ** n):
6          subset = [items[i] for i in range(n)
7                    if mask & (1 << i)]
8          result.append(subset)
9      return result
10
11 subsets = power_set(["a", "b", "c"])
12 print(len(subsets), "subsets in total")
13 for sub in sorted(subsets, key=len):
14     print("{" + ", ".join(sub) + "}")
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
8 subsets in total
{}
{a}
{b}
```

LINE 1 OF 14

Show all notes in code

```
def power_set(items):
```

Defines a function named `power_set` that takes one argument, `items`. Nothing runs yet; Python only remembers the recipe until the function is called on line 11.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Builds every subset of a list by counting from 0 to $2^n - 1$ and reading each number's binary digits as include/exclude decisions.

Code: generating a power set with bit masks: every line explained

1	`def power_set(items):`	Defines a function named `power_set` that takes one argument, `items`. Nothing runs yet; Python only remembers the recipe until the function is called on line 11.
2	`    items = list(items)`	Converts the input to a list so the elements have fixed positions 0, 1, 2, ... Bit *i* of the mask will refer to `items[i]`. This also lets the function accept a set, a tuple, or a string.
3	`    n = len(items)`	Stores the number of elements, n. We need it twice: to know how many masks exist and how many bits to inspect.
4	`    result = []`	An empty list that will collect each subset as it is built. A list is fine here because subsets come out in a predictable order.
5	`    for mask in range(2 ** n):`	`2 ** n` is 2^n. The loop visits every integer from 0 to $2^n - 1$. In binary those integers are exactly every n-bit string, from 000 to 111 when $n = 3$.
6	`        subset = [items[i] for i in range(n)`	A list comprehension that will look at every position `i` from 0 to $n - 1$ and decide whether `items[i]` belongs in this subset. The decision rule is on the next line; Python reads the two lines as one statement because the square bracket is still open.
7	`                if mask & (1 << i)]`	`1 << i` is a number with only bit *i* set (1, 2, 4, ...). `mask & (1 << i)` is nonzero exactly when bit *i* of `mask` is 1. So the comprehension keeps `items[i]` when its bit is on. For mask 5 (binary 101) it keeps items 0 and 2.
8	`        result.append(subset)`	Adds the finished subset to the result list. After the loop, `result` holds all 2^n subsets.
9	`    return result`	Sends the list of subsets back to whoever called the function and ends the function.
11	`subsets = power_set(["a", "b", "c"])`	Calls the function with three elements, so it should produce $2^3 = 8$ subsets. The returned list is stored in `subsets`.
12	`print(len(subsets), "subsets in total")`	Checks the count against the formula. The output is `8 subsets in total`.
13	`for sub in sorted(subsets, key=len):`	Loops over the subsets ordered by size. `key=len` tells `sorted` to compare subsets by their length, so ∅ comes first and the full set comes last. Ties keep their original mask order.
14	`    print("{" + ", ".join(sub) + "}")`	`", ".join(sub)` glues the elements together with commas. The braces are added as text so each line looks like set notation, for example `{a, c}`. The empty subset prints as `{}`.

Design practice: working with sets in real code

DESIGN PRACTICE

Choosing the right collection type is a design decision with real consequences for speed and correctness. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Use a set when the question is "is it in there?"**

02 Mirror set-builder notation with a comprehension

03 Name the universal set explicitly

04 Use frozenset for sets of sets and dictionary keys

05 Never change a set while looping over it

06 Know what a set throws away: order and duplicates

07 Use bit masks for small, fixed universes

Principle 1 of 7

Use a set when the question is "is it in there?"

Checking `x in some_list` scans the list from the front, so it takes time proportional to the list length: $O(n)$. Checking `x in some_set` jumps straight to the right spot using a hash, which takes about the same time no matter how big the set is: $O(1)$ on average.

With 100,000 blocked words and a million messages, that difference is the gap between seconds and hours.

✗ AVOID THIS

```
blocked = ["spam", "scam", "free-money"]  # list
for msg in messages:
    if msg.sender in blocked:   # scans every time
        reject(msg)
```

✓ DO THIS

```
blocked = {"spam", "scam", "free-money"}  # set
for msg in messages:
    if msg.sender in blocked:   # one hash lookup
        reject(msg)
```

Design practice: working with sets in real code: all principles (1/2)

Use a set when the question is "is it in there?"

Checking `x in some_list` scans the list from the front, so it takes time proportional to the list length: $O(n)$. Checking `x in some_set` jumps straight to the right spot using a hash, which takes about the same time no matter how big the set is: $O(1)$ on average.

With 100,000 blocked words and a million messages, that difference is the gap between seconds and hours.

```
blocked = {"spam", "scam", "free-money"}  # set
for msg in messages:
    if msg.sender in blocked:   # one hash lookup
        reject(msg)
```

Mirror set-builder notation with a comprehension

When a set is defined by a rule, $\{x \in U \mid P(x)\}$, write it as a set comprehension. The code then reads like the specification, which makes it easier to review and harder to get wrong.

```
evens = {x for x in U if x % 2 == 0}
# reads: { x in U | x is even }
```

Name the universal set explicitly

A complement A' is meaningless until you say what the universe is. "Users who did not log in today" depends on whether U is all users, active users, or users in one region. Store U in a named variable and compute `U - A`, so every reader knows which universe you meant.

```
ACTIVE_USERS = load_active_user_ids()   # U, defined once
inactive = ACTIVE_USERS - logged_in     # A' = U - A
```

Use frozenset for sets of sets and dictionary keys

A normal `set` can change, so Python refuses to hash it, and that means it cannot be an element of another set or a dictionary key. A `frozenset` is an immutable set and can be used in both places. That is what you need to represent a power set or to cache results by a set of inputs.

```
family = {frozenset({1, 2}), frozenset({3})}
cache[frozenset(features)] = score
```

Never change a set while looping over it

Adding or removing elements during iteration raises `RuntimeError: Set changed size during iteration`, or silently skips elements in other languages. Build a new set instead, or loop over a copy.

```
items = {x for x in items if x >= 0}
# or: for x in set(items): ...
```

Know what a set throws away: order and duplicates

Converting a list to a set loses its order and every repeated value. That is perfect for "which distinct words appear?" and wrong for "how often does each word appear?" (use `collections.Counter`, a multiset) or "in what order did events happen?" (keep a list). Before converting, ask which information later steps need.

```
from collections import Counter
counts = Counter(text.split())   # keeps how many
most_common = counts.most_common(1)
```

Design practice: working with sets in real code: all principles (2/2)

Use bit masks for small, fixed universes

When the universe has at most a few dozen known elements (days of the week, file permissions, feature flags), represent a subset as an integer where bit *i* means "element *i* is present". Union is `|`, intersection is `&`, and the whole set fits in one machine word, so operations are extremely fast. Unix file permissions (`rwx`) work exactly this way.

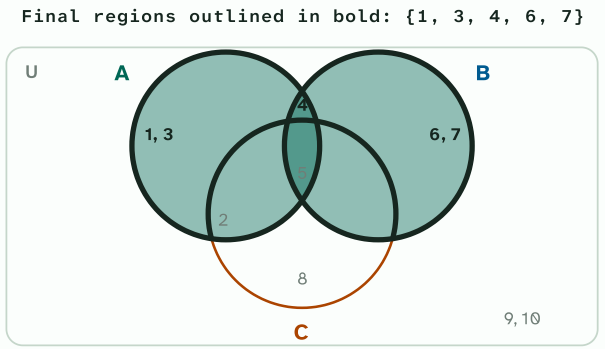
```
READ, WRITE, EXEC = 1, 2, 4      # bits 0, 1, 2
perms = READ | WRITE              # the set {read, write}
can_write = bool(perms & WRITE)   # membership test
```

Worked example: shading (A ∪ B) ∩ C′ on a Venn diagram

WORKED EXAMPLE

PROBLEM

With universe $U = \{1, 2, \ldots, 10\}$, $A = \{1, 2, 3, 4, 5\}$, $B = \{4, 5, 6, 7\}$ and $C = \{2, 5, 8\}$, compute $(A \cup B) \cap C'$, building the answer one sub-expression at a time on a Venn diagram.



(A ∪ B) ∩ C′ = {1, 3, 4, 6, 7} — check: every listed element is in A or in B, and none of them is in C

1. Plan first: place every element of U in its region, then work inside-out — shade A, grow the shading to A ∪ B, switch to the complement of C, and keep only what both shadings share. Nothing is shaded yet, so this first picture is the map for every later step.
2. Shade all of circle A, because the first sub-expression is just A = {1, 2, 3, 4, 5}. Every region inside circle A is now shaded, including the parts that also lie inside B or inside C.
3. Extend the shading to all of circle B as well, because a union keeps anything that is in either set: A ∪ B = {1, 2, 3, 4, 5, 6, 7}. The shaded area is now the whole of both circles, with the overlap shaded once, not twice.
4. Start over with a different shading: everything outside circle C, because the complement C′ = {1, 3, 4, 6, 7, 9, 10} holds all of U except 2, 5 and 8. Even the corners of the universe, outside every circle, are shaded this time.
5. Intersect the two shadings: keep only the regions that were shaded in both senses, meaning in A ∪ B and also outside C. The elements 2 and 5 drop out because they lie inside C, and 9 and 10 drop out because they were never in A ∪ B.
6. **Answer: (A ∪ B) ∩ C′ = {1, 3, 4, 6, 7}, read region by region — 1 and 3 from A only, 4 from A ∩ B outside C and 6 and 7 from B only. Check each one: it is in A or in B and it is not in C, while the only other members of A ∪ B, namely 2 and 5, fail because they lie in C.**

Worked example: De Morgan's law by a membership table

WORKED EXAMPLE

PROBLEM

Prove De Morgan's law $(A \cup B)' = A' \cap B'$ for $A = \{1, 2, 3\}$ and $B = \{3, 4\}$ in the universe $U = \{1, 2, 3, 4, 5, 6\}$, by building a membership table with one row per element of U.

Compare: the (A ∪ B)′ column and the A′ ∩ B′ column — identical

x	in A	in B	in A ∪ B	in (A ∪ B)′	in A′	in B′	in A′ ∩ B′
1	T	F	T	**F**	F	T	**F**
2	T	F	T	**F**	F	T	**F**
3	T	T	T	**F**	F	F	**F**
4	F	T	T	**F**	T	F	**F**
5	F	F	F	**T**	T	T	**T**
6	F	F	F	**T**	T	T	**T**

(A ∪ B)′ = A′ ∩ B′ = {5, 6} — check: the two highlighted columns match in all 6 rows, and 5 and 6 are exactly the elements in neither A nor B

1. Start with the raw facts only: one row for each element of U = {1, 2, 3, 4, 5, 6}, with a T where the element belongs to A = {1, 2, 3} or to B = {3, 4}. Every later column is computed from these two columns, row by row.
2. Add the union column: an element is in A ∪ B when its row has a T in the A column, in the B column, or in both. Only 5 and 6 get an F here, because they are the elements that belong to neither set.
3. Flip the union column to get (A ∪ B)′: every T becomes F and every F becomes T, because a complement keeps exactly the elements the original set missed. This column is T only for 5 and 6, so the left-hand side of the law is {5, 6}.
4. Now build the right-hand side in pieces: A′ flips the A column and B′ flips the B column, each one keeping the elements outside that single set. These two columns are the ingredients for the intersection that finishes the right-hand side.
5. Intersect the last two columns: A′ ∩ B′ is T only where a row has a T in both A′ and B′, because an intersection demands both conditions at once. That happens only for 5 and 6, the elements that are outside A and also outside B.
6. **Answer: the (A ∪ B)′ column and the A′ ∩ B′ column are identical in all 6 rows — F, F, F, F, T, T — so the two sets are equal and (A ∪ B)′ = A′ ∩ B′ = {5, 6}. The check is the row-by-row match itself, and it fits the picture: 5 and 6 are exactly the elements in neither A nor B.**

Practice: sets

PRACTICE

Type sets with braces, such as `{1, 4, 9}`. Order and spacing do not matter. Type `{}` or `∅` for the empty set.

SET OPERATIONS

Let $A = \{1, 6, 9, 10\}$ and $B = \{1, 4, 5, 7, 8, 9\}$. Find $A \Delta B$.

Type your answer

Check

Hint: keep each element that is in exactly one of A or B.

Show worked solution

New problem ↻

Go through the candidates one at a time and keep each element that is in exactly one of A or B. Result: $A \Delta B = \{4, 5, 6, 7, 8, 10\}$.

PROBLEM TYPE

- Set operations
- Complements
- Counting elements
- True or false

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: sets: sample problems with solutions

SET OPERATIONS

Let $A = \{3, 5, 8, 9, 10\}$ and $B = \{2, 4, 5, 6, 8\}$. Find $A \cup B$.

Go through the candidates one at a time and keep each element that is in A, in B, or in both. Result: $A \cup B = \{2, 3, 4, 5, 6, 8, 9, 10\}$.

COMPLEMENTS

The universal set is $U = \{1, 2, \ldots, 10\}$. With $A = \{8, 9\}$ and $B = \{5, 8\}$, find A'.

A' is everything in U that is not in A: $\{1, 2, 3, 4, 5, 6, 7, 10\}$.

COUNTING ELEMENTS

In a class, 12 students know Python, 17 know SQL, and 2 know both. How many know at least one of the two?

Inclusion–exclusion: $|P \cup S| = |P| + |S| - |P \cap S| = 12 + 17 - 2 = 27$.

TRUE OR FALSE

True or false: $\{1, 2, 3, 6\} = \{6, 3, 2, 1\}$

True. Order does not matter in a set, so listing the elements backwards gives the same set.

Check yourself: sets

CHECK YOURSELF

QUESTION 1 OF 8

Which statement is true for $A = \{1, \{2, 3\}\}$?

A $2 \in A$

B $\{2, 3\} \subseteq A$

C $|A| = 3$

D $\{2, 3\} \in A$

← Previous question

Next question →

QUESTIONS

1 2 3 4 5 6 7 8

Pick an answer to see feedback.

Start over

Check yourself: sets: answers explained

1. Which statement is true for $A = \{1, \{2, 3\}\}$?

Answer D: $\{2, 3\} \in A$

A has exactly two elements: the number 1 and the set $\{2, 3\}$. So $\{2, 3\} \in A$ and $|A| = 2$. The number 2 is not itself an element of A; it sits inside an element.

2. How many subsets does $\{a, b, c, d\}$ have?

Answer C: 16

Each of the 4 elements is independently in or out of a subset, so there are $2^4 = 16$ subsets, counting $\varnothing$ and the whole set.

3. With $U = \{1, \ldots, 6\}$, $A = \{1, 2, 3\}$ and $B = \{3, 4\}$, what is $(A \cup B)'$?

Answer C: $\{5, 6\}$

$A \cup B = \{1, 2, 3, 4\}$, and its complement in U is $\{5, 6\}$. By De Morgan's law you get the same answer from $A' \cap B' = \{4, 5, 6\} \cap \{1, 2, 5, 6\}$.

4. Which Python expression computes the symmetric difference of sets `a` and `b`?

Answer B: `a ^ b`

`^` keeps elements in exactly one of the two sets, matching $A \Delta B$. `|` is union, `&` is intersection, and `-` is difference.

5. If $|A| = 3$ and $|B| = 5$, what is $|A \times B|$?

Answer B: 15

The product rule: every one of the 3 first coordinates pairs with each of the 5 second coordinates, giving $3 \cdot 5 = 15$ ordered pairs.

6. 30 people like tea, 20 like coffee, and 8 like both. How many like at least one?

Answer D: 42

Inclusion–exclusion: $30 + 20 - 8 = 42$. Simply adding 30 and 20 counts the 8 tea-and-coffee drinkers twice.

7. You must check millions of incoming e-mail addresses against a block list of 50,000 addresses. Which design is best?

Answer B: Store the block list in a Python set and use `in`

Set membership is a hash lookup that takes about the same time whatever the set size, while list membership scans up to all 50,000 entries for every message.

8. Which pair of sets is disjoint?

Answer D: $\{1, 3, 5\}$ and $\{2, 4\}$

Disjoint means $A \cap B = \varnothing$. The odd numbers and the even numbers share nothing. The other pairs share 2, a, and both x and y ($\{x, y\}$ and $\{y, x\}$ are the same set written two ways).

Propositions and the basic connectives

CONCEPT

DEFINITION

A proposition is a sentence that is either true or false (T or F), but not both. We name propositions with letters such as p, q and r.

PROPOSITIONS

"7 is prime." (T)
"$2 + 2 = 5$." (F)
"Every even number above 2 is a sum of two primes." (Nobody knows which, yet it is one of the two.)

NOT PROPOSITIONS

"Close the door." (a command)
"Is it raining?" (a question)
"$x + 1 = 3$." (true or false only once x is chosen)

To translate English, name the simple parts first. With p: "it is raining" and q: "I take an umbrella":

It is raining and I take an umbrella.	$p \wedge q$
It is raining, but I take no umbrella.	$p \wedge \neg q$
Either it is raining or I take an umbrella, not both.	$p \oplus q$

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \oplus q$
T	T	F	T	T	F
T	F	F	F	T	T
F	T	T	F	T	T
F	F	T	F	F	F

$\neg$ is negation, $\wedge$ conjunction, $\vee$ disjunction and $\oplus$ exclusive or. One row per combination of values makes a truth table.

COMMON PITFALL

Logic's "or" is inclusive: $p \vee q$ is true when both parts are true. English often means the exclusive kind ("soup or salad?"). And "but" is just "and" with a contrast.

WHERE THIS SHOWS UP IN AI

Rule-based expert systems store knowledge as rules such as IF fever $\wedge$ cough $\wedge$ $\neg$vaccinated THEN test. They fire every rule whose condition is T.

The conditional: when is a promise broken?

CONCEPT

DEFINITION

The conditional $p \to q$ reads "if p, then q". p is the **hypothesis** (or antecedent) and q is the **conclusion** (or consequent). It is false in exactly one case: p true and q false.

p	q	$p \to q$
T	T	T
T	F	F
F	T	T
F	F	T

Think of $p \to q$ as a promise. The highlighted row is the only way to break it. When the hypothesis is false, nothing was promised, so the conditional counts as true. We say it is vacuously true.

WORKED EXAMPLE

"If $2 + 2 = 5$, then the moon is made of cheese" is true, because its hypothesis is false. "If 7 is prime, then $2 + 2 = 4$" is also true, even though the two parts have nothing to do with each other. Logic looks only at truth values; it never asks whether the parts are related.

WHY IT MATTERS

Every `if` in a program, every requirement in a specification ("if the input is valid, the output is sorted") and every step of a proof is a conditional. A test whose input never makes the hypothesis true can never catch a broken promise.

PROMISE CHECKER

"If you finish the report by Friday, then I will buy you lunch."

Here p is "you finish by Friday" and q is "I buy you lunch". For each scenario, decide whether the promise was kept or broken.

`p = T, q = T`

You finish on time. I buy you lunch.

Kept Broken

`p = T, q = F`

You finish on time. I do not buy you lunch.

Kept Broken

`p = F, q = T`

You finish late. I buy you lunch anyway.

Kept Broken

`p = F, q = F`

You finish late. I do not buy you lunch.

Kept Broken

Your score appears here.

Try again

Only if, necessary, sufficient, and the contrapositive

CONCEPT

All of these English forms mean $p \to q$. Find the hypothesis first: "if" introduces it, and "only if" introduces the conclusion.

if p, then q · p implies q	$p \to q$
q if p · q whenever p	
p only if q	
p is sufficient for q · q is necessary for p	

COMMON PITFALL

"You pass only if you submit the project" is pass → submit. Submitting is necessary, not sufficient: you can submit and still fail.

DEFINITION

The biconditional $p \leftrightarrow q$ ("p if and only if q") is true when p and q have the same truth value. It equals $(p \to q) \wedge (q \to p)$: each is necessary and sufficient for the other.

NAME	FORM	EXAMPLE
Conditional	$p \to q$	If it is a dog, then it is a mammal. **T**
Converse	$q \to p$	If it is a mammal, then it is a dog. **F** (a cat)
Inverse	$\neg p \to \neg q$	If it is not a dog, then it is not a mammal. **F** (a cat)
Contrapositive	$\neg q \to \neg p$	If it is not a mammal, then it is not a dog. **T**

THEOREM / RULE

$p \to q \equiv \neg q \to \neg p$. A conditional and its contrapositive always share a truth value. The converse and the inverse are equivalent to each other, but neither is equivalent to the original.

DESIGN TIP

Confusing a rule with its converse is a classic reasoning bug. "Every admin can delete posts" does not mean "everyone who can delete posts is an admin". When you check a permission, check the direction the rule actually states.

Lab: truth-table generator

INTERACTIVE LAB

Type any formula in p, q, r, s or pick a preset. Each sub-formula gets its own column, so you can watch the value build from the inside out. Click a row to see why it comes out that way. Keyboard forms work too: `~` `&` `|` `^` `->` `<->`.

Formula

(p → q) ∧ (q → r)

¬ ∧ ∨ ⊕ → ↔ ( ) p q r s

PRESETS

p → q | ¬p ∨ q | p ∨ ¬p | p ∧ ¬p | p ⊕ q | ¬(p ∧ q)
(p → q) ∧ (q → r) | (p → q) ∧ (q → r) → (p → r) | (p → q) ∧ q → p
(p ∨ q) ∧ ¬p → q | p ∧ q → r ∨ s

Hide sub-formula columns

CONTINGENCY (p → q) ∧ (q → r)

True in 4 of 8 rows, so it is a **contingency**: its value depends on the variables. The rows where it is T are its satisfying assignments.

ROW 2: P = T, Q = T, R = F

p → q = T (the conclusion is T)
q → r = F (hypothesis T, conclusion F: the only false case)
(p → q) ∧ (q → r) = F (q → r is F)

p	q	r	$p \to q$	$q \to r$	$(p \to q) \wedge (q \to r)$
T	T	T	T	T	T
T	T	F	T	F	F
T	F	T	F	T	F
T	F	F	F	T	F
F	T	T	T	T	T
F	T	F	T	F	F
F	F	T	T	T	T
F	F	F	T	T	T

8 rows = 2^3, one for each way to assign T or F to p, q, r. The shaded column is the whole formula.

Equivalence, tautologies, and the laws of logic

CONCEPT

DEFINITION

A tautology is true in every row, a contradiction is false in every row, and anything else is a contingency. Two formulas are logically equivalent, written $P \equiv Q$, when they agree in every row. That is the same as saying $P \leftrightarrow Q$ is a tautology.

Law	Equivalence
Double negation	$\neg\neg p \equiv p$
De Morgan	$\neg(p \wedge q) \equiv \neg p \vee \neg q \qquad \neg(p \vee q) \equiv \neg p \wedge \neg q$
Distributive	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
Implication	$p \to q \equiv \neg p \vee q$
Negated conditional	$\neg(p \to q) \equiv p \wedge \neg q$
Contrapositive	$p \to q \equiv \neg q \to \neg p$
Biconditional	$p \leftrightarrow q \equiv (p \to q) \wedge (q \to p)$
Complement, identity	$p \vee \neg p \equiv \mathrm{T} \quad p \wedge \neg p \equiv \mathrm{F} \quad p \wedge \mathrm{T} \equiv p$
Absorption	$p \vee (p \wedge q) \equiv p \qquad p \wedge (p \vee q) \equiv p$

$\wedge$ and $\vee$ are also commutative and associative. These are the set laws of Module 1 in new clothes: $\cap$ becomes $\wedge$, $\cup$ becomes $\vee$, and complement becomes $\neg$.

WORKED EXAMPLE

Simplify $\neg(p \vee \neg q) \vee (\neg p \wedge \neg q)$, one law per line:

$\equiv (\neg p \wedge \neg\neg q) \vee (\neg p \wedge \neg q)$	De Morgan
$\equiv (\neg p \wedge q) \vee (\neg p \wedge \neg q)$	double negation
$\equiv \neg p \wedge (q \vee \neg q)$	distributive
$\equiv \neg p \wedge \mathrm{T}$	complement
$\equiv \neg p$	identity

WHERE THIS SHOWS UP IN AI

SAT solvers take a huge formula and search for an assignment that makes it true. Planners, chip verifiers and schedulers encode their rules as formulas with millions of variables, and simplification laws like these keep the search fast.

Lab: equivalence checker

INTERACTIVE LAB

Two formulas are equivalent exactly when their columns match in every row. Type two formulas or pick a pair. Rows where they disagree turn red, and a single red row is enough to prove they are *not* equivalent.

Formula A

¬(p ∧ q)

Formula B

¬p ∨ ¬q

¬ ∧ ∨ ⊕ → ↔ ( )

PAIRS TO TRY

De Morgan (∧) | De Morgan (∨) | Wrong De Morgan | Implication | Contrapositive | Converse | Inverse vs converse | Negated conditional | Biconditional | Distributive | Exportation | → is not associative | Absorption | Exclusive or

Hide working columns

EQUIVALENT ≡ De Morgan (∧)

All 4 rows agree, so A ≡ B. You may replace one with the other anywhere: in a proof, in a circuit, or in an `if` condition.

"Not both" means "at least one is false". Moving ¬ inside flips ∧ to ∨.

p	q	$p \wedge q$	A: $\neg(p \wedge q)$	$\neg p$	$\neg q$	B: $\neg p \vee \neg q$	A ↔ B
T	T	T	F	F	F	F	T
T	F	F	T	F	T	T	T
F	T	F	T	T	F	T	T
F	F	F	T	T	T	T	T

The last column is A ↔ B. The formulas are equivalent exactly when that column is all T, that is, when A ↔ B is a tautology.

Predicates and quantifiers

CONCEPT

DEFINITION

A predicate $P(x)$ is a statement with a variable. It becomes a proposition once x takes a value from the domain of discourse D. With $P(x)$: "$x > 3$", $P(5)$ is true and $P(1)$ is false.

QUANTIFIERS

The universal quantifier $\forall x\, P(x)$ ("for all x") is true when $P(x)$ holds for *every* x in D. The existential quantifier $\exists x\, P(x)$ ("there exists x") is true when *at least one* x, a witness, makes $P(x)$ true.

WORKED EXAMPLE

The domain matters. $\forall x\ (x^2 \geq x)$ is true over the integers $\mathbb{Z}$, but false over the reals $\mathbb{R}$: $x = \frac{1}{2}$ gives $\frac{1}{4} < \frac{1}{2}$.

COMMON PITFALL

"Every circle is red" is $\forall x\ (\text{Circle}(x) \rightarrow \text{Red}(x))$. With $\wedge$ instead it would say every object is a red circle. "Some circle is red" uses $\wedge$: $\exists x\ (\text{Circle}(x) \wedge \text{Red}(x))$.

THEOREM / RULE

Negating a quantifier flips it and pushes ¬ inside:

$$\neg\forall x\, P(x) \equiv \exists x\, \neg P(x) \qquad \neg\exists x\, P(x) \equiv \forall x\, \neg P(x)$$

"Not everyone passed" means "someone failed".

Order matters when quantifiers are nested. Over the integers, $\forall x\, \exists y\ (y > x)$ is true: each x has a bigger y, such as $x + 1$. But $\exists y\, \forall x\ (y > x)$ is false, because no single y beats every integer. In $\forall x\, \exists y$, y may depend on x; in $\exists y\, \forall x$, one y must work for all x.

WHERE THIS SHOWS UP IN AI

Knowledge graphs and safety rules are written in this language: $\forall x\ (\text{Medication}(x) \rightarrow \exists y\, \text{Dosage}(x, y))$. A rule engine searches the facts for a counterexample, which is exactly how it flags a missing dosage.

Lab: quantifier playground

INTERACTIVE LAB

The domain is the row of objects. Pick a statement: witnesses glow green and counterexamples glow red. Click an object to change its shape, color or size, add or remove objects, and watch the truth value respond. Try emptying the domain completely.

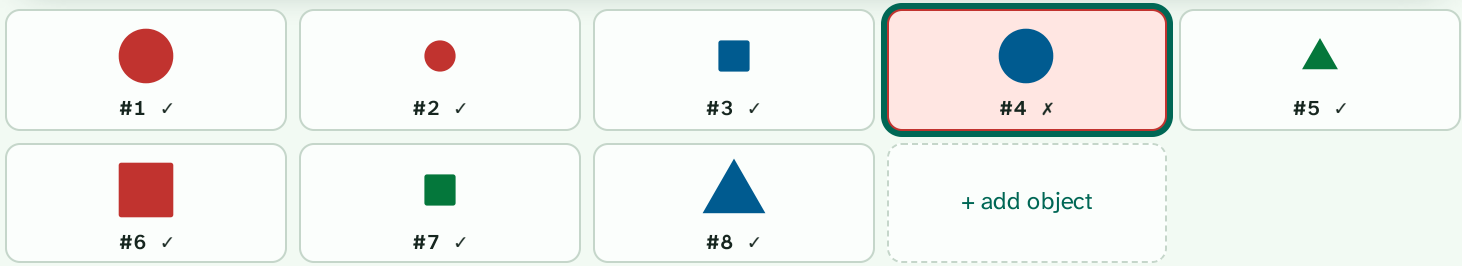
STATEMENTS

- $\forall x\ \text{Circle}(x)$
- $\exists x\ \text{Red}(x)$
- $\forall x\ (\text{Circle}(x) \rightarrow \text{Red}(x))$
- $\forall x\ (\text{Circle}(x) \wedge \text{Red}(x))$
- $\exists x\ (\text{Triangle}(x) \wedge \text{Large}(x))$
- $\exists x\ (\text{Triangle}(x) \rightarrow \text{Large}(x))$
- $\forall x\ \exists y\ (x \neq y \wedge \text{SameColor}(x, y))$
- $\forall x\ \exists y\ (\text{Square}(y) \wedge \text{SameColor}(x, y))$
- $\exists y\ \forall x\ (\text{Square}(y) \wedge \text{SameColor}(x, y))$

$\forall x\ (\text{Circle}(x) \rightarrow \text{Red}(x))$ **FALSE**

"Every circle is red." False: #4 (large blue circle) is a counterexample. One counterexample is enough to make a ∀ statement false.

Squares and triangles pass automatically: for them Circle(x) is false, so the conditional is vacuously true. Only a circle that is not red can break this statement.



#4 circle | square | triangle | red | blue | green | small | large | Remove | Reset

Negation: $\exists x\ (\text{Circle}(x) \wedge \neg\text{Red}(x))$, "Some circle is not red." It is T: a statement and its negation always have opposite values.

Logic inside your code

CONCEPT

Every `if` condition is a proposition about the program's state. Python spells the connectives as words:

LOGIC	PYTHON	NOTE
$\neg p$	`not p`	
$p \wedge q$	`p and q`	skips `q` if `p` is false
$p \vee q$	`p or q`	skips `q` if `p` is true
$p \oplus q$	`p != q`	for two Booleans
$p \to q$	`not p or q`	there is no arrow operator
$p \leftrightarrow q$	`p == q`	for two Booleans
$\forall x\, P(x)$	`all(P(x) for x in xs)`	`all([])` is `True`
$\exists x\, P(x)$	`any(P(x) for x in xs)`	`any([])` is `False`

DEFINITION

Short-circuit evaluation: Python evaluates `and` and `or` from left to right and stops as soon as the answer is known. In `a and b`, if `a` is false, `b` never runs.

WORKED EXAMPLE

Guard first, then use: `if user is not None and user.is_admin:` never touches `user.is_admin` when there is no user. Swap the two sides and a missing user crashes the program.

REFACTORING WITH DE MORGAN

`if not (is_weekend or is_holiday):` tests the same thing as `if not is_weekend and not is_holiday:`, and the second version reads as "a normal working day".

WHERE THIS SHOWS UP IN AI

Data filtering in pandas and NumPy uses Boolean masks such as `df[(df.age > 30) & ~df.churned]`. There `&`, `|` and `~` work row by row, and the parentheses are required because `&` binds tighter than `>`.

Code: a truth-table generator

CODE WALKTHROUGH

Python can build any truth table by trying every assignment. Switch to **Trace** to watch `p`, `q` and `r` run through all eight rows.

truth_table.py

Read | Edit & run | Trace

```python
from itertools import product

def implies(a, b):
    return (not a) or b

def formula(p, q, r):
    chain = implies(p, q) and implies(q, r)
    return implies(chain, implies(p, r))

col = []
for p, q, r in product([True, False], repeat=3):
    col.append(formula(p, q, r))
    print(int(p), int(q), int(r), "|", int(col[-1]))
print("tautology?", all(col), "satisfiable?", any(col))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
1 1 1 | 1
1 1 0 | 1
1 0 1 | 1
1 0 0 | 1
```

LINE 1 OF 14

Show all notes in code

```python
from itertools import product
```

Imports `product` from the standard library module `itertools`. `product` builds the Cartesian product from Module 1: every combination of values. The rows of a truth table are exactly $\{T, F\} \times \{T, F\} \times \{T, F\}$.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Prints the full truth table of $((p \to q) \wedge (q \to r)) \to (p \to r)$ and then reports whether it is a tautology and whether it is satisfiable.

Code: a truth-table generator: every line explained

1	`from itertools import product`	Imports `product` from the standard library module `itertools`. `product` builds the Cartesian product from Module 1: every combination of values. The rows of a truth table are exactly $\{T,F\} \times \{T,F\} \times \{T,F\}$.
3	`def implies(a, b):`	Python has `not`, `and` and `or` but no arrow, so we define $a \to b$ ourselves as a function with two Boolean inputs.
4	`    return (not a) or b`	The implication law, $a \to b \equiv \neg a \vee b$. When `a` is `False` the result is `True` whatever `b` is: vacuous truth. The only `False` result comes from `a = True, b = False`.
6	`def formula(p, q, r):`	The formula under test gets its own function, so testing a different formula means changing only its body. It takes the three variables as inputs.
7	`    chain = implies(p, q) and implies(q, r)`	The left part, $(p \to q) \wedge (q \to r)$: "$p$ leads to q and q leads to r". Naming it `chain` keeps the next line short enough to read.
8	`    return implies(chain, implies(p, r))`	The whole formula, $((p \to q) \wedge (q \to r)) \to (p \to r)$: if the chain holds, then p leads to r. This is the rule of inference called hypothetical syllogism, which Module 3 uses in proofs.
10	`col = []`	An empty list, `col` (short for column), that will collect the formula's value in each row, so we can classify the whole column after the loop. At the end it holds 8 Booleans.
11	`for p, q, r in product([True, False], repeat=3):`	`product([True, False], repeat=3)` yields the 8 triples `(True, True, True)`, `(True, True, False)`, …, `(False, False, False)`, in textbook order with T first. Each triple is unpacked into `p`, `q` and `r`.
12	`    col.append(formula(p, q, r))`	Evaluates the formula for this row and stores the result at the end of `col`. Python calls `formula`, which calls `implies` four times in most rows but only three times in the two rows where $p \to q$ is false: there the short-circuit `and` inside `chain` never calls `implies(q, r)`.
13	`    print(int(p), int(q), int(r), "|", int(col[-1]))`	`col[-1]` is the last item in the list: the value just added. `int(True)` is 1 and `int(False)` is 0, so each row prints as digits, such as `1 0 1 | 1`. Computer scientists often write truth tables with 1 for T and 0 for F; Module 12 does the same for circuits.
14	`print("tautology?", all(col), "satisfiable?", any(col))`	`all(col)` asks "is it T in every row?", the definition of a tautology and the quantifier ∀. `any(col)` asks "is it T in some row?", which means satisfiable, the quantifier ∃. Output: `tautology? True satisfiable? True`.

Code: checking equivalence by brute force

CODE WALKTHROUGH

Two formulas are equivalent when no row tells them apart. This program searches the rows for a disagreement, which is also how you find a counterexample by machine.

equivalence.py

Read | Edit & run | Trace

```python
from itertools import product

def implies(a, b):
    return (not a) or b

def equivalent(f, g):
    for p, q in product([True, False], repeat=2):
        if f(p, q) != g(p, q):
            return f"NO, they differ at p={p}, q={q}"
    return "yes, all 4 rows agree"

others = {"q → p": lambda p, q: implies(q, p),
          "¬q → ¬p": lambda p, q: implies(not q, not p),
          "¬p ∨ q": lambda p, q: (not p) or q}
for name, g in others.items():
    print(f"p → q  vs  {name}:", equivalent(implies, g))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
p → q  vs  q → p: NO, they differ at p=True, q=False
p → q  vs  ¬q → ¬p: yes, all 4 rows agree
p → q  vs  ¬p ∨ q: yes, all 4 rows agree
```

LINE 1 OF 16

Show all notes in code

```python
from itertools import product
```

Imports `product`, which generates every combination of truth values: the rows of the truth table.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Compares $p \to q$ with its converse, its contrapositive and $\neg p \vee q$, reporting either "equivalent" or the first row where they differ.

Code: checking equivalence by brute force: every line explained

1	`from itertools import product`	Imports `product`, which generates every combination of truth values: the rows of the truth table.
3	`def implies(a, b):`	Defines the arrow, because Python has no → operator.
4	`    return (not a) or b`	The implication law, $a \to b \equiv \neg a \vee b$. This is false only when `a` is true and `b` is false.
6	`def equivalent(f, g):`	Takes two functions, `f` and `g`, each standing for a formula in `p` and `q`. In Python, functions are values, so they can be passed as arguments like numbers.
7	`    for p, q in product([True, False], repeat=2):`	Visits the four rows (T, T), (T, F), (F, T), (F, F). Checking every row is what makes the answer certain rather than a guess based on a few examples.
8	`        if f(p, q) != g(p, q):`	Evaluates both formulas on this row. `!=` between Booleans is exclusive or: true exactly when the two formulas disagree.
9	`            return f"NO, they differ at p={p}, q={q}"`	A disagreement is a counterexample to "f ≡ g", so the function stops immediately and reports the row. There is no need to check the remaining rows: one counterexample settles it.
10	`    return "yes, all 4 rows agree"`	This line runs only if the loop finished without finding a difference. Agreement in every row is the definition of logical equivalence.
12	`others = {"q → p": lambda p, q: implies(q, p),`	A dictionary from a label to a formula. A `lambda` is a one-line nameless function: `lambda p, q: implies(q, p)` is the converse $q \to p$. The dictionary continues on the next two lines.
13	`          "¬q → ¬p": lambda p, q: implies(not q, not p),`	The contrapositive $\neg q \to \neg p$: swap the parts and negate both. The theory says it should match the original.
14	`          "¬p ∨ q": lambda p, q: (not p) or q}`	The implication law written out directly, $\neg p \vee q$. The closing brace ends the dictionary.
15	`for name, g in others.items():`	Loops over the label and formula pairs in the order they were written. `items()` gives each pair as a tuple, unpacked into `name` and `g`.
16	`    print(f"p → q  vs  {name}:", equivalent(implies, g))`	Compares the original conditional (the `implies` function itself) with each candidate and prints the verdict. Output starts `p → q vs q → p: NO, they differ at p=True, q=False`: the converse fails on the row where the original is false.

Code: quantifiers with all() and any()

CODE WALKTHROUGH

`all` is ∀ and `any` is ∃. The noisy `passed` function prints each time it is called, so you can see short-circuit evaluation stop the search early.

quantifiers.py

Read | Edit & run | Trace

```
1  def passed(name, score):
2      print("  checking", name)
3      return score >= 60
4
5  scores = [("Ana", 91), ("Ben", 58), ("Cai", 77)]
6  everyone = all(passed(n, s) for n, s in scores)
7  someone = any(passed(n, s) for n, s in scores)
8  print("∀ passed:", everyone, "| ∃ passed:", someone)
9  print("empty domain:", all([]), any([]))
10
11 D = [1, 2, 3]
12 forall_exists = all(any(x + y == 4 for y in D) for x in D)
13 exists_forall = any(all(x + y == 4 for x in D) for y in D)
14 print("∀x ∃y:", forall_exists, "| ∃y ∀x:", exists_forall)
```

OUTPUT · Runs with real Python 3 in the interactive version

▶ Run

```
Output:
  checking Ana
  checking Ben
  checking Ana
∀ passed: False | ∃ passed: True
```

LINE 1 OF 14

Show all notes in code

```
def passed(name, score):
```

Defines the predicate Passed(x). Its first line prints a message so we can see exactly which students get checked.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Evaluates ∀ and ∃ statements over a list of students, shows that `all` and `any` stop as soon as the answer is known, tests the empty domain, and shows that swapping nested quantifiers changes the answer.

Code: quantifiers with all() and any(): every line explained

1	`def passed(name, score):`	Defines the predicate $\text{Passed}(x)$. Its first line prints a message so we can see exactly which students get checked.
2	`    print("  checking", name)`	A deliberate side effect: every call announces itself. Real predicates should not print; this one does so that short-circuiting becomes visible.
3	`    return score >= 60`	The actual test. It returns `True` when the score is at least 60, so `Passed` is true for Ana (91) and Cai (77) and false for Ben (58).
5	`scores = [("Ana", 91), ("Ben", 58), ("Cai", 77)]`	The domain of discourse: a list of three (name, score) pairs. Every quantifier below ranges over this list.
6	`everyone = all(passed(n, s) for n, s in scores)`	$\forall x\ \text{Passed}(x)$. The generator calls `passed` one student at a time. Ana passes, Ben fails, and `all` stops right there, because one counterexample already makes the answer `False`. Cai is never checked, so this line prints only two "checking" messages.
7	`someone = any(passed(n, s) for n, s in scores)`	$\exists x\ \text{Passed}(x)$. Ana passes on the very first call, so `any` has its witness and stops immediately. This line prints a single "checking Ana".
8	`print("∀ passed:", everyone, "| ∃ passed:", someone)`	Prints both answers: `∀ passed: False | ∃ passed: True`. Look above it in the output: three "checking" lines in total (Ana, Ben, then Ana again), not six. That is short-circuit evaluation at work.
9	`print("empty domain:", all([]), any([]))`	Quantifiers over an empty domain. `all([])` is `True` because there is no counterexample (vacuous truth). `any([])` is `False` because there is no witness. Output: `empty domain: True False`.
11	`D = [1, 2, 3]`	A new, tiny domain for nested quantifiers.
12	`forall_exists = all(any(x + y == 4 for y in D) for x in D)`	$\forall x\ \exists y\ (x + y = 4)$. Read it from the outside in: `all` runs over each `x`, and for each one the inner `any` searches for a partner `y`. Here 1 pairs with 3, 2 with 2, and 3 with 1. Each `x` gets its own `y`, so the result is `True`.
13	`exists_forall = any(all(x + y == 4 for x in D) for y in D)`	The quantifiers swapped: $\exists y\ \forall x\ (x + y = 4)$. Now `any` tries one `y` at a time, and `all` asks whether that single `y` works for every `x` at once. No `y` does, so the result is `False`.
14	`print("∀x ∃y:", forall_exists, "| ∃y ∀x:", exists_forall)`	Prints `∀x ∃y: True | ∃y ∀x: False`. Same symbols, different order, different meaning.

Design practice: writing conditions people can trust

DESIGN PRACTICE

Every condition in a program is a small piece of logic, and the laws of this module are tools for making it clearer and safer. Click each principle (or use ↓) to compare code to avoid with code to write.

01 Name Booleans as yes/no questions
02 Never compare a Boolean to True or False
03 Push negations inward with De Morgan
04 Avoid double negatives
05 Flatten nested ifs with guard clauses
06 Say ∀ and ∃ with all() and any()
07 Order conditions for short-circuit safety
08 Make branches exhaustive and exclusive

Principle 1 of 8

Name Booleans as yes/no questions

A Boolean variable holds a proposition, so name it like one: `is_paid`, `has_access`, `can_retry`. Then `if is_paid:` reads as a sentence. A name like `flag` or `status` forces every reader to look up whether `True` means success or failure.

✗ AVOID THIS

```
flag = check(order)
if flag:
    ship(order)
```

✓ DO THIS

```
is_paid = payment_cleared(order)
if is_paid:
    ship(order)
```

Design practice: writing conditions people can trust: all principles (1/2)

Name Booleans as yes/no questions

A Boolean variable holds a proposition, so name it like one: `is_paid`, `has_access`, `can_retry`. Then `if is_paid:` reads as a sentence. A name like `flag` or `status` forces every reader to look up whether `True` means success or failure.

```
is_paid = payment_cleared(order)
if is_paid:
    ship(order)
```

Never compare a Boolean to True or False

`is_valid == True` is the proposition $p \leftrightarrow \mathrm{T}$, which is equivalent to p itself, so the comparison only adds noise. It can also mislead: a value such as `2` or a non-empty list counts as true in an `if`, yet `2 == True` and `[1] == True` are both `False`, so the comparison and the plain `if` disagree. To test for "no value", use `is None`.

```
if is_valid:
    save()
if not done:
    retry()
```

Push negations inward with De Morgan

A `not` in front of a long parenthesised condition makes the reader evaluate everything and then flip it. De Morgan's laws, $\neg(p \wedge q) \equiv \neg p \vee \neg q$ and $\neg(p \vee q) \equiv \neg p \wedge \neg q$, move the `not` onto each part, where comparisons simply flip: `>=` becomes `<`.

```
if age < 18 or not has_id:
    refuse_entry()
```

Avoid double negatives

Names such as `not_found` or `is_not_empty` lead to conditions like `if not is_not_empty(cart)`, which is $\neg\neg p$. It equals p, but every reader has to stop and work that out. Name the positive case and let a single `not` appear only where you mean it.

```
if is_empty(cart):
    show_empty_message()
```

Flatten nested ifs with guard clauses

Nested `if`s express $p \to (q \to r)$. The exportation law says this equals $(p \wedge q) \to r$, and De Morgan turns the failing case into $\neg p \vee \neg q$. Handle that case first and return early: the main work then sits at the left margin instead of several levels deep.

```
def publish(post):
    if post is None or not post.approved:
        return
    send(post)
```

Say ∀ and ∃ with all() and any()

A loop with a flag variable is a hand-written quantifier. `all(...)` is ∀ and `any(...)` is ∃. They are shorter, they cannot get the flag logic wrong, and they stop at the first counterexample or witness. Remember that `all([])` is `True`: if an empty input should fail, check for it explicitly.

```
ok = all(score >= 0 for score in scores)
```

Design practice: writing conditions people can trust: all principles (2/2)

Order conditions for short-circuit safety

Because `and` stops at the first false part, put the protective test first: check that something exists before you use it. And never hide work with side effects (saving, logging, counting) inside a condition, because short-circuiting may silently skip it.

```
if items and items[0] > 0:
    use(items)
if not is_valid(x):
    log_error(x)
```

Make branches exhaustive and exclusive

An `if/elif/else` chain is a truth table in disguise: every possible input should land in exactly one branch. Order the tests so each one may assume the earlier ones failed, and finish with an `else` that handles, or loudly rejects, whatever is left. A missing case leaves a variable unset, and that is where bugs hide.

```
if weight > 20:
    rate = "freight"
elif weight > 2:
    rate = "parcel"
else:
    rate = "letter"
```

Worked example: a truth table, column by column

WORKED EXAMPLE

PROBLEM

Build the truth table for $[(p \to q) \wedge (q \to r)] \to (p \to r)$ and classify it: tautology, contradiction, or contingency?

p	q	r	$p \to q$	$q \to r$	$(p \to q) \wedge (q \to r)$	$[(p \to q) \wedge (q \to r)] \to (p \to r)$
T	T	T	T	T	T	T
T	T	F	T	F	F	T
T	F	T	F	T	F	T
T	F	F	F	T	F	T
F	T	T	T	T	T	T
F	T	F	T	F	F	T
F	F	T	T	T	T	T
F	F	F	T	T	T	T

Answer: a tautology — T in all 8 rows

1. Start with the inputs. Three variables have 2 × 2 × 2 = 8 combinations of T and F, so the table needs 8 rows. Writing every combination first matters: one missing row could hide the single case that breaks the formula.
2. Work inside-out, smallest piece first: p → q. A conditional is false in exactly one situation — hypothesis true, conclusion false — so this column is F exactly in the rows with p = T and q = F, and T everywhere else.
3. The next piece is q → r, the same rule with the roles shifted: F only where q = T and r = F. Building one column at a time means every entry can be checked against a single rule.
4. Combine the two finished columns with ∧. A conjunction is true only where both of its pieces are true. This column is also the hypothesis of the whole formula, so it marks the rows that could possibly make the formula false.
5. Finish with the whole formula: hypothesis → (p → r). It fails only in a row where the hypothesis is true and p → r is false. Read down the new column: there is no F in it.
6. Check the dangerous rows — the highlighted ones, where the hypothesis is true. In each of them the conclusion p → r is true as well, which is exactly why the implication survived. The other rows pass automatically, because a conditional with a false hypothesis is true.
7. **Answer: the formula is a tautology — true in all 8 rows. This pattern is the hypothetical syllogism: if p → q and q → r both hold, then p → r must hold. The check in the previous step is the proof: every row that could have broken it was examined.**

Worked example: rewriting a negated conditional

WORKED EXAMPLE

PROBLEM

Rewrite $\neg(p \to q)$ as a formula with no $\to$ and no outer $\neg$, naming the law used at each step, and prove that the result is equivalent to the original formula.

$\neg(p \to q)$	starting formula
$\neg(\neg p \vee q)$	implication law: $p \to q \equiv \neg p \vee q$
$\neg\neg p \wedge \neg q$	De Morgan's law: $\neg(X \vee Y) \equiv \neg X \wedge \neg Y$
$p \wedge \neg q$	double negation: $\neg\neg p \equiv p$

p	q	$\neg(p \to q)$	$p \wedge \neg q$
T	F	T	T
T	T	F	F
F	T	F	F
F	F	F	F

Answer: ¬(p → q) ≡ p ∧ ¬q · check: both columns agree in all 4 rows — T, F, F, F

1. Start with ¬(p → q) and the plan: turn the conditional into a disjunction first, because the implication law is the only law that removes a →. Each later law then has the shape it needs to apply.
2. Apply the implication law, p → q ≡ ¬p ∨ q, inside the outer negation, so the formula becomes ¬(¬p ∨ q). The arrow is gone, but the outer ¬ still covers the whole disjunction, so one more law is needed to move it inside.
3. Apply De Morgan's law, ¬(X ∨ Y) ≡ ¬X ∧ ¬Y: the ∨ flips to ∧ and each part is negated, giving ¬¬p ∧ ¬q. Pushing ¬ inside is what turns "not a conditional" into a positive claim about p and q.
4. Apply double negation, ¬¬p ≡ p, to clean up the first part, and the formula becomes p ∧ ¬q. The rewrite is done — no → and no outer ¬ remain — but a chain of rewrites still has to be checked against the original.
5. Check the one revealing row, p = T and q = F: the original gives ¬(T → F) = ¬F = T, and the rewrite gives T ∧ ¬F = T ∧ T = T. This row matters most because it is the only row where either formula is true, so agreement here carries the proof.
6. **Answer: ¬(p → q) ≡ p ∧ ¬q, because the two columns agree in all 4 rows, reading T, F, F, F. In words, the only way to break the promise p → q is for p to be true and q false, and the table is the check that nothing else breaks it.**

Worked example: is this argument valid?

WORKED EXAMPLE

PROBLEM

Premises: $p \to q$, $q \to r$, and r. Conclusion: p. Is this argument valid? Search the truth table row by row for a counterexample.

Premises: $p \to q$, $q \to r$, r · Conclusion: p

p	q	r	$p \to q$	$q \to r$	r	p
T	T	T	T	T	T	T
T	T	F	T	F	F	T
T	F	T	F	T	T	T
T	F	F	F	T	F	T
F	T	T	T	T	T	F
F	T	F	T	F	F	F
F	F	T	T	T	T	F
F	F	F	T	T	F	F

Answer: invalid — the highlighted row is a counterexample: every premise true, the conclusion false

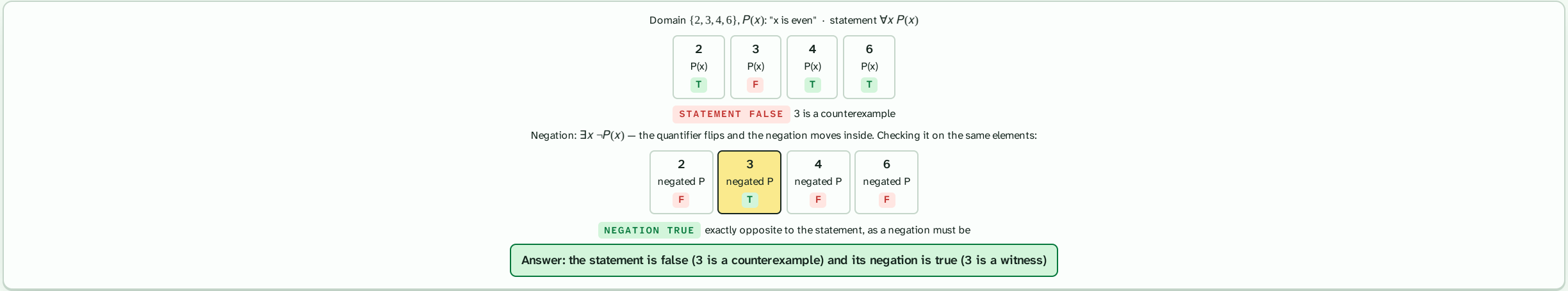
1. An argument is valid when there is no row where the premises are all true and the conclusion is false. With three variables there are 8 rows to search, and the premises here are p → q, q → r, r, with conclusion p.
2. Evaluate the premises first, one column at a time, using the same inside-out rule as any truth table. A row survives the search only if it earns a T in every premise column at once, so these columns tell us where a counterexample could possibly hide.
3. Now narrow the search: keep only the 3 rows where every premise is true, namely row 1 (p = T, q = T, r = T) and row 5 (p = F, q = T, r = T) and row 7 (p = F, q = F, r = T). Rows with a false premise can never be counterexamples, because a counterexample needs the premises to hold while the conclusion fails.
4. Test the conclusion p in each kept row: it fails in row 5 (p = F, q = T, r = T) and row 7 (p = F, q = F, r = T). In those rows every premise is true while the conclusion is false, which is exactly the shape of a counterexample.
5. Zoom in on row 5 (p = F, q = T, r = T): the premises p → q, q → r, r are all true there, yet the conclusion p is false. One such row is enough — an argument is invalid as soon as a single counterexample exists, no matter how the other rows behave.
6. **Answer: the argument is invalid — row 5 (p = F, q = T, r = T) and row 7 (p = F, q = F, r = T) are counterexamples with all premises true and the conclusion false. Nothing in the premises forces p itself to be true: the chain runs forward from p, so truth can enter at q or r without p ever holding. An argument with even one counterexample row is invalid, however many rows pass.**

Worked example: quantifiers and negation on a small domain

WORKED EXAMPLE

PROBLEM

Let the domain be $\{2, 3, 4, 6\}$ and let $P(x)$ mean "x is even". Is $\forall x\, P(x)$ true or false? Then write its negation and check the negation on the same domain.



1. The domain is {2, 3, 4, 6} and the predicate is P(x): "x is even". A quantified statement over a finite domain is checked by testing the predicate on each element in turn, so the first job is to lay the domain out where every element can be tested.
2. Evaluate the predicate element by element: P(2) is T, P(3) is F, P(4) is T, P(6) is T. No quantifier has been applied yet — this row of values is the raw evidence that both the statement and, later, its negation will be judged from.
3. Apply the universal quantifier: it demands that every element pass, but 3 fails, since P(3) is F. One counterexample is enough to make a universal statement false, so this statement is false.
4. Negate the statement by the quantifier rules: negating ∀x P(x) gives ∃x with the negation pushed inside onto P. The quantifier flips and the predicate is negated — and the flipped statement must now be checked against this same table of values, not taken on faith.
5. Evaluate the negation on the same elements: negating P flips each value, giving the negation at 2 is F, the negation at 3 is T, the negation at 4 is F, the negation at 6 is F. The element 3 makes the negated predicate true, so it witnesses the existential negation, and the negation comes out true — exactly opposite to the original statement, as a negation must.
6. **Answer: the statement ∀x P(x) over {2, 3, 4, 6} is false, and its negation ∃x with negated P is true. Both were checked against the same predicate table, and the quantifier flip is what made the negation mechanical instead of a guess.**

Practice: logic

PRACTICE

Type formulas with `~` for ¬, `&` for ∧, `|` for ∨, `^` for ⊕, `->` for → and `<->` for ↔ (the symbols themselves work too). Any formula equivalent to the answer is accepted.

EVALUATE A FORMULA

Let p be false, q be false. What is the truth value of

$$\neg q \to \neg(p \to q)$$

True | False

Hint: evaluate the innermost parts first, then work outward.

Show worked solution | New problem ↻

Work from the inside out:

1. $\neg q$ is **T**: q is F, and ¬ flips it.
2. $p \to q$ is **T**: the hypothesis is F, so it is vacuously true.
3. $\neg(p \to q)$ is **F**: p → q is T, and ¬ flips it.
4. $\neg q \to \neg(p \to q)$ is **F**: hypothesis T, conclusion F: the only false case.

So the formula is **false**.

PROBLEM TYPE

- Evaluate a formula
- English to symbols
- Equivalent or not?
- Quantifiers over a domain

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: logic: sample problems with solutions

EVALUATE A FORMULA

Let p be false, q be false, r be true. What is the truth value of

$$\neg q \leftrightarrow \neg(r \leftrightarrow p)$$

Work from the inside out:

1. $\neg q$ is **T**: q is F, and ¬ flips it.
2. $r \leftrightarrow p$ is **F**: the two sides differ.
3. $\neg(r \leftrightarrow p)$ is **T**: r ↔ p is F, and ¬ flips it.
4. $\neg q \leftrightarrow \neg(r \leftrightarrow p)$ is **T**: both sides are T.

So the formula is **true**.

ENGLISH TO SYMBOLS

Let p stand for "you practice daily" and q for "your typing speed improves". Write in symbols:

"It is not the case that either you practice daily or your typing speed improves."

$\neg(p \vee q)$, typed `~(p | q)`. This negates the whole disjunction. By De Morgan it equals $\neg p \wedge \neg q$: neither one happens.

EQUIVALENT OR NOT?

Are these two formulas logically equivalent?

$$p \to (q \to r) \qquad \text{and} \qquad (p \wedge q) \to r$$

Equivalent. They agree in all 8 rows of the truth table. The exportation law: nested ifs equal one if with "and".

QUANTIFIERS OVER A DOMAIN

The domain is $D = \{-4, 1, 2, 4, 6\}$. Is this statement true or false?

$$\forall x\,(x \text{ is even} \to x > -2)$$

False. -4 is even but not greater than -2, so it is a counterexample.

Check yourself: logic

CHECK YOURSELF

QUESTION 1 OF 10

Which sentence is a proposition?

A "Close the window."

B "17 is a prime number."

C "Is 17 prime?"

D "$x + 2 = 5$."

← Previous question | Next question →

QUESTIONS

1 2 3 4 5
6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: logic: answers explained (1/2)

1. Which sentence is a proposition?

Answer B: "17 is a prime number."

A proposition must be true or false. "17 is a prime number" is true. A command and a question have no truth value, and $x + 2 = 5$ becomes true or false only once x is chosen, which makes it a predicate.

2. When is $p \to q$ false?

Answer C: When p is true and q is false

A conditional is a promise that breaks only when the hypothesis happens and the conclusion does not. When p is false the conditional is vacuously true, whatever q is.

3. Let p: "you can board" and q: "you have a ticket". Which formula means "You can board only if you have a ticket"?

Answer D: $p \to q$

"Only if" introduces the conclusion: boarding requires a ticket, so boarding → ticket. A ticket is necessary but not sufficient (you could still miss the flight), so $q \to p$ says something different.

4. What is the contrapositive of "If a number is divisible by 4, then it is even"?

Answer D: If a number is not even, then it is not divisible by 4.

The contrapositive swaps and negates: $\neg q \to \neg p$. It is equivalent to the original, so it is true as well. "If a number is even, then it is divisible by 4" is the converse (false: 6 is even but not divisible by 4) and "If a number is not divisible by 4, then it is not even" is the inverse (also false).

5. Which formula is logically equivalent to $\neg(p \wedge q)$?

Answer A: $\neg p \vee \neg q$

De Morgan: "not both" means "at least one is false". $\neg p \wedge \neg q$ says both are false, which is stronger. In the row p = T, q = F, $\neg(p \wedge q)$ is T but $\neg p \wedge \neg q$ is F.

6. The formula $(p \wedge (p \to q)) \to q$ is true in all four rows of its truth table. It is therefore a...

Answer C: tautology

A tautology is true for every assignment of truth values. This one is modus ponens, the most used rule of inference in proofs (Module 3).

7. What is the negation of $\forall x\ (x^2 > 0)$?

Answer A: $\exists x\ (x^2 \leq 0)$

$\neg\forall x\ P(x) \equiv \exists x\ \neg P(x)$: flip the quantifier and negate the inside. The negation of $x^2 > 0$ is $x^2 \leq 0$, not $x^2 < 0$. Over the integers the negation is true, with witness $x = 0$.

Check yourself: logic: answers explained (2/2)

8. Over the integers, which statement is true?

Answer B: $\forall x\ \exists y\ (y > x)$

In $\forall x\ \exists y$, the choice of y may depend on x: take $y = x + 1$. In $\exists y\ \forall x$, one y would have to beat every integer, including itself, which is impossible. Swapping different quantifiers changes the meaning.

9. Why is `if user is not None and user.age >= 18:` safe when `user` is `None`?

Answer D: Short-circuit evaluation: once the left side is `False`, the right side never runs `and` stops at the first false operand. With no user, `user is not None` is `False`, so `user.age` is never evaluated. Reverse the two tests and the code crashes with an `AttributeError`.

10. What does `all([])` return in Python?

Answer C: `True`

`all` is ∀. Over an empty domain there is no counterexample, so the statement is vacuously true. `any([])` is `False` because there is no witness. Code that must reject empty input has to check for it separately.

What a proof is, and how to read one

CONCEPT

DEFINITION

A proof is a chain of statements. Each one is a hypothesis, an axiom, a definition, an earlier result, or follows from earlier statements by a rule of logic. The last statement is the claim being proved.

Axiom	Accepted without proof, such as: the sum and the product of two integers are integers.
Definition	The exact meaning of a word, such as: an integer n is even if $n = 2k$ for some integer k.
Theorem	An important statement that has been proved.
Lemma	A smaller theorem proved to help prove a bigger one.
Conjecture	Believed true, but not yet proved.

WHERE THIS SHOWS UP IN AI

Proof assistants such as Lean check every step of a proof by machine. AI systems now write Lean proofs, and once Lean accepts one it can be trusted, even though the AI that wrote it sometimes errs.

Definitions this module uses

- n is **even** if $n = 2k$, and **odd** if $n = 2k + 1$, for some integer k.
- a **divides** b, written $a \mid b$, if $b = ak$ for some integer k.
- An integer $p > 1$ is **prime** if its only positive divisors are 1 and p.
- r is **rational** if $r = a/b$ for integers a, b with $b \neq 0$.

HOW TO READ AND WRITE A PROOF

Reading: find the hypothesis and the conclusion, then check that each line follows from earlier ones. **Writing:** state your assumption ("Suppose..."), turn definitions into algebra, justify every step, signpost ("so", "Case 1"), and end by naming what you proved, marked ∎.

Lab: rules of inference and fallacies

INTERACTIVE LAB

DEFINITION

An argument is valid when its conclusion is true in *every* row where all its premises are true. A rule of inference is a small valid argument you may use as one step of a proof. Pick a rule: rows with every premise true are shaded, and a red row is a counterexample.

- Modus ponens — $p \to q,\ p \therefore q$
- Modus tollens — $p \to q,\ \neg q \therefore \neg p$
- Hypothetical syllogism — $p \to q,\ q \to r \therefore p \to r$
- Disjunctive syllogism — $p \lor q,\ \neg p \therefore q$
- Addition — $p \therefore p \lor q$
- Simplification — $p \land q \therefore p$
- Conjunction — $p,\ q \therefore p \land q$
- Affirming the consequent — $p \to q,\ q \therefore p$
- Denying the antecedent — $p \to q,\ \neg p \therefore \neg q$
- Your own argument — type it

$p \to q$
p
$\therefore q$

If it rains, the match is cancelled.
It rains.
∴ The match is cancelled.

VALID **Modus ponens**

p	q	P1: $p \to q$	P2: p	C: q
T	T	T	T	T
T	F	F	T	F
F	T	T	F	T
F	F	T	F	F

1 row makes every premise true (shaded), and the conclusion is true in it. So (P1 ∧ P2) → C is a tautology.

The workhorse of every proof: from "if p then q" and "p", conclude "q".

Direct proof, contrapositive, and contradiction

CONCEPT

Three ways to prove a statement. Each card gives the plan and one complete example.

Direct proof

To prove $p \to q$: assume p, then reason forward until you reach q.

WORKED EXAMPLE

Claim. If m and n are odd, then $m + n$ is even.

Proof. Write $m = 2a + 1$ and $n = 2b + 1$ for integers a, b. Then $m + n = 2a + 2b + 2 = 2(a + b + 1)$. Since $a + b + 1$ is an integer, $m + n$ is even. ∎

Proof by contrapositive

To prove $p \to q$, prove the equivalent $\neg q \to \neg p$ directly.

WORKED EXAMPLE

Claim. If n^2 is even, then n is even.

Proof. We show: if n is odd, then n^2 is odd. Write $n = 2k + 1$. Then $n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$, which is odd. So the contrapositive holds, and so does the claim. ∎

Proof by contradiction

Assume the statement is false and derive something impossible.

WORKED EXAMPLE

Claim. $\sqrt{2}$ is irrational.

Proof. Suppose $\sqrt{2} = a/b$ in lowest terms. Then $a^2 = 2b^2$, so a^2 is even and a is even: $a = 2c$. Then $b^2 = 2c^2$, so b is even too. Both even contradicts lowest terms. ∎

WHICH ONE TO TRY

Start with a direct proof. Switch to the contrapositive when "not q" gives you more to work with than p does, as with n^2 even. Reach for contradiction when the claim says something does *not* exist or is irrational or infinite.

WHERE THIS SHOWS UP IN AI

Chain-of-thought reasoning in language models is a sequence of proof-like steps. It is only as reliable as its weakest step, so systems that check each step, the way a reader checks a proof, catch errors that a final answer hides.

Lab: proof builder

INTERACTIVE LAB

Click steps on the left to add them in order, and give each one a reason. One step does not belong. A good proof states its assumption, justifies every line, and ends by naming what it proved.

Even + even | n^2 even ⇒ n even | √2 is irrational | Infinitely many primes

DIRECT PROOF **Theorem.** If m and n are even integers, then $m + n$ is even.

STEPS TO PLACE (CLICK TO ADD)

- So $m + n = 2a + 2b = 2ab$.
- So $m + n = 2a + 2b = 2(a + b)$.
- So $m + n$ is two times an integer: $m + n$ is even. ∎
- Then $m = 2a$ and $n = 2b$ for some integers a and b.
- $a + b$ is an integer.

YOUR PROOF

1 Let m and n be even integers. — Given (hypothesis) ↑ ×

Check my proof | Show the full proof | Start over

1 of 5 steps placed. Check whenever you like.

Cases, existence, and counterexamples

CONCEPT

DEFINITION

In a proof by cases, split the situation into cases that together cover every possibility, and prove the claim in each one. It rests on the law $(p_1 \lor p_2) \to q \equiv (p_1 \to q) \land (p_2 \to q)$.

WORKED EXAMPLE

Claim. For every integer n, $n^2 + n$ is even.

Case 1: n is even, $n = 2k$. Then $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$, which is even.

Case 2: n is odd, $n = 2k + 1$. Then $n^2 + n = (2k + 1)(2k + 2) = 2(2k + 1)(k + 1)$, which is even.

Every integer is even or odd, so the cases cover everything. ∎

DESIGN TIP

The cases must be exhaustive. Forgetting one, such as $n = 0$ or negative numbers, is like an `if/elif` chain with a missing branch.

EXISTENCE AND DISPROOF

To prove $\exists x\, P(x)$, show a witness: "there is a prime above 100" is proved by 101. To disprove $\forall x\, P(x)$, one counterexample suffices, because $\neg\forall x\, P(x) \equiv \exists x\, \neg P(x)$. "Every prime is odd" fails at 2.

COMMON PITFALL

Examples never prove a universal claim. $n^2 + n + 41$ is prime for $n = 0, 1, \ldots, 39$, forty cases in a row, and then $n = 40$ gives $1681 = 41^2$. The next lab lets you hunt for failures like this one.

WHY IT MATTERS

A failing test case is a counterexample to the claim "this function is correct". One is enough to prove there is a bug. A thousand passing tests are still only examples.

Lab: conjecture tester

INTERACTIVE LAB

Pick a claim and slide *N* to test it for every *n* up to *N*. Green squares hold, red squares are counterexamples. Click a square to see the arithmetic. Some claims break early, one breaks late, and some never break at all, yet only a proof can tell those last ones apart.

- $n^2 + n + 41$ is prime
- $2^n - 1$ is prime whenever n is prime
- $2^n > n^2$
- 3 never divides $n^2 + 1$
- Every even $n \geq 4$ is a sum of two primes
- Every odd composite n is a prime + $2k^2$
- Your own claim

Domain: n = 1, 2, 3, ... Testing up to **N = 50**.

DISPROVED

Checked **50** of **50** values: **46** hold, **4** fail.

First counterexample: **n = 40: 40^2 + 40 + 41 = 1681 = 41 × 41**. One is enough to make the "for all" claim false.

1	2	3	4	5	6	7	8	9	10	11
12	13	14	15	16	17	18	19	20	21	22
23	24	25	26	27	28	29	30	31	32	33
34	35	36	37	38	39	40	41	42	43	44
45	46	47	48	49	50					

✗ n = 40: 40^2 + 40 + 41 = 1681 = 41 × 41

Euler noticed in 1772 that this polynomial produces primes for n = 0 to 39: forty in a row. Then n = 40 gives 1681 = 41 × 41. Forty confirming cases proved nothing.

Mathematical induction

CONCEPT

THEOREM / RULE

The principle of mathematical induction. Let $P(n)$ be a statement about integers $n \geq n_0$. If

1. **Base case:** $P(n_0)$ is true, and
2. **Inductive step:** for every $k \geq n_0$, $P(k) \to P(k+1)$,

then $P(n)$ is true for every $n \geq n_0$.

Think of an endless row of dominoes. The base case knocks over the first one. The inductive step guarantees that each falling domino knocks over the next. Together they topple every domino, without anyone checking them one by one.

COMMON PITFALL

Inside the inductive step you assume $P(k)$ for one arbitrary k, the inductive hypothesis, and prove $P(k+1)$. You never assume the statement for all n: that would assume what you are trying to prove.

WORKED EXAMPLE

Claim. $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$ for every $n \geq 1$.

Base case ($n = 1$): the left side is 1 and the right side is $\frac{1 \cdot 2}{2} = 1$.

Inductive step: assume $1 + \cdots + k = \frac{k(k+1)}{2}$. Then

$$1 + \cdots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{(k+1)(k+2)}{2},$$

which is the formula with $n = k + 1$. ∎

DESIGN TIP

Label the parts: "Base case", "Inductive hypothesis", "Inductive step". Then point to the exact line where you use the hypothesis. If you never use it, the step is not really an induction.

WHERE THIS SHOWS UP IN AI

Training loops are induction in action. If the weights are valid before an update and every update keeps them valid, they stay valid for all epochs. Engineers rely on exactly this argument when they assert invariants inside a training loop.

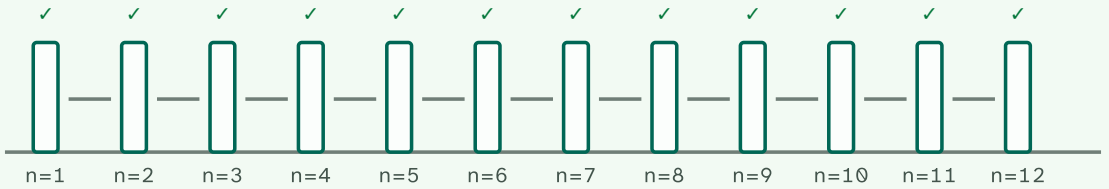
Lab: induction dominoes

INTERACTIVE LAB

Choose a claim, then push the first domino. A domino falls only if the one before it fell *and* the inductive step links them. Click a gap between dominoes (or focus it and press Enter) to break a link, or skip the base case, and see why induction needs both parts. The mark above each domino says whether $P(n)$ is actually true.

Σ i | Σ odd | $2^n \geq n+1$ | $\Sigma\, 2^i$ | Broken base | Broken step

$P(n)$: $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$



Push the first domino | Stand them up | Skip the base case

Click a gap between dominoes to break or repair that link.

Ready. Press "Push the first domino".

BASE CASE

$n = 1$: $1 = \frac{1 \cdot 2}{2}$

HOLDS

The first domino can fall.

INDUCTIVE STEP

Next line | Show all

Assume $1 + 2 + \cdots + k = \frac{k(k+1)}{2}$.

$1 + \cdots + k + (k+1)$ the left side for n = k + 1

Strong induction and well-ordering

CONCEPT

DEFINITION

In strong induction, the inductive step may assume *all* of $P(n_0), P(n_0+1), \ldots, P(k)$ to prove $P(k+1)$. Use it when $k+1$ breaks into smaller pieces that are not simply k.

WORKED EXAMPLE

Claim. Every integer $n \geq 2$ is a product of primes.

Base case: 2 is prime.

Step: take $k+1$. If it is prime, we are done. Otherwise $k+1 = ab$ with $2 \leq a, b \leq k$. By the strong hypothesis, a and b are products of primes, so their product $k+1$ is too. ∎

Ordinary induction would only give us $P(k)$, which says nothing about a and b.

WELL-ORDERING PRINCIPLE

The well-ordering principle: every nonempty set of nonnegative integers has a smallest element. It is equivalent to induction. It powers "smallest counterexample" proofs: if a claim failed, take the smallest failure and show a smaller one exists, a contradiction.

POSTAGE MACHINE

Claim: every amount of 12 cents or more can be paid with 4¢ and 5¢ stamps.

Base cases: 12 = 4 + 4 + 4, 13 = 4 + 4 + 5, 14 = 4 + 5 + 5, 15 = 5 + 5 + 5.
Step for $k+1 \geq 16$: by the strong hypothesis $k+1-4 \geq 12$ can be paid, so add one 4¢ stamp. The step reaches back 4, which is why there are four base cases.

Amount (cents)



23¢: 23 → 19 → 15 (subtract 4 until a base case is reached)



23 = 5 + 5 + 5 + 4 + 4. The base case 15 = 5 + 5 + 5, plus 2 extra 4¢ stamps.

Proofs about programs: loop invariants

CONCEPT

DEFINITION

A loop invariant is a statement about a loop's variables that is true before the loop starts and stays true after every iteration. It is induction in disguise, with the iteration count as n.

```
total, k = 0, 0
while k < n:
    k += 1
    total += k
# invariant: total == 1 + 2 + ... + k
```

CHECK	FOR THIS LOOP	INDUCTION
Initialization	Before the loop, $k = 0$ and total is 0, the empty sum.	base case
Maintenance	If total $= 1 + \cdots + k$ before a pass, then after `k += 1` and `total += k` it equals $1 + \cdots + k$ for the new k.	inductive step
Termination	The loop stops when $k = n$, so total $= 1 + \cdots + n$.	conclusion

CORRECTNESS

An algorithm is correct when every valid input produces the right output. The proof has two parts: **partial correctness** (if it stops, the answer is right, which the invariant gives) and **termination** (it does stop). For termination, find a quantity that drops each pass and can never go below 0. Here it is $n - k$.

RECURSION IS INDUCTION TOO

```
def sum_to(n):
    return 0 if n == 0 else sum_to(n - 1) + n
```

The base case of the recursion is the base case of the proof. If `sum_to(k)` is right, then `sum_to(k + 1)` adds $k + 1$ to it and is right as well.

WHERE THIS SHOWS UP IN AI

Formal verification tools prove properties of trained neural networks, such as "changing any pixel by at most 1% never changes this classification", for every input in a region. Testing samples a handful of inputs; a proof covers all infinitely many.

Code: hunting for a counterexample

CODE WALKTHROUGH

Claim: "if p is prime, then $2^p - 1$ is prime." This program tests the claim on prime after prime until it breaks. Switch to **Trace** to watch the divisor `d` climb.

counterexample.py

Read | Edit & run | Trace

```
1  def smallest_factor(m):
2      d = 2
3      while d * d <= m:
4          if m % d == 0:
5              return d
6          d += 1
7      return m
8
9  for p in [2, 3, 5, 7, 11, 13]:
10     value = 2 ** p - 1
11     d = smallest_factor(value)
12     if d < value:
13         print(f"p = {p}: {value} = {d} x {value // d}")
14         break
15     print(f"p = {p}: 2^p - 1 = {value} is prime")
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
p = 2: 2^p - 1 = 3 is prime
p = 3: 2^p - 1 = 7 is prime
p = 5: 2^p - 1 = 31 is prime
p = 7: 2^p - 1 = 127 is prime
```

LINE 1 OF 15

Show all notes in code

```
def smallest_factor(m):
```

Defines a function that returns the smallest divisor of `m` that is at least 2. If that divisor is `m` itself, then `m` is prime.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Tests the conjecture "$2^p - 1$ is prime whenever p is prime" for p = 2, 3, 5, 7, 11, 13, stopping at the first counterexample and printing its factors.

Code: hunting for a counterexample: every line explained

1	`def smallest_factor(m):`	Defines a function that returns the smallest divisor of `m` that is at least 2. If that divisor is `m` itself, then `m` is prime.
2	`    d = 2`	Starts the candidate divisor at 2, the smallest possible prime factor.
3	`    while d * d <= m:`	Only divisors up to $\sqrt{m}$ need checking: if $m = ab$ with both factors above $\sqrt{m}$, then $ab > m$, which is impossible. Writing `d * d <= m` avoids square roots and rounding errors. This is a tiny proof built into the loop condition.
4	`        if m % d == 0:`	`%` gives the remainder. A remainder of `0` means `d` divides `m`.
5	`            return d`	The first divisor found is the smallest one, because `d` counts upward. The function stops here.
6	`        d += 1`	Moves on to the next candidate. The value $\sqrt{m} - d$ shrinks every pass, so the loop must end: a termination proof in one line.
7	`    return m`	If no divisor up to $\sqrt{m}$ exists, `m` is prime, and its smallest factor is itself.
9	`for p in [2, 3, 5, 7, 11, 13]:`	Loops over the first six primes. These are the cases where the hypothesis "p is prime" is true; testing non-prime p would tell us nothing about this claim.
10	`    value = 2 ** p - 1`	`**` is exponentiation, so `value` is $2^p - 1$: 3, 7, 31, 127, 2047, ...
11	`    d = smallest_factor(value)`	Finds the smallest factor of `value`. For 127 the loop tries `d` = 2 to 11 and finds nothing, so it returns 127.
12	`    if d < value:`	A factor smaller than `value` means `value` is not prime. Here the hypothesis "p is prime" holds and the conclusion "$2^p - 1$ is prime" fails: a counterexample.
13	`        print(f"p = {p}: {value} = {d} x {value // d}")`	Prints the counterexample with its factorization, `p = 11: 2047 = 23 x 89`. `//` is whole-number division. Showing the factors lets anyone check the counterexample by multiplying.
14	`        break`	Stops the loop. One counterexample disproves a "for all" claim, so there is no reason to test p = 13.
15	`    print(f"p = {p}: 2^p - 1 = {value} is prime")`	Reached only when no smaller factor exists, so this case agrees with the claim, as in `p = 7: 2^p - 1 = 127 is prime`. Each line like this is evidence. It says nothing about the next prime.

Code: an invariant you can run

CODE WALKTHROUGH

Two ways to compute $1 + 2 + \cdots + n$. The loop carries its invariant as an `assert`; the recursive version is correct by induction. Use **Trace** and watch the invariant hold on every pass.

invariant.py

Read | Edit & run | Trace

```python
def sum_to(n):
    total, k = 0, 0
    while k < n:
        k += 1
        total += k
        assert total == k * (k + 1) // 2
    return total

def sum_rec(n):
    if n == 0:
        return 0
    return sum_rec(n - 1) + n

for n in [0, 1, 4, 10]:
    print(n, sum_to(n), sum_rec(n), n * (n + 1) // 2)
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
0 0 0 0
1 1 1 1
4 10 10 10
10 55 55 55
```

LINE 1 OF 15

Show all notes in code

```python
def sum_to(n):
```

An iterative function for $1 + 2 + \cdots + n$. Its precondition is $n \geq 0$.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Computes 1 + 2 + ... + n with a loop that asserts its invariant on every pass, computes it again recursively, and compares both with the formula n(n + 1)/2.

Code: an invariant you can run: every line explained

1	`def sum_to(n):`	An iterative function for $1 + 2 + \cdots + n$. Its precondition is $n \geq 0$.
2	`    total, k = 0, 0`	Initialization: `total` is 0 and `k` is 0. The invariant "total = 1 + ⋯ + k" holds already, because the sum of no numbers is 0. This is the base case.
3	`    while k < n:`	Runs while there are numbers left to add. The quantity `n - k` drops by 1 each pass and cannot go below 0, which proves the loop terminates.
4	`        k += 1`	Moves to the next number. For a moment the invariant is broken, because `total` is missing the new `k`.
5	`        total += k`	Adds the new `k`, which restores the invariant. This pair of lines is the maintenance step: invariant true before, invariant true after.
6	`        assert total == k * (k + 1) // 2`	`assert` stops the program with an error if the condition is false. Here it checks the invariant, in its closed form $k(k+1)/2$, after every pass. If a bug broke the invariant, the program would fail right here instead of later with a wrong answer.
7	`    return total`	Termination: the loop ended because `k == n`, and the invariant says `total` = 1 + ⋯ + n. That is the postcondition.
9	`def sum_rec(n):`	The same sum, recursively. Its correctness proof is an induction on `n`.
10	`    if n == 0:`	The base case of the recursion, which is also the base case of the proof.
11	`        return 0`	The empty sum is 0, which matches $0 \cdot 1/2 = 0$.
12	`    return sum_rec(n - 1) + n`	The inductive step: if `sum_rec(n - 1)` correctly returns $1 + \cdots + (n-1)$, then adding `n` gives $1 + \cdots + n$. Each call moves closer to the base case, so the recursion ends.
14	`for n in [0, 1, 4, 10]:`	Tests the base case 0, the first real case 1, and two larger values. Testing the base case first is a habit worth keeping: edge cases are where bugs hide.
15	`    print(n, sum_to(n), sum_rec(n), n * (n + 1) // 2)`	Prints `n`, both computed sums and the formula. All three agree on every line, for example `10 55 55 55`. Agreement on four inputs is a test; the invariant and the induction are the proof.

Design practice: reasoning about your code

DESIGN PRACTICE

Professional developers rarely write formal proofs, but they use the same ideas every day to make code that is correct by design. Click each principle (or use ↓) to compare code to avoid with code to write.

01 State preconditions and postconditions
02 Assert loop invariants while you develop
03 Tests show bugs; proofs show their absence
04 Test the base cases first
05 Check properties on many random inputs
06 Prove termination with a shrinking measure
07 Shrink a failure to the smallest counterexample
08 Make the reasoning visible

Principle 1 of 8

State preconditions and postconditions

A function is a small theorem: its precondition is the hypothesis and its postcondition is the conclusion. Write both in the docstring, so callers know what they must guarantee and what they get back. Check the precondition at the boundary, and fail loudly when it does not hold.

✗ AVOID THIS

```
def average(xs):
    return sum(xs) / len(xs)
# crashes on [] with ZeroDivisionError
```

✓ DO THIS

```
def average(xs):
    """Requires: xs is non-empty.
    Returns: the arithmetic mean of xs."""
    if not xs:
        raise ValueError("xs must be non-empty")
    return sum(xs) / len(xs)
```

Design practice: reasoning about your code: all principles (1/2)

State preconditions and postconditions

A function is a small theorem: its precondition is the hypothesis and its postcondition is the conclusion. Write both in the docstring, so callers know what they must guarantee and what they get back. Check the precondition at the boundary, and fail loudly when it does not hold.

```
def average(xs):
    """Requires: xs is non-empty.
    Returns: the arithmetic mean of xs."""
    if not xs:
        raise ValueError("xs must be non-empty")
    return sum(xs) / len(xs)
```

Assert loop invariants while you develop

An `assert` is a proof step that the computer checks for you. Writing the invariant down forces you to know why the loop works, and the assert turns a silent wrong answer into an immediate error at the exact pass where things went wrong.

```
total, k = 0, 0
while k < n:
    k += 1
    total += k
    assert total == k * (k + 1) // 2
```

Tests show bugs; proofs show their absence

Tests are examples, and examples never prove a universal claim. The version below passes tests for 2, 3, 5 and 7, yet `is_prime(1)` returns `True`: `all` over an empty range is vacuously true. Reasoning about every input, including the edges, finds what the tests missed.

```
def is_prime(n):
    if n < 2:
        return False
    return all(n % d for d in range(2, n))
```

Test the base cases first

The smallest inputs, such as 0, 1 and the empty list, are the base cases of the induction behind your code, and most off-by-one bugs live there. A test suite that checks only "typical" inputs leaves the first domino untested.

```
assert sum_to(0) == 0      # base case
assert sum_to(1) == 1
assert sum_to(10) == 55
```

Check properties on many random inputs

Property-based testing states a law that must hold for every input, then tries hundreds of random ones. It is the conjecture tester applied to your code: it cannot prove the law, but it is very good at finding a counterexample. Print the failing input so you can reproduce it.

```
import random
for _ in range(1000):
    xs = [random.randint(-9, 9) for _ in range(8)]
    assert my_sort(xs) == sorted(xs), xs
```

Design practice: reasoning about your code: all principles (2/2)

Prove termination with a shrinking measure

Every loop needs a quantity that strictly decreases each pass and cannot go below a floor. In binary search that quantity is `hi - lo`. Setting `lo = mid` can leave it unchanged when `hi = lo + 1`, which is an infinite loop; `lo = mid + 1` always shrinks it.

```
while lo < hi:
    mid = (lo + hi) // 2
    if a[mid] < x:
        lo = mid + 1   # hi - lo shrinks
    else:
        hi = mid
```

Shrink a failure to the smallest counterexample

When a test fails on a big input, look for the smallest input that still fails, just as a well-ordering proof looks at the smallest counterexample. Small failures are easy to trace by hand, and they often reveal that the specification itself was wrong.

```
def smallest_failure(claim, limit):
    for n in range(limit):
        if not claim(n):
            return n   # minimal
counterexample
    return None
```

Make the reasoning visible

Code that relies on a mathematical fact should say which fact, so a reader can check the argument instead of trusting it. A short comment that names the theorem, plus a clear expression, beats a clever trick nobody can verify.

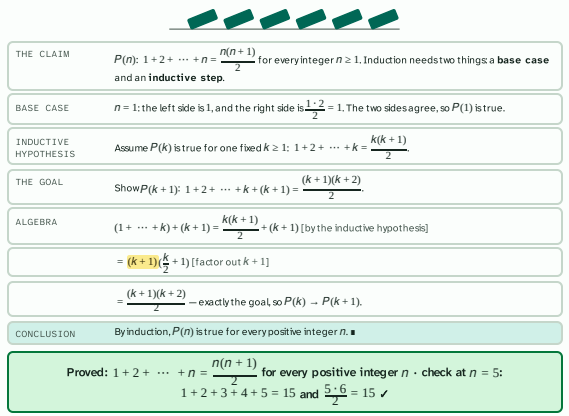
```
# 1 + 2 + ... + n = n(n + 1) / 2, proved by
induction
total = n * (n + 1) // 2
```

Worked example: the sum formula by induction

WORKED EXAMPLE

PROBLEM

Prove by mathematical induction that $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$ for every positive integer n.



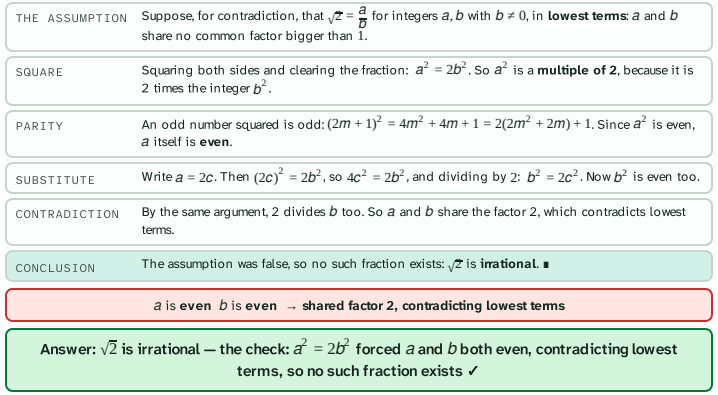
1. State the claim as P(n) and name the two things induction needs: a base case and an inductive step. In the domino picture from the induction lab, the base case tips the first domino, and the inductive step is why each falling domino knocks over the next one.
2. The base case checks n = 1 directly: the left side is 1 and the right side is 1 · 2 / 2 = 1, so the two sides agree and P(1) is true. In the picture, the first domino has fallen.
3. Next, the inductive hypothesis: assume P(k) is true for one fixed k ≥ 1, that is, 1 + ⋯ + k = k(k+1)/2. We do not prove this line; we assume it for a single k — domino k is falling — and use it to push the next domino over.
4. Before doing any algebra, write out the goal for k + 1: 1 + ⋯ + k + (k+1) = (k+1)(k+2)/2. Writing the target first matters because it gives every manipulation that follows a destination to aim at.
5. The algebra is the heart of the step: group the sum as (1 + ⋯ + k) + (k+1), replace the bracket with k(k+1)/2 by the inductive hypothesis, then factor out (k+1), highlighted in the middle line, to land exactly on the goal. Domino k has knocked over domino k + 1.
6. **Answer: by induction, 1 + 2 + ⋯ + n = n(n+1)/2 for every positive integer n, and every domino is down. Spot check at n = 5: 1 + 2 + 3 + 4 + 5 = 15 and 5 · 6 / 2 = 15, so the formula agrees with a direct count.**

Worked example: the square root of 2 is irrational

WORKED EXAMPLE

PROBLEM

Prove that $\sqrt{2}$ is irrational: there are no integers a and b with $b \neq 0$ and $\sqrt{2} = a/b$.



1. For contradiction, suppose sqrt(2) = a/b for integers a and b, with the fraction in lowest terms, so a and b share no common factor bigger than 1. Naming the assumption precisely matters, because the contradiction at the end is aimed exactly at the words "lowest terms".
2. Square both sides and clear the fraction: a squared equals 2 times b squared. That one equation is the engine of the proof, because it says a squared is even — it is 2 times the integer b squared.
3. So a itself is even. The reason is parity: the square of an odd number is odd, since (2m + 1) squared expands to 4m squared plus 4m plus 1, so an even square cannot come from an odd number.
4. Since a is even, write a = 2c and substitute: (2c) squared = 2 b squared gives 4 c squared = 2 b squared, and dividing by 2 gives b squared = 2 c squared. The same evenness has now moved from a over to b.
5. By the same parity argument, b is even too. But then a and b share the factor 2, which directly contradicts the lowest-terms assumption the proof started with — that clash is the point of the whole argument.
6. **Answer: sqrt(2) is irrational — the assumption collapses. The logic is proof by contradiction: a lowest-terms fraction forced both a and b to be even forever, an infinite descent that no lowest-terms fraction can survive, and every step used only lowest terms and basic parity.**

Worked example: a direct proof, inference by inference

WORKED EXAMPLE

PROBLEM

Prove by **direct proof**: if m and n are odd integers, then $m + n$ is even. Justify every inference.

THE CLAIM	For integers m and n: if both are odd, then $m + n$ is even. A **direct proof** assumes the hypothesis and reasons forward to the conclusion, justifying each inference.
ASSUME	Let m and n be odd integers. [Given: the hypothesis of the claim]
DEFINITION	$m = 2a + 1$ and $n = 2b + 1$ for some integers a and b. [Unpack the definition of odd]
ALGEBRA	$m + n = (2a + 1) + (2b + 1) = 2a + 2b + 2$. [Substitute, then collect like terms]
FACTOR	$m + n = 2(a + b + 1)$. [Factor out the common factor 2]
CONCLUSION	$a + b + 1$ is an integer, since sums of integers are integers, so $m + n$ is 2 times an integer: $m + n$ is **even**. ∎ [Pack the definition of even back up]

Proved directly: if m and n are odd, then $m + n$ is even · check: 5 + 7 = 12 = 2 \cdot 6 ✓

1. A direct proof starts from the hypothesis and moves forward, one justified inference at a time, until it reaches the conclusion. The claim here is that two odd integers always add to an even integer, and nothing about the conclusion may be assumed along the way.
2. Assume the hypothesis: m and n are both odd integers. A direct proof is allowed to use exactly this and nothing more, so every later line must be earned from this assumption, from a definition, or from algebra.
3. Unpack the definition of odd: write m = 2a + 1 and n = 2b + 1 for some integers a and b. This inference gives the proof something to compute with, because it turns a word, odd, into an equation.
4. Add the two forms and collect like terms: m + n = 2a + 2b + 2. The right side still shows the structure of the two odd inputs, and the total is now one factoring step away from the shape that the definition of even asks for.
5. Factor out the common factor 2 to get m + n = 2(a + b + 1). This is the key inference of the proof, because it rewrites the sum in exactly the form two times something, which is what the word even means.
6. **Answer: a + b + 1 is an integer by closure of the integers under addition, so m + n is two times an integer and is even, which completes the direct proof. Spot check: 5 + 7 = 12, and 12 is two times 6, so it is even.**

Worked example: a proof by contrapositive

WORKED EXAMPLE

PROBLEM

Prove by **contrapositive**: for every integer n, if n^2 is even, then n is even. Show the contrapositive being formed, then prove it.

THE CLAIM	For every integer n: if n^2 is even, then n is even. Call the hypothesis p and the conclusion q, so the claim is $p \to q$. Proving p directly is awkward, because a square gives little to work with.
NEGATE BOTH	Negate each part: $\neg q$ says n is **not** even, that is, n is odd; and $\neg p$ says n^2 is **not** even, that is, n^2 is odd. Negating flips even to odd and odd to even.
SWAP	Swap the negated parts to form the **contrapositive** $\neg q \to \neg p$: if n is odd, then n^2 is odd. A conditional and its contrapositive are equivalent, so proving this one proves the claim.
ASSUME	Suppose n is odd, so $n = 2k + 1$ for some integer k. [The definition of odd, applied to the new hypothesis]
ALGEBRA	$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$. [Square, then group the result into the shape that defines odd; the grouped coefficient is an integer by closure]
EQUIVALENCE	So n^2 is odd: the contrapositive $\neg q \to \neg p$ holds. Because $p \to q \equiv \neg q \to \neg p$, the original claim holds too: if n^2 is even, then n is even. ∎

Proved by contrapositive: if n^2 is even then n is even · check: $n = 5$ is odd and $n^2 = 25$ is odd ✓

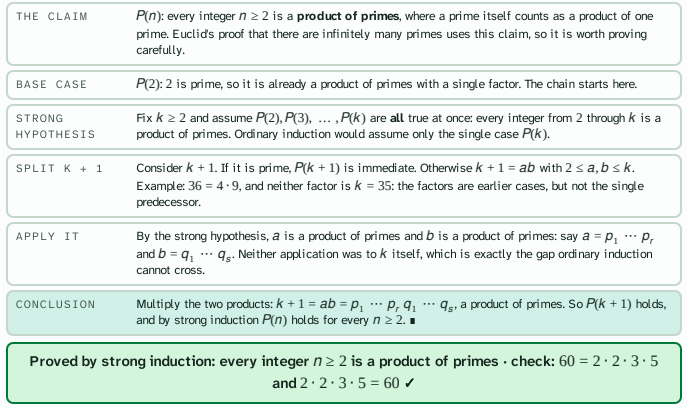
1. State the claim as p implies q, with p the statement that n squared is even and q the statement that n is even. A direct attack stalls here, because knowing a square is even hands us no equation for n itself to compute with.
2. Forming a contrapositive takes two separate moves, and this step does the first one: negate both parts. The negation of the conclusion says n is odd, and the negation of the hypothesis says n squared is odd.
3. The second move is to swap the negated parts, giving the contrapositive: if n is odd, then n squared is odd. This new claim is friendlier, because its hypothesis is about n itself, and a conditional is equivalent to its contrapositive.
4. Now prove the contrapositive directly: suppose n is odd and write n = 2k + 1 for some integer k, using the definition of odd. Just as in a direct proof, unpacking the definition turns a word into an equation we can compute with.
5. Square both sides: n squared works out to the grouped form shown in this row, which has exactly the shape that defines odd, since the grouped coefficient is an integer. So the square of a odd integer is odd, and the contrapositive is proved.
6. **Answer: because a conditional and its contrapositive are equivalent, proving the contrapositive proves the original claim: whenever n squared is even, n itself is even. Spot check the contrapositive at n = 5: n is odd and n squared is 25, which is odd too.**

Worked example: strong induction and prime factorization

WORKED EXAMPLE

PROBLEM

Prove by **strong induction**: every integer $n \geq 2$ is a product of primes. Show why the single predecessor case that ordinary induction offers is not enough.



1. State the claim P(n): every integer n from 2 on is a product of primes, counting a prime as a product with one factor. This is the canonical strong-induction statement, because the step will split a number into pieces that ordinary induction cannot reach.
2. The base case is P(2): 2 is prime, so it is a product of primes with a single factor. Every induction needs its chain anchored somewhere, and here the anchor is the smallest integer the claim covers.
3. The strong hypothesis assumes P(2) through P(k) all at once, for one fixed k: every integer from 2 through k is already known to be a product of primes. Holding all of those cases at once, not just P(k), is what makes this induction strong.
4. Consider k + 1. If it is prime we are done at once; if it is composite it splits as k + 1 = ab with both factors between 2 and k. The example 36 = 4 times 9 shows the point: the factors are earlier cases scattered below k, and neither one is the predecessor k itself.
5. Apply the strong hypothesis to each factor separately: a is a product of primes and b is a product of primes. Ordinary induction could not do this step, because its only assumption, P(k), says nothing at all about a or about b.
6. **Answer: multiplying the two prime products gives a prime product for k + 1, so P(k + 1) holds and strong induction proves the claim for every n from 2 on. Spot check: 60 = 2 times 2 times 3 times 5, and those factors multiply back to 60.**

Practice: proofs

PRACTICE

Name rules of inference, set up proofs, practice the inductive step in both ordinary and strong induction, and hunt for counterexamples. Each problem has a worked solution.

NAME THE RULE

Which rule of inference, or which fallacy, does this argument use?

If n is divisible by 4, then n is even.
If n is even, then n^2 is even.

∴ If n is divisible by 4, then n^2 is even.

Simplification | Modus tollens | Hypothetical syllogism | Denying the antecedent

Show worked solution | New problem ↻

Hypothetical syllogism: $p \to q$, $q \to r$ ∴ $p \to r$. It is a valid rule: whenever the premises are true, the conclusion is true.

PROBLEM TYPE

- Name the rule
- Set up a proof
- Inductive step
- Strong induction
- Find the counterexample

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like.
Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: proofs: sample problems with solutions

NAME THE RULE

Which rule of inference, or which fallacy, does this argument use?

If the model overfits, then validation accuracy drops.

If validation accuracy drops, then we add regularization.

∴ If the model overfits, then we add regularization.

Hypothetical syllogism: p → q, q → r ∴ p → r. It is a valid rule: whenever the premises are true, the conclusion is true.

SET UP A PROOF

You want to prove **"If 3n + 2 is odd, then n is odd."** using **a direct proof**. What do you assume at the start?

Assume **"3n + 2 is odd"**, then derive "n is odd". A direct proof starts from the hypothesis. Assuming "3n + 2 is even" would be a mistake: it leads toward the inverse, which is not equivalent.

INDUCTIVE STEP

The claim is $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$. Suppose you already know it holds for $n = 11$, so the sum up to 11 is 66. Using that fact, what is the sum up to $n = 12$?

Add the next term to the known sum: $66 + 12 = 66 + 12 = 78$. The formula at $n = 12$ also gives 78, so it survives the step from 11 to 12. The inductive step does this algebra once for a general k, which covers every step at the same time.

STRONG INDUCTION

Stamps come in 3¢ and 7¢. Claim: every amount of 12¢ or more can be paid exactly. The strong-induction step pays $k + 1$ cents by paying $k + 1 - 3$ cents and adding one 3¢ stamp. Which earlier amount does the step for $k + 1 = 20$ rely on?

The step uses $P(20 - 3) = P(17)$: once 17¢ can be paid, one more 3¢ stamp pays 20¢. Strong induction may assume every case from 12 up to k, so reaching back 3 is allowed because $17 \geq 12$. Base cases: 12 = 3 + 3 + 3 + 3, 13 = 3 + 3 + 7, 14 = 7 + 7.

Check yourself: proofs

CHECK YOURSELF

QUESTION 1 OF 10

From "If the server is down, an alert is sent" and "No alert was sent", you conclude "The server is not down". Which rule is this?

A Affirming the consequent

B Disjunctive syllogism

C Modus tollens

D Modus ponens

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: proofs: answers explained (1/2)

1. From "If the server is down, an alert is sent" and "No alert was sent", you conclude "The server is not down". Which rule is this?

Answer C: Modus tollens

The form is $p \to q$, $\neg q$, therefore $\neg p$: modus tollens. It is valid because a conditional is equivalent to its contrapositive $\neg q \to \neg p$.

2. "If it rained, the street is wet. The street is wet. Therefore it rained." What is wrong with this argument?

Answer A: It affirms the consequent

The form $p \to q$, q, therefore p is the fallacy of affirming the consequent. A street cleaner could have wet the street: premises true, conclusion false.

3. To prove "if n^2 is odd, then n is odd" by contrapositive, what do you assume?

Answer C: n is even

The contrapositive is "if n is even, then n^2 is even", so you assume n is even and derive that n^2 is even. Assuming that n^2 is odd and n is even is how a proof by contradiction would start.

4. A proof that $\sqrt{2}$ is irrational begins "Suppose $\sqrt{2} = a/b$ in lowest terms." What kind of proof is it?

Answer A: Proof by contradiction

It assumes the opposite of the claim (that √2 is rational) and derives that a and b are both even, which contradicts "lowest terms". The assumption must be false.

5. $n^2 + n + 41$ is prime for $n = 0, 1, \ldots, 39$. What can you conclude about all $n \geq 0$?

Answer D: Nothing yet; and in fact n = 40 gives 41^2

Examples never prove a universal claim. n = 40 gives 1681 = 41 × 41, a counterexample. (It is also not "never prime" afterward: n = 42 gives 1847, which is prime.)

6. In a proof by induction, what does the inductive step establish?

Answer D: P(k) → P(k + 1) for every $k \geq n_0$

The inductive step proves the implication P(k) → P(k + 1) for an arbitrary k. Combined with the base case $P(n_0)$, it topples every domino. Assuming P(k) for every k would be circular.

7. How does strong induction differ from ordinary induction?

Answer B: Its inductive step may assume $P(n_0)$, ..., P(k) all together

Strong induction lets you use any earlier case. That is what the prime-factorization proof needs, because k + 1 = ab splits into factors a and b that are smaller than k but not equal to k.

8. Using only 4¢ and 5¢ stamps, what is the largest amount that cannot be paid?

Answer D: 11¢

11 cannot be written with 4s and 5s, while 12, 13, 14 and 15 can. From those four base cases, strong induction (add a 4¢ stamp to n − 4) covers every amount from 12 up.

Check yourself: proofs: answers explained (2/2)

9. In a loop-invariant proof, which check plays the role of the base case?

Answer C: Initialization

Initialization shows the invariant holds before the first iteration, like $P(n_0)$. Maintenance is the inductive step, and termination combines the invariant with the exit condition to get the final result.

10. In Euclid's proof, $N = p_1 p_2 \cdots p_k + 1$. Must N itself be prime?

Answer A: No: 2·3·5·7·11·13 + 1 = 30031 = 59 × 509

The proof only needs N to have some prime factor, and that factor cannot be on the list. N itself can be composite, as 30031 = 59 × 509 shows. Believing that N must be prime is a common misreading of the proof.

Functions: domain, codomain and range

CONCEPT

DEFINITION

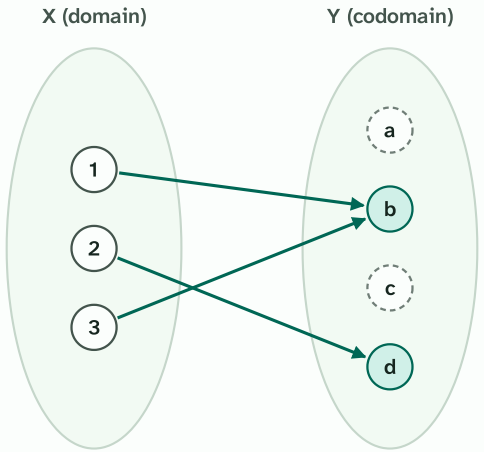
A function f from a set X to a set Y, written $f: X \to Y$, assigns **each** element $x \in X$ **exactly one** element $f(x) \in Y$.

- The domain X is the set of allowed inputs.
- The codomain Y is the set the outputs are declared to come from.
- The range, or image, is the set of outputs actually produced: $f(X) = \{f(x) \mid x \in X\} \subseteq Y$.

Formally, a function *is* a set of ordered pairs $f \subseteq X \times Y$ in which every x appears as a first coordinate exactly once. The picture on the right is the function $f = \{(1, b), (2, d), (3, b)\}$.

WORKED EXAMPLE

A function $\mathbb{R} \to \mathbb{R}$ passes the **vertical-line test**: each vertical line $x = c$ meets its graph exactly once. The circle $x^2 + y^2 = 1$ fails, because the line $x = 0$ meets it twice, at $y = 1$ and $y = -1$. So "y as a function of x" is not defined by the circle.



Each input has exactly one arrow. Two inputs may share an output (1 and 3 both go to b), and outputs may go unused (a and c). Range $= \{b, d\}$, codomain $= \{a, b, c, d\}$.

WHERE THIS SHOWS UP IN AI

Activation functions in neural networks are functions $\mathbb{R} \to \mathbb{R}$. ReLU, $f(x) = \max(0, x)$, has codomain $\mathbb{R}$ and range $[0, \infty)$. The sigmoid has range $(0, 1)$, which is why its output can be read as a probability.

One-to-one, onto, inverses and composition

CONCEPT

DEFINITION

$f: X \to Y$ is one-to-one (injective) when different inputs give different outputs: $f(x_1) = f(x_2) \Rightarrow x_1 = x_2$. No two arrows land on the same point.

DEFINITION

f is onto (surjective) when every $y \in Y$ is hit by some x. Then the range equals the codomain.

DEFINITION

A bijection is both. It pairs X with Y perfectly, so two finite sets joined by a bijection have the same size.

$f: \mathbb{Z} \to \mathbb{Z}$	one-to-one?	onto?
$f(x) = 2x$	yes	no: 3 is never hit
$f(x) = \lfloor x/2 \rfloor$	no: $f(4) = f(5) = 2$	yes
$f(x) = x + 1$	yes	yes

THEOREM / RULE

f has an inverse $f^{-1}: Y \to X$ exactly when f is a bijection. Reverse every arrow: $f^{-1}(y) = x \iff f(x) = y$, so $f^{-1}(f(x)) = x$.

DEFINITION

The composition $g \circ f$ applies f first, then g: $(g \circ f)(x) = g(f(x))$. The outputs of f must be valid inputs for g.

COMMON PITFALL

Order matters. With $f(x) = x + 1$ and $g(x) = 2x$: $(g \circ f)(3) = g(4) = 8$ but $(f \circ g)(3) = f(6) = 7$. Also, $f^{-1}(x)$ means the inverse function, never $1/f(x)$.

WHERE THIS SHOWS UP IN AI

A neural network is a composition of layers, $f_L \circ \cdots \circ f_2 \circ f_1$, so the input passes through f_1 first. A tokenizer's encoder is a bijection between its tokens and their ID numbers, which is why decoding the IDs gives back exactly the original tokens.

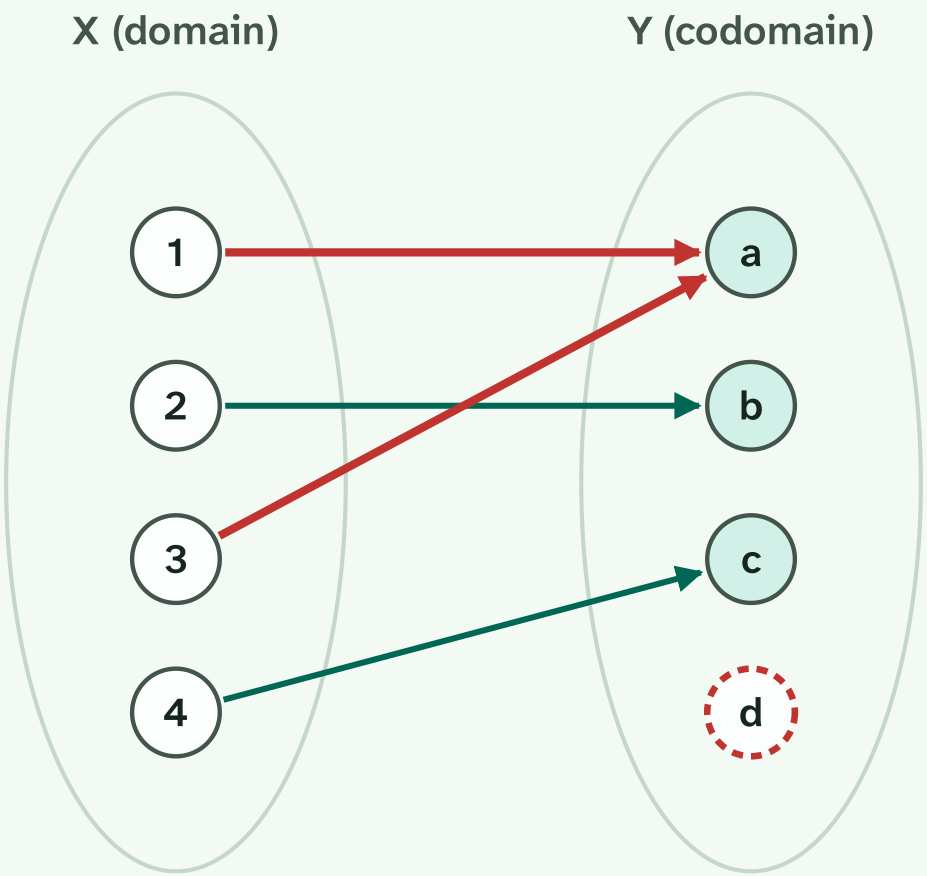
Lab: draw a function, test its properties

INTERACTIVE LAB

Click an input on the left, then an output on the right to add that arrow (click the same output again to remove it). The verdicts update instantly and name the exact element that breaks each property. Red marks the evidence.

Inputs |X| 2 3 **4** 5 Outputs |Y| 2 3 **4** 5

Bijection | One-to-one, not onto | Onto, not one-to-one | Neither | Not a function | Random function | Clear arrows



Click an input on the left (it turns yellow), then an output on the right.

✓ **Function?** Every input has exactly one arrow.

✗ **One-to-one?** $f(1) = f(3) = a$: two inputs share one output.

✗ **Onto?** Nothing maps to d, so the range is smaller than the codomain.

✗ **Bijection?** Needs to be one-to-one *and* onto.

$f = \{(1, a), (2, b), (3, a), (4, c)\}$

Range $= \{a, b, c\} \subseteq$ codomain $\{a, b, c, d\}$

No inverse: reversed, the arrows would send a to two places (1 and 3), so the reverse is not a function.

READING THE PICTURE

Solid **red arrows** are the evidence for the first failure. A **dashed red ring** marks an input with no arrow or an output nothing reaches. Shaded outputs form the range.

One-to-one asks "does any output get two arrows?" Onto asks "does every output get at least one?"

Floor, ceiling, mod and hash functions

CONCEPT

DEFINITION

The floor $\lfloor x \rfloor$ is the greatest integer $\leq x$. The ceiling $\lceil x \rceil$ is the least integer $\geq x$. Both are functions $\mathbb{R} \to \mathbb{Z}$.

x	2.3	−2.3	5	used for
$\lfloor x \rfloor$	2	−3	5	middle index $\lfloor (lo + hi)/2 \rfloor$
$\lceil x \rceil$	3	−2	5	23 people, 4 per table: $\lceil 23/4 \rceil = 6$ tables

DEFINITION

$n \bmod m$ is the remainder r with $0 \leq r < m$ and $n = m \cdot q + r$, where $q = \lfloor n/m \rfloor$. So "mod m" is a function $\mathbb{Z} \to \{0, 1, \ldots, m-1\}$: onto, never one-to-one.

COMMON PITFALL

Python follows the math: `-7 % 3` is `2`, because $-7 = 3 \cdot (-3) + 2$. C, Java and JavaScript return -1 instead.

DEFINITION

A hash function maps keys from a huge domain (all possible strings) to a small codomain of slots $\{0, \ldots, m-1\}$. A common recipe turns the key into an integer, then takes it mod m.

WORKED EXAMPLE

With $h(k) = k \bmod 11$: $h(15) = 4$, $h(26) = 4$, $h(40) = 7$. Keys 15 and 26 **collide** in slot 4.

THEOREM / RULE

If there are more keys than slots, h cannot be one-to-one, so collisions are certain (the pigeonhole principle). A good hash spreads keys evenly over the slots.

DESIGN TIP

A hash must be a function: the same key gives the same slot every time. That is why Python dictionary keys must be immutable.

WHERE THIS SHOWS UP IN AI

An embedding table is a function from token IDs $\{0, \ldots, V-1\}$ to vectors in $\mathbb{R}^d$. The "hashing trick" maps an unlimited vocabulary into m feature columns with $\text{hash}(w) \bmod m$.

Code: functions as dictionaries

CODE WALKTHROUGH

A Python `dict` is a finite function: its keys are the domain and its values are the outputs. Click any line (or press ↓) for its explanation, or switch to **Trace** to watch the comprehensions run.

functions.py

Read | Edit & run | Trace

```
1  f = {1: "b", 2: "c", 3: "a", 4: "c"}
2  Y = {"a", "b", "c", "d"}
3  image = set(f.values())
4  print("one-to-one:", len(image) == len(f))
5  print("onto:", image == Y, "never hit:", sorted(Y - image))
6
7  def compose(g, f):
8      return {x: g[f[x]] for x in f}
9
10 g = {"a": 10, "b": 20, "c": 30, "d": 40}
11 print("g∘f:", compose(g, f))
12 h = {1: "b", 2: "c", 3: "a", 4: "d"}
13 h_inv = {y: x for x, y in h.items()}
14 print("h⁻¹∘h:", compose(h_inv, h))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
one-to-one: False
onto: False never hit: ['d']
g∘f: {1: 20, 2: 30, 3: 10, 4: 30}
h⁻¹∘h: {1: 1, 2: 2, 3: 3, 4: 4}
```

LINE 1 OF 14

Show all notes in code

```
f = {1: "b", 2: "c", 3: "a", 4: "c"}
```

Defines the function $f\colon \{1,2,3,4\} \to \{a,b,c,d\}$ as a dict. Each key is an input and its value is the single output, so $f(1) = b$. A dict cannot hold the same key twice, which matches the rule that each input has exactly one output.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Stores a function as a dictionary, finds its range, tests one-to-one and onto, composes two functions, and builds an inverse.

Code: functions as dictionaries: every line explained

1	`f = {1: "b", 2: "c", 3: "a", 4: "c"}`	Defines the function f: $\{1,2,3,4\} \to \{a,b,c,d\}$ as a dict. Each key is an input and its value is the single output, so $f(1) = b$. A dict cannot hold the same key twice, which matches the rule that each input has exactly one output.
2	`Y = {"a", "b", "c", "d"}`	The codomain, stored separately as a set. A dict only records the outputs that are used, so Python cannot know the codomain unless we state it. Onto is a question about this set.
3	`image = set(f.values())`	`f.values()` lists the outputs `"b", "c", "a", "c"`; wrapping them in `set` removes the repeated `"c"`. The result is the range (image) of f: `{"a", "b", "c"}`. (The domain is just the keys, which you get by looping over `f`.)
4	`print("one-to-one:", len(image) == len(f))`	One-to-one means no two inputs share an output, so the number of distinct outputs must equal the number of inputs. `len(f)` counts the keys (4) and `len(image)` the distinct outputs (3), because 2 and 4 both map to `"c"`. Output: `one-to-one: False`.
5	`print("onto:", image == Y, "never hit:", sorted(Y - image))`	Onto means the range equals the codomain. `Y - image` is the set difference: the outputs nobody maps to. We sort it because sets of strings print in an unpredictable order. Output: `onto: False never hit: ['d']`. Reporting *which* element fails is more useful than a bare False.
7	`def compose(g, f):`	Defines a function that builds $g \circ f$. The parameter order matches the math notation: the function written first is applied last.
8	`    return {x: g[f[x]] for x in f}`	A dict comprehension: for every input `x` of f, the new value is `g[f[x]]`, which is $g(f(x))$. The inner lookup `f[x]` runs first. If some `f[x]` were not a key of `g`, Python would raise `KeyError`: the composition needs the outputs of f to lie in the domain of g.
10	`g = {"a": 10, "b": 20, "c": 30, "d": 40}`	A second function g: $\{a,b,c,d\} \to \{10,20,30,40\}$. Its domain is exactly the codomain of f, so $g \circ f$ is defined.
11	`print("g∘f:", compose(g, f))`	Composes and prints. $f(1) = b$ and $g(b) = 20$, so 1 maps to 20. Output: `g∘f: {1: 20, 2: 30, 3: 10, 4: 30}`. Dicts keep insertion order, so the keys print as 1, 2, 3, 4.
12	`h = {1: "b", 2: "c", 3: "a", 4: "d"}`	A third function, h. Unlike f, it sends the four inputs to four different outputs and hits every letter, so it is a bijection.
13	`h_inv = {y: x for x, y in h.items()}`	`h.items()` yields the ordered pairs `(1, "b")`, `(2, "c")`, ... The comprehension swaps each pair, which reverses every arrow. This only gives a correct inverse because h is one-to-one; if two keys shared a value, the later one would silently overwrite the earlier one. Result: `{"b": 1, "c": 2, "a": 3, "d": 4}` (watch it appear in Trace mode).
14	`print("h⁻¹∘h:", compose(h_inv, h))`	Reuses `compose` to apply h and then h^{-1}. Every input comes back to itself, so the result is the identity function: `h⁻¹∘h: {1: 1, 2: 2, 3: 3, 4: 4}`. That is the defining property $h^{-1}(h(x)) = x$, and exactly the test a tokenizer must pass: decoding the encoded text returns the original text.

Sequences, strings and sum notation

CONCEPT

DEFINITION

A sequence is a function whose domain is a set of consecutive integers. Its value at n is the n-th term s_n. Example: $s_n = 2n + 1$ gives $3, 5, 7, 9, \ldots$

- **Explicit** formula: compute any term directly, as in $s_n = 2^n$.
- **Recursive** formula: give a starting term and a rule built from earlier terms. With $t_1 = 1$ and $t_n = t_{n-1} + n$ we get $1, 3, 6, 10, 15, \ldots$
- **Increasing** means $s_n < s_{n+1}$ for every n; **decreasing** means $s_n > s_{n+1}$.
- A **subsequence** keeps some terms in their original order: $3, 10, 15$ is a subsequence of $1, 3, 6, 10, 15$, while $10, 3$ is not.

DEFINITION

A string over an alphabet X is a finite sequence of symbols from X. λ is the empty string, $|s|$ is the length, and X^* is the set of all strings. Over $X = \{a, b\}$ there are 2^n strings of length n.

SUM AND PRODUCT NOTATION

$$\sum_{i=m}^{n} a_i = a_m + a_{m+1} + \cdots + a_n \qquad \prod_{i=m}^{n} a_i = a_m \cdot a_{m+1} \cdots a_n$$

The index i runs from the lower limit m to the upper limit n. When $m > n$ there are no terms: the empty sum is 0 and the empty product is 1.

THEOREM / RULE

Arithmetic sum: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$. Geometric sum: $\sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}$ for $r \neq 1$.

WORKED EXAMPLE

$1 + 2 + \cdots + 100 = \frac{100 \cdot 101}{2} = 5050$, and $1 + 2 + 4 + \cdots + 2^9 = 2^{10} - 1 = 1023$.

WHERE THIS SHOWS UP IN AI

A prompt sent to a language model is a string over its vocabulary, stored as a sequence of token IDs. A time series is a sequence of readings, and a recurrent network defines its memory recursively: $h_t = f(h_{t-1}, x_t)$.

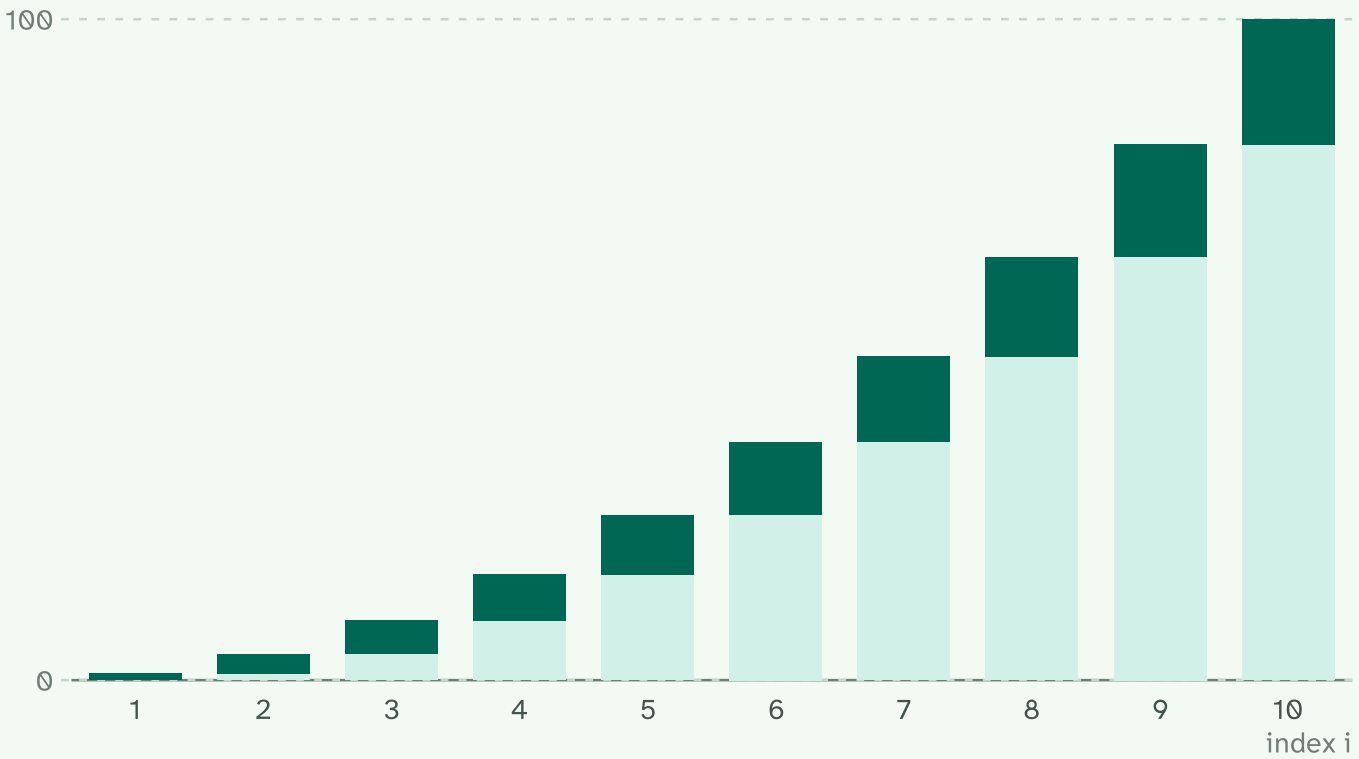
Lab: Σ and Π explorer

INTERACTIVE LAB

Pick an operator and a formula for the i-th term (or type your own using `i`, numbers, `+ - * / ^` and parentheses). Each bar is a running total: the pale part is the total so far and the strong part is the new term. The panel shows the expanded sum, the closed form and the same calculation in Python.

Σ sum | Π product | term a_i = `2i - 1` | $2i-1$ | i | i^2 | i^3 | 2^i | $\frac{1}{2^i}$

lower limit m = 1 upper limit n = 10



$$\sum_{i=1}^{10} (2i-1) = 1 + 3 + 5 + 7 + \cdots + 17 + 19 = 100$$

10 terms (from $i = 1$ to $i = 10$). Running sums: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100

Closed form: $n^2 - (m-1)^2 = 100$ ✓ agrees with the term-by-term total

SAME THING IN PYTHON

```
sum(2*i - 1 for i in range(1, 11))
# 100
```

`range(1, 11)` stops *before* its second argument, so the upper limit 10 becomes 11. A generator expression feeds the terms one at a time to `sum`.

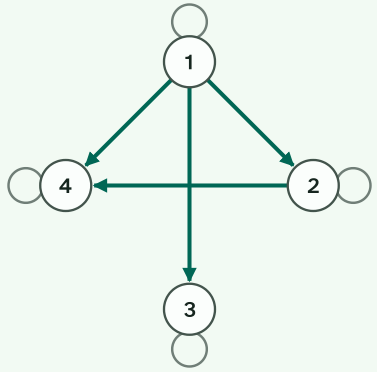
Relations and their four key properties

CONCEPT

DEFINITION

A relation R from A to B is any subset $R \subseteq A \times B$. We write $a \mathrel{R} b$ when $(a, b) \in R$. A relation on A is a subset of $A \times A$. Every function is a relation, but a relation may link an element to many partners or to none.

Example: "divides" on $A = \{1, 2, 3, 4\}$, where $x \mathrel{R} y$ means y is a multiple of x. Draw an arrow $x \to y$ for each pair (a loop for (x, x)), or fill a matrix with a 1 in row x, column y.



R	1	2	3	4
1	1	1	1	1
2	0	1	0	1
3	0	0	1	0
4	0	0	0	1

Property	Rule, for all $x, y, z \in A$	In the digraph	Fails for
reflexive	$x \mathrel{R} x$	a loop at every vertex	$<$ on $\mathbb{Z}$: $3 < 3$ is false
symmetric	$x \mathrel{R} y \Rightarrow y \mathrel{R} x$	every arrow has a return arrow	divides: $2 \mid 4$, $4 \nmid 2$
antisymmetric	$x \mathrel{R} y \wedge y \mathrel{R} x \Rightarrow x = y$	no two-way arrows between different vertices	"is a sibling of"
transitive	$x \mathrel{R} y \wedge y \mathrel{R} z \Rightarrow x \mathrel{R} z$	every two-step path has a shortcut	"is a parent of"

COMMON PITFALL

Antisymmetric does not mean "not symmetric". Equality $=$ is both symmetric and antisymmetric, and $\{(1, 2), (2, 1), (1, 3)\}$ on $\{1, 2, 3\}$ is neither.

DESIGN TIP

In the matrix, reflexive means the diagonal is all 1s, and symmetric means the matrix equals its own transpose (mirror image across the diagonal).

WHERE THIS SHOWS UP IN AI

A knowledge graph stores facts as triples (subject, predicate, object), such as (Paris, capital_of, France). Each predicate is a relation, so these properties tell a reasoner what it may infer.

Equivalence relations and partial orders

CONCEPT

DEFINITION

An equivalence relation is reflexive, symmetric and transitive. The equivalence class of a is $[a] = \{x \in A \mid x \mathrel{R} a\}$.

THEOREM / RULE

The equivalence classes **partition** A: they are nonempty, never overlap, and together cover A. Conversely, every partition defines the equivalence relation "is in the same block as".

WORKED EXAMPLE

Congruence mod n: $a \equiv b \pmod{n}$ when $n \mid (a - b)$, that is, a and b leave the same remainder. Choose n to see the classes of $0, \ldots, 11$:

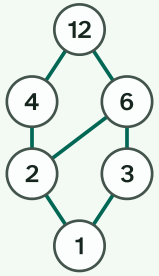
n = 2 | **3** | 4 | 5 | 6

0 1 2 3 4 5 6 7 8 9 10 11

[0] = {0, 3, 6, 9, ...} **[1]** = {1, 4, 7, 10, ...} **[2]** = {2, 5, 8, 11, ...}

DEFINITION

A partial order $\preceq$ is reflexive, antisymmetric and transitive: $\leq$ on numbers, "divides" on positive integers, $\subseteq$ on sets. It is a **total order** if every two elements are comparable. $\leq$ is total; divides is not, because $2 \nmid 3$ and $3 \nmid 2$.



A **Hasse diagram** draws a partial order with less ink: drop the loops, drop every arrow implied by transitivity, and put smaller elements lower so arrowheads are unnecessary. Left: divisors of 12 under "divides".

The **closure** of R adds the fewest pairs that give it a property: loops for reflexive, return arrows for symmetric, and (x, z) for every path from x to z for transitive. The next lab computes all three.

WHERE THIS SHOWS UP IN AI

A taxonomy such as dog $\preceq$ mammal $\preceq$ animal is a partial order, and ontologies are stored as Hasse diagrams. Entity resolution, which decides that "J. Smith" and "John Smith" are the same customer, groups records into equivalence classes.

Lab: relation matrix and digraph

INTERACTIVE LAB

Click matrix cells to add or remove pairs. The digraph redraws, and each property shows a ✓ or the exact pairs that break it (solid red = present, dashed red = missing). Click a property to highlight its evidence. The closure buttons add the missing pairs and report each one.

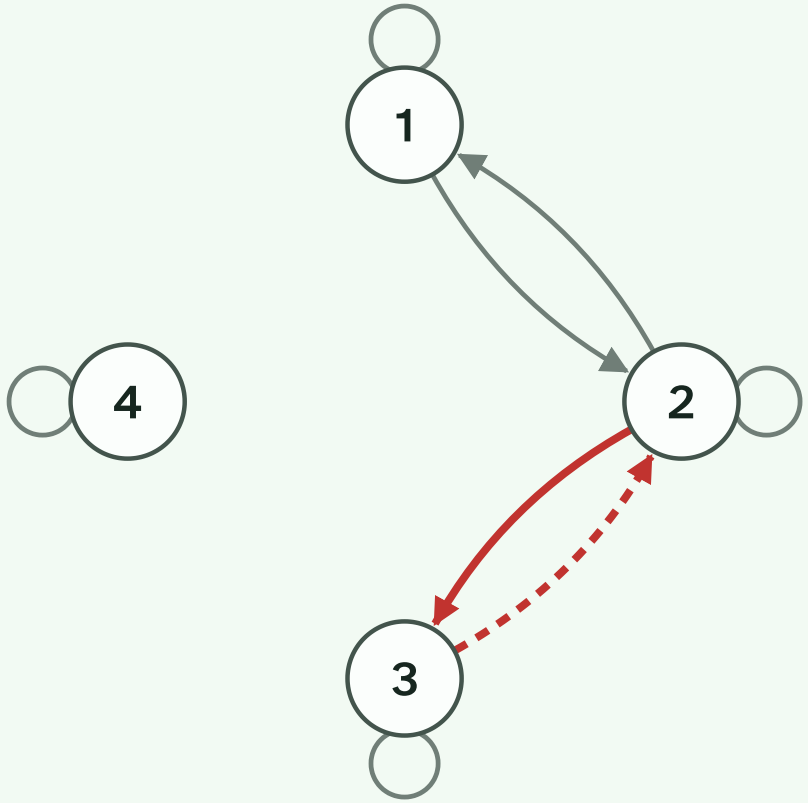
A = {1, …, n}, n = 3 | 4 | 5 | 6

Load: Start example | ≡ mod 2 | divides | ≤ | < | chain 1→2→3… | empty | random

R	1	2	3	4
1	1	1	0	0
2	1	1	1	0
3	0	?	1	0
4	0	0	0	1

Row x, column y holds 1 when x R y. |R| = 7 of 16 possible pairs.

- pair in R
- pair added by the last closure
- evidence: present pair
- evidence: missing pair (?)



Digraph | Hasse diagram

✓ **Reflexive** Every element has a loop: the diagonal is all 1s.

✗ **Symmetric** (2, 3) is in R but (3, 2) is not.

✗ **Antisymmetric** (1, 2) and (2, 1) are both in R, yet 1 ≠ 2.

✗ **Transitive** (1, 2) and (2, 3) are in R, so (1, 3) must be too, but it is missing.

Neither kind of special relation. An equivalence relation still needs to be symmetric and transitive; a partial order still needs to be antisymmetric and transitive.

Reflexive closure | Symmetric closure | Transitive closure | Undo

The closures add pairs and never remove any: each is the smallest relation containing R with that property.

Lab: a tiny relational database

INTERACTIVE LAB

DEFINITION

An n-ary relation on sets $A_1, \ldots, A_n$ is a subset of $A_1 \times \cdots \times A_n$, a set of n-tuples. A database table is one: each row is a tuple, each column an attribute. **Select** σ keeps the rows that pass a test, **project** π keeps chosen columns, and **join** $\bowtie$ glues together tuples that agree on a shared attribute (here `sid`).

Try: Who earned an A? | Which majors exist? | AI students | Discrete roster | Upper years

From: Student | Enroll | Student ⋈ Enroll — ☑ σ where grade = A — π keep ☐ sid ☑ name ☐ major ☐ year ☑ course ☐ grade

STUDENT (5 TUPLES)

SID	NAME	MAJOR	YEAR
1	Ana	AI	2
2	Ben	CS	3
3	Chloe	AI	1
4	Dev	Math	2
5	Eli	CS	1

ENROLL (7 TUPLES)

SID	COURSE	GRADE
1	Discrete	A
1	Python	B
2	Discrete	B
2	Stats	A
3	Python	A
3	Stats	A
4	Discrete	C

Highlighted rows feed the result; colored headers mark the columns kept.

RESULT (4 TUPLES)

NAME	COURSE
Ana	Discrete
Ben	Stats
Chloe	Python
Chloe	Stats

Join: checked 5 × 7 = 35 (student, enrollment) pairs; 7 share a sid. Eli (sid 5) has no enrollment, so no joined tuple mentions Eli.

Select: kept 4 of 7 tuples, a horizontal cut.

Project: kept 2 of 6 columns, a vertical cut. 4 tuples became 4 distinct ones.

RELATIONAL ALGEBRA

$$\pi_{\text{name, course}}\left(\sigma_{\text{grade='A'}}(\text{Student} \bowtie \text{Enroll})\right)$$

SQL

```
SELECT DISTINCT name, course
FROM Student JOIN Enroll USING (sid)
WHERE grade = 'A';
```

PYTHON SET COMPREHENSION

```
{(name, course)
 for (sid, name, major, year) in student
 for (sid2, course, grade) in enroll
 if sid == sid2 and grade == 'A'}
```

SQL keeps duplicate rows unless you write DISTINCT; relational algebra and a Python set remove them automatically.

Code: checking relation properties

CODE WALKTHROUGH

A relation on a finite set is a Python set of tuples. Each property becomes a search for counterexamples, so the program reports *why* a property fails. This is the start example from the matrix lab, restricted to $\{1, 2, 3\}$.

relation_properties.py

Read | Edit & run | Trace

```
1  A = [1, 2, 3]
2  R = {(1, 1), (2, 2), (3, 3), (1, 2), (2, 1), (2, 3)}
3
4  def gaps(R):
5      return {(x, z) for (x, y) in R for (w, z) in R
6              if y == w and (x, z) not in R}
7
8  no_loop = [x for x in A if (x, x) not in R]
9  one_way = {(x, y) for x, y in R if (y, x) not in R}
10 both = {(x, y) for x, y in R if x < y and (y, x) in R}
11 print("reflexive:", not no_loop, no_loop)
12 print("symmetric:", not one_way, sorted(one_way))
13 print("antisymmetric:", not both, sorted(both))
14 print("transitive:", not gaps(R), sorted(gaps(R)))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
reflexive: True []
symmetric: False [(2, 3)]
antisymmetric: False [(1, 2)]
transitive: False [(1, 3)]
```

LINE 1 OF 14

Show all notes in code

```
A = [1, 2, 3]
```

The underlying set. A list keeps the elements in a fixed order, which makes the printed results predictable. Only reflexivity needs A, because it asks about every element, including ones that appear in no pair.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Stores a relation as a set of ordered pairs and tests reflexive, symmetric, antisymmetric and transitive by listing the pairs that break each rule. Then it repairs transitivity.

Code: checking relation properties: every line explained

1	`A = [1, 2, 3]`	The underlying set. A list keeps the elements in a fixed order, which makes the printed results predictable. Only reflexivity needs A, because it asks about every element, including ones that appear in no pair.
2	`R = {(1, 1), (2, 2), (3, 3), (1, 2), (2, 1), (2, 3)}`	The relation as a set of tuples, exactly $R \subseteq A \times A$. Tuples are immutable and hashable, so they can live in a set and `(x, y) in R` is a fast hash lookup.
4	`def gaps(R):`	Defines a function that finds the transitivity counterexamples. We will call it twice, once before and once after the repair, so it is worth a name.
5	`    return {(x, z) for (x, y) in R for (w, z) in R`	A set comprehension with two loops: it pairs every pair (x, y) with every pair (w, z), which is $\lvert R \rvert^2 = 36$ combinations here. The next line filters them. The result is a set, so each missing pair is reported once.
6	`            if y == w and (x, z) not in R}`	Keeps the combinations that form a two-step path $x \to y \to z$ (the middle elements match, `y == w`) whose shortcut (x, z) is missing. That is precisely a counterexample to "$x \,R\, y \wedge y \,R\, z \Rightarrow x \,R\, z$".
8	`no_loop = [x for x in A if (x, x) not in R]`	Reflexive asks: is $(x, x) \in R$ for every $x \in A$? This list collects the elements that fail. All three loops are present, so `no_loop` is `[]`.
9	`one_way = {(x, y) for x, y in R if (y, x) not in R}`	Symmetric asks: does every pair have its reverse? This set comprehension collects the pairs whose reverse is missing. `for x, y in R` unpacks each tuple into its two coordinates. $(2, 3) \in R$ but $(3, 2) \notin R$, so the result is `{(2, 3)}`.
10	`both = {(x, y) for x, y in R if x < y and (y, x) in R}`	Antisymmetric forbids two different elements pointing at each other, so we collect the pairs present in *both* directions. `x < y` skips loops and reports each offending pair once. $(1, 2)$ and $(2, 1)$ are both present, so the result is `{(1, 2)}`.
11	`print("reflexive:", not no_loop, no_loop)`	An empty list counts as false in Python, so `not no_loop` is True exactly when there are no counterexamples. Output: `reflexive: True []`.
12	`print("symmetric:", not one_way, sorted(one_way))`	An empty set is also false, so `not one_way` is the verdict. `sorted` turns the set into an ordered list just for printing, because a set has no fixed order. Output: `symmetric: False [(2, 3)]`: the verdict and its evidence side by side.
13	`print("antisymmetric:", not both, sorted(both))`	Output: `antisymmetric: False [(1, 2)]`. This relation is neither symmetric nor antisymmetric, which shows that the two words are not opposites.
14	`print("transitive:", not gaps(R), sorted(gaps(R)))`	Calls `gaps`. The path $1 \to 2 \to 3$ exists, but the shortcut $(1, 3)$ does not. Output: `transitive: False [(1, 3)]`. Adding the reported gaps is the first step of building the transitive closure; the Try-this box below does exactly that.

Design practice: functions and relations in code

DESIGN PRACTICE

The mathematics of functions and relations maps directly onto habits that keep real code correct. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Write pure functions**
02 Store a finite mapping as a dict
03 Validate the domain at the boundary
04 Use tuples for pairs and records
05 Store each fact once (normalize)
06 Make __eq__ and __hash__ agree
07 Name functions by what they return

Principle 1 of 7

Write pure functions

A **pure** function returns the same output for the same input and changes nothing outside itself. That is the mathematical definition of a function. Pure functions are easy to test (no setup), safe to cache (memoize) and safe to run in parallel.

Hidden state turns `f(3)` into a value that depends on history, so the same call can give different answers.

✗ AVOID THIS

```
total = 0
def add_score(x):
    global total
    total += x        # hidden side effect
    return total      # depends on history
```

✓ DO THIS

```
def add_score(total, x):
    return total + x  # same input, same output
```

Design practice: functions and relations in code: all principles (1/2)

Write pure functions

A **pure** function returns the same output for the same input and changes nothing outside itself. That is the mathematical definition of a function. Pure functions are easy to test (no setup), safe to cache (memoize) and safe to run in parallel.

Hidden state turns `f(3)` into a value that depends on history, so the same call can give different answers.

```
def add_score(total, x):
    return total + x  # same input, same output
```

Validate the domain at the boundary

A function is only defined on its domain. When an input falls outside it, fail loudly at the entry point with a clear message. Returning a made-up default looks friendly but quietly corrupts every later result, and the bug surfaces far from its cause.

```
def points(g):
    if g not in POINTS:
        raise ValueError(f"unknown grade {g!r}")
    return POINTS[g]
```

Store each fact once (normalize)

In a well-designed database every fact lives in exactly one relation. Copying a student's major into every enrollment row means a change of major must update many rows, and missing one leaves the data contradicting itself. Store the key (`sid`) and **join** when you need the combined view.

```
student = {1: ("Ana", "AI")}
enroll = [(1, "Discrete"), (1, "Python")]
# join on sid when you need names
```

Store a finite mapping as a dict

A lookup table *is* a function given as data: the keys are the domain and the values are the outputs. A dict states the whole mapping in one place, answers in constant time, and makes properties checkable. A chain of `if` / `elif` hides the same function inside control flow.

```
POINTS = {"A": 4, "B": 3, "C": 2, "D": 1, "F": 0}
is_one_to_one = len(set(POINTS.values())) == len(POINTS)
```

Use tuples for pairs and records

An ordered pair (a, b) never changes once it is formed. Python tuples are immutable and hashable, so a relation can be stored as a **set of tuples** with fast membership tests, exactly matching $R \subseteq A \times B$. Lists are mutable, cannot go inside a set, and make every membership test a scan.

```
R = {(1, 2), (2, 3)}        # set of tuples
if (1, 2) in R: ...         # one hash lookup
```

Design practice: functions and relations in code: all principles (2/2)

Make __eq__ and __hash__ agree

Python assumes `==` is an **equivalence relation**: reflexive, symmetric and transitive. Sets and dicts also assume that equal objects have equal hashes, because they search by hash first. If you redefine equality, hash the same normalized key, or two "equal" objects land in different buckets and duplicates slip into your sets.

```
class Email:
    def __init__(self, s): self.key =
s.strip().lower()
    def __eq__(a, b): return a.key == b.key
    def __hash__(a): return hash(a.key)
```

Name functions by what they return

`is_onto(f, Y)` returns a bool, `range_of(f)` returns a set, `compose(g, f)` returns a function. A reader knows the type of the result from the name alone. Vague names like `check()` or `process()` could print, mutate or return anything, so every caller has to read the body.

```
def is_onto(f, Y):
    return set(f.values()) == Y
```

Worked example: transitive closure by Warshall's algorithm

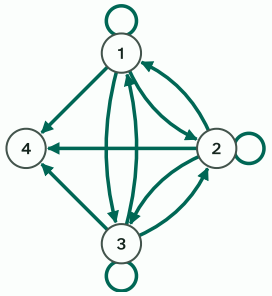
WORKED EXAMPLE

PROBLEM

Let R be the relation on $\{1, 2, 3, 4\}$ with pairs $(1, 2)$, $(2, 3)$, $(3, 1)$ and $(3, 4)$. Use Warshall's algorithm — allowing paths through one intermediate vertex at a time — to find the transitive closure of R.

R	1	2	3	4
1	1	1	1	1
2	1	1	1	1
3	1	1	1	1
4	0	0	0	0

Row i, column j is 1 when i reaches j by a path whose middle vertices are all allowed so far. A bold 1 on a tinted cell was added by this pass; the tinted row and column belong to the active vertex k.



Answer: transitive closure = [[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [0, 0, 0, 0]] — check: 1 → 2 → 3 → 4 reaches every vertex from 1, and vertex 4 reaches nothing.

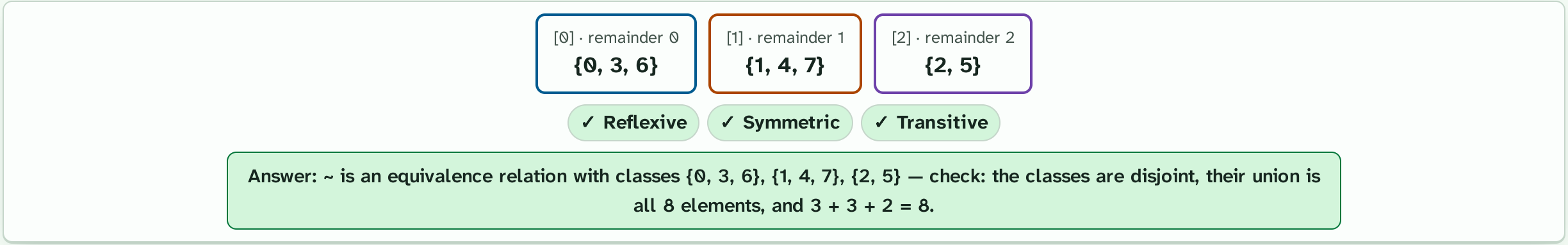
1. Start with the adjacency matrix of R: a 1 in row i, column j means the pair (i, j) is present. The rule for every pass is the same: when paths may pass through vertex k, cell (i, j) turns on whenever both (i, k) and (k, j) are already on.
2. Pass k = 1 lets paths step through vertex 1, which points to 2. Only vertex 3 reaches 1, so exactly one new pair appears: (3, 2), recording the two-step path 3 → 1 → 2 as direct pair of the closure.
3. Pass k = 2 lets paths step through vertex 2, which points to 3. Vertices 1 and 3 both reach 2, so new pairs appear: (1, 3) and (3, 3), recording the two-step paths 1 → 2 → 3 and 3 → 2 → 3 as direct pairs of the closure.
4. Pass k = 3 lets paths step through vertex 3, which points to 1, 2, 3 and 4. Vertices 1, 2 and 3 all reach 3, so new pairs appear: (1, 1), (1, 4), (2, 1), (2, 2) and (2, 4), recording the two-step paths 1 → 3 → 1, 1 → 3 → 4, 2 → 3 → 1, 2 → 3 → 2 and 2 → 3 → 4 as direct pairs of the closure.
5. Pass k = 4 changes no matrix value, and the picture shows why: row 4 is all 0s, because no pair leaves vertex 4. No path can arrive at 4 and then continue onward, so allowing paths through 4 adds no new pairs.
6. **Answer: the transitive closure has rows of all 1s for sources 1, 2 and 3; an all-0 row for source 4. Check: from 1 the path 1 → 2 → 3 → 4 reaches every vertex, the same holds starting from 2 or 3, and 4 has no outgoing pair, so it reaches nothing.**

Worked example: equivalence classes mod 3

WORKED EXAMPLE

PROBLEM

On the set $\{0, 1, 2, 3, 4, 5, 6, 7\}$, define $a \sim b$ to mean that 3 divides $a - b$. Show that $\sim$ is an equivalence relation, and find all of its equivalence classes.



1. Setup: a ~ b means a and b leave the same remainder when divided by 3, because 3 divides a – b exactly when the two remainders match. Each element below is labeled with its remainder mod 3, and the work ahead is to test three properties: reflexive, symmetric and transitive.
2. Reflexive: for every element a, the difference a – a is 0, and 3 divides 0 because 0 = 3 · 0. So every element is related to itself, which is why each chip now carries its own loop and a check mark for the first property.
3. Symmetric: if 3 divides a – b, then it also divides b – a, because b – a is just –(a – b) and flipping the sign keeps divisibility. The highlighted pair checks this with numbers: 4 – 1 = 3 and 1 – 4 = –3, and 3 divides both differences.
4. Transitive: if a – b = 3m and b – c = 3n, then adding the two equations gives a – c = 3(m + n), which is a multiple of 3 again. The highlighted triple checks this: 7 – 4 = 3 and 4 – 1 = 3, so 7 – 1 = 6, and 3 divides 6.
5. With all three properties confirmed, group the elements by their remainder mod 3, putting elements that share a remainder into one box. Elements in the same box differ by a multiple of 3, so they are related to each other, and the set splits into exactly three boxes.
6. **Answer: ~ is an equivalence relation, with classes {0, 3, 6}, {1, 4, 7} and {2, 5}. Check: the classes are disjoint, their union is all 8 elements of the set, and their sizes add up as 3 + 3 + 2 = 8, so nothing is missing and nothing is counted twice.**

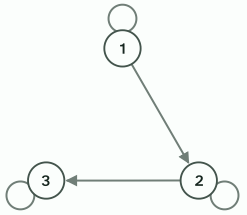
Worked example: a property audit of one relation

WORKED EXAMPLE

PROBLEM

Let R be the relation on $\{1, 2, 3\}$ with pairs $(1, 1), (2, 2), (3, 3), (1, 2)$ and $(2, 3)$. Audit R: decide whether it is reflexive, symmetric, antisymmetric and transitive, naming the witnessing pairs.

R	1	2	3
1	1	1	0
2	0	1	1
3	0	0	1



✓ reflexive ✗ symmetric ✓ antisymmetric ✗ transitive

Answer: R is reflexive, antisymmetric — neither an equivalence relation nor a partial order — check: the diagonal is all 1s and there are no two-way arrows, but (1, 2) lacks its reverse and the path 1 → 2 → 3 lacks the shortcut (1, 3).

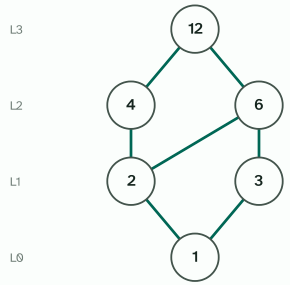
1. Setup: the relation R on {1, 2, 3} is drawn two ways at once, as a matrix and as a digraph, and the audit tests the four properties one by one. Every check below highlights the same pairs in both pictures, so the matrix pattern and the digraph feature cannot drift apart.
2. Reflexive holds: the matrix diagonal is all 1s and every vertex of the digraph carries a loop, because (1, 1), (2, 2), (3, 3) are all in R. A single missing diagonal cell would have failed this check on its own.
3. Symmetric fails: (1, 2) is in R, highlighted in the matrix and the digraph, but its mirror pair (2, 1) is missing, shown dashed. The matrix is not equal to its transpose, because those two mirror cells disagree.
4. Antisymmetric holds: no two different elements point at each other. The highlighted pair (1, 2) is present while its reverse (2, 1) is absent, which is exactly the one-way pattern this property allows between different elements.
5. Transitive fails: the digraph has the two-step path (1, 2) then (2, 3), highlighted in both pictures, but the shortcut pair (1, 3) is missing, shown dashed. One two-step path without its shortcut is enough to fail the property.
6. **Answer: R is reflexive, antisymmetric — neither an equivalence relation nor a partial order. The verdicts in the matrix and the digraph agree check by check: the diagonal is all 1s and there are no two-way arrows, but (1, 2) lacks its reverse and the path 1 → 2 → 3 lacks the shortcut (1, 3).**

Worked example: from a partial order to its Hasse diagram

WORKED EXAMPLE

PROBLEM

Take the partial order "divides" on $\{1, 2, 3, 4, 6, 12\}$. Starting from the full relation, delete the reflexive loops and the transitive edges step by step, then draw the Hasse diagram level by level.



Reachability in this diagram, plus the understood loops, reproduces all 18 original pairs.

Answer: 7 cover edges — 1 → 2, 1 → 3, 2 → 4, 2 → 6, 3 → 6, 4 → 12, 6 → 12 — check: upward reachability plus the understood loops reproduces all 18 pairs of the original order ✓

1. Start from the full relation: 18 pairs in all, counting the 6 loops where each element is related to itself. A Hasse diagram draws this same order with far less ink, by deleting exactly the pairs that the remaining pairs already imply.
2. First deletion: remove the 6 reflexive loops (1 → 1, 2 → 2, 3 → 3, 4 → 4, 6 → 6, 12 → 12), because in a Hasse diagram every element is understood to be related to itself. That leaves 12 pairs between different elements, and no information is lost.
3. Second deletion: remove the 5 transitive edges (1 → 4, 1 → 6, 1 → 12, 2 → 12, 3 → 12), because each one is already implied by a two-step path through a middle element. What remains is 7 cover relations: pairs with nothing strictly between them.
4. Now assign levels: each element sits on the level of the longest chain of cover steps below it, giving level 0: 1; level 1: 2, 3; level 2: 4, 6; level 3: 12. Levels are what let the finished diagram drop its arrowheads, because every cover edge runs from one level straight up to the next.
5. Draw the Hasse diagram: one line per cover relation (1 → 2, 1 → 3, 2 → 4, 2 → 6, 3 → 6, 4 → 12, 6 → 12), smaller elements lower, no loops, no transitive edges and no arrowheads. Reading upward along any path in the picture recovers the original order, which is the whole point of the reduction.
6. **Answer: the Hasse diagram has 7 cover edges (1 → 2, 1 → 3, 2 → 4, 2 → 6, 3 → 6, 4 → 12, 6 → 12). Check: following upward paths from any element reaches exactly the elements it was related to, and adding the understood loops back reproduces all 18 pairs of the original relation.**

Practice: functions, sequences and relations

PRACTICE

Numbers can be typed as integers or fractions (`-4`, `7/2`). Sets use braces, such as `{2, 5, 8}`.

FUNCTION PROPERTIES

Let $X = \{1, 2, 3\}$, $Y = \{a, b, c\}$ and $f = \{(1, c), (2, b), (3, a)\}$. Which describes $f: X \to Y$?

One-to-one only | Onto only | Bijection | Neither

Show worked solution | New problem ↻

Bijection. No two inputs share an output, so f is one-to-one. Every element of Y is hit, so f is onto.

PROBLEM TYPE

- Function properties
- Floor, ceiling and mod
- Sums and sequences
- Relations

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: functions, sequences and relations: sample problems with solutions

FUNCTION PROPERTIES

Let $X = \{1, 2, 3\}$, $Y = \{a, b, c, d\}$ and $f = \{(1, b), (2, c), (3, a)\}$. Which describes $f: X \to Y$?

One-to-one only. No two inputs share an output, so f is one-to-one. Nothing maps to d, so f is not onto. (Since $|X| = 3$ and $|Y| = 4$, onto was impossible from the start.)

FLOOR, CEILING AND MOD

Compute $-18 \bmod 4$ (the remainder r with $0 \le r < 4$).

$q = \lfloor -18/4 \rfloor = -5$, and $-18 = 4 \cdot (-5) + 2$ with $0 \le 2 < 4$. So $-18 \bmod 4 = 2$. Python agrees: `-18 % 4` is 2.

SUMS AND SEQUENCES

A sequence is defined by $a_1 = 1$ and $a_k = 2a_{k-1} + 1$ for $k \ge 2$. Find a_5.

Apply the rule one step at a time: $a_1 = 1$, $a_2 = 2 \cdot 1 + 1 = 3$, $a_3 = 2 \cdot 3 + 1 = 7$, $a_4 = 2 \cdot 7 + 1 = 15$, $a_5 = 2 \cdot 15 + 1 = 31$. A recursive definition only lets you reach a_5 through all the earlier terms.

RELATIONS

For congruence mod 4 on $\{0, 1, \ldots, 17\}$, list the equivalence class $[0]$.

$[0]$ holds every x with $x \equiv 0 \pmod 4$, that is, remainder 0 when divided by 4: start at 0 and add 4 repeatedly. $[0] = \{0, 4, 8, 12, 16\}$.

Check yourself: functions, sequences and relations

CHECK YOURSELF

QUESTION 1 OF 10

Let $f: \{1, 2, 3\} \to \{a, b\}$ be any function. Which statement must be true?

A f is a bijection

B f is one-to-one

C f is onto

D f is not one-to-one

← Previous question | Next question →

QUESTIONS

1 2 3 4 5
6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: functions, sequences and relations: answers explained (1/2)

1. Let $f: \{1, 2, 3\} \to \{a, b\}$ be any function. Which statement must be true?

Answer D: f is not one-to-one

Three inputs share two outputs, so at least two inputs land on the same output (the pigeonhole principle). Whether f is onto depends on the arrows: f could send everything to a.

2. For $f: \mathbb{Z} \to \mathbb{Z}$ with $f(x) = x^2$, what is the range?

Answer C: the perfect squares $\{0, 1, 4, 9, \ldots\}$

The codomain is $\mathbb{Z}$, but only perfect squares are ever produced, and $0 = 0^2$ counts. Numbers like 2 and 3 are nonnegative yet never hit, so f is not onto.

3. With $f(x) = x + 3$ and $g(x) = 2x$, what is $(g \circ f)(4)$?

Answer D: 14

$(g \circ f)(4) = g(f(4)) = g(7) = 14$: apply f first. The other order gives $(f \circ g)(4) = f(8) = 11$, so composition order matters.

4. What is $\lfloor -2.5 \rfloor + \lceil 2.5 \rceil$?

Answer D: 0

$\lfloor -2.5 \rfloor = -3$ (round down, which for a negative number moves away from zero) and $\lceil 2.5 \rceil = 3$. The sum is 0.

5. What is $\sum_{i=1}^{100} i$?

Answer A: 5050

$\frac{n(n+1)}{2} = \frac{100 \cdot 101}{2} = 5050$. Pair the terms from the outside in: $1 + 100$, $2 + 99$, ..., giving 50 pairs that each add to 101.

6. A hash table has 10 slots and uses `h(k) = k % 10`. You insert 11 different integer keys. What must happen?

Answer A: At least two keys share a slot

Eleven keys go into ten slots, so `h` cannot be one-to-one: some slot gets two keys (the pigeonhole principle). `h` is still a function, because each key has exactly one slot.

7. Which relation on a set of people is an equivalence relation?

Answer C: "has the same birthday as"

Same birthday is reflexive, symmetric and transitive, and its classes are the 366 possible birthdays. "Parent of" is not reflexive or transitive, "taller than" is not reflexive or symmetric, and friendship is usually not transitive.

8. The relation "divides" on the positive integers is a partial order. Which property does it lack?

Answer B: Symmetric

$2 \mid 4$ but $4 \nmid 2$, so it is not symmetric. It is reflexive ($a \mid a$), antisymmetric ($a \mid b$ and $b \mid a$ force $a = b$ for positive integers) and transitive.

Check yourself: functions, sequences and relations: answers explained (2/2)

9. Let $R = \{(1,2),(2,3),(3,4)\}$ on $\{1,2,3,4\}$. How many pairs does the transitive closure of R contain?

Answer A: 6

The closure adds a shortcut for every path: $(1,3)$, $(2,4)$ and $(1,4)$. That gives 6 pairs, which is exactly the relation $<$ on $\{1,2,3,4\}$.

10. Two students in the `Student` table of the database lab, Ana and Chloe, major in AI. How many rows does $\pi_{\text{major}}(\sigma_{\text{major = 'AI'}}(\text{Student}))$ contain?

Answer A: 1

Selection keeps the two AI rows (Ana and Chloe); projecting onto `major` turns them into two copies of the tuple (AI). A relation is a set, so the copies collapse into one tuple. SQL would print two rows unless you write SELECT DISTINCT.

What is an algorithm?

CONCEPT

DEFINITION

An algorithm is a finite sequence of precise instructions for solving a problem. Every algorithm has seven properties:

Input	It receives data (possibly none).
Output	It produces a result.
Precision	Every step is stated exactly, with no room for guessing.
Determinism	Each step's result depends only on the input and the steps before it.
Finiteness	It stops after finitely many steps, on every input.
Correctness	The output is right for every valid input.
Generality	It works for every input of the problem, not just one example.

WHERE THIS SHOWS UP IN AI

A language model that samples its next word at temperature above 0 deliberately gives up determinism: the same prompt can produce different answers. At temperature 0 (greedy decoding) it always picks the most likely word, so the output repeats exactly.

Pseudocode: find the largest value

```
Input:  s[1], s[2], ..., s[n]   (n ≥ 1)
Output: the largest value

max(s, n) {
  large = s[1]          // best so far
  for i = 2 to n
    if (s[i] > large)
      large = s[i]      // found a bigger one
  return large
}
```

Pseudocode conventions: indentation shows which lines belong to a loop or an `if`; `=` stores a value; `for i = 2 to n` counts up by one; `return` ends the algorithm and hands back the answer. Like the textbook, pseudocode here numbers positions from 1, while Python starts at 0.

WORKED EXAMPLE

Trace on $3, 9, 4$: `large` starts at 3, becomes 9 (because $9 > 3$), and stays 9 (because $4 > 9$ is false). The output is 9 after $n - 1 = 2$ comparisons.

Searching: linear, binary and text search

CONCEPT

Linear search

```
linear_search(s, n, key) {
  for i = 1 to n
    if (s[i] == key)
      return i
  return 0      // not found
}
```

Check the items in order. It works on **any** list. Worst case: n comparisons, when the key is last or missing.

Binary search

```
binary_search(s, n, key) {
  i = 1; j = n
  while (i ≤ j) {
    k = ⌊(i + j)/2⌋  // middle
    if (key == s[k]) return k
    if (key < s[k]) j = k − 1
    else i = k + 1
  }
  return 0
}
```

The list must be **sorted**. Each comparison discards half of what is left, so at most $\lfloor \log_2 n \rfloor + 1$ comparisons.

Text search

```
text_search(p, m, t, n) {
  for i = 1 to n − m + 1 {
    j = 1
    while (j ≤ m and t[i+j−1] == p[j])
      j = j + 1
    if (j > m) return i
  }
  return 0
}
```

Slide the pattern p along the text t. "cad" occurs in "abracadabra" at position 5. Worst case about $(n - m + 1) \cdot m$ character comparisons.

COMMON PITFALL

Binary search on unsorted data fails *silently*: it may report "not found" for a key that is there, because it threw away the half that held it. Sort first, or keep the data sorted as you insert.

WHERE THIS SHOWS UP IN AI

Nearest-neighbor search compares a query embedding with stored vectors. A linear scan computes all n distances; vector indexes organize the data so most of it can be skipped, the same idea that makes binary search fast.

Lab: search race

INTERACTIVE LAB

Both searches look for the same target in the same sorted array. Each **Step** makes one comparison in each lane. Faded cells have been ruled out. Type a target, click any cell to search for its value, or try a value that is not there. Drag the size slider and watch the worst-case counts.

size n = 20 target 46 Random present Random absent Step ▶ ▶ Play Reset

Linear search works on any data, one cell at a time

0 comparisons

1	5	9	10	14	18	19	23	27	28	32	36	37	41	45	46	50	54	55	59
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Ready. It will check a[0], a[1], a[2], … in order.

Binary search needs sorted data, halves the range each time

0 comparisons

1	5	9	10	14	18	19	23	27	28	32	36	37	41	45	46	50	54	55	59
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Ready. The whole array, indices 0 to 19, is still in play.

Worst case for n items. Linear search may check every item: **20** comparisons here. Binary search halves the range each time, so it needs at most $\lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 20 \rfloor + 1 = 5$. Doubling n adds just one comparison.

n	linear	binary
20 (this array)	20	5
1,000 (a thousand)	1,000	10
1,000,000 (a million)	1,000,000	20
1,000,000,000 (a billion)	1,000,000,000	30

Code: binary search with a step counter

CODE WALKTHROUGH

The pseudocode from the searching slide, in Python, with a counter for the number of middle elements examined. Switch to **Trace** and watch `lo`, `hi` and `mid` close in on the target.

binary_search.py

Read | Edit & run | Trace

```python
def binary_search(a, target):
    lo, hi, steps = 0, len(a) - 1, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        steps += 1
        if a[mid] == target:
            return mid, steps
        elif a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return None, steps

data = [3, 8, 15, 21, 29, 34, 42, 57, 63, 70, 88]
print(binary_search(data, 57))
print(binary_search(data, 40))
```

OUTPUT · Runs with real Python 3 in the interactive version

▶ Run

```
Output:
(7, 4)
(None, 3)
```

LINE 1 OF 16

Show all notes in code

```python
def binary_search(a, target):
```

Defines a function that takes a **sorted** list `a` and the value to find. Nothing runs until the function is called on line 15.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Searches a sorted list by repeatedly probing the middle element and discarding the half that cannot hold the target. It returns the index (or None) together with the number of probes.

Code: binary search with a step counter: every line explained

1	`def binary_search(a, target):`	Defines a function that takes a **sorted** list `a` and the value to find. Nothing runs until the function is called on line 15.
2	`    lo, hi, steps = 0, len(a) - 1, 0`	Tuple assignment sets three variables at once. `lo` and `hi` are the first and last indices still in play (0 and 10 for 11 items), and `steps` counts probes. Python indexes from 0, so the last index is `len(a) - 1`.
3	`    while lo <= hi:`	Keep going while the range *lo..hi* still contains at least one index. When `lo` passes `hi`, the range is empty and the target cannot be in the list. This condition guarantees the loop stops (finiteness).
4	`        mid = (lo + hi) // 2`	`//` is floor division, so this is $\lfloor (lo + hi)/2 \rfloor$, the middle index of the current range. On the first pass it is $\lfloor 10/2 \rfloor = 5$.
5	`        steps += 1`	One more probe. We count one step per middle element examined, even though the step may ask two questions (equal? smaller?).
6	`        if a[mid] == target:`	The lucky case: the middle element is the target.
7	`            return mid, steps`	Returns a tuple: where the target is and how many probes it took. `return` ends the function immediately, which also ends the loop.
8	`        elif a[mid] < target:`	The middle element is too small. Because the list is sorted, everything at or left of `mid` is too small as well.
9	`            lo = mid + 1`	So the search continues to the right of `mid`. The left half, including `mid`, is discarded.
10	`        else:`	Otherwise the middle element is too large.
11	`            hi = mid - 1`	Discard `mid` and everything to its right; keep searching on the left. Either way the range at least halves, which is where the $\log_2 n$ comes from.
12	`    return None, steps`	Reached only when the range became empty. `None` is Python's "no value", a clearer signal than a fake index such as -1.
14	`data = [3, 8, 15, 21, 29, 34, 42, 57, 63, 70, 88]`	Eleven sorted values. The worst case is $\lfloor \log_2 11 \rfloor + 1 = 3 + 1 = 4$ probes.
15	`print(binary_search(data, 57))`	Probes index 5 (34, too small), then 8 (63, too big), then 6 (42, too small), then 7 (57, found). Output: `(7, 4)`: index 7 after 4 probes, the worst case.
16	`print(binary_search(data, 40))`	A value that is not there. Probes 34, 63 and 42, after which `lo = 6` and `hi = 5`, so the loop stops. Output: `(None, 3)`. Plain linear search would have checked all 11 items before giving up.

Sorting, and an algorithm that uses randomness

CONCEPT

Insertion sort

```
insertion_sort(s, n) {
  for i = 2 to n {
    val = s[i]
    j = i − 1
    while (j ≥ 1 and s[j] > val) {
      s[j+1] = s[j]   // shift right
      j = j − 1
    }
    s[j+1] = val
  }
}
```

Like sorting a hand of cards: take the next card and slide it left past every larger card. Invariant: $s[1..i-1]$ is sorted. Very fast on nearly sorted data.

Selection sort

```
selection_sort(s, n) {
  for i = 1 to n − 1 {
    small = i
    for j = i + 1 to n
      if (s[j] < s[small])
        small = j
    swap(s[i], s[small])
  }
}
```

Find the smallest remaining item and swap it into position i. Invariant: $s[1..i-1]$ holds the $i-1$ smallest items in final order. Always $n(n-1)/2$ comparisons but at most $n-1$ swaps.

Shuffle (randomized)

```
shuffle(s, n) {
  for i = n down to 2 {
    j = rand(1, i)   // 1 ≤ j ≤ i
    swap(s[i], s[j])
  }
}
```

A randomized algorithm makes random choices as it runs. This Fisher–Yates shuffle produces each of the $n!$ orderings with equal probability. It gives up determinism and keeps correctness.

COMMON PITFALL

Picking `j = rand(1, n)` at every step looks equally random but is biased. It has n^n equally likely runs, and for $n \geq 3$ the number n^n is not a multiple of $n!$, so some orderings must come up more often than others.

WHERE THIS SHOWS UP IN AI

Training shuffles the examples before every pass (epoch) so a model cannot learn their order. Random weight initialization, dropout and mini-batch sampling are randomized steps too, which is why fixing the random seed makes experiments reproducible.

Lab: sorting step by step

INTERACTIVE LAB

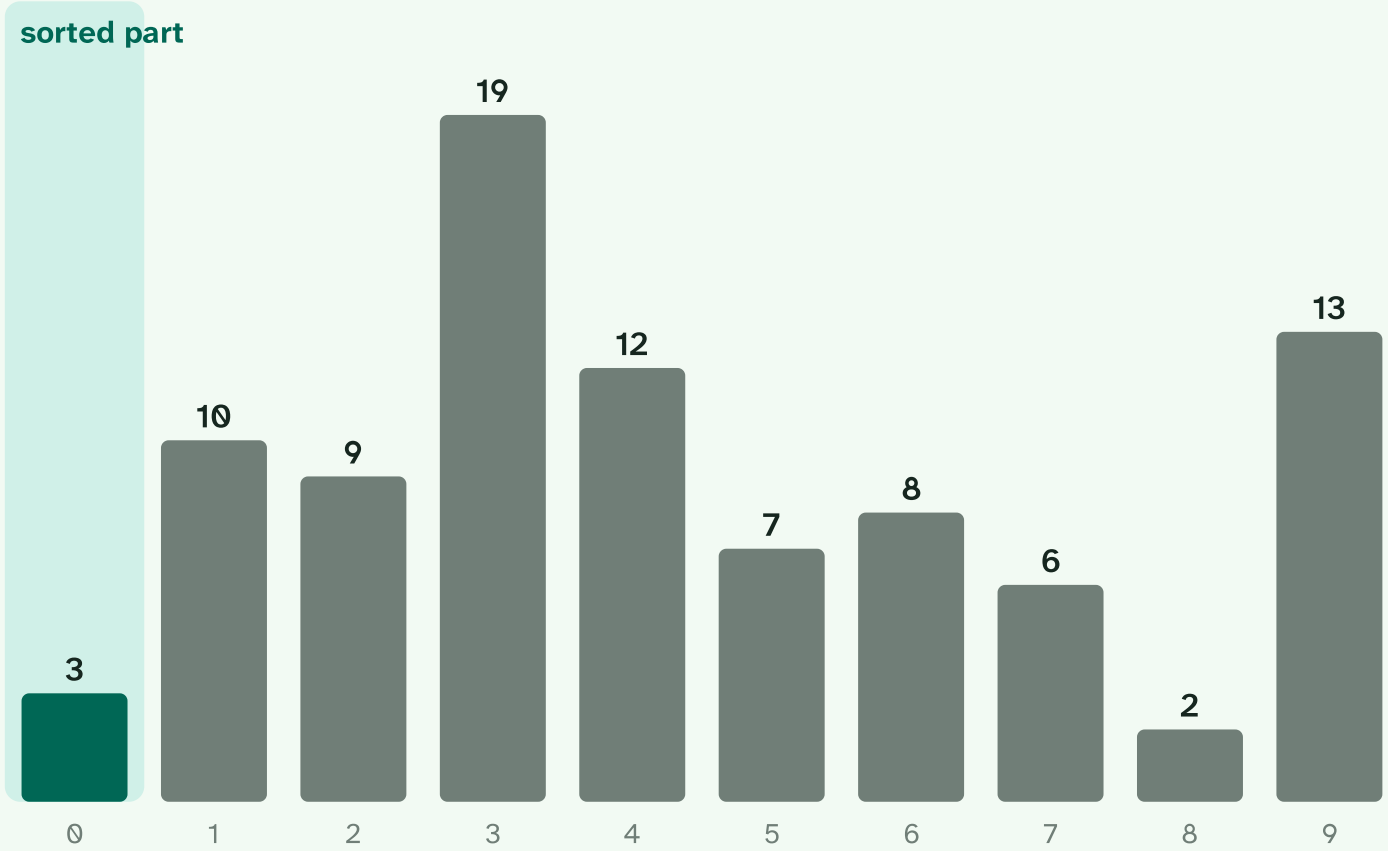
Step through either algorithm (or press ↓ / ↑). The shaded band is the part covered by the loop invariant. Compare counts across inputs: insertion sort reacts strongly to the input order, selection sort does not react at all.

Insertion sort | Selection sort

Random | Sorted | Reversed | Nearly sorted

n: 6 | 8 | 10 | 12

◀ Back | Step ▶ | ▶ Play | Reset | New random input



0	0
COMPARISONS	SHIFTS

STEP 1 OF 53

Start. A list with one item is already sorted, so the sorted part is just a[0] = 3.

sorted part · key being inserted · just compared

LOOP INVARIANT

Before each pass i, the items a[0..i–1] are sorted among themselves. Later items may still slide in between them.

For n = 10, insertion sort needs between **n – 1 = 9** comparisons (already sorted) and **n(n–1)/2 = 45** (reversed).

Counting operations: best, worst and average case

CONCEPT

Seconds depend on the machine, the language and what else is running. Analysis therefore counts a basic operation, usually a comparison, as a function of the input size n.

DEFINITION

The worst case $W(n)$ is the most operations over all inputs of size n; the best case $B(n)$ is the fewest. The average case $A(n)$ is the expected count when inputs follow a stated probability distribution.

Algorithm (comparisons)	best	worst	average
linear search	1	n	$\frac{n+1}{2}$ if present
binary search	1	$\lfloor \log_2 n \rfloor + 1$	$\approx \log_2 n$
insertion sort	$n-1$	$\frac{n(n-1)}{2}$	$\approx \frac{n^2}{4}$
selection sort	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$

Linear search average: if the target is equally likely to be in any of the n positions, the expected number of comparisons is $\frac{1+2+\cdots+n}{n} = \frac{n+1}{2}$.

WORKED EXAMPLE

How often does the inner line run?

```
for i = 1 to n
  for j = 1 to i
    x = x + 1
```

For each i the inner loop runs i times, so the total is $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$, the arithmetic sum from Module 4.

WHY IT MATTERS

The worst case is a promise: no input of size n takes longer. Deadlines need that promise. The average describes typical behavior, and the best case is rarely useful, since almost any algorithm is fast on a lucky input.

WHERE THIS SHOWS UP IN AI

A chat service might promise answers within 200 ms for 99% of requests (its "p99 latency"). That is a worst-case style guarantee; a fast average can hide many slow requests.

Code: counting comparisons in insertion sort

CODE WALKTHROUGH

Insertion sort, instrumented to count every comparison between list items. Running it on a sorted, a reversed and a mixed list shows the best, worst and an in-between case for $n = 4$.

insertion_sort.py

Read | Edit & run | Trace

```
1  def insertion_sort(a):
2      a, comps = list(a), 0
3      for i in range(1, len(a)):
4          key, j = a[i], i - 1
5          while j >= 0:
6              comps += 1
7              if a[j] <= key:
8                  break
9              a[j + 1] = a[j]
10             j -= 1
11         a[j + 1] = key
12     return a, comps
13
14 for data in ([1, 2, 3, 4], [4, 3, 2, 1], [3, 1, 4, 2]):
15     print(data, "->", insertion_sort(data))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
[1, 2, 3, 4] -> ([1, 2, 3, 4], 3)
[4, 3, 2, 1] -> ([1, 2, 3, 4], 6)
[3, 1, 4, 2] -> ([1, 2, 3, 4], 5)
```

LINE 1 OF 15

Show all notes in code

```
def insertion_sort(a):
```

Defines the function. It will return two things: the sorted list and the number of comparisons.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Sorts a copy of a list with insertion sort and returns the sorted list together with the number of item comparisons it made.

Code: counting comparisons in insertion sort: every line explained

1	`def insertion_sort(a):`	Defines the function. It will return two things: the sorted list and the number of comparisons.
2	`    a, comps = list(a), 0`	`list(a)` makes a copy, so the caller's list is not changed (no side effect). `comps` starts at `0` and counts comparisons between list items.
3	`    for i in range(1, len(a)):`	Pass `i` inserts item `a[i]` into the sorted part `a[0..i-1]`. It starts at 1 because a single item, `a[0]`, is already sorted. There are $n - 1$ passes.
4	`        key, j = a[i], i - 1`	`key` remembers the item being inserted, because the shifting below will overwrite `a[i]`. `j` points at the last item of the sorted part.
5	`        while j >= 0:`	Walk left through the sorted part. `j >= 0` stops the walk at the front of the list so we never read `a[-1]`, which in Python would silently mean the *last* item.
6	`            comps += 1`	Count the comparison the next line is about to make. Putting the counter here, before the test, counts the comparison that stops the loop too.
7	`            if a[j] <= key:`	Is the item on the left small enough? If so, the key belongs right after it. Using `<=` instead of `<` keeps equal items in their original order, which makes the sort **stable**.
8	`                break`	Stop walking left. In the best case (already sorted) this happens on the very first comparison of every pass, giving $n - 1$ comparisons in total.
9	`            a[j + 1] = a[j]`	The item is larger than the key, so shift it one place right. That opens a gap at position `j`.
10	`            j -= 1`	Move one position further left and compare again. In the worst case (reversed input) the walk goes all the way to the front on every pass.
11	`        a[j + 1] = key`	Drop the key into the gap. Now `a[0..i]` is sorted, which is the loop invariant for the next pass.
12	`    return a, comps`	Returns a tuple. The caller receives both the result and the measurement.
14	`for data in ([1, 2, 3, 4], [4, 3, 2, 1], [3, 1, 4, 2]):`	Loops over three test inputs of size $n = 4$: sorted (best case), reversed (worst case) and mixed.
15	`    print(data, "->", insertion_sort(data))`	Output: `[1, 2, 3, 4] -> ([1, 2, 3, 4], 3)`, then `[4, 3, 2, 1] -> ([1, 2, 3, 4], 6)`, then `[3, 1, 4, 2] -> ([1, 2, 3, 4], 5)`. The counts match $n - 1 = 3$ and $n(n-1)/2 = 6$, and the mixed list falls in between.

Big-O, Big-Omega and Big-Theta

CONCEPT

DEFINITION

Big-O: $f(n) = O(g(n))$ if there are constants $C > 0$ and n_0 with $|f(n)| \le C\,|g(n)|$ for all $n \ge n_0$. Read it as "f grows no faster than g".

DEFINITION

Big-Omega: $f(n) = \Omega(g(n))$ if $|f(n)| \ge C\,|g(n)|$ for all $n \ge n_0$, for some $C > 0$: "f grows at least as fast as g".

DEFINITION

Big-Theta: $f(n) = \Theta(g(n))$ if f is both $O(g(n))$ and $\Omega(g(n))$: "the same growth rate, up to constant factors".

WORKED EXAMPLE

$f(n) = 3n^2 + 5n + 2$. For $n \ge 1$, $5n \le 5n^2$ and $2 \le 2n^2$, so $f(n) \le 10n^2$: the witnesses $C = 10$, $n_0 = 1$ prove $f = O(n^2)$. Also $f(n) \ge 3n^2$, so $f = \Omega(n^2)$, and therefore $f = \Theta(n^2)$.

THEOREM / RULE

Rules that cover most code:

1. Constant factors vanish: $7n = \Theta(n)$.
2. Keep the dominant term: $n^3 + 100n^2 + 5 = \Theta(n^3)$.
3. Blocks in sequence add, and the larger wins: $O(f) + O(g) = O(\max(f, g))$.
4. Nesting multiplies: n iterations of an $O(g)$ body cost $O(n \cdot g)$.
5. Log bases differ by a constant factor, so $\log_2 n = \Theta(\log_{10} n)$.

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

COMMON PITFALL

Big-O is only an upper bound. $5n = O(n^2)$ is true but says little, like "the trip takes under a year". When you know the exact rate, say Θ.

WHERE THIS SHOWS UP IN AI

Attention in a transformer compares every token with every other token: $\Theta(n^2)$ work for a context of n tokens. Doubling the context from 4k to 8k tokens roughly quadruples that cost.

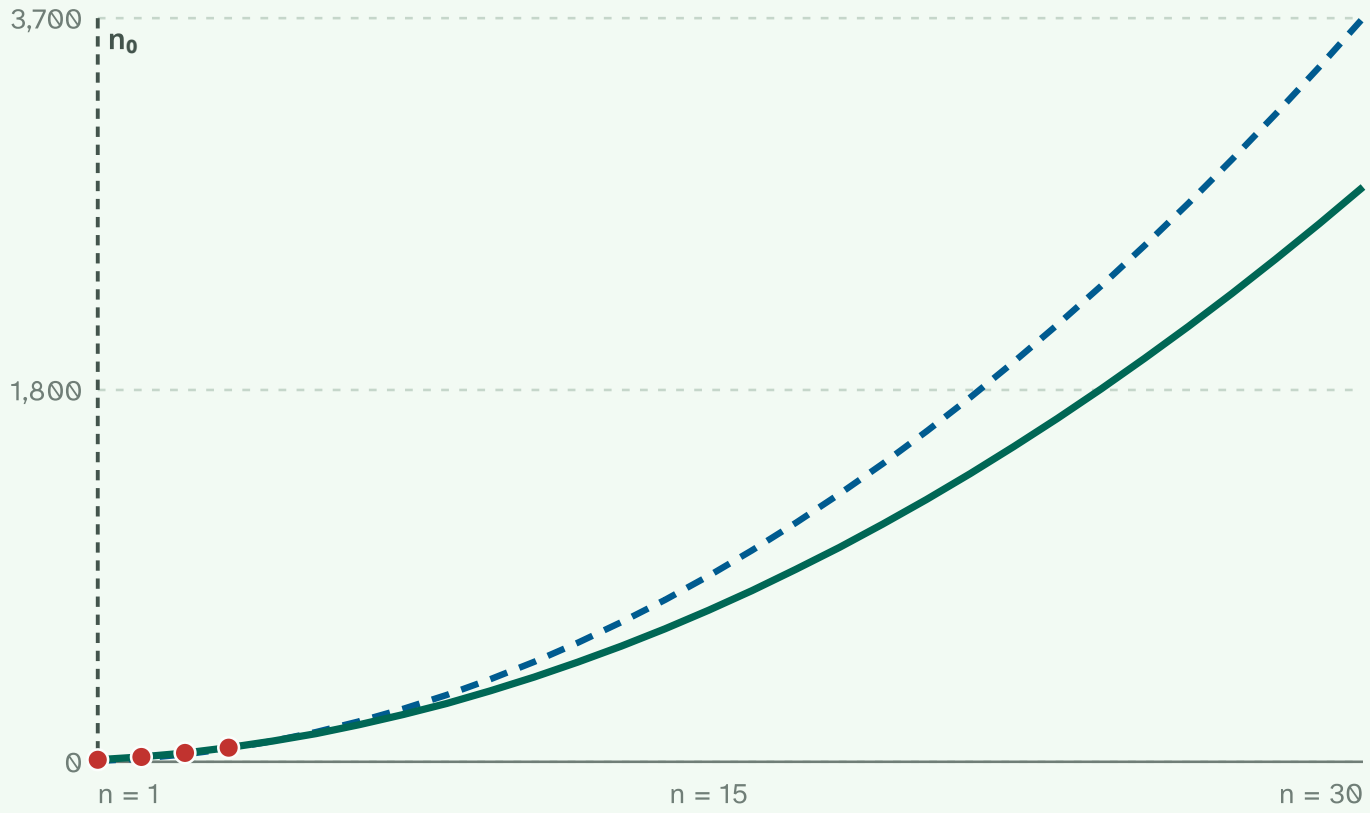
Lab: find a Big-O witness

INTERACTIVE LAB

To prove $f(n) = O(g(n))$ you must produce witnesses C and n_0 with $f(n) \le C\,g(n)$ for *every* $n \ge n_0$. Choose f and g, then adjust C and n_0. Red dots mark values of $n \ge n_0$ where the inequality fails. The panel also checks far beyond the chart.

f(n) = $3n^2 + 5n + 2$ g(n) = n^2 C = 4.1 n_0 = 1

f(n) C·g(n) with C = 4.1 failure at some n ≥ n_0



$$f(n) = 3n^2 + 5n + 2 \overset{?}{\le} C\,g(n) = 4.1 \cdot n^2$$

YOUR WITNESS ✗ **Fails** at 1, 2, 3, 4. First: $f(1) = 10 > C\,g(1) = 4.1$. Move n_0 past the last red dot, or raise C.

WHAT THEORY SAYS

$f = \Theta(g)$: $f(n)/g(n)$ approaches 3. Every C above 3 works from some n_0 on; every C below 3 fails forever.

Find a witness for me

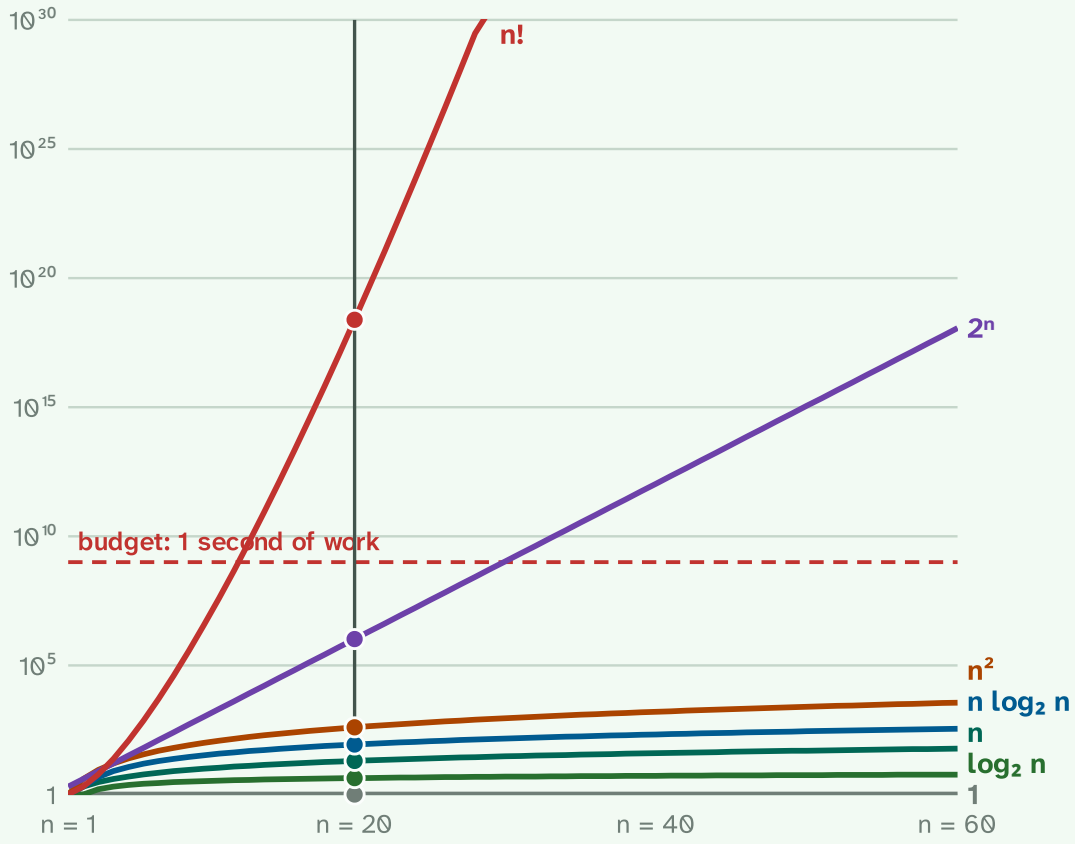
chart up to n = 20 30 60 100 y-axis linear log

Lab: growth rates and the one-second budget

INTERACTIVE LAB

Each curve counts the operations an algorithm of that growth rate performs on an input of size n, on a log scale (each grid line is 100,000 times the one below). The table assumes a fast computer doing 10^9 simple operations per second. Move n and change the time budget.

input size n = 20 time budget [1 millisecond] [1 second] [1 minute] [1 day] [1 year]



GROWTH	OPS AT N = 20	TIME	LARGEST N IN 1 SECOND
1	1	1 ns	any n
$\log_2 n$	4	4.3 ns	2^(10^9)
n	20	20 ns	1,000,000,000
$n \log_2 n$	86	86 ns	39,620,077
n^2	400	400 ns	31,622
2^n	1,048,576	1 ms	29
n!	2.4×10^{18}	77 years	12

Red times exceed the budget. With 1 second of work, a $\Theta(n^2)$ algorithm handles about **31,622** items, an $\Theta(n \log n)$ one about **39,620,077**, and an exponential one only **29**.

WHY IT MATTERS

Doubling n adds one step to $\log n$, doubles n, quadruples n^2, and *squares* 2^n. A faster computer moves the budget line up a little; a better algorithm changes the curve. That is why analysis beats hardware at scale.

Recursion and loop invariants

CONCEPT

DEFINITION

A recursive algorithm solves a problem by calling itself on a smaller input. The base case is small enough to answer directly, and every call must move toward it.

```
factorial(n) {
  if (n == 0)    // base case
    return 1
  return n * factorial(n − 1)
}
```

$4! = 4 \cdot 3!$, $3! = 3 \cdot 2!$, ..., down to $0! = 1$. Then the answers multiply on the way back up: $1, 1, 2, 6, 24$.

```
bsearch(s, i, j, key) {     // search s[i..j]
  if (i > j) return 0       // base case: empty range
  k = ⌊(i + j)/2⌋
  if (key == s[k]) return k
  if (key < s[k]) return bsearch(s, i, k − 1, key)
  return bsearch(s, k + 1, j, key)
}
```

Euclid's algorithm is the classic recursive example: $\gcd(a, b) = \gcd(b,\ a \bmod b)$, base case $\gcd(a, 0) = a$.

Open Euclid's algorithm in Module 6 →

COMMON PITFALL

A missing base case, or a call that does not shrink the input, recurses forever. Python stops it after about 1000 nested calls with `RecursionError`.

DEFINITION

A loop invariant is a statement that is true before a loop starts and stays true after every iteration.

THEOREM / RULE

To prove a loop correct, check three things (the same pattern as induction in Module 3):

1. **Initialization:** the invariant holds before the first iteration.
2. **Maintenance:** if it holds before an iteration, it holds after it.
3. **Termination:** when the loop stops, invariant plus stopping condition give the answer.

WORKED EXAMPLE

For `max`: "after processing $s[1..i]$, `large` is the largest of them". It holds at $i = 1$ since `large = s[1]`. Each step compares $s[i + 1]$ and keeps the larger. At $i = n$, `large` is the largest of all.

WHERE THIS SHOWS UP IN AI

Decision-tree learning is recursive: split the data on the best question, then build a subtree for each part, stopping at a base case such as "every example has the same label".

Design practice: algorithms in real code

DESIGN PRACTICE

Analysis pays off when it shapes everyday coding decisions. Click each principle (or use ↓) to compare code to avoid with code to write.

01 Choose the algorithm by complexity first
02 Know your input size
03 Prefer built-ins: sorted, bisect, min, heapq
04 State and test loop invariants
05 Handle the edge cases
06 Avoid accidental O(n²)
07 Write the pseudocode first

Principle 1 of 7

Choose the algorithm by complexity first

A faster inner loop speeds an $O(n^2)$ algorithm up by a constant factor; a better algorithm changes the growth rate. Pick the right complexity class first, then micro-optimize only if measurements say you must.

Below, `items.count(x)` scans the whole list for every element, which is n scans of n items. A `Counter` makes one pass.

✗ AVOID THIS

```
dups = []
for x in items:                # n times
    if items.count(x) > 1:     # n steps each
        dups.append(x)         # O(n²) total
```

✓ DO THIS

```
from collections import Counter
counts = Counter(items)        # one pass: O(n)
dups = [x for x, c in counts.items() if c > 1]
```

Design practice: algorithms in real code: all principles (1/2)

Choose the algorithm by complexity first

A faster inner loop speeds an $O(n^2)$ algorithm up by a constant factor; a better algorithm changes the growth rate. Pick the right complexity class first, then micro-optimize only if measurements say you must.

Below, `items.count(x)` scans the whole list for every element, which is n scans of n items. A `Counter` makes one pass.

```
from collections import Counter
counts = Counter(items)        # one pass: O(n)
dups = [x for x, c in counts.items() if c > 1]
```

Know your input size

An $O(n^2)$ algorithm is perfect for $n = 100$ (10^4 steps) and hopeless for $n = 10^7$ (10^{14} steps, about a day). Before choosing, write down the realistic n, estimate the operation count, and compare it with roughly 10^8 to 10^9 simple steps per second.

```
n = len(records)              # 2,000,000 rows
# n**2        = 4e12 steps -> over an hour
# n*log2(n)   = 4.2e7 steps -> under a second
# so: sort + one pass, not all pairs
```

Prefer built-ins: sorted, bisect, min, heapq

Python's built-ins are tested on millions of programs and run in optimized C. `sorted()` is $O(n \log n)$ and stable; `bisect` is binary search; `min` / `max` are one pass; `heapq.nsmallest` finds the top k without sorting everything. A hand-written bubble sort is slower and one typo away from a bug.

```
import bisect
a = sorted(data)                      # O(n log n)
i = bisect.bisect_left(a, target)     # O(log n)
found = i < len(a) and a[i] == target
```

State and test loop invariants

Write the invariant as a comment, then check it with `assert` while testing. When a change breaks the loop, the assertion fails on the exact iteration where the reasoning went wrong, instead of producing a wrong answer much later.

```
large = s[0]
for i in range(1, len(s)):
    if s[i] > large:
        large = s[i]
    # invariant: large is the max of s[0..i]
    assert large == max(s[:i + 1])
```

Handle the edge cases

Most algorithm bugs live at the edges: the empty list, a single element, duplicates, a target that is absent or sits at either end. Decide what each case should do, make the code say so, and test every one of them.

```
def largest(s):
    if not s:
        raise ValueError("largest() of empty list")
    ...
for case in ([], [7], [2, 2], [3, 1, 2]): ...
```

Design practice: algorithms in real code: all principles (2/2)

Avoid accidental O(n²)

Some innocent-looking lines hide a full scan: `x in some_list`, `list.remove(x)`, `list.pop(0)`, `list.insert(0, x)`, and building a string with `+=` in a loop. Inside a loop of n iterations, each turns the whole loop into $O(n^2)$. Use a set, a `deque`, or `"".join(parts)` instead.

```
seen = set()
for x in stream:
    if x not in seen:        # hash lookup
        seen.add(x)
text = "".join(parts)        # not s += p in a
loop
```

Write the pseudocode first

Sketch the algorithm in plain steps before typing Python. Pseudocode makes the loop structure, and therefore the cost, visible at a glance, and it is far cheaper to fix an $O(n^2)$ plan on paper than in a finished program.

```
# closest pair of scores:
# 1. sort the scores                  O(n log
n)
# 2. scan once, compare neighbors     O(n)
# 3. return the pair with least gap
# total: O(n log n), not O(n²) over all pairs
```

Worked example: binary search for 31

WORKED EXAMPLE

PROBLEM

Use binary search to find 31 in the sorted list $[3, 7, 12, 19, 25, 31, 42, 56]$, showing *lo*, *hi* and *mid* at every probe. Indices start at 0, and $mid = \lfloor (lo + hi)/2 \rfloor$.

Probe 2: lo = 4 · hi = 7 · mid = 5 · list[5] = 31 = target · found

index 0	index 1	index 2	index 3	index 4	index 5	index 6	index 7
3	7	12	19	25	**31**	42	56
ruled out	ruled out	ruled out	ruled out	lo	mid · found ★		hi

Dimmed cells stay ruled out, and the starred cell is the probe that matched the target.

Answer: 31 is at index 5 · check: the list is sorted, list[5] = 31, and the search took only 2 probes ✓

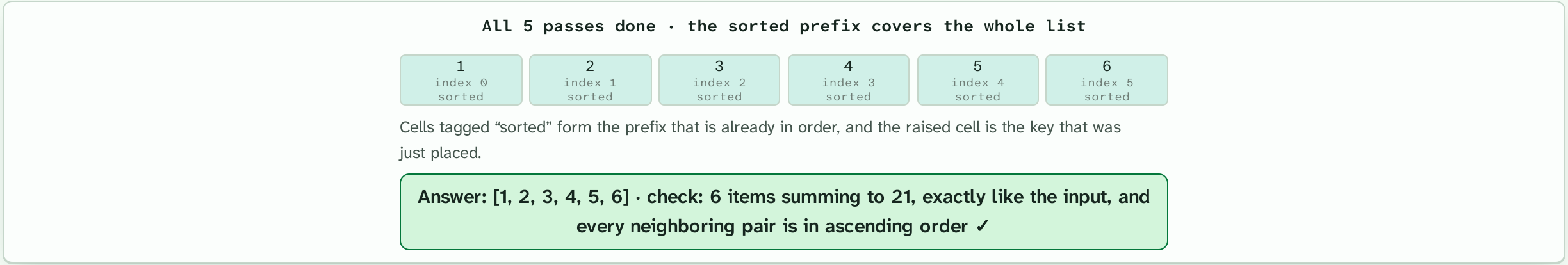
1. Setup: the list [3, 7, 12, 19, 25, 31, 42, 56] is sorted and the target is 31, with lo = 0 and hi = 7 marking the whole range as still possible. Binary search needs a sorted list and works by ruling out half of what remains with each probe — that halving is where the speed comes from.
2. Probe 1: mid = (0 + 7) // 2 = 3 and list[3] = 19, and since 19 < 31 the target, if present, is strictly to the right of mid. Cells 0 to 3 are ruled out — mid itself goes too, because we just compared it and it was too small — so lo becomes 4.
3. Probe 2: with lo = 4 and hi = 7, mid = (4 + 7) // 2 = 5 and list[5] = 31, which equals the target exactly. The search stops at once, because an equal probe is the success case and no smaller range needs to be checked.
4. Confirm the hit before reporting it: index 5 lies inside the narrowed range from lo = 4 to hi = 7, and the value stored there is exactly 31. Checking the index and the value together guards against reporting a position produced by an arithmetic slip.
5. **Answer: 31 is at index 5, found in only 2 probes where linear search could need up to 8. Check: the list really is sorted and list[5] = 31, and each probe at most halves the candidates, so the range shrank from 8 cells all the way down to the single found cell.**

Worked example: insertion sort, pass by pass

WORKED EXAMPLE

PROBLEM

Sort $[5, 2, 4, 6, 1, 3]$ with insertion sort, one pass per step, growing a sorted prefix from the left and sliding each new key into its place.



1. Setup: start with [5, 2, 4, 6, 1, 3], where the sorted prefix is just [5] because a single item is always in order by itself. Insertion sort grows that sorted prefix one key at a time, inserting each new key into its place — like sorting a hand of cards.
2. Pass 1 takes the key 2 from index 1: it is compared 1 time while sliding left past 1 larger item, and it lands at index 0, the very front of the list. The list becomes [2, 5, 4, 6, 1, 3] and the sorted prefix grows to [2, 5].
3. Pass 2 takes the key 4 from index 2: it is compared 2 times while sliding left past 1 larger item, and it lands at index 1. The list becomes [2, 4, 5, 6, 1, 3] and the sorted prefix grows to [2, 4, 5].
4. Pass 3 takes the key 6 from index 3: its only comparison is against 5, the last item of the sorted prefix, and 6 is not smaller, so nothing shifts and the key stays at index 3. The list is unchanged, [2, 4, 5, 6, 1, 3], but the sorted prefix still grows to [2, 4, 5, 6].
5. Pass 4 takes the key 1 from index 4: it is compared 4 times while sliding left past 4 larger items, and it lands at index 0, the very front of the list. The list becomes [1, 2, 4, 5, 6, 3] and the sorted prefix grows to [1, 2, 4, 5, 6].
6. Pass 5 takes the key 3 from index 5: it is compared 4 times while sliding left past 3 larger items, and it lands at index 2. The list becomes [1, 2, 3, 4, 5, 6] and the sorted prefix grows to [1, 2, 3, 4, 5, 6].
7. **Answer: the sorted list is [1, 2, 3, 4, 5, 6], in ascending order at every neighboring pair. Check: it is the same multiset as the input — both lists have 6 elements and both sum to 21, so nothing was lost, added or duplicated while sorting. The whole sort took 12 comparisons and 9 shifts.**

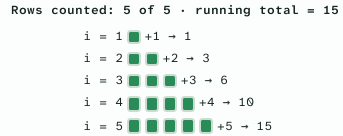
Worked example: counting a nested loop exactly

WORKED EXAMPLE

PROBLEM

Count exactly how many times the line `x = x + 1` runs in the nested loop below when $n = 5$, counting row by row, and check the total against its closed form.

```
for i = 1 to n
  for j = 1 to i
    x = x + 1
```



i	inner runs	running total
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

Closed form check: n(n + 1)/2 = 5 · 6/2 = 15 ✓

Answer: 15 executions · check: 1 + 2 + 3 + 4 + 5 = 15 and the closed form n(n + 1)/2 = 5 · 6/2 = 15 ✓

1. Setup: count how many times the inner line x = x + 1 runs when n = 5. The outer loop takes i = 1 to 5 in turn; for each fixed i the inner loop runs j from 1 to i, so each outer value contributes one row of executions, and the rows accumulate into the total we are after.
2. Row i = 1: the inner loop runs j = 1 to i = 1, which is 1 execution, because both inner bounds are fixed once i is 1. Adding 1 to the running total gives 0 + 1 = 1, and the picture gains a row of 1 dot — one longer than the row before.
3. Row i = 2: the inner loop runs j = 1 to i = 2, which is 2 executions, because both inner bounds are fixed once i is 2. Adding 2 to the running total gives 1 + 2 = 3, and the picture gains a row of 2 dots — one longer than the row before.
4. Row i = 3: the inner loop runs j = 1 to i = 3, which is 3 executions, because both inner bounds are fixed once i is 3. Adding 3 to the running total gives 3 + 3 = 6, and the picture gains a row of 3 dots — one longer than the row before.
5. Row i = 4: the inner loop runs j = 1 to i = 4, which is 4 executions, because both inner bounds are fixed once i is 4. Adding 4 to the running total gives 6 + 4 = 10, and the picture gains a row of 4 dots — one longer than the row before.
6. Row i = 5: the inner loop runs j = 1 to i = 5, which is 5 executions, because both inner bounds are fixed once i is 5. Adding 5 to the running total gives 10 + 5 = 15, and the picture gains a row of 5 dots — one longer than the row before.
7. **Answer: the inner line runs 15 times in all: 1 + 2 + 3 + 4 + 5 = 15. Check: the closed form n(n + 1)/2 = 5 · 6/2 = 15 agrees, and actually running both loops with a counter ends at the same 15, so the row-by-row count, the formula and the simulation all match. The count is Theta(n^2) either way, because the total grows like a constant times n^2.**

Worked example: a growth race at doubling inputs

WORKED EXAMPLE

PROBLEM

Three algorithms use n, $n \log_2 n$ and n^2 operations. Compute the exact counts at $n = 4, 8, 16, 32$, track each family's doubling factor, and name the family that grows fastest.

Final column: n = 32 · winner: n²

family	n = 4	n = 8	n = 16	n = 32
n	4	8	16	32
n log₂ n	8	24	64	160
n²	16	64	256	1024

Answer: n² grows fastest · check: at n = 32 it is 1024, ahead of n = 32 and n log₂ n = 160, and its doubling factor never shrinks ✓

1. Setup: race the operation counts n, n log₂ n, n² as n doubles through 4, 8, 16, 32. Each new column recomputes every count at the doubled input, and the doubling factor of each family — how many times bigger its count gets — is what decides the race in the long run.
2. First column, n = 4: the counts are n = 4, n log₂ n = 8, n² = 16. At this small size the families sit close together and the lead looks modest, but a race between growth rates is decided by the doubling factors, not by who leads first — so read the columns left to right.
3. Doubling to n = 8: n: 4 → 8 (×2, because doubling the input doubles the count); n log₂ n: 8 → 24 (×3, because log₂ n grows by exactly 1 when n doubles, so the factor 2(k+1)/k shrinks toward 2); n²: 16 → 64 (×4, because (2n)² = 4n², so the count quadruples). The family with the stubbornly largest doubling factor pulls further ahead at every doubling, which is exactly why constant factors stop mattering for large n.
4. Doubling to n = 16: n: 8 → 16 (×2, because doubling the input doubles the count); n log₂ n: 24 → 64 (×2.67, because log₂ n grows by exactly 1 when n doubles, so the factor 2(k+1)/k shrinks toward 2); n²: 64 → 256 (×4, because (2n)² = 4n², so the count quadruples). The family with the stubbornly largest doubling factor pulls further ahead at every doubling, which is exactly why constant factors stop mattering for large n.
5. Doubling to n = 32: n: 16 → 32 (×2, because doubling the input doubles the count); n log₂ n: 64 → 160 (×2.5, because log₂ n grows by exactly 1 when n doubles, so the factor 2(k+1)/k shrinks toward 2); n²: 256 → 1024 (×4, because (2n)² = 4n², so the count quadruples). The family with the stubbornly largest doubling factor pulls further ahead at every doubling, which is exactly why constant factors stop mattering for large n.
6. **Answer: n² grows fastest of the 3 families. At n = 32 it already needs 1024 operations against n = 32 and n log₂ n = 160, and its doubling factor never shrinks, so the gap only widens from here. A counted total with this shape is Theta(n²): that family is both an upper bound and a lower bound for its growth.**

Practice: algorithms and their analysis

PRACTICE

Lists are typed in brackets, such as `[2, 5, 9]`. Indices start at 0, as in Python.

SEARCHING

What is the largest number of comparisons binary search can make on a sorted list of $n = 1,000,000$ items?

Type your answer

Check

Hint: the formula is $\lfloor \log_2 n \rfloor + 1$.

Show worked solution

New problem ↻

$2^{19} = 524,288 \le 1,000,000 < 2^{20} = 1,048,576$, so $\lfloor \log_2 n \rfloor = 19$ and the worst case is $19 + 1 = 20$ comparisons. Linear search could need all 1,000,000.

PROBLEM TYPE

- Searching
- Counting operations
- Big-O
- Sorting passes

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: algorithms and their analysis: sample problems with solutions

SEARCHING

What is the largest number of comparisons binary search can make on a sorted list of $n = 4,096$ items?

$2^{12} = 4,096 \le 4,096 < 2^{13} = 8,192$, so $\lfloor \log_2 n \rfloor = 12$ and the worst case is $12 + 1 = 13$ comparisons. Linear search could need all 4,096.

COUNTING OPERATIONS

With $n = 14$ and `count = 0` at the start, what is `count` after this code runs?

```
for i in range(n):
    count += 1
for j in range(n):
    for k in range(3):
        count += 1
```

Blocks in sequence add: $14 + 3 \cdot 14 = 56$. The constant 3 does not change the growth rate: $\Theta(n)$.

BIG-O

Which is the tightest description of $f(n) = 60 \log_2 n + 5n^2 + 29$?

Keep the fastest-growing term and drop its constant. The order is $1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n$, so $5n^2$ dominates and $f(n) = \Theta(n^2)$.

SORTING PASSES

Apply **selection sort** to $[26, 10, 33, 3, 34, 25]$. What does the list look like after 1 pass of the outer loop?

Each pass finds the smallest remaining item and swaps it into the next position.

After pass 1: [3, 10, 33, 26, 34, 25]

Check yourself: algorithms

CHECK YOURSELF

QUESTION 1 OF 10

A recipe says "add salt until it tastes right". Which property of an algorithm does that step violate?

A Generality

B Precision

C Input

D Output

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: algorithms: answers explained (1/2)

1. A recipe says "add salt until it tastes right". Which property of an algorithm does that step violate?

Answer B: Precision

"Until it tastes right" is not stated exactly; two cooks could stop at different points. Precision requires every step to be unambiguous.

2. A loop starts with `x = 1`, adds 1 to `x` each time, and stops when `x` is negative. Which property fails?

Answer A: Finiteness

`x` only grows, so it never becomes negative and the loop never stops. An algorithm must finish after finitely many steps on every input.

3. What is the largest number of comparisons binary search makes on 1,000,000 sorted items?

Answer C: 20

$2^{19} = 524{,}288 \le 10^6 < 2^{20}$, so $\lfloor \log_2 10^6 \rfloor + 1 = 19 + 1 = 20$. Each comparison halves the remaining range.

4. What must be true of a list before binary search can be used on it?

Answer A: It is sorted

Binary search throws away half the list based on one comparison, which is only safe when the list is sorted. Duplicates and odd lengths are fine, and an absent target is reported as not found.

5. How many comparisons does insertion sort make on a list of n items that is already sorted?

Answer D: $n - 1$

Each of the $n - 1$ passes compares the new item with its left neighbor once, finds it is not smaller, and stops. That is the best case.

6. Which is the tightest description of $3n^2 + 100n + 7$?

Answer A: $\Theta(n^2)$

Drop the constant factors and keep the dominant term: n^2 eventually outgrows $100n$, so the expression is $\Theta(n^2)$. $O(n^3)$ would also be true, but it is not tight.

7. Which function grows fastest as n becomes large?

Answer A: $n!$

$n! = 1 \cdot 2 \cdots n$ eventually beats 2^n (it has more and more factors larger than 2), and every exponential beats every polynomial.

8. How does the running time of this code grow?

```
for i in range(n):
    for j in range(i):
        total += 1
```

Answer B: $\Theta(n^2)$

The inner line runs $0 + 1 + \cdots + (n - 1) = n(n - 1)/2$ times. Dropping the constant $\frac{1}{2}$ and the lower-order term leaves $\Theta(n^2)$.

Check yourself: algorithms: answers explained (2/2)

9. Which statement is a loop invariant of selection sort?

Answer B: After i passes, the first i positions hold the i smallest items in final order

Each pass swaps the smallest remaining item into the next position, so the sorted prefix grows by one and never changes again. When $i = n - 1$, the whole list is sorted.

10. Self-attention costs $\Theta(n^2)$ in the context length n. Roughly how much more attention work does a 16,000-token context need than a 4,000-token one?

Answer A: 16 times

The context is 4 times longer, and the cost grows with the square: $4^2 = 16$. That quadratic growth is why long-context models use tricks to avoid comparing every pair of tokens.

Divisibility and the division algorithm

CONCEPT

DEFINITION

For integers $a \neq 0$ and b, a divides b, written $a \mid b$, when $b = ak$ for some integer k. Then a is a **divisor** (or factor) of b, and b is a **multiple** of a. So $3 \mid 12$ because $12 = 3 \cdot 4$, while $5 \nmid 12$.

THEOREM / RULE

The division algorithm. For any integer a and any integer $d > 0$ there is exactly one pair of integers q, the quotient, and r, the remainder, with

$$a = dq + r, \qquad 0 \le r < d.$$

We write $q = \lfloor a/d \rfloor$ and $r = a \bmod d$.

Three facts follow straight from the definition, and Euclid's algorithm later relies on them. If $a \mid b$ and $a \mid c$, then $a \mid (b \pm c)$. If $a \mid b$, then $a \mid bc$. If $a \mid b$ and $b \mid c$, then $a \mid c$.

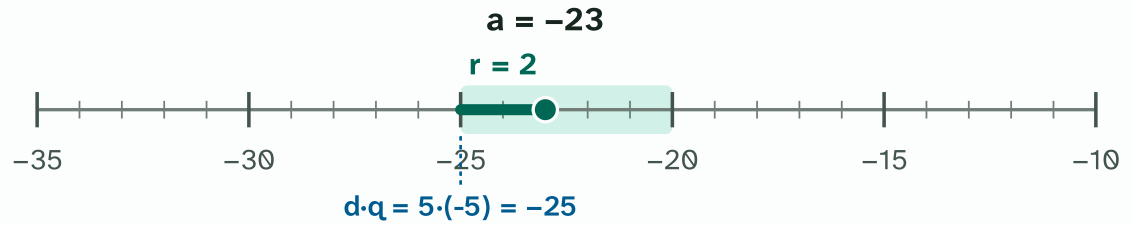
WHERE THIS SHOWS UP IN AI

Splitting $N = 1000$ training examples into batches of $d = 64$ gives $q = 15$ full batches and $r = 40$ examples left over. Data loaders compute exactly this with `N // 64` and `N % 64`, then decide whether to keep or drop the short last batch.

NUMBER-LINE MODEL · DRAG a, PICK d

a = −23

d = 2 3 4 **5** 6 7 8 9



$-23 = 5 \cdot (-5) + 2$, so $q = \lfloor -23/5 \rfloor = -5$ and $r = 2$. The point a always sits in the shaded block from dq up to $d(q+1)$; r is how far it lies past the left end. Rounding toward zero would give $q = -4$ and a negative remainder -3, which the theorem forbids.

COMMON PITFALL

The remainder is never negative. For $a = -23$ and $d = 5$ the answer is $q = -5$, $r = 2$, because $-23 = 5 \cdot (-5) + 2$. Python agrees: `-23 // 5` is `-5` and `-23 % 5` is `2`. C, Java and JavaScript round the quotient toward zero instead, so in those languages `-23 % 5` is `-3`.

Primes and unique factorization

CONCEPT

DEFINITION

An integer $p > 1$ is prime when its only positive divisors are 1 and p. An integer $n > 1$ that is not prime is composite: it factors as $n = ab$ with $1 < a, b < n$. The number 1 is neither prime nor composite.

THEOREM / RULE

If n is composite, then n has a prime divisor $p \le \sqrt{n}$.

Here is why. Write $n = ab$ with $a \le b$. Then $a \cdot a \le a \cdot b = n$, so $a \le \sqrt{n}$, and any prime factor of a is at most a. So **trial division** tests only the primes up to $\sqrt{n}$. For $n = 97$, $\sqrt{97} \approx 9.8$, and none of 2, 3, 5, 7 divides 97, so 97 is prime. For $n = 91$, the test finds $7 \mid 91$, since $91 = 7 \cdot 13$.

DESIGN TIP

Stopping at $\sqrt{n}$ is a huge saving. For a 12-digit number it means about a million trial divisions instead of a trillion.

FUNDAMENTAL THEOREM OF ARITHMETIC

Every integer $n > 1$ is a product of primes in exactly one way, apart from the order of the factors.

$$360 = 2 \cdot 180 = 2 \cdot 2 \cdot 90 = \cdots = 2^3 \cdot 3^2 \cdot 5$$

Whichever factor you split off first, you end with the same primes.

THEOREM (EUCLID)

There are infinitely many primes.

This is the proof by contradiction from Module 3. Suppose $p_1, \ldots, p_k$ were all the primes, and let $N = p_1 p_2 \cdots p_k + 1$. Dividing N by any p_i leaves remainder 1, so no p_i divides N. Yet $N > 1$ has a prime factor, and that prime is missing from the list. Contradiction.

WHERE THIS SHOWS UP IN AI

The encryption that protects every call to a hosted AI model rests on a gap between two problems. Multiplying two 300-digit primes takes a computer microseconds. No known method recovers the two primes from their product in any reasonable time.

Lab: the Sieve of Eratosthenes

INTERACTIVE LAB

The sieve finds every prime up to N without a single division. Each pass takes the smallest number that is still standing (it must be prime) and crosses out its multiples. **Step** runs one pass; **Play** shows each number fall. Colors show which prime removed each number.

Numbers up to N = 30 | 60 | **100** | 200

◀ Back | Step ▶ | ▶ Play | Finish | Reset

1	**2**	3	~~4~~	5	~~6~~	7	~~8~~	9	~~10~~
11	~~12~~	13	~~14~~	15	~~16~~	17	~~18~~	19	~~20~~
21	~~22~~	23	~~24~~	25	~~26~~	27	~~28~~	29	~~30~~
31	~~32~~	33	~~34~~	35	~~36~~	37	~~38~~	39	~~40~~
41	~~42~~	43	~~44~~	45	~~46~~	47	~~48~~	49	~~50~~
51	~~52~~	53	~~54~~	55	~~56~~	57	~~58~~	59	~~60~~
61	~~62~~	63	~~64~~	65	~~66~~	67	~~68~~	69	~~70~~
71	~~72~~	73	~~74~~	75	~~76~~	77	~~78~~	79	~~80~~
81	~~82~~	83	~~84~~	85	~~86~~	87	~~88~~	89	~~90~~
91	~~92~~	93	~~94~~	95	~~96~~	97	~~98~~	99	~~100~~

Pass 1 of 4: $p = 2$

2 is the first number still standing, so it is prime. Every larger even number has 2 as a factor, so cross out $4, 6, 8, \ldots$ up to 100.

CROSSED IN THIS PASS (49 NEW, 0 ALREADY GONE)

4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40
42 44 46 48 50 52 54 56 58 …

1	49	49
PRIMES FOUND	CROSSED OUT	UNDECIDED

Next candidate: 3, the smallest number still standing. Press **Step** to run its pass.

multiples of 2 · multiples of 3 · multiples of 5 · multiples of 7 · prime

GCD, LCM, and Euclid's algorithm

CONCEPT

DEFINITION

The greatest common divisor $\gcd(a, b)$ is the largest integer dividing both a and b (not both 0). The least common multiple $\operatorname{lcm}(a, b)$ is the smallest positive integer that both divide. If $\gcd(a, b) = 1$, then a and b are relatively prime.

WORKED EXAMPLE

From the factorizations $360 = 2^3 \cdot 3^2 \cdot 5$ and $84 = 2^2 \cdot 3 \cdot 7$: the gcd takes the *smaller* power of each prime, and the lcm takes the *larger*.

$$\gcd = 2^2 \cdot 3 = 12, \qquad \operatorname{lcm} = 2^3 \cdot 3^2 \cdot 5 \cdot 7 = 2520$$

THEOREM / RULE

For positive a, b: $\gcd(a, b) \cdot \operatorname{lcm}(a, b) = a \cdot b$. Check: $12 \cdot 2520 = 30240 = 360 \cdot 84$.

BÉZOUT'S IDENTITY

There are integers s, t with $\gcd(a, b) = s\,a + t\,b$. Running Euclid's steps backwards finds them; that is the extended Euclidean algorithm.

EUCLID'S ALGORITHM

$\gcd(a, b) = \gcd(b,\ a \bmod b)$, and $\gcd(a, 0) = a$.

Why it works: write $a = qb + r$. Any d that divides a and b also divides $r = a - qb$. Any d that divides b and r also divides $a = qb + r$. Both pairs have exactly the same common divisors, so they have the same greatest one.

WORKED EXAMPLE

$\gcd(252, 198)$, dividing until the remainder is 0:

$$\begin{aligned} 252 &= 1 \cdot 198 + 54 \\ 198 &= 3 \cdot 54 + 36 \\ 54 &= 1 \cdot 36 + \mathbf{18} \\ 36 &= 2 \cdot 18 + 0 \end{aligned}$$

The last nonzero remainder is 18. Backwards:
$18 = 54 - 36 = 4 \cdot 54 - 198 = 4 \cdot 252 - 5 \cdot 198$.

WHY IT MATTERS

Euclid's algorithm is fast. Every two steps at least halve the remainder, so the number of steps grows with the number of *digits*. Lamé's theorem caps it at five steps per decimal digit of the smaller number: at most 1,500 steps for two 300-digit numbers. Testing every possible divisor would take longer than the age of the universe.

Lab: Euclid's algorithm, step by step

INTERACTIVE LAB

Each division step cuts the largest possible squares from an $a \times b$ rectangle, and the leftover strip is the next, smaller problem. The final square size tiles the whole rectangle exactly: that is the gcd. Switch to the **Bézout table** to see every remainder written as $s\,a + t\,b$.

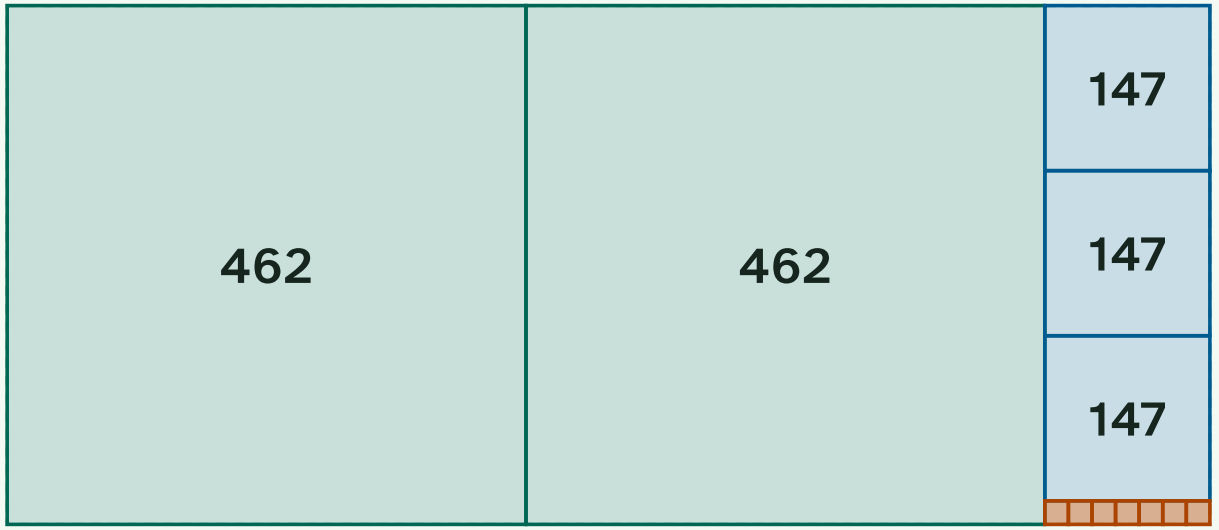
a = 1071 b = 462 Try: 1071, 462 | 252, 198 | 89, 55 | 120, 84 | 17, 5 | 462, 1071 ◀ Back | Step ▶ | Reset

Division steps | Bézout table

1 1071 = 2 · 462 + 147
2 462 = 3 · 147 + 21
3 147 = 7 · 21 + 0

The remainder reached 0, so $\gcd(1071, 462) = 21$, the last nonzero remainder. 3 division steps; Lamé's bound for a 3-digit number allows at most 15.

THE a × b RECTANGLE, CUT INTO SQUARES



2 × (462 × 462) 3 × (147 × 147) 7 × (21 × 21) **smallest square: 21 × 21 = gcd**

Step 3: 7 squares of side 21 fill the 147×21 rectangle exactly. Every earlier square is tiled by 21×21 squares too, so 21 divides both numbers: $\gcd = 21$.

Code: Euclid's algorithm and Bézout coefficients

CODE WALKTHROUGH

A loop finds the gcd, and a recursive version also returns the Bézout coefficients. **Trace** shows the recursive calls stacking up.

euclid.py

Read | Edit & run | Trace

```
1  def gcd(a, b):
2      while b != 0:
3          a, b = b, a % b
4      return a
5
6  def extended_gcd(a, b):
7      if b == 0:
8          return a, 1, 0
9      g, s, t = extended_gcd(b, a % b)
10     return g, t, s - (a // b) * t
11
12 print(gcd(1071, 462))
13 g, s, t = extended_gcd(1071, 462)
14 print(g, s, t)
15 print(s * 1071 + t * 462 == g)
16 print(1071 * 462 // g)
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
21
21 -3 7
True
23562
```

LINE 1 OF 16

Show all notes in code

```
def gcd(a, b):
```

Defines a function that implements Euclid's algorithm with a loop. Nothing runs yet; Python stores the recipe until line 12 calls it.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Computes gcd(1071, 462) with Euclid's loop, then finds integers s and t with gcd = s·1071 + t·462, and finally the lcm.

Code: Euclid's algorithm and Bézout coefficients: every line explained

1	`def gcd(a, b):`	Defines a function that implements Euclid's algorithm with a loop. Nothing runs yet; Python stores the recipe until line 12 calls it.
2	`    while b != 0:`	The loop keeps going while there is a nonzero number to divide by. When `b` reaches 0, `a` holds the answer, because $\gcd(a, 0) = a$.
3	`        a, b = b, a % b`	The heart of the algorithm: replace the pair (a, b) by $(b,\ a \bmod b)$, which has the same gcd. Python evaluates the whole right side first, so both new values come from the old pair. For our numbers: $(1071, 462) \to (462, 147) \to (147, 21) \to (21, 0)$.
4	`    return a`	The loop has ended, so `b` is 0 and `a` is the gcd. For 1071 and 462 this returns 21 after three trips around the loop.
6	`def extended_gcd(a, b):`	A second function that returns three numbers (g, s, t) with $g = \gcd(a, b) = s \cdot a + t \cdot b$. It is **recursive**: it calls itself on the smaller pair $(b,\ a \bmod b)$, following the same recurrence as the loop.
7	`    if b == 0:`	The base case, where the recursion stops. Without it the function would call itself forever.
8	`        return a, 1, 0`	When $b = 0$ the gcd is a, and $a = 1 \cdot a + 0 \cdot 0$, so $s = 1$ and $t = 0$. Python packs the three values into a tuple.
9	`    g, s, t = extended_gcd(b, a % b)`	Solves the smaller problem first and unpacks its answer. Afterwards $g = s \cdot b + t \cdot (a \bmod b)$. The calls go $(1071, 462) \to (462, 147) \to (147, 21) \to (21, 0)$; Trace shows each one nested deeper.
10	`    return g, t, s - (a // b) * t`	Turns the smaller answer into one for (a, b). Since $a \bmod b = a - (a//b) \cdot b$, substituting gives $g = t \cdot a + (s - (a//b)\,t) \cdot b$. So the new coefficients are `t` and `s - (a // b) * t`. This is the hand back-substitution, done automatically.
12	`print(gcd(1071, 462))`	Calls the loop version. Output: `21`. You can confirm with factorizations: $1071 = 3^2 \cdot 7 \cdot 17$ and $462 = 2 \cdot 3 \cdot 7 \cdot 11$ share $3 \cdot 7 = 21$.
13	`g, s, t = extended_gcd(1071, 462)`	Calls the extended version and unpacks the returned tuple into three names: `g = 21`, `s = -3`, `t = 7`.
14	`print(g, s, t)`	Output: `21 -3 7`, meaning $21 = (-3) \cdot 1071 + 7 \cdot 462$. The coefficients are not unique: $s = 19,\ t = -44$ works too, because adding $462/21 = 22$ to s and subtracting $1071/21 = 51$ from t keeps the sum the same.
15	`print(s * 1071 + t * 462 == g)`	Checks the identity: $-3213 + 3234 = 21$, so this prints `True`. Checking an answer by plugging it back in is a habit worth building into tests.
16	`print(1071 * 462 // g)`	The lcm, from $\gcd \cdot \operatorname{lcm} = a \cdot b$: 494802//21 = 23562. Integer division `//` keeps the result an exact integer instead of a float.

Modular arithmetic and inverses

CONCEPT

DEFINITION

a is congruent to b modulo n, written $a \equiv b \pmod{n}$, when $n \mid (a - b)$. Equivalently, a and b leave the same remainder: $a \bmod n = b \bmod n$. The number n is the modulus.

WORKED EXAMPLE

$38 \equiv 14 \equiv 2 \pmod{12}$, because 12 divides $38 - 14 = 24$ and $38 - 2 = 36$. A clock works this way: 38 hours after midnight it shows 2. Likewise, 100 days after a Wednesday is a Friday, since $100 \bmod 7 = 2$.

THEOREM / RULE

If $a \equiv c$ and $b \equiv d \pmod{n}$, then $a + b \equiv c + d$, $a - b \equiv c - d$ and $ab \equiv cd \pmod{n}$. So you may reduce at every step: $(123 \cdot 456) \bmod 10 = (3 \cdot 6) \bmod 10 = 8$.

COMMON PITFALL

Division does not carry over. $2 \cdot 3 \equiv 2 \cdot 8 \pmod{10}$, yet $3 \not\equiv 8 \pmod{10}$. You may cancel a factor only when it has an inverse.

DEFINITION

An inverse of a modulo n is a number x with $ax \equiv 1 \pmod{n}$. We write it a^{-1}.

THEOREM / RULE

a has an inverse modulo n exactly when $\gcd(a, n) = 1$. Bézout supplies it: if $sa + tn = 1$, then $sa \equiv 1 \pmod{n}$, so $a^{-1} \equiv s$.

WORKED EXAMPLE

$3^{-1} \equiv 5 \pmod{7}$, since $3 \cdot 5 = 15 = 2 \cdot 7 + 1$. To solve $3x \equiv 4 \pmod{7}$, multiply both sides by 5: $x \equiv 20 \equiv 6$. On the other hand, 4 has no inverse mod 6: $\gcd(4, 6) = 2$, and $4x \bmod 6$ is always 0, 2 or 4. In Python, `pow(3, -1, 7)` returns 5.

WHERE THIS SHOWS UP IN AI

Time features are modular: hour 23 and hour 0 are one hour apart. Models encode hour h as the point $(\cos \frac{2\pi h}{24}, \sin \frac{2\pi h}{24})$ on a circle, so the step from 23 to 0 looks like any other one-hour step.

Lab: the modular clock

INTERACTIVE LAB

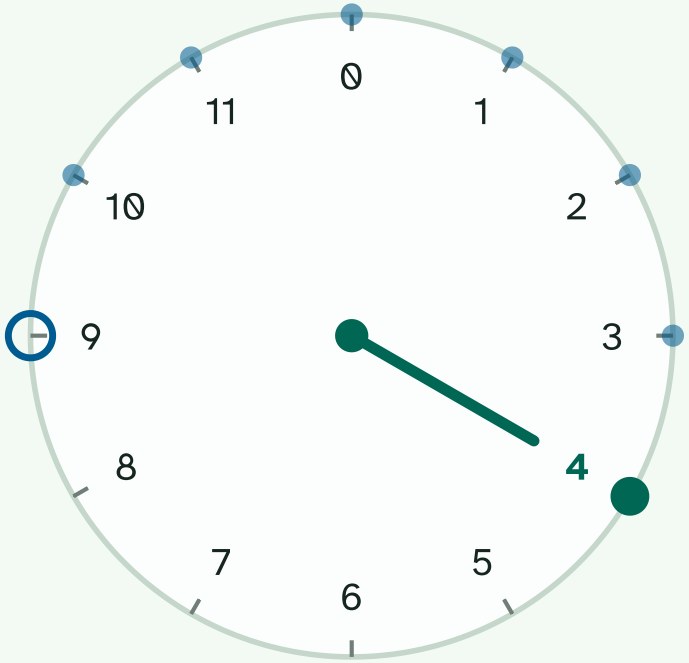
Pick a modulus n, two numbers and an operation. The hand walks around the clock one step at a time, so $a + b$, $a - b$ and $a \times b$ (as b hops of size a) all wrap around at n. The table is $a \cdot x \bmod n$ for every pair; green cells hold a 1, so their row has an inverse. Click any cell to try it.

modulus n 12

a 9

b 7

Operation: a + b | a − b | a × b



MULTIPLICATION TABLE MOD n: ROW a, COLUMN x, CELL a·x mod n

·	0	1	2	3	4	5	6	7	8	9	10	11	a^{-1}
0	0	0	0	0	0	0	0	0	0	0	0	0	—
1	0	1	2	3	4	5	6	7	8	9	10	11	1
2	0	2	4	6	8	10	0	2	4	6	8	10	—
3	0	3	6	9	0	3	6	9	0	3	6	9	—
4	0	4	8	0	4	8	0	4	8	0	4	8	—
5	0	5	10	3	8	1	6	11	4	9	2	7	5
6	0	6	0	6	0	6	0	6	0	6	0	6	—
7	0	7	2	9	4	11	6	1	8	3	10	5	7
8	0	8	4	0	8	4	0	8	4	0	8	4	—
9	0	9	6	3	0	9	6	3	0	9	6	3	—
10	0	10	8	6	4	2	0	10	8	6	4	2	—
11	0	11	10	9	8	7	6	5	4	3	2	1	11

Row 9: $\gcd(9, 12) = 3$. Every entry is a multiple of 3 (only 0, 3, 6, 9 appear), so 1 never shows up and 9 has no inverse mod 12. Rows with an inverse: 1, 5, 7, 11, which is 4 of 12.

$9 + 7 = 16 = 1 \cdot 12 + 4$, so $9 + 7 \equiv 4 \pmod{12}$. Reducing first gives the same answer: $9 + 7 = 16 \equiv 4$.

Number bases and fast powers

CONCEPT

DEFINITION

In base $b \geq 2$ the digits $d_k \ \ldots \ d_1 \, d_0$, each with $0 \leq d_i < b$, stand for $n = d_k \, b^k + \cdots + d_1 \, b + d_0$. So $(1011)_2 = 8 + 2 + 1 = 11$ and $(2F)_{16} = 2 \cdot 16 + 15 = 47$, with A to F standing for 10 to 15.

Decimal to base b: divide by b again and again, which is the division algorithm on repeat. The remainders are the digits, last one first. Try it:

Convert `156` to base **2** | 3 | 8 | 16

n ÷ 2	156	78	39	19	9	4	2	1
remainder	0	0	1	1	1	0	0	1

Reading the remainders right to left: $156 = (10011100)_2$. Check: $1 \cdot 2^7 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 = 156$.

Binary to hex or octal: group the bits in fours (hex) or threes (octal) from the right. $(1001\,1100)_2 = (9C)_{16}$, because $16 = 2^4$.

FAST MODULAR EXPONENTIATION

To find $a^e \mod n$, square repeatedly to get $a, a^2, a^4, a^8, \ldots$, reducing mod n each time. Then multiply together the powers that match the 1-bits of e in binary. That takes about $2 \log_2 e$ multiplications instead of $e - 1$.

WORKED EXAMPLE

$3^{13} \mod 7$, with $13 = (1101)_2 = 8 + 4 + 1$:

power	3^1	3^2	3^4	3^8
mod 7	3	$9 \equiv 2$	$2^2 = 4$	$4^2 = 16 \equiv 2$
bit of 13	1	0	1	1

$3^{13} = 3^8 \cdot 3^4 \cdot 3^1 \equiv 2 \cdot 4 \cdot 3 = 24 \equiv 3 \pmod 7$.

DESIGN TIP

Python's `pow(a, e, n)` uses this method. Writing `a ** e % n` builds the full power first. For 2048-bit RSA numbers that power would have more digits than there are atoms in the universe.

Lab: toy RSA, every step explained

INTERACTIVE LAB

RSA is a public-key cryptography system: anyone can encrypt with the public key (n, e), and only the holder of the private key d can decrypt. Change the primes, the exponent or the message. Click a row of the table to see that letter encrypted by repeated squaring.

1 · Choose two different primes

p = 61 q = 53

$n = pq = 61 \cdot 53 = 3233$ is public. $\varphi(n) = (p-1)(q-1) = 60 \cdot 52 = 3120$ stays secret: computing it requires knowing p and q.

2 · Choose the public exponent e

e = 17

$\gcd(17, 3120) = 1$, so e is valid: it has an inverse modulo 3120.

3 · Compute the private key d

$d = e^{-1} \bmod \varphi(n) = 2753$, found with the extended Euclidean algorithm. Check: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$.

WHY DECRYPTION WORKS

$ed = 1 + 15\,\varphi(n)$. Euler's theorem says $m^{\varphi(n)} \equiv 1 \pmod{n}$ when $\gcd(m, n) = 1$, so $c^d \equiv m^{ed} = m \cdot (m^{\varphi(n)})^{15} \equiv m$. Public key $(3233, 17)$; private key 2753.

4 · Encrypt with $c = m^e \bmod n$, decrypt with $m = c^d \bmod n$

Message HELLO A = 1, B = 2, ..., Z = 26

plain	m	$c = m^e \bmod n$	$c^d \bmod n$	back
H	8	**2041**	8	H
E	5	**3086**	5	E
L	12	**1730**	12	L
L	12	**1730**	12	L
O	15	**3031**	15	O

Equal letters give equal ciphertexts. Textbook RSA is deterministic, so an eavesdropper can count letter frequencies. Real systems add random padding (OAEP) and use a 2048-bit n.

Encrypting H: 8^17 mod 3233 by repeated squaring

$e = 17 = (10001)_2$. Square, reduce mod 3233, and keep the columns whose bit is 1.

power of 8	1	2	4	8	16
value mod n	**8**	64	863	1179	**3084**
bit of e	**1**	0	0	0	**1**

$c \equiv 8 \cdot 3084 \equiv 2041 \pmod{3233}$. That took 4 squarings and 1 extra multiplication, instead of 16 multiplications.

Code: fast powers and a toy RSA

CODE WALKTHROUGH

Fast modular exponentiation written from scratch, then used to generate an RSA key, encrypt a number and decrypt it again. **Trace** shows `exp` losing one binary digit per loop.

toy_rsa.py

Read | Edit & run | Trace

```
1  def power_mod(base, exp, n):
2      result = 1
3      base = base % n
4      while exp > 0:
5          if exp % 2 == 1:
6              result = result * base % n
7          base = base * base % n
8          exp = exp // 2
9      return result
10
11 p, q, e = 61, 53, 17
12 n, phi = p * q, (p - 1) * (q - 1)
13 d = pow(e, -1, phi)
14 c = power_mod(65, e, n)
15 print("public key:", (n, e), "private key:", d)
16 print("cipher:", c, "decrypted:", power_mod(c, d, n))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
public key: (3233, 17) private key: 2753
cipher: 2790 decrypted: 65
```

LINE 1 OF 16

Show all notes in code

```
def power_mod(base, exp, n):
```

Defines fast modular exponentiation (repeated squaring) from scratch, so you can see what Python's built-in `pow(base, exp, n)` does inside. It returns $\text{base}^{\text{exp}} \bmod n$.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Implements repeated squaring as power_mod, builds the classic RSA key p = 61, q = 53, e = 17, and encrypts and decrypts the message 65.

Code: fast powers and a toy RSA: every line explained

1	`def power_mod(base, exp, n):`	Defines fast modular exponentiation (repeated squaring) from scratch, so you can see what Python's built-in `pow(base, exp, n)` does inside. It returns $\text{base}^{\text{exp}} \bmod n$.
2	`    result = 1`	The running product starts at 1, the "empty product". Each 1-bit of `exp` will multiply one power of the base into it.
3	`    base = base % n`	Reduces the base first. The multiplication rule for congruences allows it, and it keeps every number below n.
4	`    while exp > 0:`	Processes `exp` one binary digit at a time, lowest bit first. The loop runs about $\log_2(\text{exp})$ times: 5 times for 17, which is $(10001)_2$, and 12 times for 2753.
5	`        if exp % 2 == 1:`	`exp % 2` is the lowest bit of `exp`. When it is 1, the current power of the base belongs in the answer.
6	`            result = result * base % n`	Multiplies that power into the result and reduces mod n at once, so `result` never grows past n^2. Python's `*` and `%` have equal precedence and run left to right, so this means `(result * base) % n`.
7	`        base = base * base % n`	Squares the base: it goes $a \to a^2 \to a^4 \to a^8 \to \cdots$, always reduced mod n. Squaring is what turns $e - 1$ multiplications into about $2\log_2 e$.
8	`        exp = exp // 2`	Drops the lowest bit of `exp` by integer division: $17 \to 8 \to 4 \to 2 \to 1 \to 0$. When it reaches 0, every bit has been used and the loop ends.
9	`    return result`	Returns $\text{base}^{\text{exp}} \bmod n$, the product of the powers whose bit was 1.
11	`p, q, e = 61, 53, 17`	Key generation, steps 1 and 2: two primes and a public exponent with $\gcd(e, (p-1)(q-1)) = 1$. Real keys use random primes with hundreds of digits each.
12	`n, phi = p * q, (p - 1) * (q - 1)`	$n = 3233$ is published as part of the public key. $\varphi = 60 \cdot 52 = 3120$ stays secret, because computing it requires knowing p and q.
13	`d = pow(e, -1, phi)`	The private key: the inverse of e modulo φ. The three-argument `pow` with exponent -1 runs the extended Euclidean algorithm (Python 3.8 or later). $d = 2753$, and $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$.
14	`c = power_mod(65, e, n)`	Encrypts the message $m = 65$ (the letter A in ASCII) with the public key: $c = 65^{17} \bmod 3233 = 2790$. Anyone who knows (n, e) can do this.
15	`print("public key:", (n, e), "private key:", d)`	Output: `public key: (3233, 17) private key: 2753`. Python prints the tuple with its parentheses.
16	`print("cipher:", c, "decrypted:", power_mod(c, d, n))`	Decrypts with the private key: $2790^{2753} \bmod 3233 = 65$, the original message. Output: `cipher: 2790 decrypted: 65`. It works because $ed \equiv 1 \pmod{\varphi(n)}$.

Mod at work: hashing, check digits, random numbers

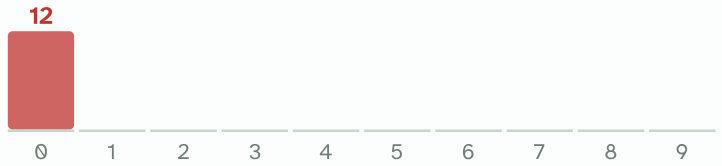
CONCEPT

Hashing

A hash function sends each key to one of m buckets. The simplest is $h(k) = k \bmod m$. With $m = 11$, the IDs 25, 36 and 47 all land in bucket 3. That is a **collision**, and it happens because they differ by multiples of 11.

Real keys often share a pattern. Below, every key is a multiple of 10. A prime m shares no factor with the pattern, so the keys spread out. Python's `dict` and `set` are hash tables.

keys 10, 20, ..., 120, m = 10 | 12 | 15 | 11 | 13



gcd(10, 10) = 10: the keys use only 1 of the 10 buckets, so 11 keys collide.

Check digits

The Luhn algorithm guards card numbers. Starting from the right, double every second digit, subtract 9 from any result above 9, and add everything up. A valid number has a sum $\equiv 0 \pmod{10}$. It catches every single mistyped digit.

79927398713

7	9	9	2	7	3	9	8	7	1	3
7	9	9	4	7	6	9	7	7	2	3

Sum 70: a multiple of 10, so the number passes. Change one digit to see it flip.

Pseudo-random numbers

A linear congruential generator repeats $x_{k+1} = (ax_k + c) \bmod m$. With $a = 5$, $c = 3$, $m = 16$ and seed $x_0 = 7$ it produces 7, 6, 1, 8, 11, 10, 5, 12, 15, 14, ... The numbers look random, yet the seed fixes all of them, and they repeat after at most m steps.

a=5 c=3 m=16 | a=3 c=0 m=16 | seed 7

7 6 1 8 11 10 5 12 15 14 9 0 3 2 13 4 → 7 again

Period 16: every value 0 to 15 appears before the cycle repeats.

That makes them ideal for repeatable experiments and useless for passwords or keys.

WHERE THIS SHOWS UP IN AI

The **hashing trick** in machine learning sends every word to column $h(\text{word}) \bmod 2^{20}$ of a fixed-size feature vector, so the model needs no vocabulary list. A fixed seed (`random.seed(42)`) makes a training run repeatable. A checksum such as SHA-256 confirms that downloaded model weights arrived unchanged.

Design practice: integers in real software

DESIGN PRACTICE

Number theory turns up in money handling, indexing, hashing and security. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Use integers for money, counts and IDs**
02 Know how % treats negative numbers
03 Use pow(a, e, m) for modular powers
04 Never roll your own cryptography
05 Validate inputs before the math
06 Watch for overflow outside plain Python
07 Size hash tables with a prime
08 Seed random generators for repeatable experiments

Principle 1 of 8

Use integers for money, counts and IDs

Floats store binary fractions, and most decimal amounts such as 0.10 have no exact binary form. Sums drift: `0.1 + 0.2` is `0.30000000000000004`. Store money as a whole number of cents and format it only for display.

Identifiers such as card numbers are better kept as strings of digits. They can start with 0, and nobody does arithmetic on them.

✗ AVOID THIS

```
price = 0.10
total = price * 3          # 0.30000000000000004
if total == 0.30:
    ship()                 # never runs
```

✓ DO THIS

```
price_cents = 10
total_cents = price_cents * 3   # exactly 30
print(f"${total_cents // 100}.{total_cents % 100:02d}")
```

Design practice: integers in real software: all principles (1/2)

Use integers for money, counts and IDs

Floats store binary fractions, and most decimal amounts such as 0.10 have no exact binary form. Sums drift: `0.1 + 0.2` is `0.30000000000000004`. Store money as a whole number of cents and format it only for display.

Identifiers such as card numbers are better kept as strings of digits. They can start with 0, and nobody does arithmetic on them.

```
price_cents = 10
total_cents = price_cents * 3   # exactly 30
print(f"${total_cents // 100}.{total_cents % 100:02d}")
```

Know how % treats negative numbers

Python's `%` returns a result with the sign of the divisor, which matches the division algorithm: `-7 % 3 == 2` and `-7 // 3 == -3`. C, C++, Java, JavaScript and Go round toward zero, so there `-7 % 3` is `-1`.

When you port code, or compute a circular index such as "one slot before slot 0", force the result into the range 0 to n – 1.

```
prev = ((i - 1) % n + n) % n   # always 0..n-1
# in Python, (i - 1) % n is already 0..n-1
```

Use pow(a, e, m) for modular powers

`a ** e % m` computes the entire power and reduces only at the end. For a 2048-bit exponent no computer could store that power. The three-argument `pow` uses repeated squaring and reduces after every multiplication, so it finishes in microseconds. `pow(a, -1, m)` (Python 3.8 or later) returns a modular inverse.

```
x = pow(7, 1_000_000, 13)  # repeated squaring, instant
inv = pow(3, -1, 7)        # 5, the inverse of 3 mod 7
```

Never roll your own cryptography

Textbook RSA, the version in this module, is insecure in practice. It is deterministic, so equal messages give equal ciphertexts, and an attacker can transform a ciphertext without decrypting it. Production systems use vetted libraries with random padding (RSA-OAEP), constant-time arithmetic and keys of at least 2048 bits.

For tokens and passwords use the `secrets` module. The `random` module is predictable by design.

```
import secrets
token = secrets.token_urlsafe(32)  # unguessable
# encryption: a vetted library with OAEP padding
```

Validate inputs before the math

Some inputs have no answer. $\gcd(0, 0)$ is undefined because every integer divides 0. A modulus must be positive. An inverse exists only when $\gcd(a, n) = 1$. Check these cases and raise an error that names the problem, instead of returning a wrong value or failing deep inside a loop.

```
def inverse(a, n):
    if n <= 0:
        raise ValueError("modulus must be positive")
    if math.gcd(a, n) != 1:
        raise ValueError(f"{a} has no inverse mod {n}")
    return pow(a, -1, n)
```

Design practice: integers in real software: all principles (2/2)

Watch for overflow outside plain Python

Python integers grow as large as memory allows. NumPy arrays, C, Java and GPU code use fixed-width integers that silently wrap around: a 32-bit signed integer holds at most 2,147,483,647. Products overflow first, so reduce modulo m before multiplying, and choose a 64-bit type when values can grow.

```
a = np.int64(50_000)
print(a * a)                    # 2500000000
prod = (x % m) * (y % m) % m    # stays below m*m
```

Seed random generators for repeatable experiments

Pseudo-random generators are deterministic: the same seed always gives the same sequence, just like the generator on the previous slide. Seed them in experiments so a training run, a data split or a bug can be reproduced exactly, and record the seed next to the results.

```
import random
random.seed(42)
split = random.sample(range(1000), 200)  # same every run
```

Size hash tables with a prime

With `key % m`, keys that share a factor with m crowd into a few buckets. If every key is a multiple of 8 and $m = 1024$, only 128 of the 1,024 buckets are ever used. A prime m shares no factor with such patterns, so the keys spread over all the buckets.

```
buckets = 1021          # a prime close to 1024
slot = key % buckets    # patterns spread out evenly
```

Worked example: Euclid’s algorithm for gcd(252, 198)

WORKED EXAMPLE

PROBLEM

Use Euclid's algorithm to find $\gcd(252, 198)$, showing every division.

$$\gcd(252,\ 198) = 18$$

a	q	b	r	division
252	1	198	54	252 = 1 · 198 + 54
198	3	54	36	198 = 3 · 54 + 36
54	1	36	18	54 = 1 · 36 + 18
36	2	18	0	36 = 2 · 18 + 0

gcd(252, 198) = 18 · check: 18 · 14 = 252 and 18 · 11 = 198, with gcd(14, 11) = 1

1. The plan: replace the pair (a, b) by (b, the remainder when a is divided by b). The gcd survives every replacement, because any common divisor of a and b also divides that remainder — and the numbers shrink each time, so the process must stop. Here the starting pair is (252, 198).
2. 252 = 1 · 198 + 54. One copy of 198 fits into 252 and 54 is left over, so gcd(252, 198) = gcd(198, 54): the same question about smaller numbers. Each step keeps only the remainder, which is why the algorithm is fast.
3. 198 = 3 · 54 + 36. 3 copies of 54 fit into 198 and 36 is left over, so gcd(198, 54) = gcd(54, 36): the same question about smaller numbers. Each step keeps only the remainder, which is why the algorithm is fast.
4. 54 = 1 · 36 + 18. One copy of 36 fits into 54 and 18 is left over, so gcd(54, 36) = gcd(36, 18): the same question about smaller numbers. Each step keeps only the remainder, which is why the algorithm is fast.
5. 36 = 2 · 18 + 0. This time 18 fits exactly and nothing is left over. A remainder of 0 is the stop signal: dividing further is impossible, and the last nonzero remainder is the answer.
6. **Answer: gcd(252, 198) = 18. Check: 252 = 18 · 14 and 198 = 18 · 11, so 18 divides both numbers — and 14 and 11 share no factor bigger than 1, so no number larger than 18 can divide both. That confirms 18 is the greatest common divisor.**

Worked example: the inverse of 7 modulo 26

WORKED EXAMPLE

PROBLEM

Find the modular inverse of 7 modulo 26 — the number x with $7x \equiv 1 \pmod{26}$ — by running Euclid's algorithm forward and then back-substituting.

$$7x \equiv 1 \pmod{26}$$

Forward $26 = 3 \cdot 7 + 5$

Forward $7 = 1 \cdot 5 + 2$

Forward $5 = 2 \cdot 2 + 1$

Back $1 = 5 - 2 \cdot 2$

Back $1 = 5 - 2(7 - 5) = 3 \cdot 5 - 2 \cdot 7$

Back $1 = 3(26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7$

Read off $-11 \cdot 7 \equiv 1 \pmod{26}$, and $-11 + 26 = 15$

Modulo 26, the term $3 \cdot 26$ counts as 0, so only $-11 \cdot 7$ remains.

$7^{-1} \equiv 15 \pmod{26}$ · check: $7 \cdot 15 = 105 = 4 \cdot 26 + 1$, so $105 \equiv 1 \pmod{26}$

1. Setup: an inverse of 7 modulo 26 is a number x with 7x ≡ 1 (mod 26), and it can exist only because gcd(7, 26) = 1. The plan is to run Euclid forward to remainder 1, then work backward to write 1 as a combination of 26 and 7.
2. Forward pass: 26 = 3 · 7 + 5, then 7 = 1 · 5 + 2, then 5 = 2 · 2 + 1, so the 3 division lines are now on the page. We run Euclid only until remainder 1 appears, because 1 is exactly the number the backward pass has to express.
3. Back-substitution starts by rearranging the last division, 5 = 2 · 2 + 1, into 1 = 5 – 2 · 2. Every backward step replaces a remainder by the expression the forward pass gave for it, so the value stays equal to 1 throughout.
4. Replace the remainder 2 by 7 – 5, rearranging 7 = 1 · 5 + 2, giving 1 = 5 – 2(7 – 5) = 3 · 5 – 2 · 7 after the terms are collected. The highlighted piece is the substitution itself, and now only the remainders 5 and 7 remain in the expression.
5. Replace the remainder 5 by 26 – 3 · 7, rearranging 26 = 3 · 7 + 5, giving 1 = 3(26 – 3 · 7) – 2 · 7 = 3 · 26 – 11 · 7 after the terms are collected. The highlighted piece is the substitution itself, and now only the remainders 26 and 7 remain in the expression — the original numbers 26 and 7 themselves.
6. Read off the inverse: modulo 26 the term 3 · 26 is 0, so –11 · 7 ≡ 1 (mod 26) and the coefficient of 7 is the inverse, still in negative form. Adding 26 turns –11 into the standard residue 15, because residues are named by a number from 0 to 25.
7. **Answer: $7^{-1} \equiv 15 \pmod{26}$. Check: $7 \cdot 15 = 105 = 4 \cdot 26 + 1$, so $105 \equiv 1 \pmod{26}$ and the inverse works, with no smaller positive candidate left to test because an inverse modulo n is unique when it exists.**

Worked example: 5^11 mod 13 by repeated squaring

WORKED EXAMPLE

PROBLEM

Compute $5^{11} \mod 13$ by repeated squaring: expand 11 in binary, build the squaring chain $5^1, 5^2, 5^4, 5^8 \mod 13$, and multiply in only the powers the 1-bits select.

5^11 mod 13

Done: 5^11 ≡ 8 (mod 13)

power of 5	1	2	4	8
value mod 13	5	12	1	1
bit of 11	1	1	0	1

Product of the kept powers: 5 · 12 · 1 ≡ 8 (mod 13)

5^11 ≡ 8 (mod 13) · check: multiplying 5 by itself 11 times, reducing mod 13 each time, also ends at 8 ✓

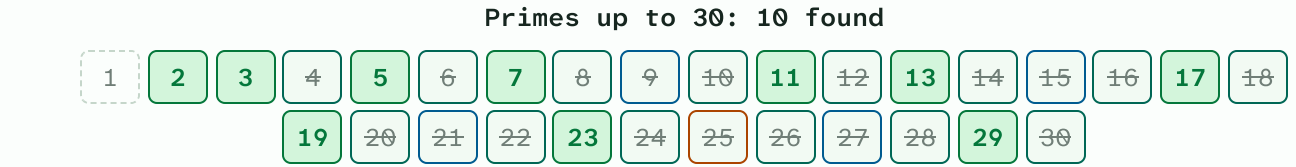
1. Setup: compute 5^11 mod 13 without ever forming the enormous number 5^11 itself. Write the exponent in binary, 11 = $(1011)_2$ = 8 + 2 + 1, square the base repeatedly to get the powers 5^1, 5^2, 5^4, ... mod 13, and multiply in only the powers that match the 1-bits.
2. First power: 5^1 mod 13 = 5, just the base reduced mod 13. This value anchors the whole chain, because every later power is the square of the one before it, reduced at once, so no entry in the table ever reaches 13^2.
3. Square the previous power: 5^2 = $(5^1)^2 \equiv 5^2 = 25 \equiv 12 \pmod{13}$. Reducing right after squaring is what keeps the method fast: the arithmetic stays tiny no matter how large the exponent grows.
4. Square the previous power: 5^4 = $(5^2)^2 \equiv 12^2 = 144 \equiv 1 \pmod{13}$. Reducing right after squaring is what keeps the method fast: the arithmetic stays tiny no matter how large the exponent grows.
5. Square the previous power: 5^8 = $(5^4)^2 \equiv 1^2 = 1 \equiv 1 \pmod{13}$. Reducing right after squaring is what keeps the method fast: the arithmetic stays tiny no matter how large the exponent grows.
6. Read the bits of 11 = $(1011)_2$ from the right: the 1-bits sit at positions 0, 1, 3, so the powers that belong in the answer are 5^1 ≡ 5, 5^2 ≡ 12, 5^8 ≡ 1, while the power at bit position 2 (bit 0) are skipped. Only 3 of the 4 squares survive into the product.
7. Multiply the kept powers in, reducing after each step: 1 · 5 = 5, then 5 · 12 = 60 ≡ 8, then 8 · 1 = 8 ≡ 8. Reducing at every step gives the same final residue as reducing once at the end — the multiplication rule for congruences guarantees it — but the intermediate numbers stay below 13 · 13 the whole way.
8. **Answer: 5^11 ≡ 8 (mod 13). Check: multiplying 5 by itself 11 times, reducing mod 13 after every single multiplication, also ends at 8 — repeated squaring reached the same residue with just 3 squarings and 2 extra multiplications instead of 10 multiplications.**

Worked example: the sieve up to 30

WORKED EXAMPLE

PROBLEM

Use the sieve of Eratosthenes to find every prime up to 30: start with all numbers standing, then for each prime in turn cross out its multiples, and list the primes that remain standing.



Green cells are the primes: every composite was crossed by its smallest prime factor.

Primes up to 30: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 · check: trial division of every number up to its square root finds the same 10 primes ✓

1. Setup: find every prime up to 30 with the sieve, which needs no division at all. Every number from 2 to 30 starts out standing; each pass takes the smallest number still standing — which must be prime — and crosses out its multiples, and whatever is left standing at the end is exactly the list of primes.
2. Pass p = 2: 2 is the smallest number still standing, so it is prime, because a smaller prime factor would already have crossed it out. Cross out its multiples starting at $2^2 = 4$, since every smaller multiple of 2 is either 2 itself or has a smaller prime factor and fell earlier: this pass newly crosses 4, 6, 8, 10, 12, … (14 in all). Now 15 numbers are still standing.
3. Pass p = 3: 3 is the smallest number still standing, so it is prime, because a smaller prime factor would already have crossed it out. Cross out its multiples starting at $3^2 = 9$, since every smaller multiple of 3 is either 3 itself or has a smaller prime factor and fell earlier: this pass newly crosses 9, 15, 21, 27. Now 11 numbers are still standing.
4. Pass p = 5: 5 is the smallest number still standing, so it is prime, because a smaller prime factor would already have crossed it out. Cross out its multiples starting at $5^2 = 25$, since every smaller multiple of 5 is either 5 itself or has a smaller prime factor and fell earlier: this pass newly crosses 25. Now 10 numbers are still standing.
5. Stop: the next candidate would be 7, and $7^2 = 49 > 30$, so every composite up to 30 already lost a prime factor at or below $\sqrt{30} \approx 5.5$ in an earlier pass. Everything still standing is therefore prime: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29.
6. **Answer: the primes up to 30 are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29 — 10 primes in all. Check: trial division, a different method, tests each number from 2 to 30 against divisors up to its own square root and finds exactly these 10 primes, and every composite in the range was crossed by its smallest prime factor during the passes.**

Practice: integers and number theory

PRACTICE

Numbers can be typed plainly (`-8`). Base answers accept upper or lower case, with or without a `0x` or `0b` prefix.

DIVISION ALGORITHM

Apply the division algorithm to $a = -52$ and $d = 8$. Find the integers q and r with $a = dq + r$ and $0 \le r < 8$. Type them as `q, r`.

q, r

Check

Hint: for negative a, q rounds down (toward minus infinity), so that r stays non-negative.

Show worked solution

New problem ↻

$-52/8 \approx -6.50$, so $q = \lfloor -52/8 \rfloor = -7$. Then $r = a - dq = -52 - 8 \cdot (-7) = 4$. Check: $8 \cdot (-7) + 4 = -52$ and $0 \le 4 < 8$. Answer: **-7, 4**. Python agrees: `divmod(-52, 8)` is `(-7, 4)`. Rounding toward zero would give $q = -6$ and $r = -4$, a negative remainder, which the theorem does not allow.

PROBLEM TYPE

- Division algorithm
- GCD and LCM
- Modular arithmetic
- Number bases

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: integers and number theory: sample problems with solutions

DIVISION ALGORITHM

Apply the division algorithm to $a = -58$ and $d = 10$. Find the integers q and r with $a = dq + r$ and $0 \le r < 10$. Type them as `q, r`.

$-58/10 \approx -5.80$, so $q = \lfloor -58/10 \rfloor = -6$.

Then $r = a - dq = -58 - 10 \cdot (-6) = 2$.

Check: $10 \cdot (-6) + 2 = -58$ and $0 \le 2 < 10$.

Answer: **-6, 2**. Python agrees: `divmod(-58, 10)` is `(-6, 2)`. Rounding toward zero would give $q = -5$ and $r = -8$, a negative remainder, which the theorem does not allow.

GCD AND LCM

Use Euclid's algorithm to find $\gcd(42, 8)$.

Euclid's steps:

$42 = 5 \cdot 8 + 2$

$8 = 4 \cdot 2 + 0$

The last nonzero remainder is $\gcd(42, 8) = 2$.

MODULAR ARITHMETIC

Compute $5^9 \bmod 23$.

$9 = (1001)_2 = 8 + 1$. Squaring and reducing mod 23: $5^1 \equiv 5$, $5^2 \equiv 2$, $5^4 \equiv 4$, $5^8 \equiv 16$.

Multiply the powers whose bit is 1:

$5 \cdot 16 \equiv 11 \pmod{23}$.

NUMBER BASES

Convert $(376)_{10}$ to base 8.

Divide by 8 repeatedly:

$376 = 8 \cdot 47 + 0$

$47 = 8 \cdot 5 + 7$

$5 = 8 \cdot 0 + 5$

Reading the remainders from the last to the first gives $(570)_8$.

Check yourself: integers and number theory

CHECK YOURSELF

QUESTION 1 OF 10

The division algorithm with $a = -17$ and $d = 5$ gives which q and r?

A $q = -3,\ r = -2$

B $q = 3,\ r = -2$

C $q = -4,\ r = 3$

D $q = -3,\ r = 2$

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: integers and number theory: answers explained (1/2)

1. The division algorithm with $a = -17$ and $d = 5$ gives which q and r?

Answer C: $q = -4$, $r = 3$

$-17 = 5 \cdot (-4) + 3$ with $0 \le 3 < 5$. The pair $q = -3$, $r = -2$ is what C or Java produce by rounding toward zero, but a remainder may not be negative. Python's `divmod(-17, 5)` returns `(-4, 3)`.

2. To decide whether 221 is prime by trial division, what is the largest prime you must try?

Answer C: 13

$\sqrt{221} \approx 14.9$, so only primes up to 14 need testing, and the largest is 13. It succeeds: $221 = 13 \cdot 17$.

3. $\gcd(a, b) = 6$ and $a \cdot b = 432$. What is $\operatorname{lcm}(a, b)$?

Answer A: 72

$\gcd \cdot \operatorname{lcm} = ab$, so $\operatorname{lcm} = 432/6 = 72$. For example $a = 18$, $b = 24$.

4. One step of Euclid's algorithm replaces $\gcd(91, 35)$ with which pair?

Answer B: $\gcd(35, 21)$

$91 = 2 \cdot 35 + 21$, so $\gcd(91, 35) = \gcd(35, 21)$. Continuing: $\gcd(21, 14) = \gcd(14, 7) = \gcd(7, 0) = 7$.

5. Which number has an inverse modulo 12?

Answer B: 7

An inverse exists exactly when $\gcd(a, 12) = 1$. Only 7 qualifies, and $7 \cdot 7 = 49 = 4 \cdot 12 + 1$, so 7 is its own inverse. The others share the factor 2 with 12.

6. What is $(97 \cdot 46 + 13) \bmod 9$?

Answer A: 2

Reduce first: $97 \equiv 7$, $46 \equiv 1$ and $13 \equiv 4 \pmod 9$. Then $7 \cdot 1 + 4 = 11 \equiv 2$. The full value 4475 leaves the same remainder.

7. What is $(2F)_{16}$ in decimal?

Answer C: 47

F stands for 15, so $2 \cdot 16 + 15 = 47$. In binary that is $0010\,1111$, one hex digit per four bits.

8. How many multiplications does repeated squaring need to compute $a^{64} \bmod n$?

Answer C: 6

$64 = 2^6$, so six squarings do it: $a \to a^2 \to a^4 \to a^8 \to a^{16} \to a^{32} \to a^{64}$. Multiplying a by itself 63 times is the slow way.

9. A toy RSA key uses $p = 5$, $q = 11$ and $e = 3$. What is the private key d?

Answer D: 27

$\varphi(n) = 4 \cdot 10 = 40$, and d must satisfy $3d \equiv 1 \pmod{40}$. $3 \cdot 27 = 81 = 2 \cdot 40 + 1$, so $d = 27$.

Check yourself: integers and number theory: answers explained (2/2)

10. Every key is a multiple of 10. Which table size spreads them best with $h(k) = k \bmod m$?

Answer A: 97

With $m = 100$ the keys land only in buckets 0, 10, 20, ..., 90: ten of the hundred. The prime 97 shares no factor with 10, so the keys spread over every bucket.

The product rule and the sum rule

CONCEPT

PRODUCT RULE

If a task is a sequence of steps, step 1 can be done in n_1 ways, step 2 in n_2 ways, and so on, and each count does not depend on the earlier choices, then the whole task can be done in $n_1 \cdot n_2 \cdots n_k$ ways. This is the product rule.

WORKED EXAMPLE

A license plate has 3 letters followed by 4 digits: $26^3 \cdot 10^4 = 175{,}760{,}000$ plates. If the letters may not repeat, the letter choices shrink step by step: $26 \cdot 25 \cdot 24 \cdot 10^4 = 156{,}000{,}000$.

SUM RULE

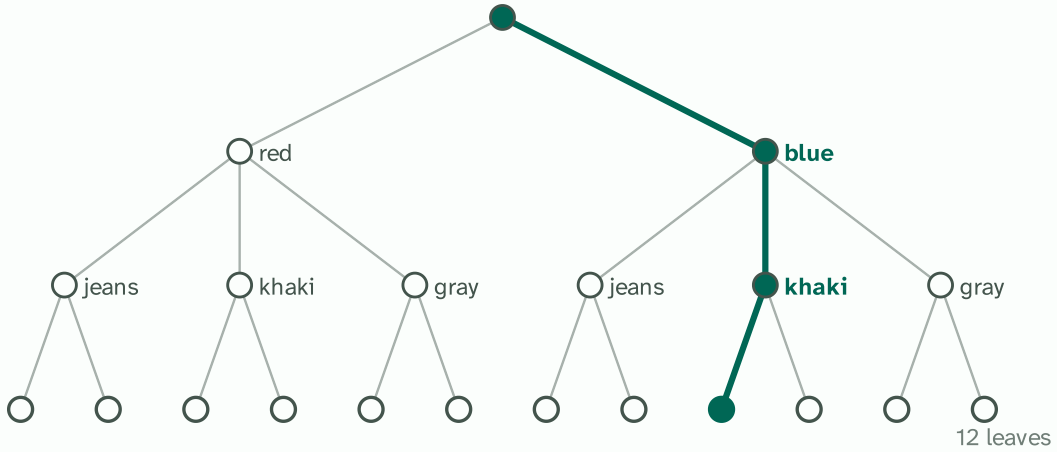
If a choice comes from one of several groups that do not overlap, with $n_1, n_2, \ldots$ options, then there are $n_1 + n_2 + \cdots$ choices in all. This is the sum rule.

WORKED EXAMPLE

Bit strings of length at most 3: split them by length (the groups cannot overlap), then count each length with the product rule: $2 + 4 + 8 = 14$.

TREE OF CHOICES · SET THE OPTIONS PER STEP, CLICK A LEAF

Shirt	1	**2**	3	4	red, blue
Pants	1	2	**3**	4	jeans, khaki, gray
Shoes	1	**2**	3	4	sneakers, boots



Each step multiplies the branches: 2 → 2 × 3 = 6 → 6 × 2 = **12 outfits**. Highlighted leaf: blue shirt, khaki pants, sneakers. Every leaf is one complete path of choices.

WHERE THIS SHOWS UP IN AI

Combinatorial explosion: a game with about 30 legal moves per turn has $30^{10} \approx 5.9 \times 10^{14}$ ten-move sequences. A hyperparameter grid of 5 learning rates, 4 batch sizes and 6 depths already needs 120 training runs. AI search and planning methods exist because the product rule makes trying everything hopeless.

Inclusion–exclusion: counting overlapping groups

CONCEPT

INCLUSION-EXCLUSION

Two sets: $|A \cup B| = |A| + |B| - |A \cap B|$. Three sets:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

Why the signs alternate: follow one element and count how often the formula counts it. Every element of the union ends up counted exactly once.

An element that lies in	singles	pairs	triple	total
exactly 1 set	+1	0	0	**1**
exactly 2 sets	+2	−1	0	**1**
all 3 sets	+3	−3	+1	**1**

COMMON PITFALL

The sum rule needs groups that do not overlap. When they do overlap, adding the sizes counts shared elements more than once. Inclusion–exclusion is the correction.

WORKED EXAMPLE

Bit strings of length 8 that start with 1 or end with 00: start with 1 gives $2^7 = 128$, end with 00 gives $2^6 = 64$, and both gives $2^5 = 32$. So $128 + 64 - 32 = 160$.

CALCULATOR · HOW MANY INTEGERS ARE DIVISIBLE BY a, b OR c?

Integers 1 to 100 divisible by 2 or 3 or 5

+	|A|	⌊100/2⌋	50
+	|B|	⌊100/3⌋	33
+	|C|	⌊100/5⌋	20
−	|A ∩ B|	⌊100/lcm(2, 3)⌋ = ⌊100/6⌋	16
−	|A ∩ C|	⌊100/lcm(2, 5)⌋ = ⌊100/10⌋	10
−	|B ∩ C|	⌊100/lcm(3, 5)⌋ = ⌊100/15⌋	6
+	|A ∩ B ∩ C|	⌊100/30⌋	3
=	**|A ∪ B ∪ C|**		**74**

Brute-force check, testing all 100 numbers: 74 ✓. Numbers divisible by none of them: 26. A number is divisible by both a and b exactly when lcm(a, b) divides it (Module 6).

Permutations and combinations

CONCEPT

DEFINITION

An r-permutation of n distinct objects is an *ordered* arrangement of r of them. There are n choices for the first position, $n-1$ for the second, and so on, so by the product rule

$$P(n,r) = n(n-1)\cdots(n-r+1) = \frac{n!}{(n-r)!}.$$

Here $n! = n(n-1)\cdots 2\cdot 1$ is n factorial, and $0! = 1$.

WORKED EXAMPLE

Gold, silver and bronze among 8 runners: $P(8,3) = 8\cdot 7\cdot 6 = 336$. Every order of 5 books on a shelf: $5! = 120$.

DEFINITION

An r-combination is an *unordered* selection of r of the n objects. Their number is the binomial coefficient

$$C(n,r) = \binom{n}{r} = \frac{n!}{r!\,(n-r)!}.$$

WHERE THIS SHOWS UP IN AI

Picking 10 of 100 input features for a model gives $C(100,10) \approx 1.7\times 10^{13}$ feature subsets, far too many to test one by one.

THE DIVIDE-BY-R! IDEA

Each unordered group of r objects appears $r!$ times in the list of ordered arrangements, once for each of its orders. So $C(n,r) = P(n,r)/r!$.

choose 2 of {A, B, C}	ordered: $P(3,2) = 6$	unordered: $6/2! = 3$
group 1	AB, BA	{A, B}
group 2	AC, CA	{A, C}
group 3	BC, CB	{B, C}

WORKED EXAMPLE

A 3-person committee from 8 people: $C(8,3) = 336/3! = 56$. A president, treasurer and secretary from 8 people: 336, because the roles make order matter.

DESIGN TIP

Ask one question first: would swapping two chosen items give a different outcome? If yes, order matters, so count permutations. If no, count combinations. Also $C(n,r) = C(n,n-r)$: choosing who is in is the same as choosing who is out.

Lab: four ways to choose

INTERACTIVE LAB

Draw r letters from the first n letters of the alphabet. The two switches give the four basic counting models. Every arrangement is listed in lexicographic order (the order Python's `itertools` uses), next to the formula that counts them.

n = 2 | 3 | 4 | **5** | 6 r = 1 | 2 | **3** | 4

Order matters: no | Repetition allowed: no

	NO REPETITION	REPETITION
ORDER MATTERS	permutations $P(n,r) = \frac{n!}{(n-r)!}$	strings n^r
ORDER DOES NOT	**combinations** $C(n,r) = \frac{n!}{r!\,(n-r)!}$	multisets $C(n+r-1,\, r)$

$C(5,3) = \frac{P(5,3)}{3!} = \frac{60}{6} = 10$

Each group of 3 letters appears 6 times among the ordered lists, once per order (shown faded after each group when r ≤ 3). Dividing by 3! leaves one entry per group.

10 groups of 3 from {A, B, C, D, E}.

{A,B,C} ABC ACB BAC BCA CAB CBA	**{A,B,D}** ABD ADB BAD BDA DAB DBA	**{A,B,E}** ABE AEB BAE BEA EAB EBA
{A,C,D} ACD ADC CAD CDA DAC DCA	**{A,C,E}** ACE AEC CAE CEA EAC ECA	**{A,D,E}** ADE AED DAE DEA EAD EDA
{B,C,D} BCD BDC CBD CDB DBC DCB	**{B,C,E}** BCE BEC CBE CEB EBC ECB	**{B,D,E}** BDE BED DBE DEB EBD EDB
{C,D,E} CDE CED DCE DEC ECD EDC		

HOW A PROGRAM GENERATES THIS LIST

Find the rightmost letter that can still increase (it is not yet as large as its position allows). Increase it by one, and make every letter to its right the next letters in order.

ABC → ABD → ABE → ACD → ACE → …

Repetition: multisets and stars and bars

CONCEPT

ARRANGEMENTS WITH REPEATED LETTERS

If n objects include n_1 identical copies of one kind, n_2 of another, ..., n_k of the last kind, the number of different arrangements is

$$\frac{n!}{n_1!\,n_2!\cdots n_k!}.$$

WORKED EXAMPLE

MISSISSIPPI has 11 letters: one M, four I, four S and two P.

$$\frac{11!}{1!\,4!\,4!\,2!} = 34{,}650$$

Why: tag the copies $I_1, I_2, \ldots$ so all 11 letters differ, which gives $11!$ arrangements. Erasing the tags merges every $4!\cdot 4!\cdot 2!$ of them into one word. Choosing positions gives the same count: $C(11,1)\,C(10,4)\,C(6,4)\,C(2,2) = 11\cdot 210\cdot 15\cdot 1$.

choose r from n	no repetition	repetition allowed
order matters	$P(n,r)$	n^r
order does not matter	$C(n,r)$	$C(n+r-1,\ r)$

COMBINATIONS WITH REPETITION

Choosing r items from n types, where types may repeat and order does not matter, can be done in $C(n+r-1,\ r)$ ways. Each choice is a multiset.

WORKED EXAMPLE

Buy 5 donuts from 3 flavors: glazed, chocolate, jelly. Write each order as 5 stars and 2 bars: $\star\ \star\ |\ \star\ |\ \star\ \star$ means 2 glazed, 1 chocolate and 2 jelly. Every order is one arrangement of 7 symbols, so there are $C(7,2) = 21$ orders. This stars and bars count also gives the number of solutions of $x_1 + x_2 + x_3 = 5$ in non-negative integers.

WHERE THIS SHOWS UP IN AI

A language model with a vocabulary of $V = 50{,}000$ tokens allows $V^3 = 1.25\times 10^{14}$ three-token sequences (order matters, repeats allowed). No text collection contains more than a tiny fraction of them, so models must generalize instead of storing a count for every n-gram.

Code: counting formulas, checked by brute force

CODE WALKTHROUGH

The formulas from scratch, Python's built-ins, and a brute-force count with `itertools`: three independent answers that must agree.

counting.py

Read | Edit & run | Trace

```python
from math import comb, factorial, perm
from itertools import combinations, permutations

def P(n, r):
    return factorial(n) // factorial(n - r)

ordered = list(permutations("ABCDE", 3))
groups = list(combinations("ABCDE", 3))
print(P(5, 3), perm(5, 3), len(ordered))
print(P(5, 3) // factorial(3), comb(5, 3), len(groups))
print(groups[:4])
word = factorial(11) // (factorial(4) ** 2 * factorial(2))
print("MISSISSIPPI:", word, "donuts:", comb(5 + 3 - 1, 5))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
60 60 60
10 10 10
[('A', 'B', 'C'), ('A', 'B', 'D'), ('A', 'B', 'E'), ('A', 'C', 'D')]
MISSISSIPPI: 34650 donuts: 21
```

LINE 1 OF 13

Show all notes in code

```python
from math import comb, factorial, perm
```

Imports three functions from the standard `math` module: `comb(n, r)` for $C(n, r)$, `perm(n, r)` for $P(n, r)$ (both Python 3.8 or later), and `factorial(n)` for $n!$. All three return exact integers.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Computes P(5, 3) and C(5, 3) three ways each, shows the first combinations in lexicographic order, and counts the arrangements of MISSISSIPPI and the donut orders.

Code: counting formulas, checked by brute force: every line explained

1	`from math import comb, factorial, perm`	Imports three functions from the standard `math` module: `comb(n, r)` for $C(n, r)$, `perm(n, r)` for $P(n, r)$ (both Python 3.8 or later), and `factorial(n)` for $n!$. All three return exact integers.
2	`from itertools import combinations, permutations`	Imports two generators from `itertools`. They do not count anything; they produce the actual arrangements one at a time, in lexicographic order.
4	`def P(n, r):`	Defines our own permutation count from the formula, so we can compare it with the built-in version.
5	`    return factorial(n) // factorial(n - r)`	The formula $P(n, r) = n!/(n-r)!$. The `//` operator is integer division, so the result stays an exact integer. For $P(5, 3)$: $120//2 = 60$.
7	`ordered = list(permutations("ABCDE", 3))`	Builds every ordered arrangement of 3 of the 5 letters: `('A', 'B', 'C')`, `('A', 'B', 'D')`, and so on. `list(...)` collects everything the generator produces. That is fine for 60 items; design principle 4 explains why you would not do it for millions.
8	`groups = list(combinations("ABCDE", 3))`	Builds every unordered group of 3. Each group appears once, with its letters in sorted order, so `('B', 'A', 'C')` never shows up.
9	`print(P(5, 3), perm(5, 3), len(ordered))`	Three independent answers to "how many ordered arrangements of 3 of the 5 letters?": our formula, the built-in, and a brute-force count of the list. Output: `60 60 60`.
10	`print(P(5, 3) // factorial(3), comb(5, 3), len(groups))`	The same check for unordered groups. The first value is the divide-by-$r!$ idea written out: $C(5, 3) = P(5, 3)/3! = 60//6 = 10$, because every group appears $3!$ times among the ordered arrangements. The division is always exact, so `//` loses nothing. Output: `10 10 10`.
11	`print(groups[:4])`	The slice `[:4]` shows the first four groups: ABC, ABD, ABE, ACD. That is lexicographic (dictionary) order. To get the next group, a program increases the rightmost letter that can still increase and resets the letters after it, the method the lab on slide 4 describes.
12	`word = factorial(11) // (factorial(4) ** 2 * factorial(2))`	Arrangements of a multiset: $11!/(4!\,4!\,2!)$. `factorial(4) ** 2` covers the four I's and the four S's; the single M contributes $1! = 1$. The result, 34650, is stored in `word`.
13	`print("MISSISSIPPI:", word, "donuts:", comb(5 + 3 - 1, 5))`	Prints the MISSISSIPPI count and the stars-and-bars count for 5 donuts from 3 flavors, $C(5 + 3 - 1, 5) = C(7, 5) = 21$, which equals $C(7, 2)$ by symmetry. Output: `MISSISSIPPI: 34650 donuts: 21`.

Pascal's triangle and Pascal's identity

CONCEPT

Arrange the binomial coefficients in rows: row n lists $\binom{n}{0}, \binom{n}{1}, \ \dots\ , \binom{n}{n}$. The result is Pascal's triangle. Row numbers are on the left.

0				1			
1			1		1		
2		1		2		1	
3	1		3		3		1
4		1		4		6		4		1
5	1		5		10		10		5		1
6		1		6		15		20		15		6		1

PASCAL'S IDENTITY

$\binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k}$ for $1 \le k \le n$. Each inner entry is the sum of the two entries above it: $10 = 4 + 6$ in row 5.

A COMBINATORIAL ARGUMENT

A club has $n + 1$ members, one of them named Ada. How many k-person committees are there? Directly, $\binom{n+1}{k}$. Now split the committees by Ada. Committees that include Ada choose $k - 1$ more people from the other n: $\binom{n}{k-1}$. Committees without Ada choose all k from the other n: $\binom{n}{k}$. The sum rule says the two counts agree, which proves the identity without any algebra. This style of proof is a combinatorial argument.

THEOREM / RULE

$\sum_{k=0}^{n} \binom{n}{k} = 2^n$. Count the subsets of an n-element set two ways: grouped by size, or by an in/out decision for each element. Row 5: $1 + 5 + 10 + 10 + 5 + 1 = 32$.

THEOREM / RULE

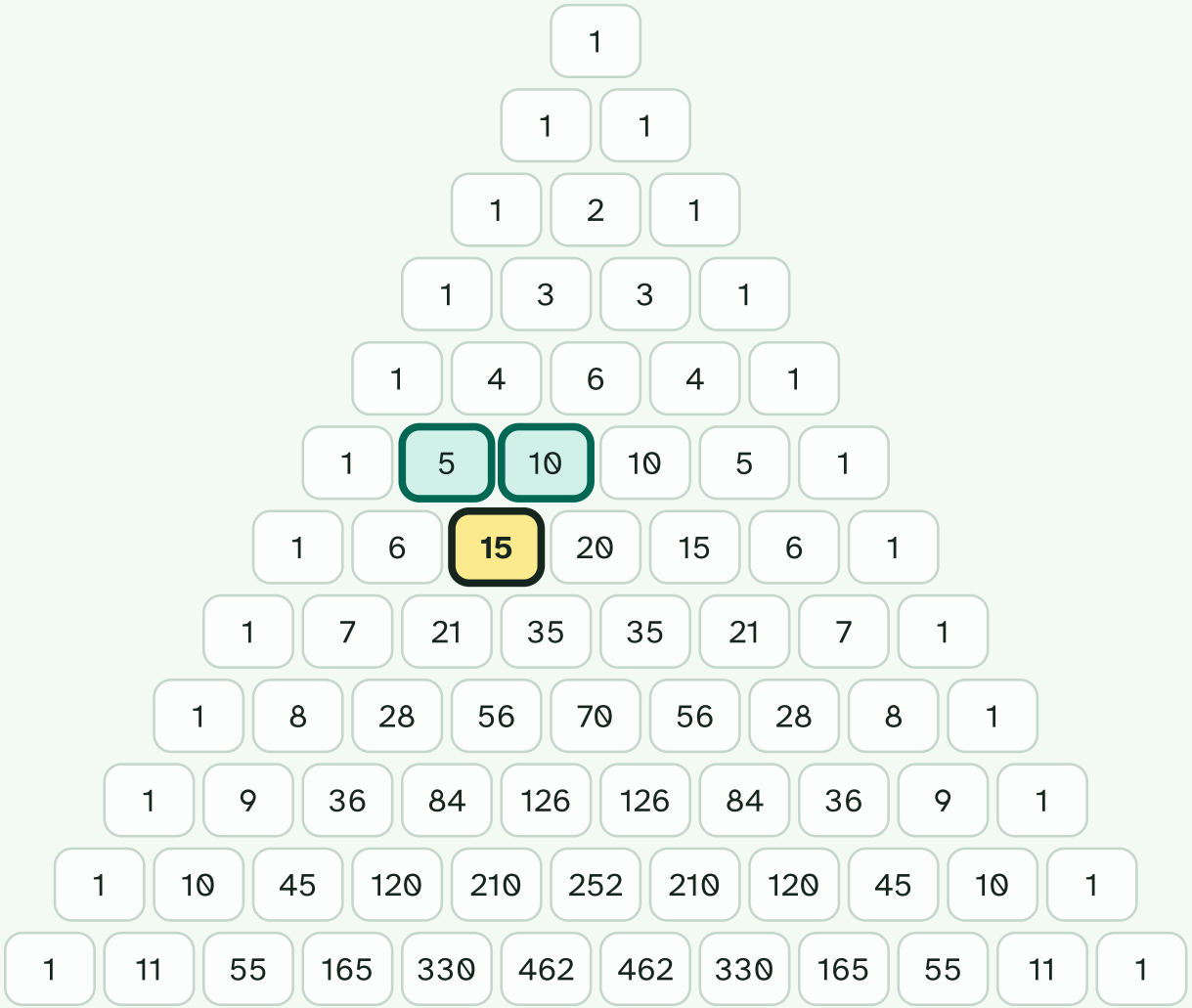
Symmetry: $\binom{n}{k} = \binom{n}{n-k}$. Choosing who is in is the same as choosing who is out, which is why every row reads the same in both directions.

Lab: Pascal's triangle explorer

INTERACTIVE LAB

Click any entry to see its two parents (Pascal's identity) and its row expanded as $(a+b)^n$. Color by remainder to reveal hidden structure: odd entries form the Sierpinski triangle.

Rows: 8 | 12 | 16 | 32 — Color: plain | odd / even | mod 3 | mod 5



ROW 6, POSITION 2

$$\binom{6}{2} = 15$$

Pascal's identity: $\binom{6}{2} = \binom{5}{1} + \binom{5}{2} = 5 + 10$. The two parents are outlined.

From the formula: $\frac{6!}{2!\,4!} = \frac{720}{2 \cdot 24} = 15$.

Meaning: 15 ways to choose 2 of 6 items. It is also the number of paths from the top to this entry that step down-left or down-right: each path makes 6 steps, and you choose which 2 of them go right.

THIS ROW AS A BINOMIAL EXPANSION (CLICK A TERM)

$$(a+b)^6 = a^6 + 6a^5b + 15a^4b^2 + 20a^3b^3 + 15a^2b^4 + 6ab^5 + b^6$$

Row 6 adds up to $2^6 = 64$.

The Binomial Theorem

CONCEPT

THE BINOMIAL THEOREM

For any numbers a, b and any integer $n \geq 0$,

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k} b^k .$$

Why it holds: $(a+b)^n$ is the product of n factors $(a+b)(a+b)\cdots(a+b)$. Multiplying out means picking a or b from each factor and multiplying the picks. A term $a^{n-k}b^k$ comes from picking b in exactly k of the n factors, and there are $\binom{n}{k}$ ways to choose which factors those are.

WORKED EXAMPLE

For $n = 3$ the 8 pick sequences are aaa, aab, aba, baa, abb, bab, bba, bbb. Grouping them by the number of b's gives

$$(a+b)^3 = a^3 + 3a^2 b + 3ab^2 + b^3 ,$$

and the coefficients 1, 3, 3, 1 are row 3 of Pascal's triangle.

WORKED EXAMPLE: ONE COEFFICIENT

Find the coefficient of x^3 in $(2x-3)^6$. Take $a = 2x$ and $b = -3$. The general term is $\binom{6}{k}(2x)^{6-k}(-3)^k$. For x^3 we need $6-k = 3$, so $k = 3$:

$$\binom{6}{3} 2^3 (-3)^3 = 20 \cdot 8 \cdot (-27) = -4320.$$

THEOREM / RULE

Plugging in numbers gives identities. With $a = b = 1$: $\sum_k \binom{n}{k} = 2^n$. With $a = 1,\ b = -1$: $\sum_k (-1)^k \binom{n}{k} = 0$ for $n \geq 1$, so every nonempty set has as many even-sized subsets as odd-sized ones.

WHERE THIS SHOWS UP IN AI

With $a = 1-p$ and $b = p$, each term $\binom{n}{k} p^k (1-p)^{n-k}$ is the probability of exactly k successes in n independent tries (Module 13). That is how you find the chance that a majority vote of 5 independent classifiers, each right 80% of the time, is right: about 94%.

Lab: binomial expansion calculator

INTERACTIVE LAB

Set a, b and n to expand $(ax + by)^n$, or switch off y for $(ax + b)^n$. Pick a power of x to watch its coefficient computed from the general term. Click a row of the table to pick it.

$$(2x - 3)^6$$

a 2

b -3

n 6

power of x 3

Second term: constant b (click to add y)

COEFFICIENT OF x^3, STEP BY STEP

1. General term: $T_k = \binom{6}{k}(2x)^{6-k}(-3)^k$.
2. The power of x is $6 - k$. Set $6 - k = 3$, so $k = 3$.
3. Substitute: $\binom{6}{3} \cdot 2^3 \cdot (-3)^3 = 20 \cdot 8 \cdot (-27)$.
4. Coefficient: $-4{,}320$.

The C(n,k) column is row 6 of Pascal's triangle: 1, 6, 15, 20, 15, 6, 1. Because b is negative, b^k is negative for odd k, so the signs alternate. Every term is one way of choosing x from n − k factors and the other part from the remaining k.

FULL EXPANSION

$$64x^6 - 576x^5 + 2{,}160x^4 - 4{,}320x^3 + 4{,}860x^2 - 2{,}916x + 729$$

k	C(n,k)	a^(n−k)	b^k	coefficient	term
0	1	64	1	64	x^6
1	6	32	-3	-576	x^5
2	15	16	9	2,160	x^4
3	**20**	**8**	**-27**	**-4,320**	x^3
4	15	4	81	4,860	x^2
5	6	2	-243	-2,916	x
6	1	1	729	729	1

Check: setting $x = 1$ adds up all the coefficients, 1, which must equal $(2 - 3)^6 = 1$. ✓

Code: building Pascal's triangle row by row

CODE WALKTHROUGH

Each new row comes from Pascal's identity applied to the row above. Use **Trace** to watch `rows` grow one row per loop.

pascal.py

Read | Edit & run | Trace

```python
def pascal(n):
    rows = [[1]]
    for i in range(1, n + 1):
        up = rows[-1]
        mid = [up[k - 1] + up[k] for k in range(1, i)]
        rows.append([1] + mid + [1])
    return rows

for row in pascal(4):
    print(" ".join(map(str, row)).center(13).rstrip())

terms = [f"{c}·a^{4-k}·b^{k}" for k, c in enumerate(row)]
print(" + ".join(terms))
print("x^3 coefficient:", pascal(6)[6][3] * 2**3 * (-3)**3)
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
      1
     1 1
    1 2 1
   1 3 3 1
1·a^4·b^0 + 4·a^3·b^1 + 6·a^2·b^2 + 4·a^1·b^3 + 1·a^0·b^4
x^3 coefficient: -4320
```

LINE 1 OF 14

Show all notes in code

```python
def pascal(n):
```

Defines a function that returns rows 0 through *n* of Pascal's triangle as a list of lists.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Builds rows 0 to n of Pascal's triangle with additions only, prints the first five rows as a triangle, expands $(a + b)^4$, and finds a coefficient of $(2x - 3)^6$.

Code: building Pascal's triangle row by row: every line explained

1	`def pascal(n):`	Defines a function that returns rows 0 through n of Pascal's triangle as a list of lists.
2	`    rows = [[1]]`	Starts with row 0, which is just $[1]$ because $\binom{0}{0} = 1$. Every later row is built from the one before it.
3	`    for i in range(1, n + 1):`	Builds rows 1, 2, ..., n in order. `range(1, n + 1)` stops before $n + 1$, so the last row built is row n.
4	`        up = rows[-1]`	`rows[-1]` is the last row built so far, the row *up* above the one we are about to make. For $i = 5$ it is `[1, 4, 6, 4, 1]`.
5	`        mid = [up[k - 1] + up[k] for k in range(1, i)]`	Pascal's identity in one line: inner entry k is the sum of entries $k - 1$ and k of the row above. For row 5: $1 + 4,\ 4 + 6,\ 6 + 4,\ 4 + 1$, giving `[5, 10, 10, 5]`. Row i has $i - 1$ inner entries.
6	`        rows.append([1] + mid + [1])`	Puts a 1 on each end (the edges are always $\binom{i}{0} = \binom{i}{i} = 1$) and adds the finished row to the list.
7	`    return rows`	Returns the whole triangle. Each entry was computed once, with a single addition. Storing smaller answers to reuse them is called dynamic programming.
9	`for row in pascal(4):`	Builds rows 0 to 4 and loops over them to print each one.
10	`    print(" ".join(map(str, row)).center(13).rstrip())`	`map(str, row)` turns the numbers into text, `" ".join` glues them with spaces, `.center(13)` pads both sides so the rows line up as a triangle, and `.rstrip()` drops the padding on the right. The first line printed is `1`.
12	`terms = [f"{c}·a^{4-k}·b^{k}" for k, c in enumerate(row)]`	After a loop ends, its variable keeps its last value, so `row` is still row 4, `[1, 4, 6, 4, 1]`: the coefficients of $(a + b)^4$. `enumerate` pairs each coefficient `c` with its position `k`, and the f-string writes the term $\binom{4}{k}a^{4-k}b^k$ as text, such as `6·a^2·b^2`. This is the Binomial Theorem as code.
13	`print(" + ".join(terms))`	Joins the five terms with plus signs. Output: `1·a^4·b^0 + 4·a^3·b^1 + 6·a^2·b^2 + 4·a^1·b^3 + 1·a^0·b^4`, which is $a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4$.
14	`print("x^3 coefficient:", pascal(6)[6][3] * 2**3 * (-3)**3)`	The worked example from the Binomial Theorem slide, the coefficient of x^3 in $(2x - 3)^6$: $\binom{6}{3} \cdot 2^3 \cdot (-3)^3$. `pascal(6)[6][3]` is 20, so the product is $20 \cdot 8 \cdot (-27) = -4320$. The parentheses in `(-3)**3` matter: `-3**3` means $-(3^3)$. Output: `x^3 coefficient: -4320`.

Lab: the pigeonhole principle

INTERACTIVE LAB

DEFINITION

The pigeonhole principle. If n items go into k boxes and $n > k$, then some box holds at least two items.

GENERALIZED PIGEONHOLE PRINCIPLE

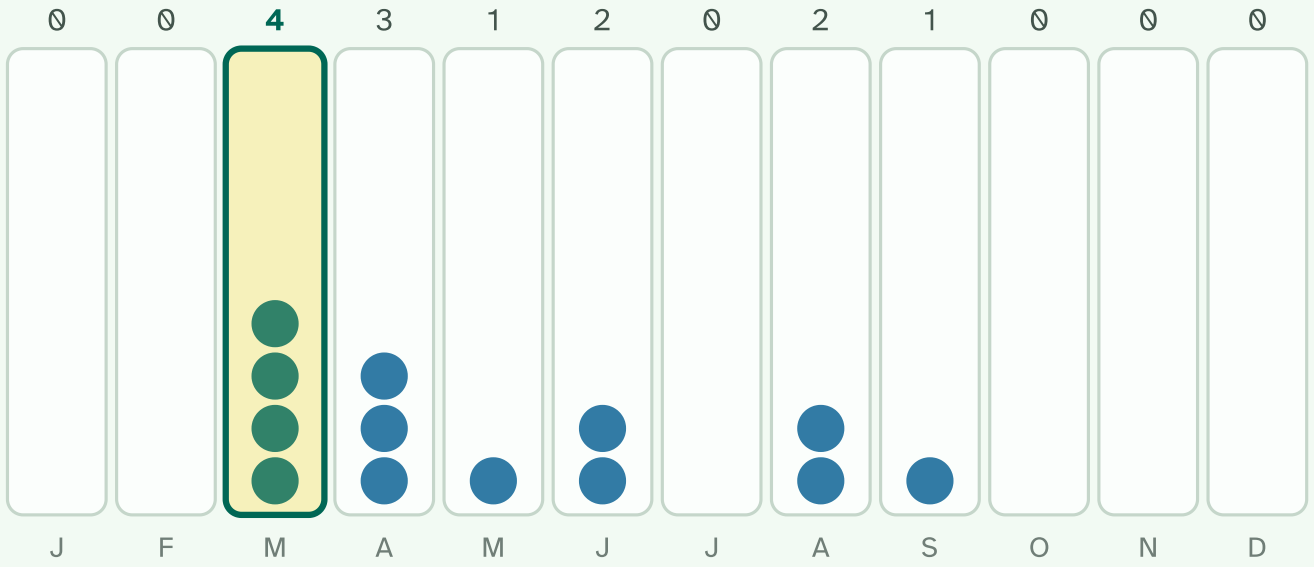
If n items go into k boxes, some box holds at least $\lceil n/k \rceil$ items. If every box held fewer, the total would be below $k \cdot n/k = n$.

Birth months | Sock colors | Remainders mod 5 | Custom

Drop randomly ↻ | Spread as evenly as possible

items n 13

boxes k 12



$$\lceil 13/12 \rceil = 2$$

With 13 people in 12 boxes, some box must hold at least **2**. This random drop put **4** in the fullest box, more than the guaranteed 2. Press **Spread as evenly as possible** to see the best case still reach 2.

WHERE THE PRINCIPLE IS USED

- **Birth months:** 13 people always include two with the same birth month: 13 items, 12 boxes.
- **Socks:** With 3 colors in a drawer, 4 socks always include a matching pair.
- **Remainders:** Any 6 integers include two with the same remainder mod 5, so their difference is a multiple of 5.
- **Compression:** No lossless compressor shrinks every file: there are 2^n files of n bits but only $2^n - 1$ shorter bit strings.
- **Hashing:** A hash table with more keys than buckets must have a collision, however clever the hash function.

Design practice: counting before computing

DESIGN PRACTICE

Counting is the cheapest analysis you can run on a problem. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Count before you brute-force**
02 Decide order and repetition first
03 Use math.comb, not factorial division
04 Generate lazily with itertools
05 Avoid double counting with a canonical form
06 Check formulas against brute force on small cases
07 Memoize repeated binomials
08 Size secrets with the product rule

Principle 1 of 8

Count before you brute-force

Before looping over every possibility, compute how many there are. Ordering 20 cities gives $20! \approx 2.4 \times 10^{18}$ routes; at a billion routes per second that takes 77 years. A one-line count tells you to choose a smarter algorithm before you waste a day of computing.

✗ AVOID THIS

```
for route in permutations(cities):    # 20 cities
    best = min(best, length(route))    # 77 years
```

✓ DO THIS

```
from math import factorial
print(f"{factorial(len(cities)):.2e} routes")
# 2.43e+18: use a heuristic or dynamic programming
```

Design practice: counting before computing: all principles (1/2)

Count before you brute-force

Before looping over every possibility, compute how many there are. Ordering 20 cities gives $20! \approx 2.4 \times 10^{18}$ routes; at a billion routes per second that takes 77 years. A one-line count tells you to choose a smarter algorithm before you waste a day of computing.

```
from math import factorial
print(f"{factorial(len(cities)):.2e} routes")
# 2.43e+18: use a heuristic or dynamic programming
```

Decide order and repetition first

Most counting bugs come from the wrong model, not from bad arithmetic. Before you write a formula, answer two questions: does order matter, and may items repeat? The answers pick the formula. Writing the decision in a comment lets a reviewer check it.

```
# order?  repeat?  formula      Python
# yes     no       n!/(n-r)!    perm(n, r)
# yes     yes      n**r         n ** r
# no      no       C(n, r)      comb(n, r)
# no      yes      C(n+r-1, r)  comb(n + r - 1, r)
```

Use math.comb, not factorial division

`factorial(n) / (factorial(k) * factorial(n - k))` uses true division and returns a float. A float keeps only about 16 significant digits, so large counts come out wrong, and the huge factorials waste time. `math.comb` returns the exact integer quickly.

```
from math import comb
c = comb(100, 50)   # exact: 100891344545564193334812497256
```

Generate lazily with itertools

`itertools.combinations` and `permutations` produce one item at a time. Wrapping them in `list()` builds everything in memory at once: all pairs of a million users is about 5×10^{11} tuples. Loop over the generator directly and stop as soon as you have what you need.

```
for a, b in combinations(users, 2):   # one at a time
    if similar(a, b):
        report(a, b)
```

Avoid double counting with a canonical form

Unordered things need one standard way to be written down, or each gets counted several times. A pair found by a double loop appears twice, as (a, b) and (b, a). Loop over `combinations` (or require `i < j`), and store unordered groups as sorted tuples, so each has exactly one representation.

```
for a, b in combinations(people, 2):   # each pair once
    ...
seen.add(tuple(sorted(hand)))   # one key per hand
```

Design practice: counting before computing: all principles (2/2)

Check formulas against brute force on small cases

Before trusting a formula for $n = 1000$, compare it with direct enumeration for $n = 1, 2, 3, 4$. If they disagree, the formula or the model is wrong. This catches off-by-one mistakes such as $C(n + r, r)$ in place of $C(n + r - 1, r)$.

```
from itertools import
combinations_with_replacement as cwr
for n in range(1, 6):
    for r in range(5):
        assert comb(n + r - 1, r) ==
len(list(cwr(range(n), r)))
```

Size secrets with the product rule

A password space holds $\text{alphabet}^{\text{length}}$ strings, so length beats complexity. Eight characters from all 94 printable symbols give $94^8 \approx 6.1 \times 10^{15}$. Five random words from a 7,776-word list give $7776^5 \approx 2.8 \times 10^{19}$, over 4,000 times more, and they are easier to remember. Every pick must be truly random.

```
print(f"{94 ** 8:.1e}")     # 6.1e+15: 8
random symbols
print(f"{7776 ** 5:.1e}")   # 2.8e+19: 5
random words
# pick with secrets.choice, never with random
```

Memoize repeated binomials

A recursion written straight from Pascal's identity recomputes the same entries again and again: $C(30, 15)$ takes over 300 million calls. Caching each answer (memoization) or building the triangle row by row computes every entry once.

```
from functools import cache
@cache
def C(n, k):
    if k in (0, n):
        return 1
    return C(n - 1, k - 1) + C(n - 1, k)   #
each once
```

Worked example: ordered picks collapse into committees

WORKED EXAMPLE

PROBLEM

A club has 4 members: A, B, C and D. How many different 3-person committees can the club form from these 4 members, that is, what is $C(4, 3)$?

THE 4 COMMITTEES

`{A, B, C}` `{A, B, D}` `{A, C, D}` `{B, C, D}`

Formula: C(4, 3) = 4! / (3! · 1!) = 24 / 6 = 4.

Answer: C(4, 3) = 4 committees — ABC, ABD, ACD, BCD. Check: 4! / (3! · 1!) = 4 and the explicit list has exactly 4 entries.

Each committee listed once, with no order inside a committee.

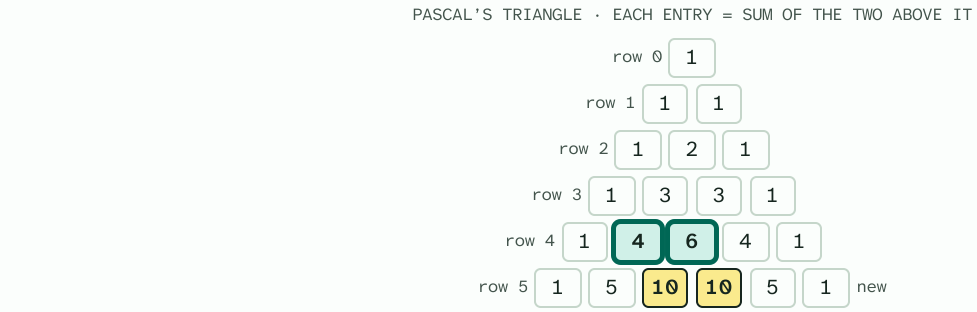
1. We count ordered picks on purpose first, even though a committee has no order. The first pick has 4 options, one for each member A, B, C and D, and the tree starts with those 4 branches.
2. Each first pick splits into the 3 members still left, because nobody can be picked twice. The product rule multiplies the stages, so there are now 4 × 3 = 12 partial picks.
3. Each partial pick splits once more into the 2 members still left, and the tree stops at full triples. The product rule gives 4 × 3 × 2 = 24 ordered picks, which is exactly P(4, 3).
4. Look at the highlighted leaves: ABC, ACB, BAC, BCA, CAB and CBA are six different leaves but one and the same committee. Every committee is counted exactly 3! = 6 times this way, once for each order of its 3 members.
5. Since every committee was counted the same number of times, dividing undoes the overcount fairly. There are 24 / 6 = 4 committees, and the leaves now sit in 4 groups, one group per committee.
6. **Answer: C(4, 3) = 4, and the committees are ABC, ABD, ACD and BCD. Check with the formula: 4! / (3! · 1!) = 24 / 6 = 4, which matches the explicit list of exactly 4 committees.**

Worked example: building Pascal's triangle to find C(5, 2)

WORKED EXAMPLE

PROBLEM

Build Pascal's triangle down to row 5, one row at a time. Then use the finished triangle to find $C(5, 2)$, the number of ways to choose 2 books from a set of 5 books.



Row 5 sums to 1 + 5 + 10 + 10 + 5 + 1 = 32 = 2^5, one subset count per entry.

Answer: C(5, 2) = 10 ways to choose 2 books from 5. Check: 5! / (2! · 3!) = 120 / 12 = 10, and C(5, 3) = 10 is the same 10 mirrored in row 5.

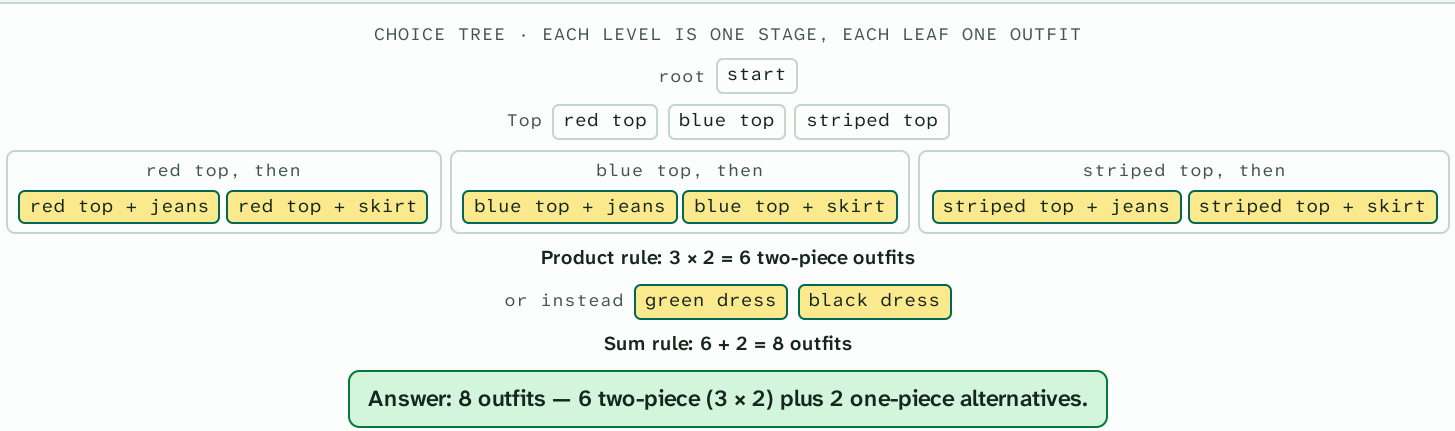
1. Rows 0 and 1 are already in place: a single 1, then 1 and 1. Every later entry is the sum of the two entries above it, and the edges stay 1 because there is exactly one way to choose all of a set or none of it.
2. Row 2 appears as 1, 2, 1, and its middle entry is the first one that must be computed. It comes from its two parents in row 1: 2 = 1 + 1, while the two edge entries are copied down as 1.
3. Row 3 appears as 1, 3, 3, 1, built by adding neighboring pairs from row 2. The marked entry shows the pattern clearly: 3 = 1 + 2, the sum of the entry above-left and the entry above-right.
4. Row 4 appears as 1, 4, 6, 4, 1, and its middle entry is worth remembering. It is 6 = 3 + 3 from the two 3s above it, and it is C(4, 2), the count for choosing 2 of 4.
5. Row 5 appears as 1, 5, 10, 10, 5, 1, completing the triangle we need. The marked third entry is C(5, 2), and it comes from its parents 4 and 6 in row 4: 10 = 4 + 6.
6. **Answer: C(5, 2) = 10, so there are 10 ways to choose 2 books from 5. Check with factorials: 5! / (2! · 3!) = 120 / 12 = 10, and symmetry gives the same 10 at C(5, 3), the mirrored entry in row 5.**

Worked example: a choice tree for the product and sum rules

WORKED EXAMPLE

PROBLEM

An outfit is either a two-piece outfit — one of 3 tops (red, blue, striped), then one of 2 bottoms (jeans, skirt) — or one of 2 one-piece dresses (green, black). How many different outfits are there?



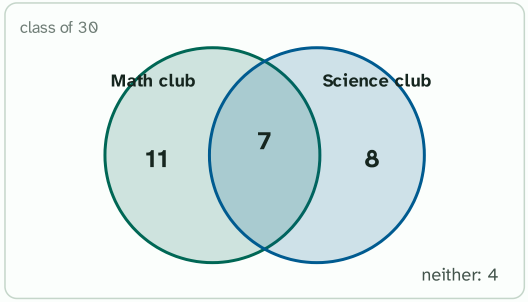
1. Start at the root of the choice tree. An outfit is built in stages: first a top, then a bottom. Nothing has branched yet, so the tree is a single point and the count so far is 1 partial outfit.
2. Level 1 branches by the first stage: there are 3 choices for a top (red top, blue top, striped top), so the root splits into 3 branches. Each branch is one partial outfit waiting for its second stage.
3. Level 2 completes each branch: every first-stage choice splits into the 2 choices for a bottom (jeans, skirt). The product rule multiplies independent stages, giving 3 × 2 = 6 leaves, one per complete two-piece outfit.
4. Now add the disjoint alternative: 2 one-piece options (green dress, black dress) that are not two-piece outfits at all. Because the groups do not overlap — no outfit is both two-piece and one-piece — the sum rule, not the product rule, combines them.
5. Combine the two counts with the sum rule: 6 two-piece outfits + 2 one-piece alternatives = 8 outfits in all. The product rule worked inside one staged task; the sum rule works across separate groups that share no members.
6. **Answer: 8 outfits — 6 from the product rule (3 × 2 = 6) plus 2 alternatives by the sum rule. Listing the leaves confirms it: red top + jeans; red top + skirt; blue top + jeans; blue top + skirt; striped top + jeans; striped top + skirt, plus green dress and black dress: exactly 8 outfits, each counted once.**

Worked example: inclusion-exclusion on a Venn diagram

WORKED EXAMPLE

PROBLEM

In a class of 30 students, 18 are in the math club and 15 are in the science club, and 7 students are in both clubs. How many students are in at least one of the two clubs?



Regions: 11 + 7 + 8 + 4 = 30 students

Answer: 26 students are in at least one club (18 + 15 – 7 = 26), and 4 are in neither.

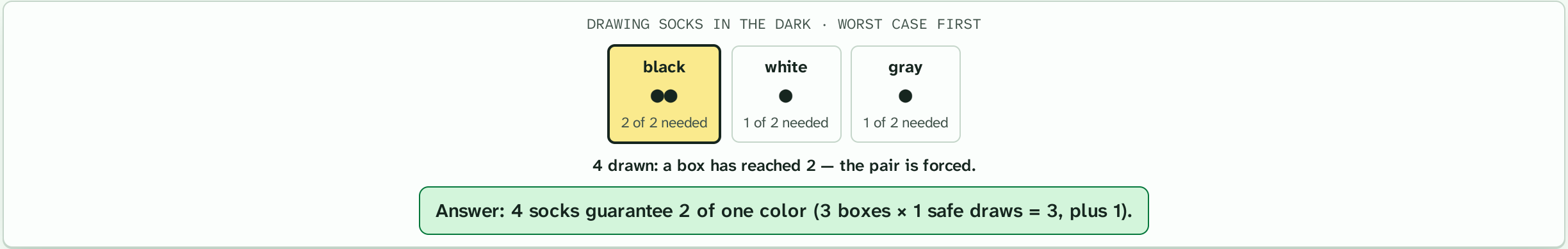
1. The universe is a class of 30 students. Two groups overlap on the Venn diagram: Math club and Science club. Before any counting, the diagram only frames the question — which regions hold how many students is what we have to find.
2. Add the two group sizes the naive way: Math club has 18 students and Science club has 15, for a running total of 18 + 15 = 33. This is the sum rule applied too early, because the groups are not yet known to be disjoint.
3. Look at the overlap region in the middle of the diagram: it holds 7 students. Those students sit inside both circles, so the naive total counted each of them twice — the 7 students in both clubs were counted twice, once inside each club total, so that overlap must come back out exactly once.
4. Apply inclusion-exclusion: 18 + 15 – 7 = 26 students are in at least one club. Region by region that is 11 in Math club only, 7 in both, and 8 in Science club only, and 11 + 7 + 8 = 26 checks the arithmetic.
5. The students in neither club are what is left of the class: 30 – 26 = 4. Every student now sits in exactly one region of the diagram — Math club only, both, Science club only, or neither — and the four regions add back to the whole class of 30.
6. **Answer: 26 students are in at least one club, by inclusion-exclusion: 18 + 15 – 7 = 26. Direct enumeration of the class list gives the same regions — 11 only, 7 both, 8 only, 4 neither — totaling 26 in the union.**

Worked example: pigeonhole — how many socks guarantee a pair?

WORKED EXAMPLE

PROBLEM

A drawer holds socks in 3 colors: black, white and gray. You draw socks in the dark, one at a time. How many socks must you draw to guarantee that two of them have the same color?



Answer: 4 socks guarantee 2 of one color (3 boxes × 1 safe draws = 3, plus 1).

1. The drawer holds socks in 3 colors — black, white, gray — and you draw in the dark, so an unfriendly order is possible. The boxes start empty; the question is how many socks guarantee that some color reaches 2.
2. Place sock number 1 in the worst possible way, into a box that does not yet hold 2: the boxes now hold 1 black, 0 white, 0 gray. Spreading the draws out this way delays the target as long as possible, which is exactly the worst case the guarantee must survive.
3. Place sock number 2 in the worst possible way, into a box that does not yet hold 2: the boxes now hold 1 black, 1 white, 0 gray. Spreading the draws out this way delays the target as long as possible, which is exactly the worst case the guarantee must survive.
4. Place sock number 3 in the worst possible way, into a box that does not yet hold 2: the boxes now hold 1 black, 1 white, 1 gray. Every box sits at 1, the most that avoids the target — the next draw has nowhere safe to go.
5. Draw number 4 must land in one of the 3 boxes, and every box already holds 1. Whichever color it is, that box reaches 2: the guarantee moment. No arrangement of 4 socks can avoid a box with 2.
6. **Answer: 4 socks guarantee 2 of one color. The worst case fills each of the 3 boxes with 1 first, using 3 × 1 = 3 socks, and one more draw forces the target: 3 + 1 = 4, matching the ceiling formula for 4 items in 3 boxes.**

Practice: counting and the Binomial Theorem

PRACTICE

Every answer is a whole number. Large numbers can be typed with or without commas.

COUNTING RULES

A code is 3 letters (A to Z) followed by 2 digits (0 to 9). How many codes are possible?

Type your answer

Check

Hint: one factor per position (product rule).

Show worked solution

New problem ↻

Product rule, one factor per position: $26^3 \cdot 10^2 = 1{,}757{,}600$.

PROBLEM TYPE

- Counting rules
- Permutations & combinations
- Repetition
- Binomial theorem

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: counting and the Binomial Theorem: sample problems with solutions

COUNTING RULES

What is the smallest number of people that guarantees at least 4 of them share the same day of the week (7 possibilities)?

Up to $7 \cdot 3 = 21$ people can be spread with at most 3 per box. One more person forces some box to reach 4: 22. Check with the generalized principle: $\lceil 22/7 \rceil = 4$.

PERMUTATIONS & COMBINATIONS

In how many ways can a committee of 3 be chosen from 13 people?

Order does not matter:

$$C(13, 3) = \frac{P(13, 3)}{3!} = \frac{1{,}716}{6} = 286.$$

REPETITION

How many different arrangements of the letters of BOOKKEEPER are there?

BOOKKEEPER has 10 letters with repeats O ×2, K ×2, E ×3. Arrangements:

$$\frac{10!}{2!\,2!\,3!} = \frac{3{,}628{,}800}{24} = 151{,}200.$$

BINOMIAL THEOREM

What is the sum of all the coefficients of $(x + 2)^4$ after it is fully expanded?

Setting $x = 1$ turns every term into its coefficient, so the sum is $(1 + 2)^4 = 3^4 = 81$.

Check yourself: counting

CHECK YOURSELF

QUESTION 1 OF 10

How many 4-digit PINs are possible if each digit is 0 to 9 and digits may repeat?

A 40

B 5,040

C 210

D 10,000

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: counting: answers explained (1/2)

1. How many 4-digit PINs are possible if each digit is 0 to 9 and digits may repeat?

Answer D: 10,000

Four positions with 10 choices each: $10^4 = 10{,}000$ by the product rule. 5,040 would be the count without repeats, $P(10, 4)$.

2. In how many ways can 8 runners take gold, silver and bronze?

Answer C: 336

Order matters and nobody wins twice: $P(8, 3) = 8 \cdot 7 \cdot 6 = 336$. 56 is $C(8, 3)$, which ignores who got which medal.

3. How many 5-card hands can be dealt from a 52-card deck?

Answer B: 2,598,960

A hand is unordered: $C(52, 5) = 311{,}875{,}200/5! = 2{,}598{,}960$. The count 311,875,200 is $P(52, 5)$, which treats the same five cards dealt in a different order as a different hand; 380,204,032 is 52^5, which allows the same card to be dealt twice.

4. How many different arrangements do the letters of BANANA have?

Answer B: 60

Six letters with A three times and N twice: $6!/(3!\,2!) = 720/12 = 60$.

5. In how many ways can you buy 6 donuts from 4 flavors (order does not matter, flavors may repeat)?

Answer B: 84

Stars and bars: 6 stars and 3 bars, so $C(9, 3) = 84$. 4,096 is 4^6, which counts ordered sequences.

6. Which binomial coefficient equals $\binom{9}{3} + \binom{9}{4}$?

Answer A: $\binom{10}{4}$

Pascal's identity with $n = 9$, $k = 4$: $\binom{9}{3} + \binom{9}{4} = \binom{10}{4} = 84 + 126 = 210$.

7. What is the coefficient of x^2y^3 in $(x + 2y)^5$?

Answer A: 80

The term with y^3 is $\binom{5}{3}x^2(2y)^3 = 10 \cdot 8\,x^2y^3 = 80x^2y^3$. Forgetting the 2^3 gives 10.

8. How many integers from 1 to 60 are divisible by 2 or by 3?

Answer D: 40

Inclusion–exclusion: $30 + 20 - 10 = 40$, where the 10 multiples of 6 were counted in both groups.

9. What is the smallest group that guarantees 3 people share a birth month?

Answer C: 25

24 people can be spread 2 per month. The 25th forces a third into some month: $\lceil 25/12 \rceil = 3$.

Check yourself: counting: answers explained (2/2)

10. Which Python expression counts the 5-card hands exactly and efficiently?

Answer C: `math.comb(52, 5)`

`math.comb` returns the exact integer directly. The first uses float division, the third builds 2.6 million tuples in memory just to count them, and the last uses the wrong model.

What is a recurrence relation?

CONCEPT

DEFINITION

A recurrence relation for a sequence $a_0, a_1, a_2, \ldots$ is a rule that gives each term a_n in terms of earlier terms. The initial conditions state the first term or terms outright. A rule plus enough initial conditions defines exactly one sequence.

WORKED EXAMPLE

You deposit \$1000 at 5% interest per year. Let A_n be the balance after n years. Each year adds 5% of the previous balance:

$$A_n = 1.05\,A_{n-1} \quad (n \geq 1), \qquad A_0 = 1000.$$

The rule says how to take one step. The initial condition says where to start.

COMMON PITFALL

A rule alone is not enough. $a_n = 2a_{n-1}$ fits $1, 2, 4, 8, \ldots$ and also $3, 6, 12, 24, \ldots$. Only the initial condition a_0 tells you which sequence you mean.

Following the rule one year at a time

n	A_n	computed as
0	1000.00	initial condition
1	1050.00	1.05×1000.00
2	1102.50	1.05×1050.00
3	1157.63	1.05×1102.50
4	1215.51	1.05×1157.63

Reaching A_{30} this way takes 30 steps. A closed form, $A_n = 1000 \cdot 1.05^n$, jumps straight to any year. Finding closed forms is a main goal of this module.

WHY IT MATTERS

Every recursive program hides a recurrence. Count the work a recursive function does on an input of size n and you get a rule in terms of smaller inputs. Solving that rule tells you how fast the program runs.

WHERE THIS SHOWS UP IN AI

A recurrent neural network reads text one token at a time and updates its hidden state with $h_t = f(h_{t-1}, x_t)$: the new state depends on the previous state and the current input x_t. That is a recurrence relation, and the starting state h_0 is its initial condition.

Three classic recurrences

CONCEPT

Tower of Hanoi

Move a stack of n disks from peg A to peg C, one disk at a time, never placing a larger disk on a smaller one. To move n disks: move the top $n-1$ to the spare peg, move the largest disk, then move the $n-1$ disks back on top of it.

$$H_n = 2H_{n-1} + 1,\ H_1 = 1$$

Terms: 1, 3, 7, 15, 31, 63, … The two copies of H_{n-1} are the two smaller towers; the $+1$ is the largest disk.

Fibonacci numbers

Each term is the sum of the two before it. The rule looks back two steps, so it needs two initial conditions to get started.

$$F_n = F_{n-1} + F_{n-2}$$

$$F_0 = 0,\ F_1 = 1$$

Terms: 0, 1, 1, 2, 3, 5, 8, 13, 21, … They model branching growth and appear again and again when we analyze recursive algorithms.

Bit strings with no "11"

Let S_n count the length-n bit strings with no two 1s in a row. Split them by their **last bit**:

- ends in 0: any good string of length $n-1$, then 0. That is S_{n-1} strings.
- ends in 1: the bit before must be 0, so a good string of length $n-2$, then 01. That is S_{n-2} strings.

$$S_n = S_{n-1} + S_{n-2}$$

$S_1 = 2$ (0, 1) and $S_2 = 3$ (00, 01, 10). Then 5, 8, 13, …: the Fibonacci numbers again, $S_n = F_{n+2}$.

WORKED EXAMPLE

Check S_3 by listing every good string of length 3: 000, 001, 010, 100, 101. The three ending in 0 (000, 010, 100) are the good strings of length 2 with a 0 added. The two ending in 1 (001, 101) are the good strings of length 1 with 01 added. So $S_3 = S_2 + S_1 = 3 + 2 = 5$.

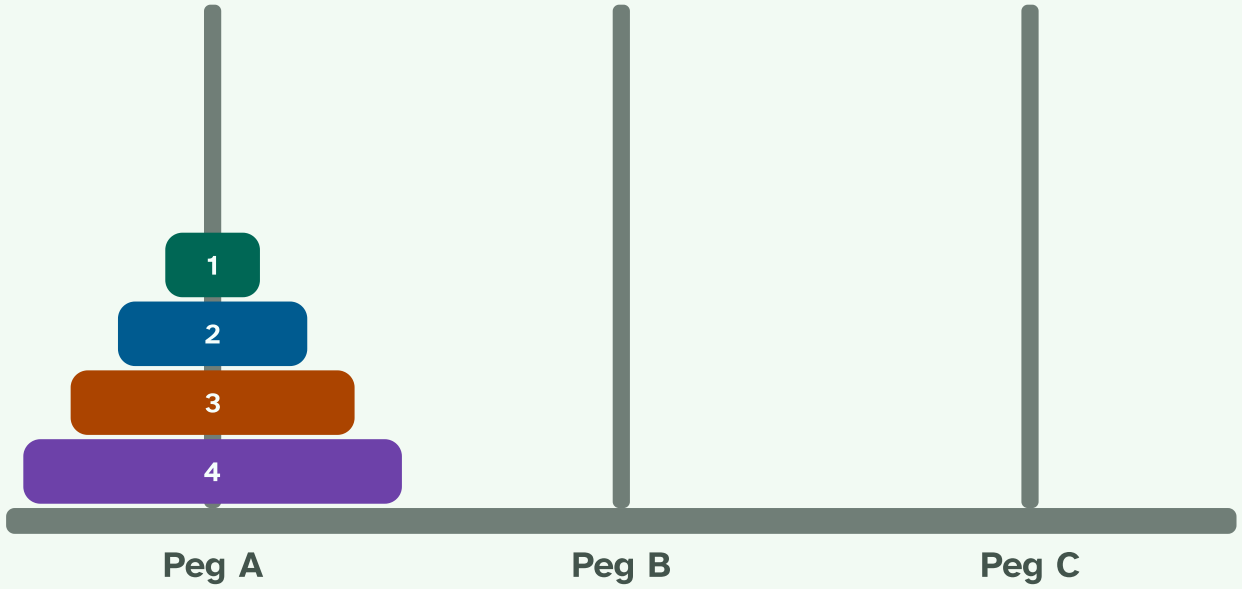
DESIGN TIP

To find a recurrence, ask how a problem of size n breaks into smaller problems of the same kind. Then count each case separately and add the counts. Splitting by the first or the last choice works for a great many counting problems.

Lab: the Tower of Hanoi

INTERACTIVE LAB

Click a peg to lift its top disk, then click another peg to drop it. A disk may land only on an empty peg or on a larger disk. Press **Step** or **Play** to watch the recursive solution: the panel lists every call that is running and which of its three phases it is in.



Disks 1 2 3 **4** 5 6 7

Speed Slow **Normal** Fast

Step ▶ ▶ Play Reset

0	15
MOVES MADE	MINIMUM: $2^4 - 1$

CALLS RUNNING RIGHT NOW (OUTERMOST FIRST)

Nothing is running yet. The first call will be `hanoi(4, A→C)`: move 4 disks from A to C using B as the spare.

n	1	2	3	**4**	5	6	7
H_n	1	3	7	**15**	31	63	127

Each value is twice the one before, plus 1: $H_n = 2H_{n-1} + 1$.

Code: a recursive Tower of Hanoi solver

CODE WALKTHROUGH

The program below is the recurrence $H_n = 2H_{n-1} + 1$ written as code. Switch to **Trace** to watch the calls nest four levels deep (down to `n = 0`) and unwind again.

hanoi.py

Read | Edit & run | Trace

```
1  def hanoi(n, source, target, spare, moves):
2      if n == 0:
3          return
4      hanoi(n - 1, source, spare, target, moves)
5      moves.append((n, source, target))
6      hanoi(n - 1, spare, target, source, moves)
7
8  moves = []
9  hanoi(3, "A", "C", "B", moves)
10 for step, (disk, src, dst) in enumerate(moves, start=1):
11     print(f"{step}: disk {disk} {src} -> {dst}")
12 print(len(moves), "moves; 2**3 - 1 =", 2**3 - 1)
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
1: disk 1 A -> C
2: disk 2 A -> B
3: disk 1 C -> B
4: disk 3 A -> C
```

LINE 1 OF 12

Show all notes in code

```
def hanoi(n, source, target, spare, moves):
```

Defines the recursive solver. It moves `n` disks from the peg named `source` to the peg named `target`, using `spare` as the helper peg, and records each move in the list `moves`. Nothing runs until the function is called on line 9.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

A recursive solver for the Tower of Hanoi that records every move in a list, prints the moves, and checks that 3 disks need exactly $2^3 - 1 = 7$ moves.

Code: a recursive Tower of Hanoi solver: every line explained

1	`def hanoi(n, source, target, spare, moves):`	Defines the recursive solver. It moves `n` disks from the peg named `source` to the peg named `target`, using `spare` as the helper peg, and records each move in the list `moves`. Nothing runs until the function is called on line 9.
2	`    if n == 0:`	The base case: zero disks need zero moves. Every recursive function needs an input it can answer without calling itself, or the calls would never stop. Each call works on one disk fewer, so this case is always reached.
3	`        return`	Leaves the function at once. A bare `return` hands back `None`. That is fine here because the useful result is the list of moves, not a return value.
4	`    hanoi(n - 1, source, spare, target, moves)`	Phase 1: move the top $n-1$ disks out of the way, from `source` onto `spare`. Notice the roles swap: the old spare becomes the target and the old target becomes the helper. This one call makes H_{n-1} moves.
5	`    moves.append((n, source, target))`	Phase 2: disk `n`, the largest one here, now sits alone on `source` and `target` is empty, so it moves directly. The move is recorded as a tuple (disk, from, to). This single move is the $+1$ in $H_n = 2H_{n-1} + 1$.
6	`    hanoi(n - 1, spare, target, source, moves)`	Phase 3: move the $n-1$ disks from `spare` onto the largest disk at `target`, now using `source` as the helper. That is another H_{n-1} moves. Phases 1 and 3 together explain the $2H_{n-1}$.
8	`moves = []`	Creates an empty list to collect the moves. It is created once, outside the function, and the same list is passed down to every call, so every level of the recursion appends to one shared record.
9	`hanoi(3, "A", "C", "B", moves)`	Solves the 3-disk puzzle: move everything from A to C with B as the spare. This single call triggers 15 calls in total: 7 that move a disk and 8 with `n = 0` that return at once. In Trace mode the call depth on the right shows how deeply they nest.
10	`for step, (disk, src, dst) in enumerate(moves, start=1):`	Loops over the recorded moves. `enumerate(..., start=1)` numbers them 1, 2, 3, ..., and the pattern `(disk, src, dst)` unpacks each tuple into three variables in one step.
11	`    print(f"{step}: disk {disk} {src} -> {dst}")`	Prints one move per line, for example `1: disk 1 A -> C`. The `f` before the quote makes it an f-string, so each `{...}` is replaced by the variable's value. Notice that disk 1 moves on every odd-numbered step.
12	`print(len(moves), "moves; 2**3 - 1 =", 2**3 - 1)`	Checks the count against the closed form and prints `7 moves; 2**3 - 1 = 7`. `len(moves)` counts the recorded moves, and `**` is Python's power operator. Comparing a closed form with real output on small cases is a cheap and powerful test.

Solving by iteration, then proving it by induction

CONCEPT

Step 1: iterate. Iteration substitutes the rule into itself until a pattern appears. For the Tower of Hanoi:

$$
\begin{aligned}
H_n &= 2H_{n-1} + 1 && \text{the rule} \\
&= 2(2H_{n-2} + 1) + 1 = 2^2 H_{n-2} + 2 + 1 && \text{rule for } H_{n-1} \\
&= 2^3 H_{n-3} + 2^2 + 2 + 1 && \text{rule for } H_{n-2} \\
&\;\;\vdots \\
&= 2^{n-1} H_1 + 2^{n-2} + \cdots + 2 + 1 && \text{after } n-1 \text{ steps} \\
&= 2^{n-1} + 2^{n-2} + \cdots + 2 + 1 = 2^n - 1 && H_1 = 1
\end{aligned}
$$

The last step is a geometric sum: $1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1$. In binary it is n ones in a row, which is one less than a 1 followed by n zeros.

WORKED EXAMPLE

The same trick solves compound interest:
$A_n = 1.05A_{n-1} = 1.05^2 A_{n-2} = \cdots = 1.05^n A_0$. After 10 years,
$A_{10} = 1000 \cdot 1.05^{10} \approx 1628.89$.

Step 2: prove the guess by induction. Iteration suggests a formula but does not prove it, because the ⋮ hides infinitely many steps. Induction closes that gap.

THEOREM / RULE

Claim: $H_n = 2^n - 1$ for every $n \geq 1$.

Base case. $H_1 = 1 = 2^1 - 1$.

Inductive step. Assume $H_k = 2^k - 1$ for some $k \geq 1$. Then

$$H_{k+1} = 2H_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1.$$

So the formula holds for $k + 1$, and by induction for every $n \geq 1$.

DESIGN TIP

Test a guess on small cases before proving it: $H_1, \ldots, H_4 = 1, 3, 7, 15$, and $2^n - 1$ gives the same values. A guess that fails at $n = 3$ saves you from trying to prove something false.

WHY IT MATTERS

At one move per second, 64 disks take $2^{64} - 1$ seconds, about 585 billion years. The closed form answers that question instantly. Running the recursion never would.

Linear homogeneous recurrences of order 2

CONCEPT

DEFINITION

A linear homogeneous recurrence of order 2 with constant coefficients has the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2}, \qquad c_2 \neq 0.$$

Linear: earlier terms are only multiplied by constants. *Homogeneous*: no extra term such as +1. *Order 2*: it looks back two steps.

The key idea. Guess a solution of the form $a_n = r^n$. Substituting gives $r^n = c_1 r^{n-1} + c_2 r^{n-2}$. Dividing by r^{n-2} leaves the characteristic equation

$$r^2 - c_1 r - c_2 = 0.$$

Each root r gives a solution r^n, and any sum of multiples of solutions is again a solution.

THEOREM / RULE

Distinct roots $r_1 \neq r_2$: $a_n = b r_1^n + d r_2^n$.
Repeated root r: $a_n = b r^n + d n r^n$.
Find the constants b and d from the two initial conditions.

WORKED EXAMPLE

Fibonacci. $r^2 - r - 1 = 0$ has roots $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ and $\psi = \frac{1-\sqrt{5}}{2} \approx -0.618$. The conditions $F_0 = 0$ and $F_1 = 1$ give $b = \frac{1}{\sqrt{5}}$ and $d = -\frac{1}{\sqrt{5}}$, so

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

This is Binet's formula. Because $|\psi| < 1$, the ψ^n term fades away and F_n is the integer nearest to $\varphi^n/\sqrt{5}$. Fibonacci numbers grow by a factor of about 1.618, the golden ratio, at every step.

WORKED EXAMPLE

Repeated root. $a_n = 4a_{n-1} - 4a_{n-2}$ with $a_0 = 1$, $a_1 = 4$. Here $r^2 - 4r + 4 = (r-2)^2 = 0$, so 2 is a root twice and $a_n = b\,2^n + d\,n\,2^n$. Then $a_0 = b = 1$, and $a_1 = 2b + 2d = 4$ gives $d = 1$. So $a_n = (n+1)\,2^n$. Check: $a_2 = 4 \cdot 4 - 4 \cdot 1 = 12 = 3 \cdot 2^2$.

Lab: characteristic-equation solver

INTERACTIVE LAB

Enter the coefficients and initial conditions of $a_n = c_1 a_{n-1} + c_2 a_{n-2}$, or pick a preset. The lab solves the characteristic equation, finds the constants, and checks the closed form against the recurrence term by term. A root outside the dashed unit circle contributes exponential growth only when its term has a nonzero coefficient.

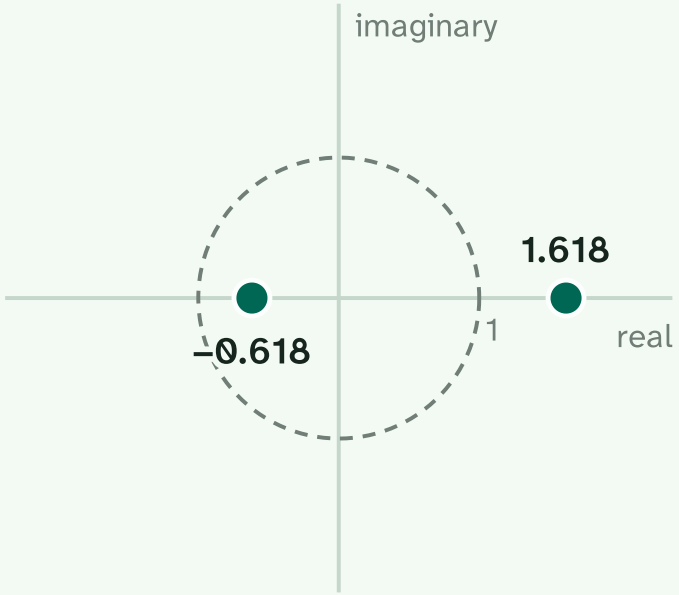
c_1 (multiplies a_{n-1}): 1

c_2 (multiplies a_{n-2}): 1

a_0 (first term): 0

a_1 (second term): 1

Fibonacci | Lucas numbers | No "11" strings | Roots 2 and 3 | Roots 3 and –2 | Repeated root | Complex roots



1 · RECURRENCE

$a_n = a_{n-1} + a_{n-2},\ a_0 = 0,\ a_1 = 1$

2 · CHARACTERISTIC EQUATION

$r^2 - r - 1 = 0$

3 · DISCRIMINANT

$\Delta = c_1^2 + 4c_2 = 5$: positive, so there are two different real roots.

4 · ROOTS

$r_1 = \frac{c_1 + \sqrt{\Delta}}{2} = 1.618, \quad r_2 = \frac{c_1 - \sqrt{\Delta}}{2} = -0.618$

5 · CONSTANTS FROM a_0, a_1

$b + d = 0, \qquad 1.618\,b - 0.618\,d = 1$

$b = 0.4472, \quad d = -0.4472$

6 · CLOSED FORM

$a_n = 0.4472 \cdot (1.618)^n - 0.4472 \cdot (-0.618)^n = \frac{\varphi^n - \psi^n}{\sqrt{5}}$

The dominant active root is 1.618. Their magnitude eventually grows by a factor approaching 1.618 per step. The ratio approaches 1.618.

Try this. Change only a_0 or a_1. The roots stay put, but a constant can become zero. Initial conditions determine which root terms appear and can change the sequence's growth.

n	from rule	closed form	ratio
0	0	0	–
1	1	1	–
2	1	1	1
3	2	2	2
4	3	3	1.5
5	5	5	1.6667
6	8	8	1.6
7	13	13	1.625
8	21	21	1.6154
9	34	34	1.619
10	55	55	1.6176

The closed form matches the rule on every row (up to rounding).

The ratio column divides each term by the one before it; a zero denominator has no ratio. A unique dominant root with a nonzero contribution controls the limiting ratio.

Recurrences hiding inside algorithms

CONCEPT

To analyze a recursive algorithm, let $T(n)$ be the number of basic steps on an input of size n. Write down what *one* call does: the calls it makes, plus its own extra work. That sentence is the recurrence.

Algorithm	One call on size n does…	Recurrence	Solution
Recursive binary search	1 comparison, then searches one half	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
Merge sort	sorts both halves, then merges them with up to n comparisons	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
Tower of Hanoi	two $(n-1)$-disk solves plus 1 move	$H_n = 2H_{n-1} + 1$	$2^n - 1$
Naive recursive Fibonacci	calls itself on $n-1$ and on $n-2$	$C_n = C_{n-1} + C_{n-2} + 1$	$2F_{n+1} - 1 = \Theta(\varphi^n)$

WORKED EXAMPLE

Unroll binary search with $T(1) = 1$: $T(n) = T(n/2) + 1 = T(n/4) + 2 = \cdots = T(n/2^k) + k$. The halving stops when $n/2^k = 1$, that is $k = \log_2 n$, so $T(n) = \log_2 n + 1$. A sorted list of a million items needs only about 20 comparisons.

COMMON PITFALL

Look at *how* the size shrinks. $T(n) = T(n-1) + 1$ removes one item per call: n steps. $T(n) = T(n/2) + 1$ halves the problem: $\log_2 n$ steps. Two calls on $n-1$, as in Hanoi, double the work at every level: 2^n. Small changes to a recurrence change the speed enormously.

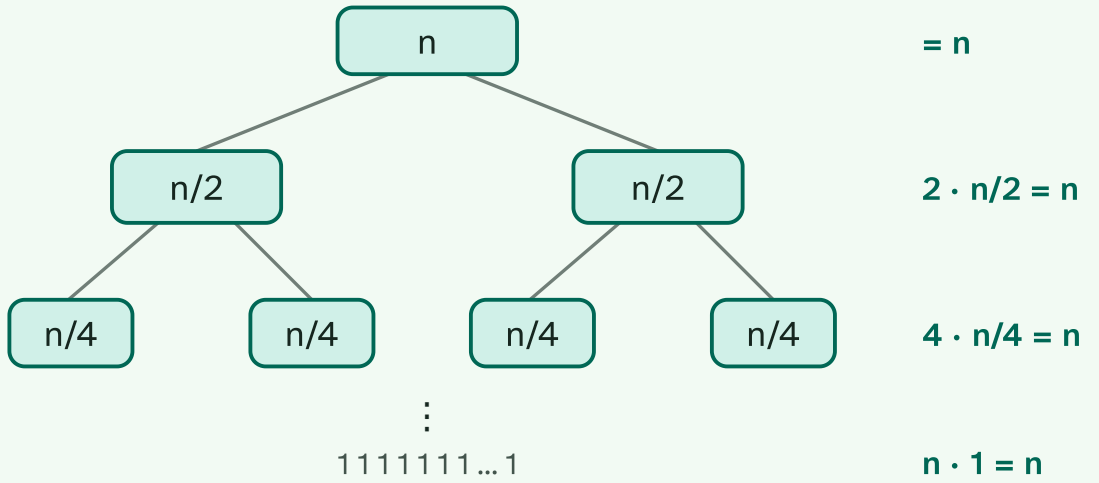
WHERE THIS SHOWS UP IN AI

Training-data pipelines sort, deduplicate and shard billions of examples with divide and conquer: split the data, process the pieces in parallel, merge the results. The recurrence $T(n) = 2T(n/2) + n$ is why that costs about $n \log n$ work instead of n^2.

Recursion trees and the master theorem

CONCEPT

A recursion tree draws every call as a node labeled with the work it does outside its recursive calls. Add up each level, then add the levels. For merge sort, $T(n) = 2T(n/2) + n$:



Every level costs n. Halving reaches size 1 after $\log_2 n$ steps, so there are $\log_2 n + 1$ levels and $T(n) = n(\log_2 n + 1) = \Theta(n \log n)$.

Recurrence	a, b, d	$\log_b a$	Result
$T(n/2) + 1$	1, 2, 0	0	$\Theta(\log n)$
$2T(n/2) + n$	2, 2, 1	1	$\Theta(n \log n)$
$3T(n/2) + n$	3, 2, 1	1.585	$\Theta(n^{1.585})$
$2T(n/2) + n^2$	2, 2, 2	1	$\Theta(n^2)$

THEOREM / RULE

The master theorem (simple form). If $T(n) = aT(n/b) + \Theta(n^d)$ with $a \geq 1$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } d > \log_b a \\ \Theta(n^d \log n) & \text{if } d = \log_b a \\ \Theta(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Why it works. Level k of the tree has a^k calls, each on size n/b^k, so the level costs $n^d (a/b^d)^k$. Everything depends on the ratio a/b^d:

- below 1: each level is cheaper than the one above, so the root's n^d dominates;
- exactly 1: every level costs n^d, and the $\log n$ levels multiply it;
- above 1: each level is costlier, so the $n^{\log_b a}$ leaves at the bottom dominate.

COMMON PITFALL

The theorem needs subproblems that shrink by a constant *factor* b. It does not apply to Hanoi or Fibonacci, where the size drops by subtraction. Use iteration or the characteristic equation for those.

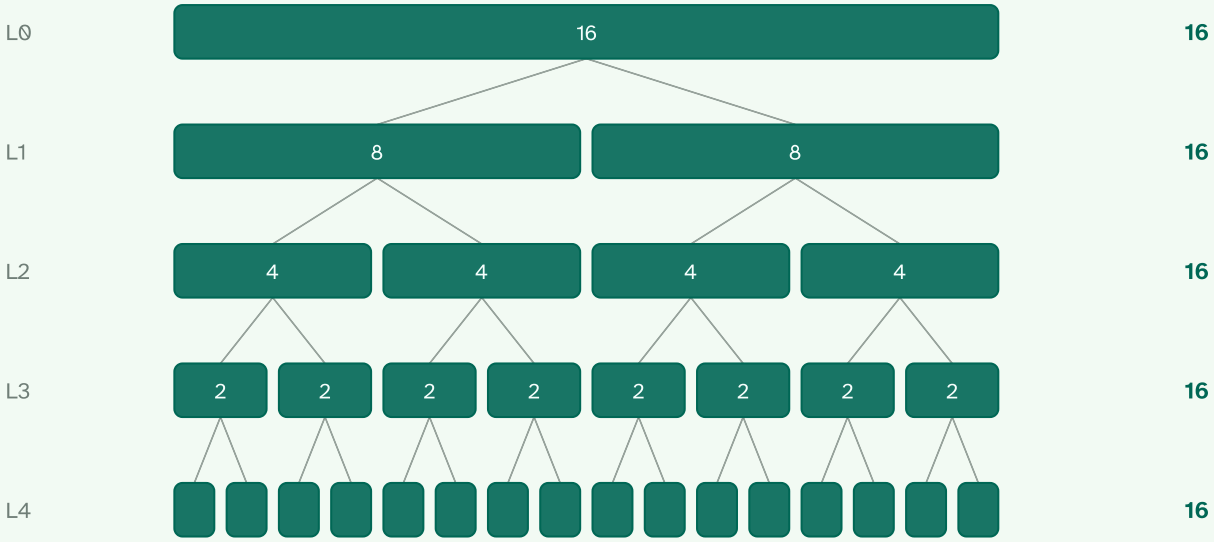
DESIGN TIP

Before you ship a recursive function, write its recurrence in a comment and solve it. Thirty seconds with the master theorem can reveal a hidden n^2.

Lab: recursion-tree explorer

INTERACTIVE LAB

For $T(n) = a\,T(n/b) + n^d$, choose a (calls per call), b (how much smaller each call is) and d (each call on size m does m^d work). Each bar is one level of the tree, and its length is that level's **total** work. Watch where the work piles up: at the top, evenly, or at the bottom.



LEVEL TOTAL

16
16
16
16
16

a 1 **2** 3 4 7 8 b **2** 3 4

d **0** **1** 2 3 levels 2 3 **4** 5

Binary search | **Merge sort** | Karatsuba | Strassen | $2T(n/2) + n^2$

$T(n) = 2T(n/2) + n, n = 2^4 = 16$

level	calls	size	work each	level total
0	1	16	16	**16**
1	2	8	8	**16**
2	4	4	4	**16**
3	8	2	2	**16**
4	16	1	1	**16**
sum of all levels				**80**

$\log_2 2 = 1 = d = 1$, so **all levels are equal**: $T(n) = \Theta(n \log n)$.

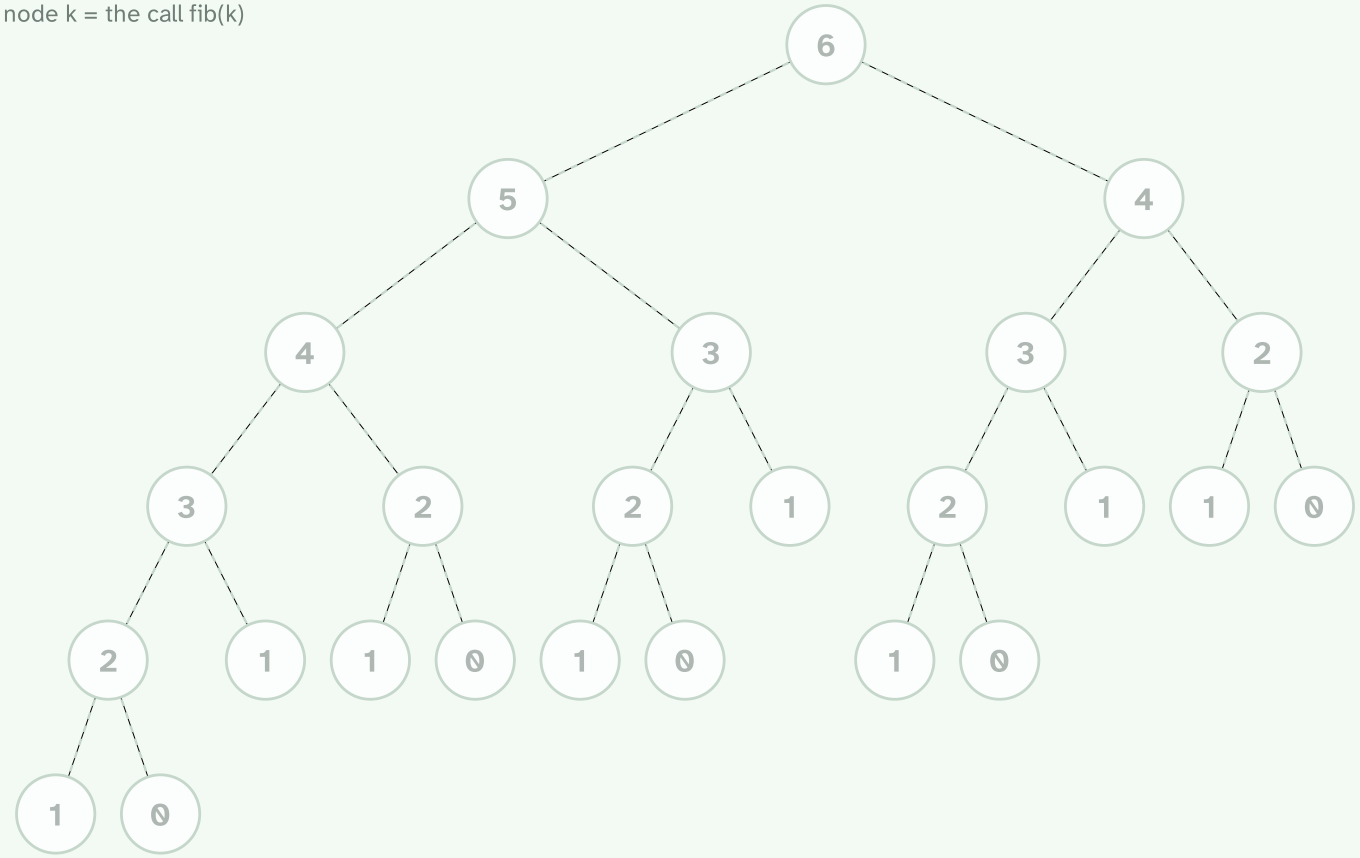
The ratio $a/b^d = 1$, so every level does exactly the same work, n. There are $\log_2 n + 1 = 5$ levels, which contributes the $\log n$ factor.

Lab: Fibonacci call tree and memoization

INTERACTIVE LAB

Memoization stores each answer the first time it is computed and returns the stored copy on every later call. Step through naive recursion and count how often each call repeats (the overlapping subproblems), then switch to the memoized version. Nodes with the same number are the same question asked again.

node k = the call fib(k)



Version: Naive | Memoized

n: 2 | 3 | 4 | 5 | 6 | 7 | 8

◀ Back | Step ▶ | ▶ Play | Show all | Reset

0	25
CALLS SO FAR (OF 25)	NAIVE TOTAL = 2F7 − 1

HOW OFTEN EACH CALL HAS RUN SO FAR

fib(6)		0/1
fib(5)		0/1
fib(4)		0/2
fib(3)		0/3
fib(2)		0/5
fib(1)		0/8
fib(0)		0/5

MEMO TABLE

none: the naive version forgets every answer

The counts in the bars are Fibonacci numbers: fib(1) runs F6 = 8 times. The total 25 = 2F7 − 1 roughly multiplies by 1.618 each time n grows by 1, the golden ratio from Binet's formula.

Press **Step** or **Play**. The first call is `fib(6)`.

Code: naive versus memoized Fibonacci

CODE WALKTHROUGH

Both functions compute the same numbers. One counts its calls, the other caches its answers. Run it, then compare the call counts with the lab on the previous slide.

fib_memo.py

Read | Edit & run | Trace

```
1  from functools import lru_cache
2  calls = 0
3  def fib(n):
4      global calls
5      calls += 1
6      if n < 2:
7          return n
8      return fib(n - 1) + fib(n - 2)
9
10 @lru_cache(maxsize=None)
11 def fib_memo(n):
12     if n < 2:
13         return n
14     return fib_memo(n - 1) + fib_memo(n - 2)
15 print("fib(8) =", fib(8), "using", calls, "calls")
16 print("memo:", fib_memo(8), fib_memo.cache_info())
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
fib(8) = 21 using 67 calls
memo: 21 CacheInfo(hits=6, misses=9, maxsize=None, currsize=9)
```

LINE 1 OF 16

Show all notes in code

```
from functools import lru_cache
```

Imports `lru_cache` from the standard-library module `functools`. It is a **decorator** that adds memoization to a function. LRU stands for "least recently used", the rule it follows if a size limit forces it to forget old entries.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Computes $F_8 = 21$ twice: once with plain recursion while counting every call, and once with Python's built-in memoization decorator `lru_cache`, which reports how many calls it answered from memory.

Code: naive versus memoized Fibonacci: every line explained

1	`from functools import lru_cache`	Imports `lru_cache` from the standard-library module `functools`. It is a **decorator** that adds memoization to a function. LRU stands for "least recently used", the rule it follows if a size limit forces it to forget old entries.
2	`calls = 0`	A counter for the naive version, starting at zero. Counting calls turns the recurrence $C_n = C_{n-1} + C_{n-2} + 1$ into a number you can see.
3	`def fib(n):`	The naive recursive Fibonacci function, written exactly like the mathematical definition. It is clear and correct, and very slow for large `n`.
4	`    global calls`	Tells Python that `calls` inside this function means the variable defined at the top of the file. Without this line, `calls += 1` would try to update a local variable that does not exist yet and fail with an `UnboundLocalError`.
5	`    calls += 1`	Counts this call. Every call adds one, including the calls that stop at a base case.
6	`    if n < 2:`	The base cases $F_0 = 0$ and $F_1 = 1$. The test `n < 2` covers both at once and guarantees the recursion stops.
7	`        return n`	Returns 0 when `n` is 0 and 1 when `n` is 1, which are exactly the initial conditions.
8	`    return fib(n - 1) + fib(n - 2)`	The recurrence $F_n = F_{n-1} + F_{n-2}$. The problem is that `fib(n - 1)` will itself compute `fib(n - 2)` again, so the same subproblems are solved over and over. The number of calls grows by a factor of about 1.618 every time `n` grows by 1.
10	`@lru_cache(maxsize=None)`	Decorates the function defined on the next line. `maxsize=None` means the cache never forgets anything. From now on, a call with an argument seen before returns the stored result immediately, without running the body.
11	`def fib_memo(n):`	The same function under a new name, so the two versions can be compared side by side. Apart from the counter, its body is identical to `fib`: the decorator on the line above is the only real change.
12	`    if n < 2:`	The same base-case test as before, covering F_0 and F_1.
13	`        return n`	Returns 0 or 1 for the base cases. With the cache in place, each base case runs at most once.
14	`    return fib_memo(n - 1) + fib_memo(n - 2)`	The same recurrence. The difference is invisible in the code: when `fib_memo(n - 2)` runs after `fib_memo(n - 1)` has finished, its answer is already stored, so the call returns at once. Each value from 0 to `n` is computed only once: $n + 1$ real computations instead of exponentially many.
15	`print("fib(8) =", fib(8), "using", calls, "calls")`	Runs the naive version and prints `fib(8) = 21 using 67 calls`. 67 matches the formula $2F_9 - 1 = 2 \cdot 34 - 1$. Python evaluates the arguments left to right, so `calls` is read after `fib(8)` has finished.
16	`print("memo:", fib_memo(8), fib_memo.cache_info())`	Runs the memoized version and prints its cache statistics. `misses=9` are the real computations, one for each of $n = 0, \ldots, 8$. `hits=6` are calls answered from the cache. That is 15 calls instead of 67, and for $n = 30$ it is 59 calls instead of 2,692,537.

Design practice: recursion you can trust

DESIGN PRACTICE

Recurrences are how professionals reason about recursive code before it runs. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Write the base case first**

02 Make every call move toward a base case

03 Memoize overlapping subproblems

04 Prefer bottom-up dynamic programming for predictable memory

05 Convert deep recursion into iteration

06 Derive the recurrence before you ship

07 Test the closed form against the recurrence

Principle 1 of 7

Write the base case first

A recursive function without a reachable base case never stops: Python gives up after about 1000 nested calls with a `RecursionError`. Write the base case before the recursive line and test it on its own. It is the initial condition of the recurrence, written in code.

✗ AVOID THIS

```
def total(xs):
    return xs[0] + total(xs[1:])
# no base case: crashes when xs runs out
```

✓ DO THIS

```
def total(xs):
    if not xs:          # base case first
        return 0
    return xs[0] + total(xs[1:])
```

Design practice: recursion you can trust: all principles (1/2)

Write the base case first

A recursive function without a reachable base case never stops: Python gives up after about 1000 nested calls with a `RecursionError`. Write the base case before the recursive line and test it on its own. It is the initial condition of the recurrence, written in code.

```
def total(xs):
    if not xs:          # base case first
        return 0
    return xs[0] + total(xs[1:])
```

Make every call move toward a base case

Each recursive call must work on a strictly smaller input, and the shrinking must land exactly on a base case. A step of 2 can jump straight past a base case of 0 when the input is odd. Ask two questions: what gets smaller on every call, and can it skip over the base case?

```
def countdown(n):
    if n <= 0:          # catches every value
        return
    countdown(n - 2)
```

Memoize overlapping subproblems

When the call tree asks the same question many times, cache the answers. `@lru_cache` turns naive Fibonacci from $\Theta(\varphi^n)$ calls into $\Theta(n)$. The arguments must be hashable (numbers, strings, tuples), because the cache is a dictionary keyed by them.

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n < 2: return n
    return fib(n - 1) + fib(n - 2)
```

Prefer bottom-up dynamic programming for predictable memory

Top-down memoization still nests calls as deep as n and stores every answer. Bottom-up dynamic programming computes the subproblems in order from smallest to largest with a loop, so there is no call stack to overflow, and you can keep only what the next step needs. Fibonacci needs just the last two values: constant memory.

```
def fib(n):
    a, b = 0, 1             # F(0), F(1)
    for _ in range(n):
        a, b = b, a + b     # slide the window
    return a
```

Convert deep recursion into iteration

Python limits recursion depth (1000 by default) because every call uses memory on the call stack. A recursion that goes one level deeper per item crashes on a list of 100,000 items. Rewrite it as a loop, or keep your own stack in a list. Raising the limit with `sys.setrecursionlimit` only moves the crash further away.

```
def length(node):
    count = 0
    while node is not None:
        count, node = count + 1, node.next
    return count
```

Design practice: recursion you can trust: all principles (2/2)

Derive the recurrence before you ship

Write the recurrence for each recursive function and solve it. Hidden costs show up immediately. Binary search that slices the list copies half of it on every call: $T(n) = T(n/2) + n$, which is $\Theta(n)$ and no faster than a plain scan. Passing indices instead keeps $T(n) = T(n/2) + 1 = \Theta(\log\ n)$.

```
def has(xs, x, lo, hi):   # T(n) = T(n/2) + 1
    if lo >= hi: return False
    m = (lo + hi) // 2
    if xs[m] == x: return True
    if x < xs[m]: return has(xs, x, lo, m)
    return has(xs, x, m + 1, hi)
```

Test the closed form against the recurrence

A closed form derived by hand can hide a slip: an exponent off by one, a wrong constant. Compute the first twenty terms straight from the recurrence and compare them with the formula. It takes three lines and catches most algebra mistakes before they reach a report or a capacity estimate.

```
def hanoi_moves(n):           # straight from
the rule
    return 1 if n == 1 else 2 * hanoi_moves(n
- 1) + 1

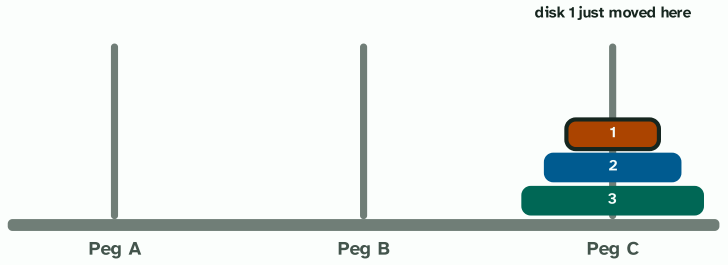
for n in range(1, 20):
    assert hanoi_moves(n) == 2**n - 1, n
```

Worked example: Tower of Hanoi in 7 moves

WORKED EXAMPLE

PROBLEM

The Tower of Hanoi starts with 3 disks on peg A, largest at the bottom, and no disk may ever rest on a smaller one. List and picture every move of the fastest solution that transfers the whole tower to peg C, and count the moves.



After move 7 of 7: disk 1 moved A → C

Moves so far: 1. disk 1 A→C · 2. disk 2 A→B · 3. disk 1 C→B · 4. disk 3 A→C · 5. disk 1 B→A · 6. disk 2 B→C · 7. disk 1 A→C

Answer: 7 moves = 2^3 − 1 · check: T(3) = 2·T(2) + 1 = 7, and the tower stands in size order on peg C

1. All three disks start on peg A, largest at the bottom, and the goal is the same tower on peg C. To free the largest disk, the two smaller disks above it must first become one tower on the spare peg B.
2. Move 1: disk 1 goes from A to C, because it is the top disk on peg A and peg C is empty, so it can land there. It belongs to the first smaller tower, gathering the 2 smaller disks on one peg so the largest disk can move.
3. Move 2: disk 2 goes from A to B, because it is the top disk on peg A and peg B is empty, so it can land there. It belongs to the first smaller tower, gathering the 2 smaller disks on one peg so the largest disk can move.
4. Move 3: disk 1 goes from C to B, because it is the top disk on peg C and it lands on the larger disk 2, so the move is legal. It belongs to the first smaller tower, gathering the 2 smaller disks on one peg so the largest disk can move.
5. Move 4: disk 3 goes from A to C, because it is the top disk on peg A and peg C is empty, so it can land there. This is the largest disk's one move, the +1 between the two smaller towers in the recurrence T(n) = 2T(n−1) + 1.
6. Move 5: disk 1 goes from B to A, because it is the top disk on peg B and peg A is empty, so it can land there. It belongs to the second smaller tower, rebuilding the 2 smaller disks on top of the largest disk.
7. Move 6: disk 2 goes from B to C, because it is the top disk on peg B and it lands on the larger disk 3, so the move is legal. It belongs to the second smaller tower, rebuilding the 2 smaller disks on top of the largest disk.
8. Move 7: disk 1 goes from A to C, because it is the top disk on peg A and it lands on the larger disk 2, so the move is legal. It belongs to the second smaller tower, rebuilding the 2 smaller disks on top of the largest disk.
9. **Answer: the tower takes 7 moves, checked by the recurrence T(3) = 2·T(2) + 1 = 2·3 + 1 = 7, which is also 2^3 − 1. Every disk ended in size order on peg C, and no disk ever rested on a smaller one during the sequence.**

Worked example: iterating a recurrence to a closed form

WORKED EXAMPLE

PROBLEM

A sequence starts with $a_0 = 0$ and follows $a_n = 2a_{n-1} + 1$ for $n \geq 1$. Compute the first terms by iteration, guess a closed form for a_n, and prove the guess by induction.

$a_0 = 0, \quad a_n = 2a_{n-1} + 1$

n	a_n	$a_n + 1$
0	0	1
1	1	2
2	3	4
3	7	8
4	15	16

Guess: $a_n + 1 = 2^n$, so $a_n = 2^n - 1$

Base case: $a_0 = 2^0 - 1 = 0$ ✓

Assume $a_k = 2^k - 1$ for some $k \geq 0$.

Then $a_{k+1} = 2a_k + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$.

Answer: a(n) = 2^n – 1 · check: a4 = 2^4 – 1 = 15 matches the table, and T(3) = 7 = a3

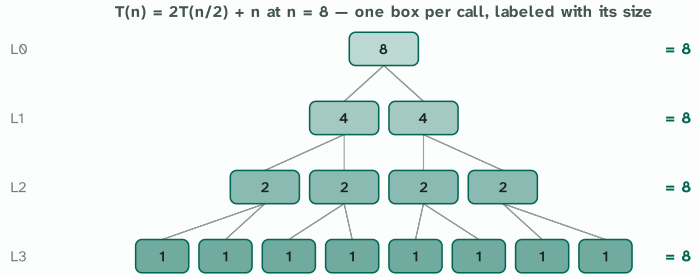
1. Setup: the starting value is a0 = 0 and every later term follows a(n) = 2·a(n–1) + 1, so iteration means just applying the rule repeatedly. The extra a(n) + 1 column is there because the shifted values turn out to follow a much cleaner pattern than the terms themselves.
2. Two terms appear: a1 = 2·0 + 1 = 1 and a2 = 2·1 + 1 = 3, filling the table down from the starting value. Each term needs only the previous one, which is what makes iteration cheap.
3. Two more terms appear: a3 = 2·3 + 1 = 7 and a4 = 2·7 + 1 = 15, completing the first five terms. The a(n) + 1 column now reads 1, 2, 4, 8, 16 — pure powers of 2.
4. The pattern suggests the guess a(n) + 1 = 2^n, so a(n) = 2^n – 1 for every n. A pattern in five terms is a conjecture, not a proof, and induction is the right tool because the claim covers all n while the recurrence links each n to the one before.
5. Induction checks the guess: the base case a0 = 2^0 – 1 = 0 matches the starting value outright. If a(k) = 2^k – 1 holds for some k, then a(k+1) = 2(2^k – 1) + 1 = 2^(k+1) – 1 after simplifying, so the formula reproduces itself through the recurrence.
6. **Answer: a(n) = 2^n – 1, and the spot check a4 = 2^4 – 1 = 15 matches the iterated table exactly. This is the same count as the Hanoi moves with a shift of index, since the three-disk tower takes T(3) = 7 = a3 moves.**

Worked example: a recursion tree with level costs

WORKED EXAMPLE

PROBLEM

Merge sort on 8 items satisfies $T(n) = 2T(n/2) + n$ with $T(1) = 1$: two half-size sorts plus n units of merging work. Expand the recursion tree at $n = 8$, compute the work on each level and the leaf total, and add the levels to find $T(8)$.



Level totals: 8 + 8 + 8 + 8 = 32

Answer: T(8) = 32 · check: iterating the recurrence gives T(2) = 4, T(4) = 12, T(8) = 32, the same total

1. Setup: solve T(n) = 2T(n/2) + n with T(1) = 1 at n = 8 by drawing the recursion tree. Each node is one call, labeled with the work that call does outside its recursive calls; the plan is to total every level separately and then add the levels.
2. Level 0 is the root: a single call on size 8. It does 8 units of its own work and hands the rest to 2 calls on size 4, so the level total is 1 × 8 = 8 before any child is even drawn.
3. Level 1 has 2 calls, each on size 4 doing 4 units of work, so its total is 2 × 4 = 8. The call count doubled while the size halved, so this level costs exactly what level 0 cost.
4. Level 2 has 4 calls, each on size 2 doing 2 units of work, for a total of 4 × 2 = 8. The pattern per level is now visible: multiply the call count by 2, halve the size, and the level total follows from those two numbers alone.
5. Level 3 is the bottom: the size has halved down to 1, so these 8 calls are base cases with T(1) = 1 each, and the leaf total is 8 × 1 = 8. The tree has log2 8 + 1 = 4 levels in all: one level per halving, plus the root level.
6. Add the level totals: 8 + 8 + 8 + 8 = 32. Every level costs the same n = 8 and there are log2 8 + 1 = 4 levels, which is exactly why the closed form for this shape is n(log2 n + 1).
7. **Answer: T(8) = 32. The independent check skips the tree and iterates the recurrence itself: T(2) = 2·T(1) + 2 = 4, T(4) = 2·4 + 4 = 12, and T(8) = 2·12 + 8 = 32, matching the level sum exactly.**

Worked example: characteristic roots with initial conditions

WORKED EXAMPLE

PROBLEM

Solve $a_n = 5a_{n-1} - 6a_{n-2}$ with $a_0 = 7$ and $a_1 = 16$ by the characteristic-equation method: form the equation, factor it, write the general solution, and use the initial conditions to fix the constants.

$$a_n = 5a_{n-1} - 6a_{n-2}, \qquad a_0 = 7, \quad a_1 = 16$$

Characteristic equation: $r^2 - 5r + 6 = 0$

Factored: $(r-2)(r-3) = 0$, so $r = 2$ and $r = 3$

General solution: $a_n = b \cdot 2^n + d \cdot 3^n$

$n = 0$: $\quad b + d = 7$

$n = 1$: $\quad 2b + 3d = 16, \quad$ so $\quad b = 5,\ d = 2$

n	0	1	2	3	4
from the rule	7	16	38	94	242
closed form	7	16	38	94	242

Answer: a(n) = 5·2^n + 2·3^n · check: the closed form matches the iterated terms 7, 16, 38, 94, 242

1. Setup: a(n) = 5a(n–1) – 6a(n–2) with a(0) = 7 and a(1) = 16 is linear, homogeneous and order 2, so the characteristic-equation method applies. The plan: guess a solution of the form a(n) = r^n, find which r work, then let the two initial conditions fix the two constants.
2. Substituting the guess a(n) = r^n into the recurrence and dividing by r^(n–2) leaves the characteristic equation r^2 – 5r + 6 = 0. Its roots are exactly the values of r for which r^n solves the recurrence, so factoring this quadratic is the whole next job.
3. It factors as (r – 2)(r – 3) = 0, so the roots are r = 2 and r = 3, two different real numbers. Each root contributes one exponential solution, 2^n and 3^n, and any combination of them also solves the recurrence.
4. The general solution combines the two root solutions with unknown constants: a(n) = b·2^n + d·3^n. Every choice of b and d satisfies the recurrence; the initial conditions pick out the one sequence we actually want.
5. Use a(0) = 7 first: at n = 0 the formula gives b·2^0 + d·3^0 = b + d = 7. That is one equation in two unknowns, so it narrows the possibilities to a line rather than fixing the constants; the second initial condition is still needed.
6. Now a(1) = 16 gives 2b + 3d = 16. Subtracting 2 times the first equation removes b: (3 – 2)d = 16 – 2·7 = 2, so d = 2, and then b = 7 – 2 = 5. Both constants are fixed.
7. **Answer: a(n) = 5·2^n + 2·3^n. Check it against plain iteration of the recurrence, which gives 7, 16, 38, 94, 242, 646, while the closed form gives 7, 16, 38, 94, 242, 646: identical on every term shown, including a(4) = 242, which was never used to fit the constants.**

Practice: recurrence relations

PRACTICE

Numbers are typed plainly. For two answers, separate them with a comma, as in `3, -2`. Sets of roots can be typed like `{2, 3}`.

COMPUTE TERMS

Let $a_n = 2a_{n-1} + 2a_{n-2}$ for $n \geq 2$, with $a_0 = 0$ and $a_1 = 2$. Find a_4.

Type your answer

Check

Hint: each new term uses the two terms just before it.

Show worked solution

New problem ↻

The rule looks back two steps, so start from a_0 and a_1:

$a_2 = 2 \cdot 2 + 2 \cdot 0 = 4$

$a_3 = 2 \cdot 4 + 2 \cdot 2 = 12$

$a_4 = 2 \cdot 12 + 2 \cdot 4 = 32$

So $a_4 = 32$.

PROBLEM TYPE

- Compute terms
- Characteristic equation
- Master theorem
- Recurrences from algorithms

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: recurrence relations: sample problems with solutions

COMPUTE TERMS

Let $a_n = 3a_{n-1} - 2$ for $n \geq 1$, with $a_0 = 2$. Find a_4.

Apply the rule repeatedly:

$a_1 = 3 \cdot 2 - 2 = 4$

$a_2 = 3 \cdot 4 - 2 = 10$

$a_3 = 3 \cdot 10 - 2 = 28$

$a_4 = 3 \cdot 28 - 2 = 82$

So $a_4 = 82$.

CHARACTERISTIC EQUATION

Find both roots of the characteristic equation of $a_n = -a_{n-1} + 6a_{n-2}$. Type them as a set, for example {2, 3}.

Move every term to one side: $r^2 + r - 6 = 0$. Two numbers that add to -1 and multiply to -6 are 2 and -3, so it factors as $(r - 2)(r + 3) = 0$. The roots are 2 and -3, and the general solution is $a_n = b \cdot 2^n + d \cdot (-3)^n$.

MASTER THEOREM

Use the master theorem to find the growth rate of $T(n) = 4T(n/4) + n$.

Here $a = 4$, $b = 4$, $d = 1$. Compute $\log_4 4 = 1$, because $4^1 = 4$. Since $d = 1 = \log_4 4$, the ratio $a/b^d = 1$: every level costs the same n, and there are about $\log n$ levels, so $T(n) = \Theta(n \log n)$.

RECURRENCES FROM ALGORITHMS

The naive recursive `fib(n)` makes $C_n = C_{n-1} + C_{n-2} + 1$ calls, with $C_0 = C_1 = 1$. How many calls does `fib(12)` make?

Building up: $C_0 = 1$, $C_1 = 1$, $C_2 = 3$, $C_3 = 5$, $C_4 = 9$, $C_5 = 15$, $C_6 = 25$, $C_7 = 41$, $C_8 = 67$, $C_9 = 109$, $C_{10} = 177$, $C_{11} = 287$, $C_{12} = 465$. The closed form $C_n = 2F_{n+1} - 1$ agrees: $2 \cdot 233 - 1 = 465$.

Check yourself: recurrence relations

CHECK YOURSELF

QUESTION 1 OF 10

Which recurrence and initial condition produce the sequence $3, 7, 15, 31, \ldots$?

A $a_n = 2a_{n-1} + 1,\ a_0 = 3$

B $a_n = a_{n-1} + 4,\ a_0 = 3$

C $a_n = 2a_{n-1} + 1,\ a_0 = 1$

D $a_n = 2a_{n-1},\ a_0 = 3$

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: recurrence relations: answers explained (1/2)

1. Which recurrence and initial condition produce the sequence $3, 7, 15, 31, \ldots$?

Answer A: $a_n = 2a_{n-1} + 1,\ a_0 = 3$

Check each step: $2 \cdot 3 + 1 = 7$, $2 \cdot 7 + 1 = 15$, $2 \cdot 15 + 1 = 31$. The same rule with $a_0 = 1$ gives $1, 3, 7, 15$: the rule alone does not fix the sequence.

2. What is the minimum number of moves for the Tower of Hanoi with 6 disks?

Answer C: 63

$H_n = 2^n - 1$, so $H_6 = 64 - 1 = 63$. Each extra disk roughly doubles the work: two copies of the smaller tower plus one move.

3. What is the general solution of $a_n = 5a_{n-1} - 6a_{n-2}$?

Answer A: $a_n = b \cdot 2^n + d \cdot 3^n$

The characteristic equation is $r^2 - 5r + 6 = 0$, which factors as $(r - 2)(r - 3) = 0$. Two distinct roots, 2 and 3, give $b \cdot 2^n + d \cdot 3^n$.

4. The characteristic equation of $a_n = 6a_{n-1} - 9a_{n-2}$ is $(r - 3)^2 = 0$. Which form does the solution take?

Answer A: $a_n = b \cdot 3^n + d \cdot n \cdot 3^n$

A repeated root r contributes both r^n and $n\,r^n$. With only $b \cdot 3^n$ you could not match two arbitrary initial conditions.

5. Recursive binary search satisfies $T(n) = T(n/2) + 1$. How does $T(n)$ grow?

Answer C: $\Theta(\log\ n)$

Each call does constant work and halves the problem, so there are about $\log_2\ n$ calls before the size reaches 1. By the master theorem, $a = 1$, $b = 2$, $d = 0 = \log_2\ 1$: the balanced case, $\Theta(n^0\ \log\ n)$.

6. By the master theorem, what is $T(n) = 4T(n/2) + n$?

Answer D: $\Theta(n^2)$

$\log_2\ 4 = 2 > d = 1$, so the leaves dominate: $\Theta(n^{\log_2\ 4}) = \Theta(n^2)$. Each level costs twice the one above it.

7. Which recurrence describes merge sort?

Answer B: $T(n) = 2T(n/2) + n$

Merge sort sorts two halves ($2T(n/2)$) and then merges them with up to n comparisons ($+\,n$). Every level of its recursion tree costs n, so it runs in $\Theta(n\ \log\ n)$.

8. Why is the naive recursive Fibonacci function so slow?

Answer C: It solves the same subproblems over and over, so the number of calls grows exponentially

`fib(n)` recomputes `fib(n-2)` inside `fib(n-1)`, and so on down the tree: $2F_{n+1} - 1$ calls in total. Memoization computes each value once, which makes the work linear in n.

Check yourself: recurrence relations: answers explained (2/2)

9. How many bit strings of length 5 have no two consecutive 1s?

Answer C: 13

$S_n = S_{n-1} + S_{n-2}$ with $S_1 = 2$, $S_2 = 3$ gives $S_3 = 5$, $S_4 = 8$, $S_5 = 13$.

10. Iterating $H_n = 2H_{n-1} + 1$ suggests $H_n = 2^n - 1$. What turns that guess into a proof?

Answer B: A proof by mathematical induction

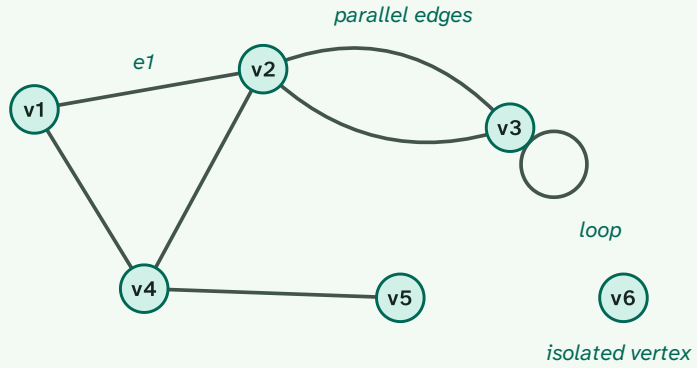
Small cases and experiments build confidence but cover only finitely many n. Induction proves the base case and shows that truth for k forces truth for $k + 1$, which covers every n.

The vocabulary of graphs

CONCEPT

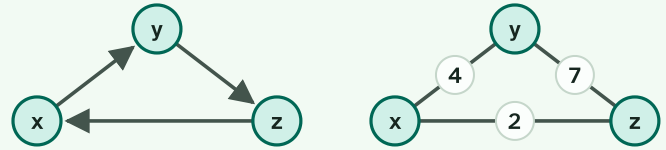
DEFINITION

A graph $G = (V, E)$ has a set V of vertices (also called nodes) and a set E of edges. Each edge joins two vertices, called its endpoints. The edge is *incident* on its endpoints, and two vertices joined by an edge are adjacent.



Edge e_1 is incident on v_1 and v_2, so v_1 and v_2 are adjacent. v_6 touches no edge at all. Where the vertices sit on the page means nothing: only which pairs are joined.

- A loop joins a vertex to itself (at v_3 above).
- Parallel edges join the same pair of vertices (v_2 and v_3 above).
- A simple graph has no loops and no parallel edges. Most graphs in this course are simple.
- In a directed graph every edge has a direction, drawn as an arrow. "Follows" on a social network is directed; a two-way road is not.
- In a **weighted graph** every edge carries a number: a distance, a cost, a travel time.



WHERE THIS SHOWS UP IN AI

A **knowledge graph** stores facts as directed, labeled edges such as `Paris —capital of→ France`. Search engines and AI assistants follow these edges to answer questions, and graph neural networks learn from the same structure.

Degree, the handshake theorem, and special graphs

CONCEPT

DEFINITION

The degree $\deg(v)$ of a vertex is the number of edges incident on it, with a loop counted twice because it touches v at both ends. Johnsonbaugh writes $\delta(v)$.

THEOREM / RULE

Handshake theorem. In every graph,

$$\sum_{v \in V} \deg(v) = 2|E|.$$

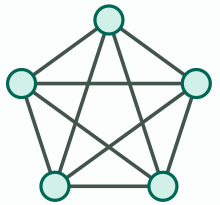
Each edge has two ends, and each end adds 1 to the degree of the vertex it touches.

COROLLARY

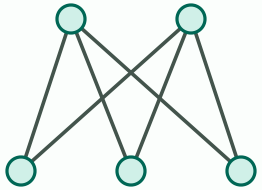
Every graph has an **even** number of vertices of odd degree. An odd number of odd degrees would make the sum odd, but $2|E|$ is even.

WORKED EXAMPLE

Can 7 people each shake hands with exactly 3 of the others? The degrees would add up to $7 \cdot 3 = 21$, an odd number. No graph has an odd degree sum, so it is impossible, however the people try.



K5: 10 edges



K2,3: 6 edges

The complete graph K_n joins every pair of its n vertices, so it has $\binom{n}{2} = \frac{n(n-1)}{2}$ edges.

A bipartite graph splits its vertices into two groups V_1, V_2 so that every edge joins V_1 to V_2. In the complete bipartite graph $K_{m,n}$ each of the m vertices on one side is joined to all n on the other: $m \cdot n$ edges.

WHERE THIS SHOWS UP IN AI

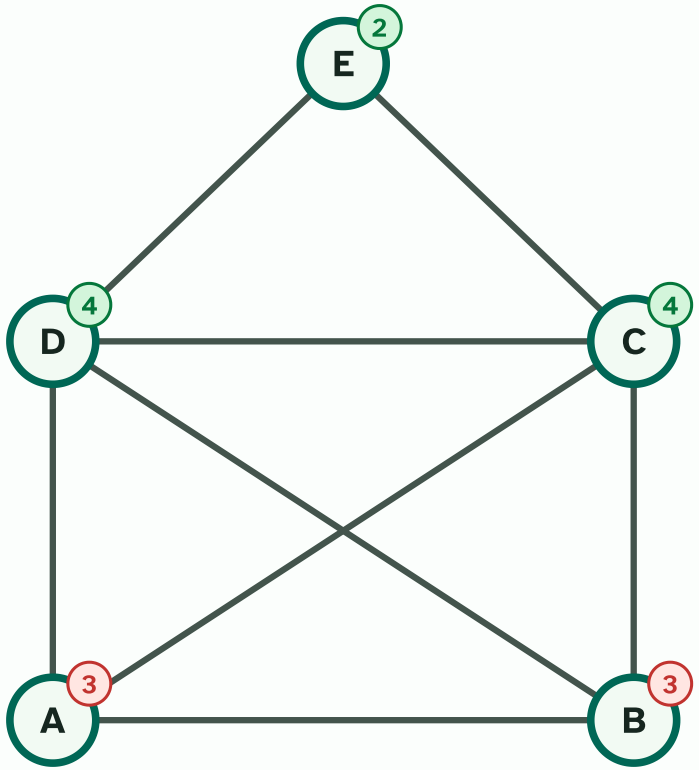
Recommender systems model users and products as a bipartite graph: an edge means "this user bought or rated this product". Suggesting an item means predicting a missing edge, and graph-based models do that by looking at the neighbors of your neighbors.

Lab: graph editor

INTERACTIVE LAB

Click empty space to add a vertex. Click one vertex, then another, to add or remove the edge between them. Drag a vertex to move it. The panel recomputes degrees, the handshake check, the adjacency matrix, the components and the Euler verdict after every change. Red badges mark odd degrees.

Envelope | K_5 | $K_{3,3}$ | Cycle C_6 | Two pieces | Star | Delete selected | Clear



5	8	1
|V| VERTICES	|E| EDGES	COMPONENTS

DEGREES

A: 3 | B: 3 | C: 4 | D: 4 | E: 2

Handshake check: 3 + 3 + 4 + 4 + 2 = 16 = 2 × 8 edges ✓

Odd-degree vertices: A, B (2, always an even count)

ADJACENCY MATRIX (1 = EDGE)

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	1	1	0
C	1	1	0	1	1
D	1	1	1	0	1
E	0	0	1	1	0

Exactly two vertices, A and B, have odd degree. An Euler path exists from one to the other, but no Euler cycle.

Show an Euler trail

Paths, connectedness, and Euler cycles

CONCEPT

DEFINITION

A path of length k from v_0 to v_k is a sequence of vertices $v_0, v_1, \ldots, v_k$ in which each consecutive pair is joined by an edge. The length counts edges. A **simple path** repeats no vertex.

DEFINITION

A cycle is a path of nonzero length that starts and ends at the same vertex and repeats no edge. A **simple cycle** repeats no vertex either, apart from the shared start and end.

DEFINITION

A graph is connected if every pair of vertices is joined by a path. The largest connected pieces of a graph are its **components**. The "Two pieces" preset in the editor has two components.

WHERE THIS SHOWS UP IN AI

Genome assemblers read millions of short, overlapping DNA fragments. They build a graph whose edges are the fragments and rebuild the genome as an Euler path through it, which degree counts make fast to find.

DEFINITION

An Euler cycle uses every edge exactly once and returns to its start. An **Euler path** uses every edge exactly once but may end at a different vertex.

THEOREM / RULE

Euler's theorem. Let G be a connected graph.

- G has an Euler cycle if and only if every vertex has even degree.
- G has an Euler path from v to $w \neq v$, but no Euler cycle, if and only if v and w are the only vertices of odd degree.

WHY IT MATTERS

Why degrees decide. Each time a walk passes through a vertex it uses one edge to arrive and another to leave: two edge ends at a time. Only the start and the end of an open walk can be left with an unmatched edge end. So more than two odd vertices make the walk impossible.

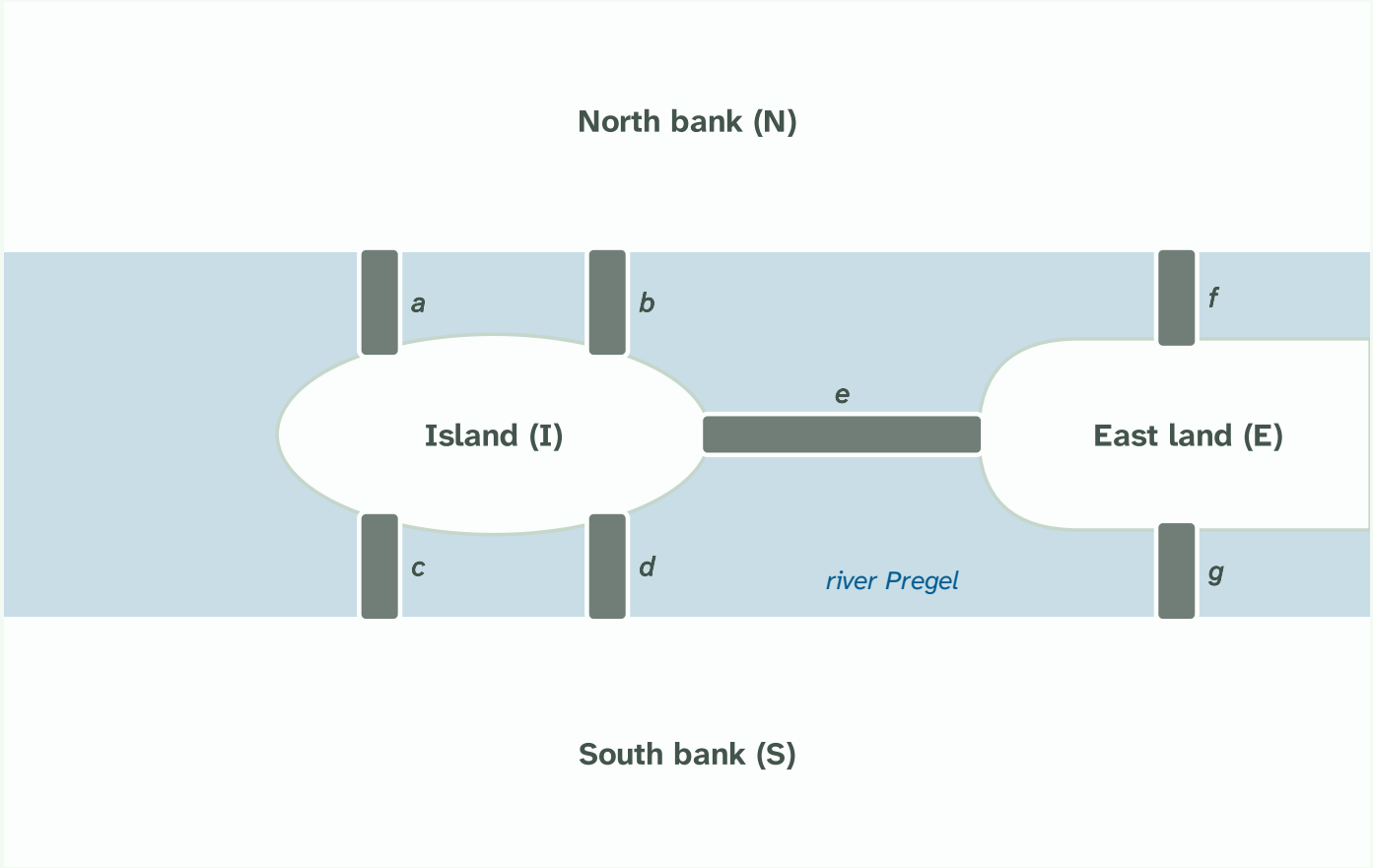
WORKED EXAMPLE

Königsberg (1736): four land masses joined by seven bridges have degrees 5, 3, 3 and 3. All four are odd, so no walk crosses every bridge exactly once. Euler's argument started graph theory. Try it yourself on the next slide.

Lab: the bridges of Königsberg

INTERACTIVE LAB

Click a land mass to choose where your walk starts, then click bridges to cross them. Each bridge may be crossed once. Orange bridges are the ones you can take from where you stand. When you get stuck, open the explanation, or add an eighth bridge and try again.



7 bridges (1736) | Add bridge h

Undo | Restart | Show as a graph | Explain

BRIDGES CROSSED: 0 OF 7

Click a land mass to start.

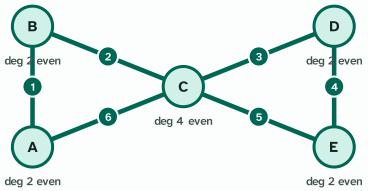
Goal: cross all 7 bridges, each exactly once. You may finish anywhere.

Worked example: Euler audit and building a circuit

WORKED EXAMPLE

PROBLEM

Two triangles share the vertex C: the edges are A-B, B-C, C-A, C-D, D-E and E-C. First audit every degree, then either build an Euler circuit that uses every edge exactly once or explain why none exists.



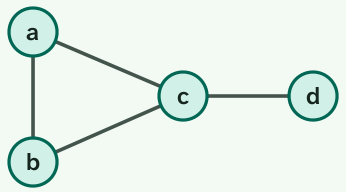
Trail so far: A-B-C-D-E-C-A · 6 of 6 edges used

Answer: all degrees even · Euler circuit A-B-C-D-E-C-A · check: the numbered edges run 1 to 6 with no repeat, and the trail ends back at A ✓

1. Start with the graph and the question: can one trail use every one of its 6 edges exactly once? Do not guess from the picture. First audit every degree, because Euler's theorem says the parity pattern alone decides whether such a trail can exist at all.
2. Audit A: it is joined to B, C, so its degree is 2, which is even, and the running count of odd-degree vertices is now 0. Parity is what matters because a trail passing through a vertex spends its edges two at a time, one arriving and one leaving, so only a trail's two ends can sit at odd-degree vertices.
3. Audit B: it is joined to A, C, so its degree is 2, which is even, and the running count of odd-degree vertices is now 0. Parity is what matters because a trail passing through a vertex spends its edges two at a time, one arriving and one leaving, so only a trail's two ends can sit at odd-degree vertices.
4. Audit C: it is joined to A, B, D, E, so its degree is 4, which is even, and the running count of odd-degree vertices is now 0. Parity is what matters because a trail passing through a vertex spends its edges two at a time, one arriving and one leaving, so only a trail's two ends can sit at odd-degree vertices.
5. Audit D: it is joined to C, E, so its degree is 2, which is even, and the running count of odd-degree vertices is now 0. Parity is what matters because a trail passing through a vertex spends its edges two at a time, one arriving and one leaving, so only a trail's two ends can sit at odd-degree vertices.
6. Audit E: it is joined to C, D, so its degree is 2, which is even, and the running count of odd-degree vertices is now 0. Parity is what matters because a trail passing through a vertex spends its edges two at a time, one arriving and one leaving, so only a trail's two ends can sit at odd-degree vertices.
7. Cross edge A-B, edge 1 of 6, and mark it used so it can never be crossed again. The trail so far is A-B, and building it by splicing detours (Hierholzer's method) guarantees that no unused edge is ever stranded in a part of the graph the trail has left.
8. Cross edge B-C, edge 2 of 6, and mark it used so it can never be crossed again. The trail so far is A-B-C, and building it by splicing detours (Hierholzer's method) guarantees that no unused edge is ever stranded in a part of the graph the trail has left.
9. Cross edge C-D, edge 3 of 6, and mark it used so it can never be crossed again. The trail so far is A-B-C-D, and building it by splicing detours (Hierholzer's method) guarantees that no unused edge is ever stranded in a part of the graph the trail has left.
10. Cross edge D-E, edge 4 of 6, and mark it used so it can never be crossed again. The trail so far is A-B-C-D-E, and building it by splicing detours (Hierholzer's method) guarantees that no unused edge is ever stranded in a part of the graph the trail has left.
11. Cross edge E-C, edge 5 of 6, and mark it used so it can never be crossed again. The trail so far is A-B-C-D-E-C, and building it by splicing detours (Hierholzer's method) guarantees that no unused edge is ever stranded in a part of the graph the trail has left.
12. Cross edge C-A, edge 6 of 6, and mark it used so it can never be crossed again. The trail so far is A-B-C-D-E-C-A, and building it by splicing detours (Hierholzer's method) guarantees that no unused edge is ever stranded in a part of the graph the trail has left.
13. **Answer: every degree is even (odd count 0), so an Euler circuit exists, and here it is: A-B-C-D-E-C-A. It crosses each of the 6 edges exactly once and ends where it started, which is exactly what the audit promised. Check the used-edge numbers on the picture: they run from 1 to 6 with no repeat and no gap.**

Representing a graph in a computer

CONCEPT



A	a	b	c	d
a	0	1	1	0
b	1	0	1	0
c	1	1	0	1
d	0	0	1	0

a: b, c
b: a, c
c: a, b, d
d: c

The adjacency matrix (middle) has a 1 in row i, column j when v_i and v_j are adjacent. For an undirected graph it is symmetric, and each row sums to that vertex's degree. The adjacency list (right) stores only the neighbors of each vertex.

Task	Matrix	List
memory	$\lvert V\rvert^2$ entries	$\lvert V\rvert + 2\lvert E\rvert$ entries
is u adjacent to v?	one lookup	scan u's list
list the neighbors of u	scan a whole row	$\deg(u)$ steps
best for	small or dense graphs	sparse graphs (most real ones)

COMMON PITFALL

A social network with 100 million users and about 200 friends each would need 10^{16} matrix cells, almost all zero. Its adjacency list needs about 2×10^{10} entries. Match the representation to the density of the graph.

THEOREM / RULE

Powers of the adjacency matrix count walks. The entry in row i, column j of A^k is the number of walks of length k from v_i to v_j. (A walk may repeat vertices and edges.)

WORKED EXAMPLE

For the graph on the left,

$$A^2 = \begin{pmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 3 & 0 \\ 1 & 1 & 0 & 1 \end{pmatrix}$$

Row a, column d is 1: the single walk a → c → d. The diagonal holds the degrees, because a walk of length 2 from a vertex back to itself goes out along one edge and straight back.

WHERE THIS SHOWS UP IN AI

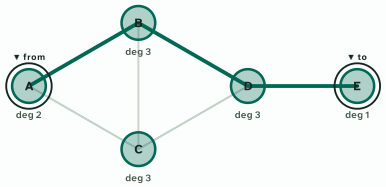
A graph neural network layer updates every node from its neighbors by multiplying the feature table by A. Stacking k layers mixes in information from k steps away, exactly the walks that A^k counts.

Worked example: adjacency list, degrees and a path

WORKED EXAMPLE

PROBLEM

The graph has vertices A, B, C, D, E and edges A-B, A-C, B-C, B-D, C-D and D-E. Build its adjacency list, give the degree of every vertex and check the handshake sum, then decide: is there a path from A to E, and if so, exhibit one?



ADJACENCY LIST

A: B, C
B: A, C, D
C: A, B, D
D: B, C, E
E: D

DEGREES AND HANDSHAKE CHECK

A: 2 B: 3 C: 3 D: 3 E: 1

2 + 3 + 3 + 3 + 1 = 12 = 2 x 6 edges

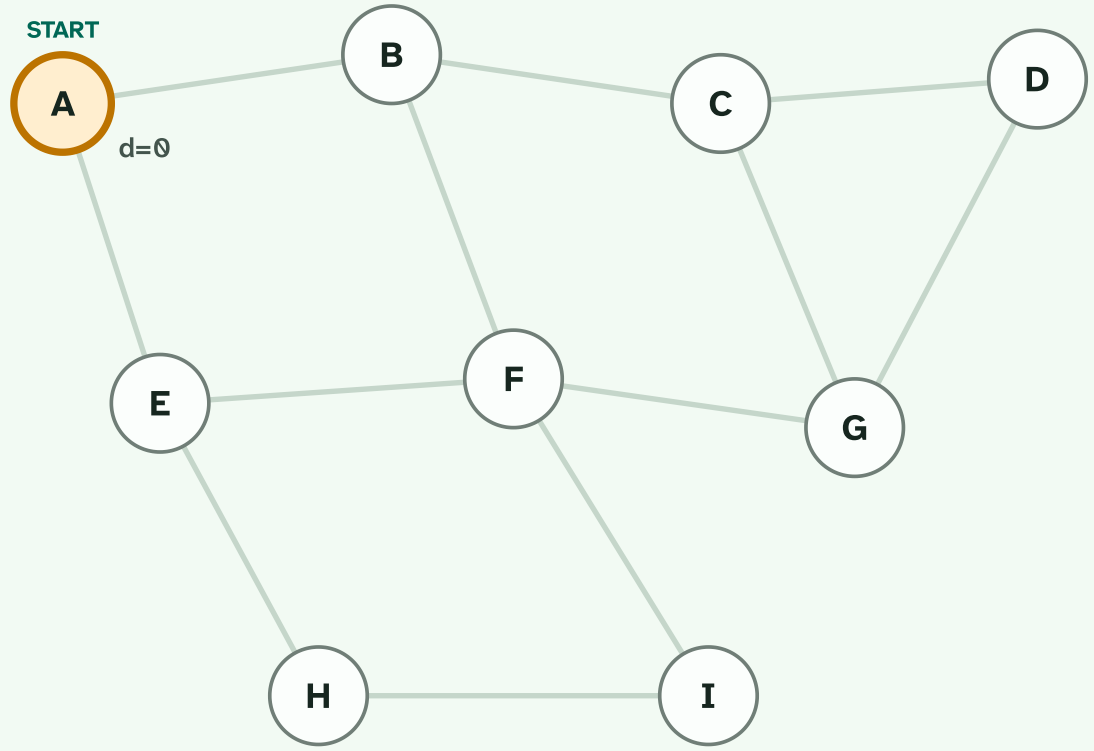
Answer: degrees A 2, B 3, C 3, D 3, E 1 · sum 12 = 2 x 6 · path from A to E: A-B-D-E ✓

1. Start with only the picture: 5 vertices and 6 edges, and no written record of the graph yet. A computer cannot store a picture, so the first job is to turn the drawing into an adjacency list without losing or inventing a single edge.
2. Write the row for A: its neighbors are B, C, because those are exactly the vertices joined to A by an edge in the picture. Every undirected edge will appear twice in the finished list, once in the row of each endpoint, which is what makes the list a faithful copy of the graph.
3. Write the row for B: its neighbors are A, C, D, because those are exactly the vertices joined to B by an edge in the picture. Every undirected edge will appear twice in the finished list, once in the row of each endpoint, which is what makes the list a faithful copy of the graph.
4. Write the row for C: its neighbors are A, B, D, because those are exactly the vertices joined to C by an edge in the picture. Every undirected edge will appear twice in the finished list, once in the row of each endpoint, which is what makes the list a faithful copy of the graph.
5. Write the row for D: its neighbors are B, C, E, because those are exactly the vertices joined to D by an edge in the picture. Every undirected edge will appear twice in the finished list, once in the row of each endpoint, which is what makes the list a faithful copy of the graph.
6. Write the row for E: its neighbors are D, because those are exactly the vertices joined to E by an edge in the picture. Every undirected edge will appear twice in the finished list, once in the row of each endpoint, which is what makes the list a faithful copy of the graph.
7. Count the entries in each row to get the degrees: A has degree 2, B has degree 3, C has degree 3, D has degree 3, E has degree 1. A row length is a degree because each entry is one edge end at that vertex. The degrees add to 12, and the handshake check demands exactly twice the edge count: 2 times 6 is 12, so the count closes with nothing double-counted or missed.
8. Now use the list for the path question: is there a path from A to E? Work outward from A through the rows, marking every vertex you can reach, because a path exists exactly when E gets marked before the reachable set stops growing.
9. **Answer: yes, there is a path: A-B-D-E uses 3 edges, and every consecutive pair in it is an edge, since each step follows one entry of the adjacency list. The full reachable set from A is {A, B, C, D, E}, which contains E, and the degrees and the handshake sum from the earlier steps are part of the same answer.**

Lab: breadth-first versus depth-first search

INTERACTIVE LAB

Both searches visit every vertex reachable from the start. **Breadth-first search** (BFS) keeps a queue and spreads out in rings of equal distance. **Depth-first search** (DFS) keeps a stack and dives as deep as it can before backing up. Click a vertex to start there, then step through and watch the container.



Search: Breadth-first (queue) | Depth-first (stack)

Start: A B C D E F G H I

◀ Back | Step ▶ | ▶ Play | Reset | step 0 of 9

QUEUE: FIRST IN, FIRST OUT

front → A ← back

VISIT ORDER

nothing visited yet

Start: put A in the queue and mark it discovered at distance 0.

Complete orders from A:

BFS: A B E C F H D G I

DFS: A B C D G F E H I

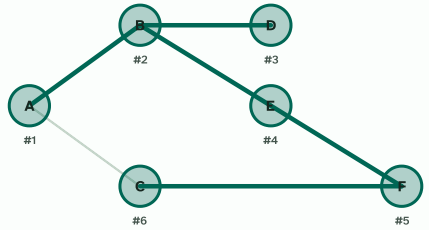
BFS reaches vertices in order of distance, so the thick tree edges are shortest paths (fewest edges) from the start. Use it for "fewest hops" questions: degrees of separation, network routing, and solving puzzles in the fewest moves.

Worked example: depth-first search with a stack

WORKED EXAMPLE

PROBLEM

Run depth-first search from A in the graph with edges A-B, A-C, B-D, B-E, C-F and E-F, using a stack: push the source, then repeatedly pop the top vertex, visit it if it is new, and push its unvisited neighbors in reverse alphabetical order so the smallest neighbor lands on top. Give the discovery order.



STACK (LAST IN, FIRST OUT)

bottom empty ← top

DISCOVERY ORDER

1A 2B 3D 4E 5F 6C

Answer: discovery order A → B → D → E → F → C · check: a recursive DFS that always takes the smallest unvisited neighbor first gives the same order, and the last stale entry was skipped, not revisited ✓

1. Start with A alone on the stack and nothing visited. Depth-first search always takes the vertex on top of the stack next, last in, first out, which is the opposite discipline from the BFS queue and is exactly what makes it dive along one branch instead of spreading in layers.
2. Pop A from the top of the stack and visit it, number 1 in the discovery order. Push its unvisited neighbors C, B in that order, which leaves B on top, so the search dives to B next instead of returning to older waiting vertices.
3. Pop B from the top of the stack and visit it, number 2 in the discovery order. Push its unvisited neighbors E, D in that order, which leaves D on top, so the search dives to D next instead of returning to older waiting vertices.
4. Pop D from the top of the stack and visit it, number 3 in the discovery order. It has no unvisited neighbors, so nothing is pushed: this is a dead end, and the search backtracks to whatever is waiting underneath in the stack.
5. Pop E from the top of the stack and visit it, number 4 in the discovery order. Push its unvisited neighbors F in that order, which leaves F on top, so the search dives to F next instead of returning to older waiting vertices.
6. Pop F from the top of the stack and visit it, number 5 in the discovery order. Push its unvisited neighbors C in that order, which leaves C on top, so the search dives to C next instead of returning to older waiting vertices.
7. Pop C from the top of the stack and visit it, number 6 in the discovery order. It has no unvisited neighbors, so nothing is pushed: this is a dead end, and the search backtracks to whatever is waiting underneath in the stack.
8. Pop C from the top and find it already visited, so skip it and change nothing. A vertex can be pushed by two different neighbors before it is visited, and this visited check on popping is what keeps those stale entries from restarting the search or looping forever.
9. **Answer: the discovery order from A is A, B, D, E, F, C, and the stack is empty, so every vertex the search pushed has been accounted for. Every vertex of the graph was reached, because this graph is connected, so a single DFS from any source visits all of it.**

Code: breadth-first search with a queue

CODE WALKTHROUGH

The adjacency list is a dictionary of sets, and the queue is a `deque`. Switch to **Trace** and watch `queue` and `dist` change: the vertices come out of the queue in order of distance.

bfs.py

Read | Edit & run | Trace

```
1  from collections import deque
2  graph = {"A": {"B", "C"}, "B": {"A", "D"},
3           "C": {"A", "D", "E"}, "D": {"B", "C", "F"},
4           "E": {"C", "F"}, "F": {"D", "E"}}
5
6  def bfs(graph, start):
7      dist = {start: 0}
8      queue = deque([start])
9      while queue:
10         v = queue.popleft()
11         for w in sorted(graph[v]):
12             if w not in dist:
13                 dist[w] = dist[v] + 1
14                 queue.append(w)
15     return dist
16 print(bfs(graph, "A"))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
{'A': 0, 'B': 1, 'C': 1, 'D': 2, 'E': 2, 'F': 3}
```

LINE 1 OF 16

Show all notes in code

```
from collections import deque
```

Imports `deque`, a double-ended queue from the standard library. Adding at the back and removing from the front both take constant time. A plain list would be slow here, because `list.pop(0)` shifts every remaining element.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Breadth-first search from vertex A on a 6-vertex graph, computing the distance (number of edges on a shortest path) from A to every reachable vertex.

Code: breadth-first search with a queue: every line explained (1/2)

1	`from collections import deque`	Imports `deque`, a double-ended queue from the standard library. Adding at the back and removing from the front both take constant time. A plain list would be slow here, because `list.pop(0)` shifts every remaining element.
2	`graph = {"A": {"B", "C"}, "B": {"A", "D"},`	Starts the adjacency list: a dictionary that maps each vertex to the **set** of its neighbors. A is adjacent to B and C. Each undirected edge appears twice, once in each endpoint's set.
3	`"C": {"A", "D", "E"}, "D": {"B", "C", "F"},`	Continues the dictionary. Python lets a dictionary run over several lines as long as the braces are still open. C has degree 3 and so does D.
4	`"E": {"C", "F"}, "F": {"D", "E"}}`	Closes the dictionary: 6 vertices and 7 edges. Check with the handshake theorem: the set sizes add up to $2+2+3+3+2+2 = 14 = 2 \cdot 7$.
6	`def bfs(graph, start):`	Defines the search. It takes the adjacency list and the start vertex and returns a dictionary of distances. Passing the graph in, rather than reading a global, makes the function reusable on any graph.
7	`    dist = {start: 0}`	`dist` maps every discovered vertex to its distance from `start`. It doubles as the **visited set**: a vertex is in `dist` exactly when it has been discovered, so nothing is ever added to the queue twice.
8	`    queue = deque([start])`	The queue of discovered vertices waiting to be processed, first in, first out. It starts with just the start vertex.
9	`    while queue:`	Repeats as long as the queue is not empty. An empty deque counts as false, so the loop stops when every reachable vertex has been processed.
10	`        v = queue.popleft()`	Removes the vertex at the **front** of the queue, the one that has waited longest. Because vertices enter in order of distance, they also leave in order of distance: first A, then the distance-1 vertices, then distance 2, and so on.
11	`        for w in sorted(graph[v]):`	Loops over the neighbors of `v`. `sorted` fixes the order alphabetically, because the order of a set of strings can change from run to run, and a predictable order makes the output reproducible.
12	`            if w not in dist:`	Skips neighbors that were already discovered. Without this check the search would go back and forth along edges forever, because every edge leads back to where it came from.
13	`                dist[w] = dist[v] + 1`	Records that `w` is one step further from the start than `v`. The first time BFS reaches a vertex is always along a shortest path, so this value never needs correcting.
14	`                queue.append(w)`	Adds `w` to the **back** of the queue, so it is processed after every vertex that is closer to the start.
15	`    return dist`	Returns the finished distances once the queue is empty. Vertices that cannot be reached from `start` would simply be missing from the dictionary.

Code: breadth-first search with a queue: every line explained (2/2)

16	`print(bfs(graph, "A"))`	Runs the search from A and prints `{'A': 0, 'B': 1, 'C': 1, 'D': 2, 'E': 2, 'F': 3}`. Dictionaries keep insertion order, so the keys appear in the order BFS discovered them.

Hamiltonian cycles and the traveling salesperson

CONCEPT

DEFINITION

A Hamiltonian cycle visits every *vertex* exactly once and returns to the start. It is named after William Rowan Hamilton, who sold a puzzle in 1857 that asked for such a tour of the corners of a dodecahedron. Try to build one yourself on the next slide.

	Euler cycle	Hamiltonian cycle
uses each...	edge exactly once	vertex exactly once
simple test?	yes: every degree even	none known; Dirac's theorem is only a sufficient condition
fast algorithm?	yes, linear time	none known

DEFINITION

The **traveling salesperson problem** (TSP): in a weighted complete graph, find a Hamiltonian cycle of smallest total weight. Think of the shortest round trip through every city once.

COMMON PITFALL

The two cycle problems sound alike, yet one is easy and one is hard. Always ask which question you are really facing before you promise a fast program.

Why brute force fails

Fix the starting city. The other $n-1$ cities can follow in $(n-1)!$ orders, and each tour is counted twice (once per direction).

cities n	tours $(n-1)!/2$	at a billion tours per second
5	12	instant
10	181,440	under a millisecond
15	43,589,145,600	44 seconds
20	6.1×10^{16}	about 2 years

WHY IT MATTERS

The TSP is **NP-hard**: nobody knows an algorithm that is fast (polynomial time) on every input, and finding one would settle the P versus NP question. Practical solvers use heuristics such as "nearest unvisited city first" and get very close to the best tour.

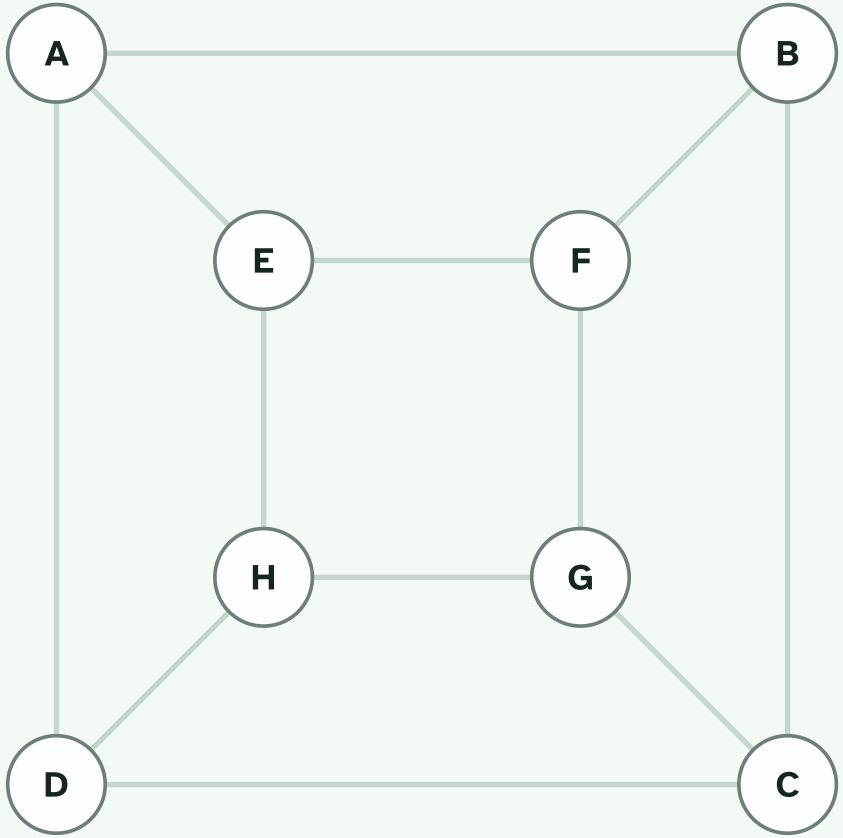
WHERE THIS SHOWS UP IN AI

Delivery companies plan routes every day with TSP heuristics, and researchers train neural networks to build tours. The shortest path between two places is easy by contrast: see the Dijkstra lab.

Lab: find a Hamiltonian cycle

INTERACTIVE LAB

Click a vertex to start, then click neighbors one at a time to build a cycle that visits every vertex once and returns to the start. Dashed edges lead to the legal next moves. Some of these graphs have no Hamiltonian cycle at all: **Explain** says why, and the panel checks Dirac's theorem, the one easy guarantee.



Graph: Prism | Cube | Octahedron | 3 × 3 grid | $K_{2,3}$ | Petersen

Undo | Restart | Show a cycle | Explain

8	12	3
|V| VERTICES	|E| EDGES	SMALLEST DEGREE

Dirac check: smallest degree 3 < n/2 = 4, so a Hamiltonian cycle is **not guaranteed** (one may still exist).

PATH SO FAR: 0 OF 8 VERTICES

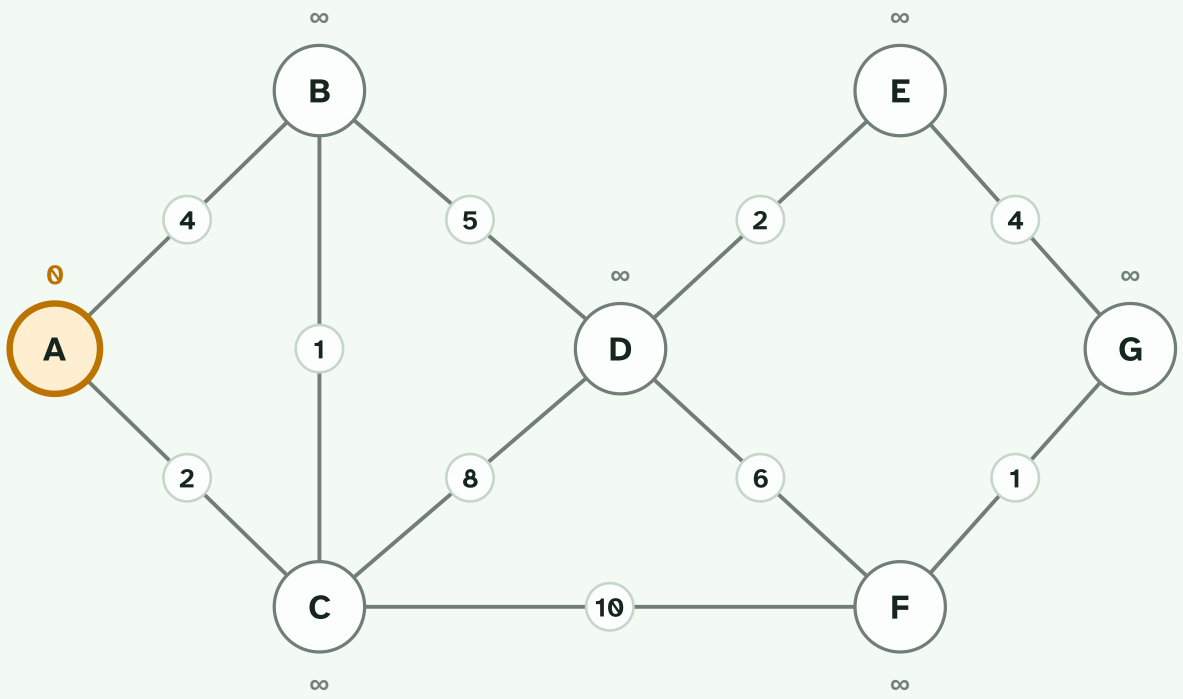
```
click a vertex to start
```

Compare with the Euler check: "all degrees even" is a complete, instant test. No such test is known for Hamiltonian cycles, which is why the traveling salesperson problem is hard.

Lab: Dijkstra's shortest paths, step by step

INTERACTIVE LAB

Dijkstra's algorithm finds the shortest distance from a source to every vertex when all weights are non-negative. Each step **finalizes** the unfinished vertex with the smallest tentative distance, then **relaxes** its edges: if going through it gives a neighbor a shorter route, that neighbor's distance is lowered. Click any vertex to highlight its current best route.



Source A B C D E F G

Step ▶ | Run to the end | Reset | 0 of 7 final

PRIORITY QUEUE (SMALLEST FIRST)

`A:0`

vertex	distance	via	status
A	0	–	tentative
B	∞		unseen
C	∞		unseen
D	∞		unseen
E	∞		unseen
F	∞		unseen
G	∞		unseen

Start: the source **A** has distance 0 and every other vertex ∞ (no route known yet). Press **Step**.

Code: Dijkstra's algorithm with a heap

CODE WALKTHROUGH

The graph is the left half of the lab's graph. A **heap** (Python's `heapq`) always hands back the smallest distance first. Trace it and watch two outdated heap entries get skipped.

dijkstra.py

Read | Edit & run | Trace

```python
import heapq
def dijkstra(graph, source):
    dist, heap = {source: 0}, [(0, source)]
    while heap:
        d, v = heapq.heappop(heap)
        if d > dist[v]:
            continue
        for w, weight in graph[v].items():
            if d + weight < dist.get(w, float("inf")):
                dist[w] = d + weight
                heapq.heappush(heap, (dist[w], w))
    return dist
roads = {"A": {"B": 4, "C": 2}, "D": {"B": 5, "C": 8},
         "B": {"A": 4, "C": 1, "D": 5},
         "C": {"A": 2, "B": 1, "D": 8}}
print(dijkstra(roads, "A"))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
{'A': 0, 'B': 3, 'C': 2, 'D': 8}
```

LINE 1 OF 16

Show all notes in code

```python
import heapq
```

Imports `heapq`, the standard-library heap. A heap is a list kept in a special order so that the smallest item can always be removed in $O(\log\ n)$ time. That is exactly the "closest unfinished vertex" question Dijkstra asks at every step.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Dijkstra's algorithm on a 4-vertex weighted graph, stored as a dictionary of dictionaries, computing the shortest distance from A to every vertex.

Code: Dijkstra's algorithm with a heap: every line explained (1/2)

1	`import heapq`	Imports `heapq`, the standard-library heap. A heap is a list kept in a special order so that the smallest item can always be removed in $O(\log\ n)$ time. That is exactly the "closest unfinished vertex" question Dijkstra asks at every step.
2	`def dijkstra(graph, source):`	Defines the algorithm. `graph` maps each vertex to a dictionary of `{neighbor: weight}`, and `source` is where the distances are measured from.
3	`    dist, heap = {source: 0}, [(0, source)]`	Two variables at once. `dist` holds the best distance found so far for each vertex (the source is 0 from itself). `heap` starts with one entry, the pair (distance 0, vertex `source`). Tuples compare by their first item, so the heap orders entries by distance.
4	`    while heap:`	Keeps going while there are entries to examine. Each entry is a claim "this vertex can be reached at this distance".
5	`        d, v = heapq.heappop(heap)`	Removes the entry with the smallest distance and unpacks it into `d` and `v`. This is the vertex Dijkstra finalizes: nothing still in the heap is closer, and with non-negative weights no later route can be shorter.
6	`        if d > dist[v]:`	Checks whether this entry is **stale**. When a vertex's distance improves, a new entry is pushed and the old one stays in the heap. If the popped distance is larger than the best known, the entry is outdated.
7	`            continue`	Skips the stale entry and goes back to the top of the loop. Ignoring old entries this way ("lazy deletion") is simpler than finding and removing them from the heap.
8	`        for w, weight in graph[v].items():`	Loops over every neighbor `w` of `v` together with the weight of the edge between them. `.items()` yields the (key, value) pairs of the inner dictionary.
9	`            if d + weight < dist.get(w, float("inf")):`	The **relaxation** test: is going through `v` shorter than the best route to `w` known so far? `dist.get(w, float("inf"))` returns infinity for a vertex that has not been reached yet, so any real route beats it.
10	`                dist[w] = d + weight`	Records the shorter distance for `w`. In the lab this is an "update" line: for example, B drops from 4 to 3 when the route through C is found.
11	`                heapq.heappush(heap, (dist[w], w))`	Pushes a new entry for `w` with its improved distance. The heap reorders itself so the smallest entry is still on top, which costs $O(\log\ n)$.
12	`    return dist`	Returns the dictionary of shortest distances once the heap is empty, meaning every reachable vertex has been finalized.
13	`roads = {"A": {"B": 4, "C": 2}, "D": {"B": 5, "C": 8},`	The weighted graph as a dictionary of dictionaries, back at the left margin so it is outside the function. From A the road to B costs 4 and the road to C costs 2; from D, roads lead to B (5) and C (8). The order of the keys does not matter to the algorithm.
14	`         "B": {"A": 4, "C": 1, "D": 5},`	The roads from B. The graph is undirected, so every road is listed at both ends: A lists B with weight 4, and B lists A with weight 4.
15	`         "C": {"A": 2, "B": 1, "D": 8}}`	The roads from C, which close the dictionary. These four vertices and five edges are the left half of the lab's graph, so you can check the result against the lab.

Code: Dijkstra's algorithm with a heap: every line explained (2/2)

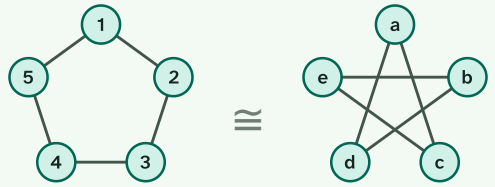
16	`print(dijkstra(roads, "A"))`	Runs the algorithm from A and prints `{'A': 0, 'B': 3, 'C': 2, 'D': 8}`. B is 3, not 4, because A → C → B costs 2 + 1. D is 8 through B, not 10 through C.

Isomorphism and planar graphs

CONCEPT

DEFINITION

Graphs G_1 and G_2 are isomorphic if there is a one-to-one, onto function f from the vertices of G_1 to the vertices of G_2 such that u and v are adjacent exactly when $f(u)$ and $f(v)$ are. They are the same graph, drawn and labeled differently.



pentagon five-point star

f: 1 → a, 2 → c, 3 → e, 4 → b, 5 → d maps every edge to an edge.

DESIGN TIP

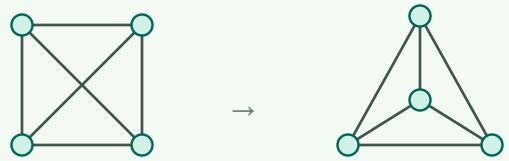
To **disprove** isomorphism, find a graph invariant that differs: vertex or edge count, degree sequence, number of components or triangles. To **prove** it, exhibit f. Matching invariants alone prove nothing; the next lab asks for both.

WHERE THIS SHOWS UP IN AI

Whether two molecules share a bond structure is an isomorphism question in AI-driven drug discovery; circuit-board layout is a planarity question, since wires on one layer must not cross.

DEFINITION

A graph is planar if it can be drawn with no edges crossing. The drawing cuts the plane into faces, including the unbounded outer face.



K4 with a crossing redrawn: no crossing

THEOREM / RULE

Euler's formula. A connected planar graph drawn without crossings has $V - E + F = 2$ (for K_4: $4 - 6 + 4 = 2$). Each face has at least 3 boundary edges and each edge borders at most 2 faces, so $3F \le 2E$; with Euler's formula this gives $E \le 3V - 6$ when $V \ge 3$. K_5 has $10 > 9$ edges: not planar.

WHY K3,3 IS NOT PLANAR

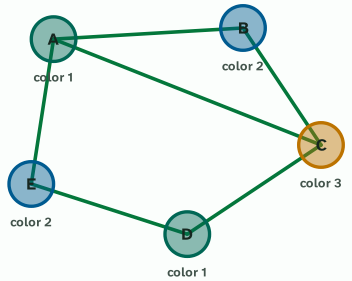
The general bound allows $K_{3,3}$ ($9 \le 12$). But $K_{3,3}$ is bipartite, so it has no triangles, and every face would need at least 4 boundary edges: $4F \le 2E$, so $E \le 2V - 4 = 8$. Nine edges are one too many. Kuratowski's theorem says these are the only two obstacles: a graph is planar if and only if no subgraph is a subdivision of K_5 or $K_{3,3}$ (edges replaced by paths).

Worked example: coloring a graph, checking conflicts

WORKED EXAMPLE

PROBLEM

Color the graph with vertices A, B, C, D, E and edges A-B, B-C, C-D, D-E, E-A and A-C, going in the order A, B, C, D, E and giving each vertex the smallest color (numbered 1, 2, 3, ...) that no already colored neighbor uses. Give the final coloring and the number of colors used.



COLORS USED SO FAR

A: color 1 · B: color 2 · C: color 3 · D: color 1 · E: color 2

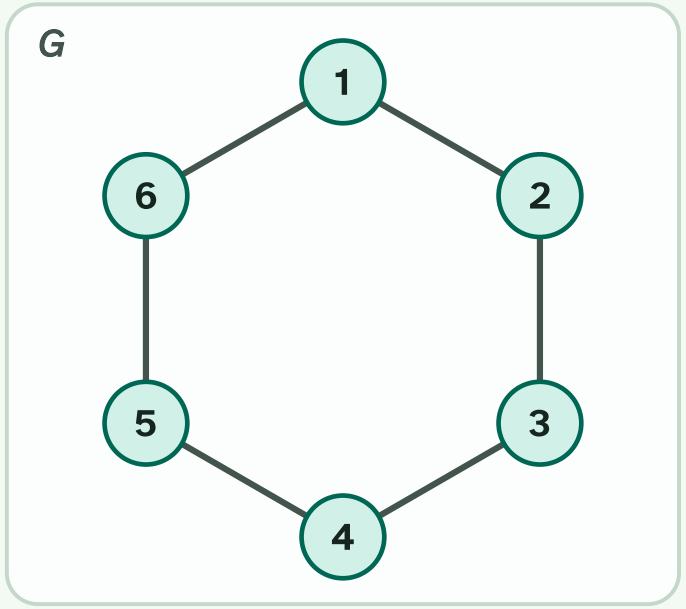
Answer: A color 1 · B color 2 · C color 3 · D color 1 · E color 2 · 3 colors used · check: every edge joins different colors, and A, B and C are pairwise adjacent (a triangle), so they need three different colors all by themselves. ✓

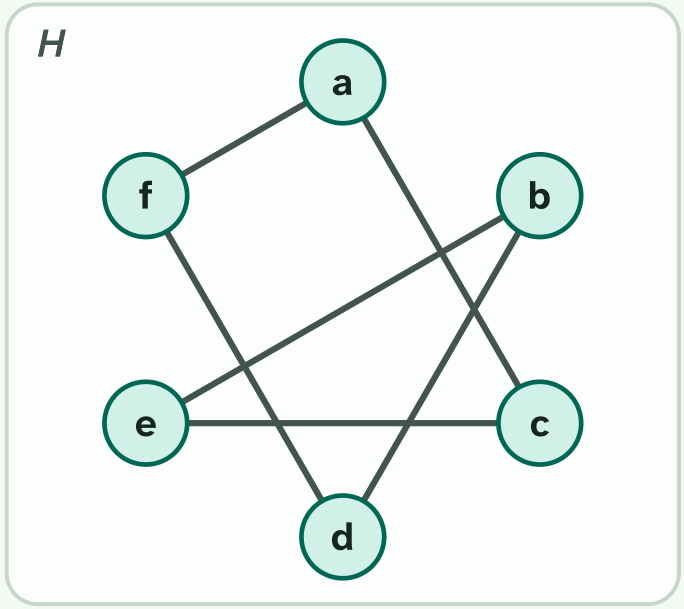
1. Start with every vertex uncolored and the goal of using as few colors as possible, in the order A, B, C, D, E. The rule is greedy: give each vertex the smallest color that none of its already colored neighbors uses, and check the conflicts out loud each time, because a coloring only counts if the two ends of every edge end up different.
2. Color A: none of its neighbors is colored yet, so no color is ruled out and the smallest color, 1, is available. A takes color 1, and later neighbors will have to work around this choice.
3. Color B: its colored neighbors are A with color 1, which rule out color 1 — using any of those would put the same color on both ends of an edge. The smallest color not ruled out is 2, so B takes color 2.
4. Color C: its colored neighbors are A with color 1 and B with color 2, which rule out colors 1 and 2 — using any of those would put the same color on both ends of an edge. The smallest color not ruled out is 3, so C takes color 3.
5. Color D: its colored neighbors are C with color 3, which rule out color 3 — using any of those would put the same color on both ends of an edge. The smallest color not ruled out is 1, so D takes color 1.
6. Color E: its colored neighbors are A with color 1 and D with color 1, which rule out color 1 — using any of those would put the same color on both ends of an edge. The smallest color not ruled out is 2, so E takes color 2.
7. **Answer: the coloring is A gets color 1, B gets color 2, C gets color 3, D gets color 1, E gets color 2, using 3 colors, and every edge joins two different colors, so the coloring is proper. Fewer colors cannot work here: A, B and C are pairwise adjacent (a triangle), so they need three different colors all by themselves.**

Lab: isomorphic or not?

INTERACTIVE LAB

Two drawings, one question: is *H* just *G* with its vertices renamed and moved around? Reveal the invariants one at a time. A single mismatch settles the question with **no**. When everything matches, only an actual vertex mapping settles it with **yes**, and the lab can show one.





Pair 1 2 3 4 5 6 7

invariant	G	H	same?
vertices	?	?	
edges	?	?	
degree sequence	?	?	
triangles	?	?	
components	?	?	

Reveal the first invariant | Reveal all

Your call: Isomorphic | Not isomorphic

Reveal invariants until one differs, or until you are ready to decide. Where a vertex sits on the page means nothing; only which pairs are joined.

Design practice: graphs in real software

DESIGN PRACTICE

Graph code goes wrong in predictable ways: unclear modeling, the wrong representation, missing visited checks. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Model the domain before you write code**

02 Adjacency lists for sparse graphs, matrices for dense ones

03 Add undirected edges in both directions, in one place

04 Always keep a visited set

05 Use a priority queue for Dijkstra

06 Never run Dijkstra with negative weights

07 Use stable, hashable IDs and keep vertex data separate

08 Use a tested library in production

Principle 1 of 8

Model the domain before you write code

Decide on paper what a vertex is, what an edge is, whether edges have a direction, and whether they carry weights. "Ana follows Ben" is directed; "Ana and Ben are friends" is not. Write the decision down next to the data structure, because every algorithm you run later depends on it.

✗ AVOID THIS

```
g = {"ana": ["ben"], "ben": []}
# follows? friends? which direction?
```

✓ DO THIS

```
# Directed: an edge u -> v means "u follows v"
follows = {"ana": {"ben"}, "ben": set()}
```

Design practice: graphs in real software: all principles (1/2)

Model the domain before you write code

Decide on paper what a vertex is, what an edge is, whether edges have a direction, and whether they carry weights. "Ana follows Ben" is directed; "Ana and Ben are friends" is not. Write the decision down next to the data structure, because every algorithm you run later depends on it.

```
# Directed: an edge u -> v means "u follows v"
follows = {"ana": {"ben"}, "ben": set()}
```

Adjacency lists for sparse graphs, matrices for dense ones

Most real graphs are **sparse**: each vertex has a few neighbors out of millions of vertices. An adjacency matrix spends $|V|^2$ memory on mostly zeros, while an adjacency list spends memory only on edges that exist. Choose a matrix for small or dense graphs, or when you need matrix algebra such as A^k.

```
from collections import defaultdict
adj = defaultdict(set)              # only real edges
adj["ana"].add("ben")
```

Add undirected edges in both directions, in one place

An undirected edge must appear in both endpoints' neighbor sets. Forgetting one half creates a graph that is secretly directed: B can no longer reach A, and degree counts break the handshake theorem. Put the rule in one helper function and always use it.

```
def add_edge(adj, u, v):
    adj[u].add(v)
    adj[v].add(u)      # the edge goes both ways
```

Always keep a visited set

Graphs have cycles, so a traversal without a visited check walks around a cycle forever, or until Python raises `RecursionError`. Mark each vertex when you first discover it and check the mark before visiting again. A set gives constant-time checks.

```
def dfs(v, seen):
    seen.add(v)
    for w in adj[v]:
        if w not in seen:
            dfs(w, seen)
```

Use a priority queue for Dijkstra

Finding the closest unfinished vertex by scanning all of them costs $O(|V|)$ per step and $O(|V|^2)$ overall. A heap answers the same question in $O(\log |V|)$, for $O((|V| + |E|) \log |V|)$ in total. On a road map with millions of intersections that is the difference between milliseconds and minutes.

```
d, u = heapq.heappop(heap)        # O(log V)
if d > dist[u]:
    continue                      # stale entry
```

Design practice: graphs in real software: all principles (2/2)

Never run Dijkstra with negative weights

Dijkstra finalizes the closest vertex for good, which is only safe if no later edge can make a route shorter. One negative edge breaks that promise. Example: A→B costs 2, A→C costs 3 and C→B costs −2. Dijkstra finalizes B at 2, but A→C→B costs 1. Check the weights first, and use Bellman–Ford when negatives are possible.

```
if any(w < 0 for nbrs in graph.values()
       for w in nbrs.values()):
    raise ValueError("negative weight: use
Bellman-Ford")
```

Use stable, hashable IDs and keep vertex data separate

Vertices should be small immutable values, such as integers or strings, so they can be dictionary keys and set members. Store the facts about each vertex (name, city, profile) in a separate dictionary keyed by the same ID. Renaming a user then changes one record instead of every edge that mentions them.

```
info = {17: {"name": "Ana", "city": "Miami"}}
adj = {17: {42, 99}}       # structure uses
IDs only
```

Use a tested library in production

For real analysis, use an established library such as `networkx` or a graph database. They provide tested shortest paths, components, centrality, PageRank and isomorphism checks. Write the algorithms yourself to understand them, then rely on a library for code you ship. (This course sticks to the standard library so every example runs anywhere.)

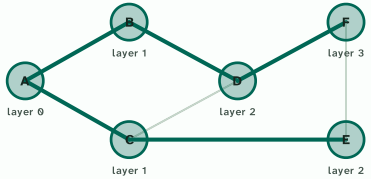
```
import networkx as nx
G = nx.Graph()
G.add_weighted_edges_from([("A", "B", 4),
("A", "C", 2)])
print(nx.shortest_path(G, "A", "B",
weight="weight"))
```

Worked example: breadth-first search order and layers

WORKED EXAMPLE

PROBLEM

Run breadth-first search from vertex A in the graph with vertices A, B, C, D, E, F and edges A-B, A-C, B-D, C-D, C-E, D-F and E-F, visiting neighbors in alphabetical order. Give the visit order and the distance (layer) of each vertex from A.



QUEUE

front → empty ← back

VISIT ORDER

1 A 2 B 3 C 4 D 5 E 6 F

vertex	A	B	C	D	E	F
distance	0	1	1	2	2	3

Answer: visit order A → B → C → D → E → F · distances A 0, B 1, C 1, D 2, E 2, F 3 · check: every edge joins layers at most 1 apart (A-B: 0-1, D-F: 2-3), and F = 3 matches the path A-C-E-F ✓

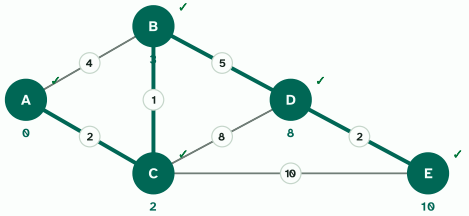
1. Start with the graph and the queue holding only A, marked as the source at layer 0. Nothing is visited yet, because BFS visits a vertex only when it reaches the front of the queue, and that first-in, first-out order is what processes nearer vertices before farther ones.
2. Visit A and enqueue B at layer 1 and C at layer 1, discovered now for the first time along a shortest path from A. A is processed at this moment because it reached the front of the queue, and everything enqueued behind it sits at the same layer or one layer farther out. The queue now holds B, C, and its front is the next vertex to be visited.
3. Visit B and enqueue D at layer 2, discovered now for the first time along a shortest path from A. B is processed at this moment because it reached the front of the queue, and everything enqueued behind it sits at the same layer or one layer farther out. The queue now holds C, D, and its front is the next vertex to be visited.
4. Visit C and enqueue E at layer 2, discovered now for the first time along a shortest path from A. C is processed at this moment because it reached the front of the queue, and everything enqueued behind it sits at the same layer or one layer farther out. The queue now holds D, E, and its front is the next vertex to be visited.
5. Visit D and enqueue F at layer 3, discovered now for the first time along a shortest path from A. D is processed at this moment because it reached the front of the queue, and everything enqueued behind it sits at the same layer or one layer farther out. The queue now holds E, F, and its front is the next vertex to be visited.
6. Visit E and enqueue nothing new, because every neighbor of E has already been discovered. E is processed at this moment because it reached the front of the queue, and everything enqueued behind it sits at the same layer or one layer farther out. The queue now holds F, and its front is the next vertex to be visited.
7. Visit F and enqueue nothing new, because every neighbor of F has already been discovered. F is processed at this moment because it reached the front of the queue, and everything enqueued behind it sits at the same layer or one layer farther out. The queue is now empty, so no discovered vertex is left waiting to be visited.
8. **Answer: the visit order is A, B, C, D, E, F, with distances A 0, B 1, C 1, D 2, E 2, F 3. Check: every edge between two reached vertices joins layers differing by at most 1, which is exactly what a breadth-first layering must satisfy.**

Worked example: Dijkstra shortest distances from A

WORKED EXAMPLE

PROBLEM

Find the shortest distances from A in the weighted graph with edges A-B 4, A-C 2, C-B 1, B-D 5, C-D 8, C-E 10 and D-E 2. Show the tentative distance of every vertex after each vertex is settled, and give the shortest path to E.



SETTLED ORDER

`1A:0` `2C:2` `3B:3` `4D:8` `5E:10`

vertex	A	B	C	D	E
distance	0	3	2	8	10

Answer: distances from A — A 0, B 3, C 2, D 8, E 10 · shortest path to E: A-C-B-D-E = 2 + 1 + 5 + 2 = 10 · check: D via C would be 2 + 8 = 10 > 8, so the route through B is better ✓

1. Setup: A starts at distance 0 and every other vertex is at infinity, meaning no route to it is known yet. Dijkstra settles the closest unsettled vertex next, because with all weights positive no alternative route could reach that vertex more cheaply.
2. Settle A at 0, the smallest tentative label among the unsettled vertices, and relax its edges: B appears at 4 via A, and C appears at 2 via A. The label of A is final from this moment, since any other route to A would have to leave through an unsettled vertex whose label is already at least 0.
3. Settle C at 2, the smallest tentative label among the unsettled vertices, and relax its edges: B improves from 4 to 3 via C, and D appears at 10 via C, and E appears at 12 via C. The label of C is final from this moment, since any other route to C would have to leave through an unsettled vertex whose label is already at least 2.
4. Settle B at 3, the smallest tentative label among the unsettled vertices, and relax its edges: D improves from 10 to 8 via B. The label of B is final from this moment, since any other route to B would have to leave through an unsettled vertex whose label is already at least 3.
5. Settle D at 8, the smallest tentative label among the unsettled vertices, and relax its edges: E improves from 12 to 10 via D. The label of D is final from this moment, since any other route to D would have to leave through an unsettled vertex whose label is already at least 8.
6. Settle E at 10, the smallest tentative label among the unsettled vertices, and relax its edges: no tentative label improves, because every neighbor of E is already settled or already carries a label the route through E cannot beat. The label of E is final from this moment, since any other route to E would have to leave through an unsettled vertex whose label is already at least 10.
7. **Answer: the shortest distances from A are A 0, B 3, C 2, D 8, E 10. The shortest path to E is A-C-B-D-E, and its edges sum to 10, matching the distance of E. Check: the settled order A, C, B, D, E has nondecreasing distances, which is exactly the pattern Dijkstra guarantees.**

Practice: graphs

PRACTICE

Answer with a number unless buttons are shown. Every problem is freshly generated, and the worked solution explains each step.

DEGREES AND EDGES

How many edges does the complete graph K_7 have?

Type your answer

Check

Hint: every pair of vertices is joined by exactly one edge.

Show worked solution

New problem ↻

Each of the $\binom{7}{2}$ pairs of vertices is joined by one edge: $\frac{7 \cdot 6}{2} = 21$. Check with the handshake theorem: each vertex has degree 6, and $7 \cdot 6 = 42 = 2 \cdot 21$.

PROBLEM TYPE

- Degrees and edges
- Euler paths and cycles
- Shortest path (Dijkstra)
- Adjacency matrices
- Isomorphism
- Planar & Hamiltonian

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: graphs: sample problems with solutions

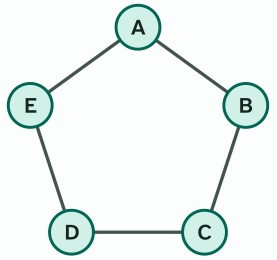
DEGREES AND EDGES

How many edges does the complete graph K_{11} have?

Each of the $\binom{11}{2}$ pairs of vertices is joined by one edge: $\frac{11 \cdot 10}{2} = 55$. Check with the handshake theorem: each vertex has degree 10, and $11 \cdot 10 = 110 = 2 \cdot 55$.

EULER PATHS AND CYCLES

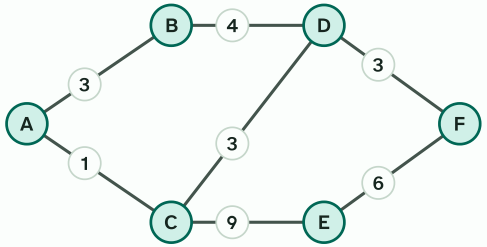
This connected graph has edges AB, AE, BC, CD, DE. Which does it have?



Degrees: A = 2, B = 2, C = 2, D = 2, E = 2. Every degree is even, so by Euler's theorem the graph has an **Euler cycle**.

SHORTEST PATH (DIJKSTRA)

Using Dijkstra's algorithm, find the length of the shortest path from A to F.



Dijkstra finalizes the vertices in this order: A (0), C (1), B (3), D (4), F (7), E (10). The shortest route is A → C → D → F, with length 1 + 3 + 3 = **7**.

ADJACENCY MATRICES

This is the adjacency matrix of a simple graph. How many edges does the graph have?

	a	b	c	d
a	0	0	1	0
b	0	0	1	1
c	1	1	0	0
d	0	1	0	0

Each edge appears twice in the matrix (row u, column v and row v, column u), so count the 1s and halve: 6 / 2 = **3**.

Check yourself: graphs

CHECK YOURSELF

QUESTION 1 OF 12

A graph has five vertices with degrees 3, 3, 2, 2 and 2. How many edges does it have?

A 6

B 12

C It cannot exist

D 5

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

11 12

Pick an answer to see feedback.

Start over

Check yourself: graphs: answers explained (1/2)

1. A graph has five vertices with degrees 3, 3, 2, 2 and 2. How many edges does it have?

Answer A: 6

By the handshake theorem the degrees add to $2|E|$: $3 + 3 + 2 + 2 + 2 = 12$, so $|E| = 6$.

2. Is there a graph with five vertices that all have degree 3?

Answer B: No: the degrees would add up to 15, an odd number

The degree sum is always $2|E|$, which is even. Five vertices of degree 3 add up to 15. (In K_5 every vertex has degree 4, not 3.)

3. How many edges does K_6 have?

Answer B: 15

$\binom{6}{2} = \frac{6 \cdot 5}{2} = 15$. Each of the 6 vertices has degree 5, and $6 \cdot 5 = 30 = 2 \cdot 15$.

4. A connected graph has exactly two vertices of odd degree, u and v. What does Euler's theorem tell you?

Answer B: It has an Euler path from u to v, but no Euler cycle

An Euler cycle needs every degree even. With exactly two odd vertices, an Euler path exists and must start at one of them and end at the other. Euler's theorem says nothing about Hamiltonian cycles.

5. Why can no walk cross each of the seven bridges of Königsberg exactly once?

Answer C: All four land masses touch an odd number of bridges

Each pass through a land mass uses two bridges, so only the start and the finish may touch an odd number. In Königsberg all four land masses touch an odd number (5, 3, 3, 3).

6. What is the difference between an Euler cycle and a Hamiltonian cycle?

Answer A: An Euler cycle uses every edge exactly once; a Hamiltonian cycle visits every vertex exactly once

Euler cycles are about edges and have an instant test (every degree even). Hamiltonian cycles are about vertices and have no known fast test. The two are independent: K_4 has a Hamiltonian cycle but no Euler cycle, because every degree is 3.

7. Why does Dijkstra's algorithm fail when some edge weights are negative?

Answer B: A vertex already finalized might later be reachable more cheaply through a negative edge

Dijkstra never revisits a finalized vertex. That is safe only if adding more edges can never shorten a route. A negative edge discovered later can, so the finalized distance may be wrong. Bellman–Ford handles negative weights.

8. A social network has 100 million users, each with about 200 friends. Which representation fits?

Answer A: An adjacency list

A matrix would need 10^{16} cells, almost all zero. The list stores only the real friendships, about 2×10^{10} entries. Sparse graphs call for adjacency lists.

Check yourself: graphs: answers explained (2/2)

9. In an unweighted graph, what does breadth-first search from a vertex s find?

Answer D: The path from s with the fewest edges to every reachable vertex

BFS processes vertices in order of distance from s, so the first time it reaches a vertex it does so along a path with the fewest possible edges.

10. What does the entry in row i, column j of A^2 count, where A is the adjacency matrix?

Answer D: The number of walks of length 2 from v_i to v_j

$(A^2)_{ij} = \sum_k A_{ik} A_{kj}$ counts the middle vertices k adjacent to both v_i and v_j, that is, the walks $v_i \to v_k \to v_j$. On the diagonal it gives the degree.

11. A connected planar graph has 10 vertices and 15 edges. How many faces does a crossing-free drawing of it have?

Answer A: 7

Euler's formula $V - E + F = 2$ gives $10 - 15 + F = 2$, so $F = 7$, counting the outer face.

12. Why is $K_{3,3}$ not planar?

Answer A: It is bipartite, so it has no triangles, and a planar graph without triangles needs $E \le 2V - 4 = 8$; it has 9 edges

The general bound gives $9 \le 12$ and does not decide. Without triangles every face has at least 4 boundary edges, so $4F \le 2E$ and $E \le 2V - 4 = 8$, one short of the 9 edges of $K_{3,3}$. Kuratowski's theorem names K_5 and $K_{3,3}$ as the two basic obstacles to planarity. Odd degrees matter for Euler cycles, not for planarity.

Trees: connected graphs with no cycles

CONCEPT

DEFINITION

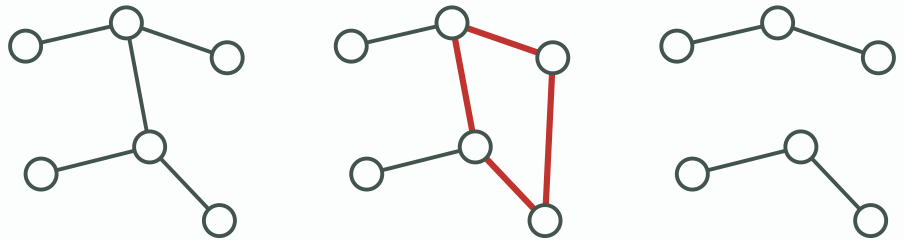
A tree is a connected graph with no cycles. A graph with no cycles that may fall apart into several pieces is a forest: each piece is a tree.

THEOREM / RULE

For a graph T with n vertices, these four statements are equivalent (each one implies the others):

1. T is connected and has no cycles.
2. Between any two vertices there is **exactly one** simple path.
3. T is connected and has exactly $n - 1$ edges.
4. T has no cycles and has exactly $n - 1$ edges.

Why $n - 1$? Grow a tree one vertex at a time. A single vertex needs 0 edges, and each new vertex hooks on with exactly one edge. An extra edge between two vertices that are already connected would create a second path between them, which is a cycle.



Tree
6 vertices, 5 edges

Not a tree: cycle
6 vertices, 6 edges

Forest of 2 trees
6 vertices, 4 edges

COMMON PITFALL

The edge count alone is not enough. A triangle plus one isolated vertex has 4 vertices and 3 edges, yet it has a cycle and is disconnected. Check a second condition too.

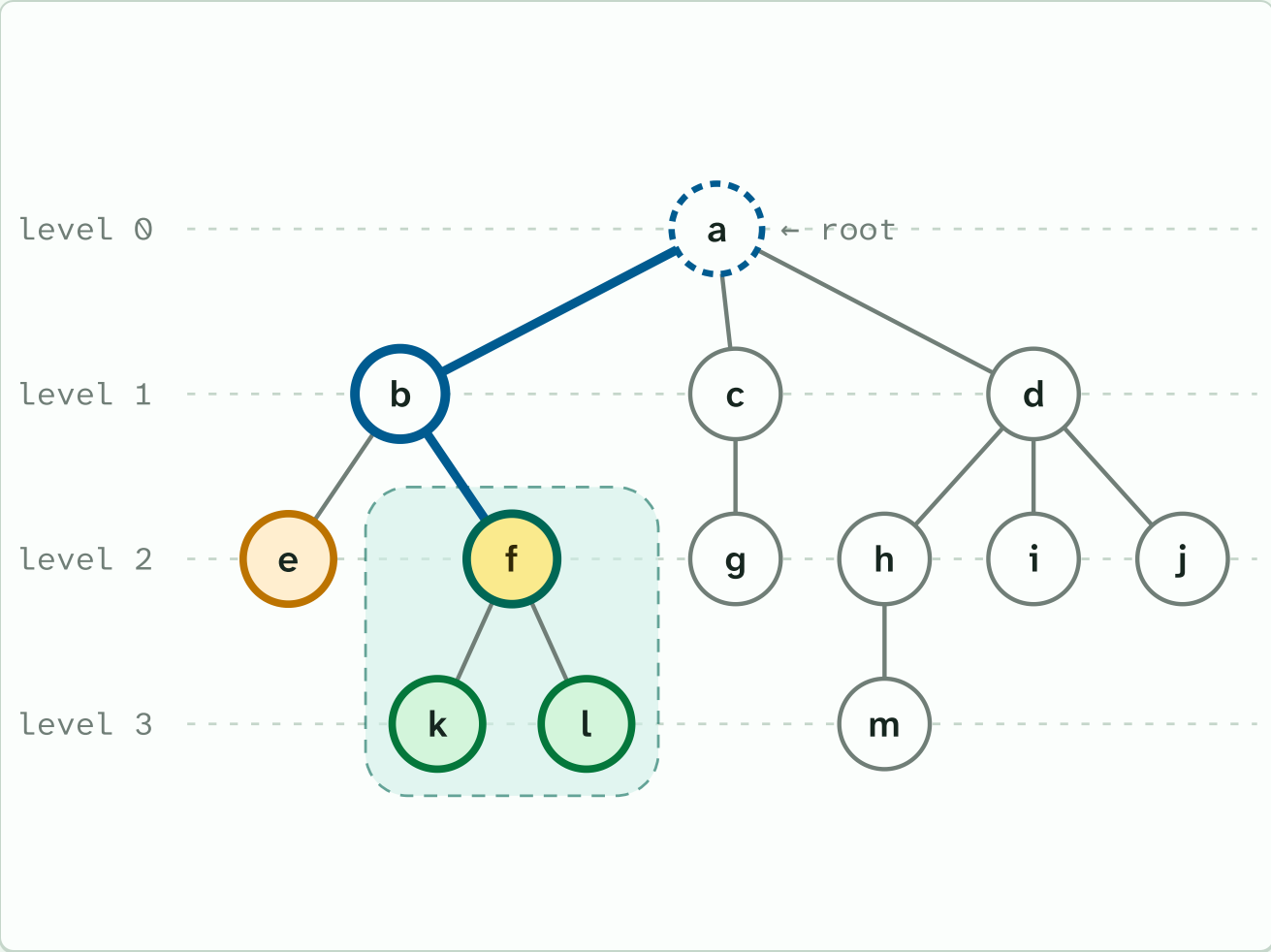
WHERE THIS SHOWS UP IN AI

Folders on a disk, the elements of a web page and the grammar of a sentence are all trees. A decision tree classifier routes each input from the root to one leaf, and a random forest is a forest of such trees whose votes are combined.

Lab: anatomy of a rooted tree

INTERACTIVE LAB

A rooted tree is a tree with one vertex chosen as the root, drawn at the top. Every other vertex then has a parent and a level. Click any vertex to see its relatives and its subtree. Then make it the root: the same tree hangs differently, and every family word changes with it.



Selected · Parent · Children · Siblings · Ancestors (dashed) · Lower descendants (dashed)

Make f the root | Put the root back at a | 13 vertices, 12 edges · 7 leaves · 6 internal

TERM	FOR THE SELECTED VERTEX
Root the vertex with no parent	a
Parent one step closer to the root	b
Child one step farther from the root	k, l
Sibling other children of its parent	e
Ancestor on the path up to the root	b, a
Descendant anywhere below it	k, l
Leaf a vertex with no children	no: an internal vertex
Level edges from the root down to it	2
Subtree f plus all its descendants	{f, k, l}
Height the largest level in the tree	3

When are two trees the same tree?

CONCEPT

DEFINITION

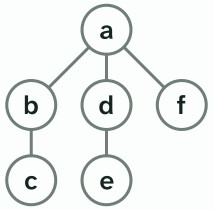
Two trees are **isomorphic** (tree isomorphism) when one can be relabeled into the other: a one-to-one correspondence between their vertices such that two vertices are adjacent in the first tree exactly when their partners are adjacent in the second. Isomorphic trees are one tree drawn twice. For **rooted** trees the correspondence must also send root to root, and therefore children to children.

THEOREM / RULE

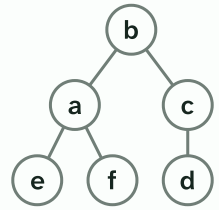
Isomorphic trees agree on every property that ignores the names: the number of vertices and edges, the degree sequence, the length of the longest path, and the height when each is hung from its center. So a single mismatch proves two trees different. Agreement proves nothing: the two trees on the right share all of these numbers and are still different.

WORKED EXAMPLE

A path of three vertices hung from its middle vertex has height 1; hung from an end it has height 2. As plain trees the two drawings are isomorphic. As rooted trees they are not, because the roots are vertices of different degree.



degrees 3, 2, 2, 1, 1, 1 · name `(()(()))(())`



degrees 3, 2, 2, 1, 1, 1 · name `((()))(()())`

Both have 6 vertices, 5 edges, the same degree sequence, a longest path of 4 edges and height 2 from the center. Yet on the left the degree-3 vertex touches two vertices of degree 2, on the right only one, and no relabeling can change that.

CANONICAL NAME

A canonical name is a string that depends only on the shape. Name a leaf `()`, and name every other vertex by writing its children's names in sorted order inside one pair of parentheses. Two rooted trees are isomorphic exactly when their roots get the same name. For unrooted trees, first hang each tree from its center (strip all the leaves, repeat, and the last one or two vertices standing are the center), then compare the names.

WHERE THIS SHOWS UP IN AI

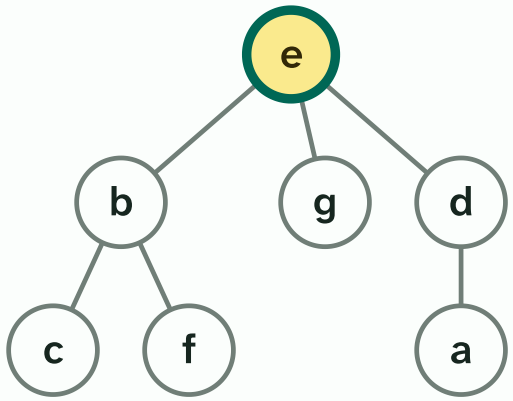
Canonical forms turn "same structure?" into string equality. Chemical databases match molecules that way, compilers find duplicated code by naming syntax trees, and query engines recognize repeated sub-expressions by their canonical form.

Lab: same tree or not?

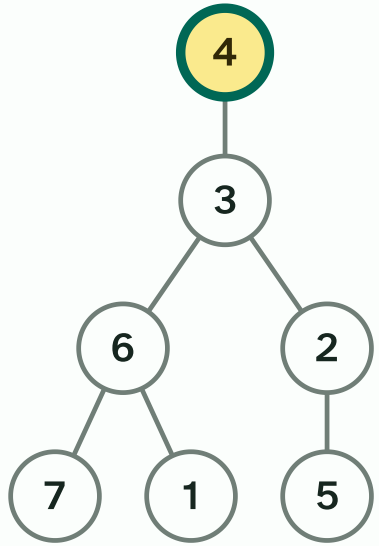
INTERACTIVE LAB

Two trees, drawn differently. Reveal one invariant at a time, then give your verdict. Click a vertex to hang that tree from it: the height changes with the root, which is why the fair comparison hangs both trees from their centers. Some pairs pass every quick check and are still different.

Tree A · root e · height 2



Tree B · root 4 · height 3



Hang both from the center | New pair

REVEAL AN INVARIANT

Vertices, edges | Degree sequence | Height from the center | Canonical name

INVARIANT	TREE A	TREE B
Vertices, edges	?	?
Degree sequence	?	?
Height from the center	?	?
Canonical name	?	?

Your verdict: Isomorphic | Not isomorphic

Spanning trees and minimal spanning trees

CONCEPT

DEFINITION

A spanning tree of a connected graph G is a subgraph that contains **every** vertex of G and is a tree. It keeps $n - 1$ edges: just enough to connect everything, with no redundant loops. Every connected graph has one, because you can delete edges on cycles until no cycle is left.

DEFINITION

In a weighted graph every edge carries a number, such as a cost or a distance. A minimal spanning tree (MST) is a spanning tree T whose total weight $w(T)$ is as small as possible.

THEOREM / RULE

The cut property. Split the vertices into any two groups. The lightest edge between the two groups belongs to some minimal spanning tree. Kruskal and Prim only ever add such an edge, which is why a greedy choice never needs to be undone.

Kruskal's algorithm

1. Sort all the edges from lightest to heaviest.
2. Take each edge in turn. Keep it unless its two ends are already connected, because then it would close a cycle.
3. Stop when you have $n - 1$ edges.

Prim's algorithm

1. Start a tree at any vertex.
2. Add the cheapest edge with exactly one end in the tree.
3. Repeat until every vertex is in the tree.

WHERE THIS SHOWS UP IN AI

Single-link clustering builds an MST over the data points and deletes its $k - 1$ heaviest edges, which leaves k clusters. MSTs also plan the cheapest cabling for a data center or a sensor network.

COMMON PITFALL

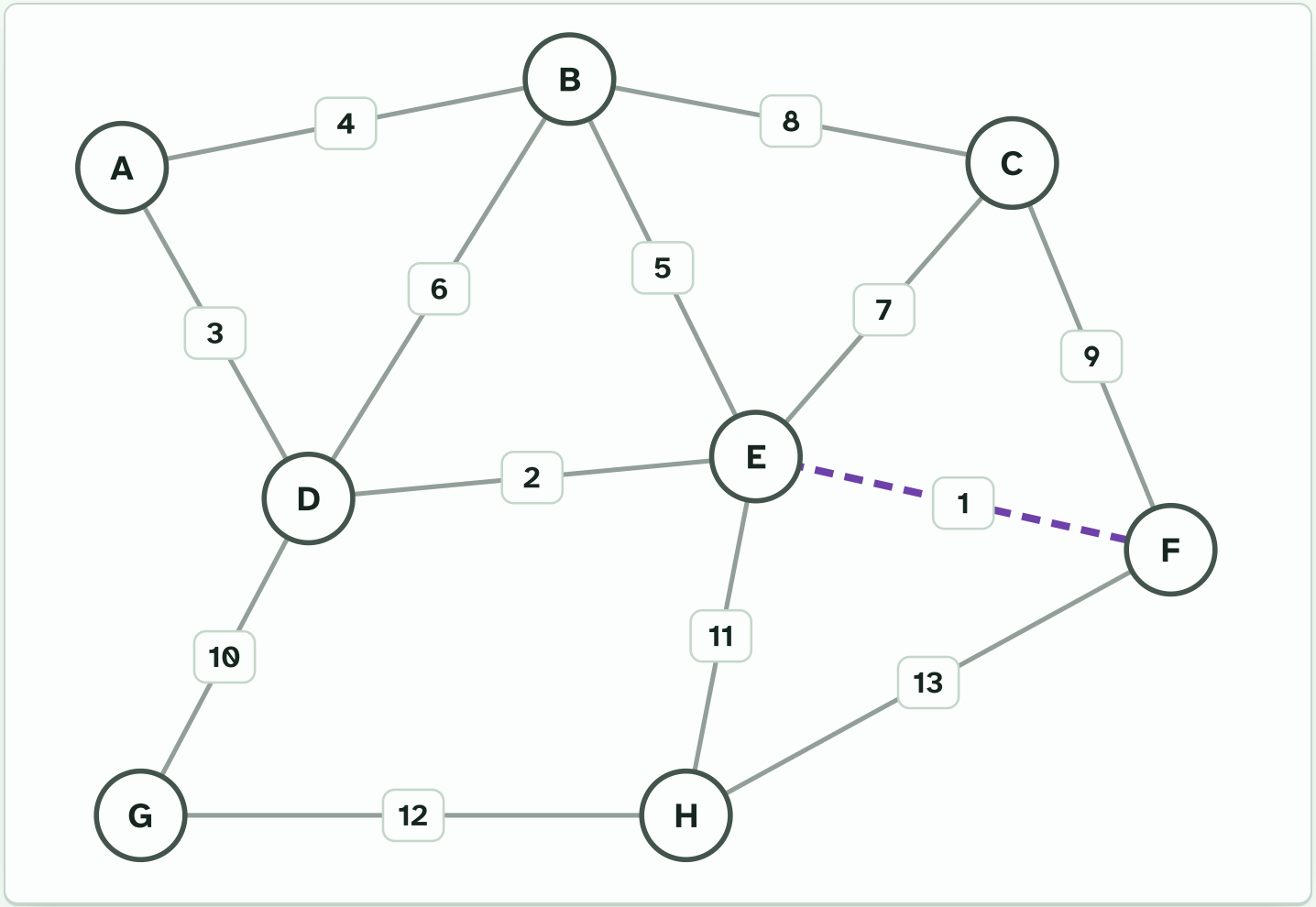
Greedy is not always safe. Planning a delivery route by always driving to the nearest unvisited city can give a tour far longer than the best one. The cut property is what makes greedy choices correct for spanning trees.

Lab: Kruskal and Prim, step by step

INTERACTIVE LAB

Press **Step** to make one greedy choice at a time. Kruskal walks the sorted edge list (the dashed purple edge is next) and rejects any edge that would close a cycle. Prim grows one tree from a start vertex (click a vertex to change it). Both always reach the same total weight when the weights are all different.

Kruskal | Prim | Step ▶ | ▶ Play | Reset | New weights



EDGE	WEIGHT	STATUS
E–F	1	next
D–E	2	
A–D	3	
A–B	4	
B–E	5	
B–D	6	
C–E	7	
B–C	8	
C–F	9	
D–G	10	
E–H	11	
G–H	12	
F–H	13	

Total weight so far: 0 · edges chosen: 0 of 7

Pieces (union-find sets): {A} {B} {C} {D} {E} {F} {G} {H}

Kruskal looks at edges from lightest to heaviest. Every vertex starts as its own piece. Press Step.

Binary trees and binary search trees

CONCEPT

DEFINITION

A binary tree is a rooted tree in which every vertex has at most two children, each labeled **left child** or **right child**. In a full binary tree every internal vertex has exactly two children.

THEOREM / RULE

A full binary tree with i internal vertices has $i + 1$ leaves and $2i + 1$ vertices.
A binary tree of height h has at most 2^h leaves, so a binary tree with t leaves has $h \geq \lceil \log_2 t \rceil$.

Why 2^h? Level 0 holds one vertex, and each level can at most double the one above: 1, 2, 4, 8, ... After h levels there is room for 2^h leaves.

WHERE THIS SHOWS UP IN AI

Databases index millions of rows with B-trees, balanced search trees with many children per vertex. Nearest-neighbor search in machine learning libraries uses k-d trees, which split on one coordinate per level, much like a BST.

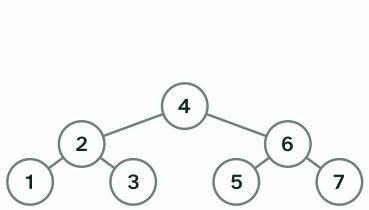
DEFINITION

A binary search tree (BST) holds a key in each vertex. For every vertex v, all keys in its left subtree are smaller than v's key and all keys in its right subtree are larger.

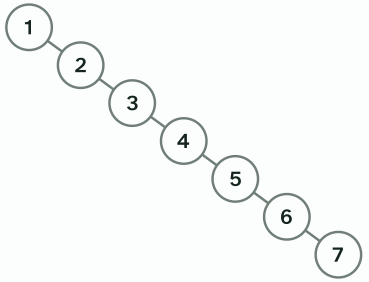
Search starts at the root. Equal means found, smaller means go left, larger means go right, and falling off the tree means the key is absent. **Insert** searches, then attaches the new key where the search fell off. Both cost at most $h + 1$ comparisons.

COMMON PITFALL

Insertion order shapes the tree. Inserting 1, 2, 3, ..., n in order builds a path of height $n - 1$, so a search may compare against every key. Shuffled input usually stays close to balanced. Self-balancing trees (AVL, red-black) rotate after inserts to keep h near $\log_2 n$.



insert 4 2 6 1 3 5 7: height 2



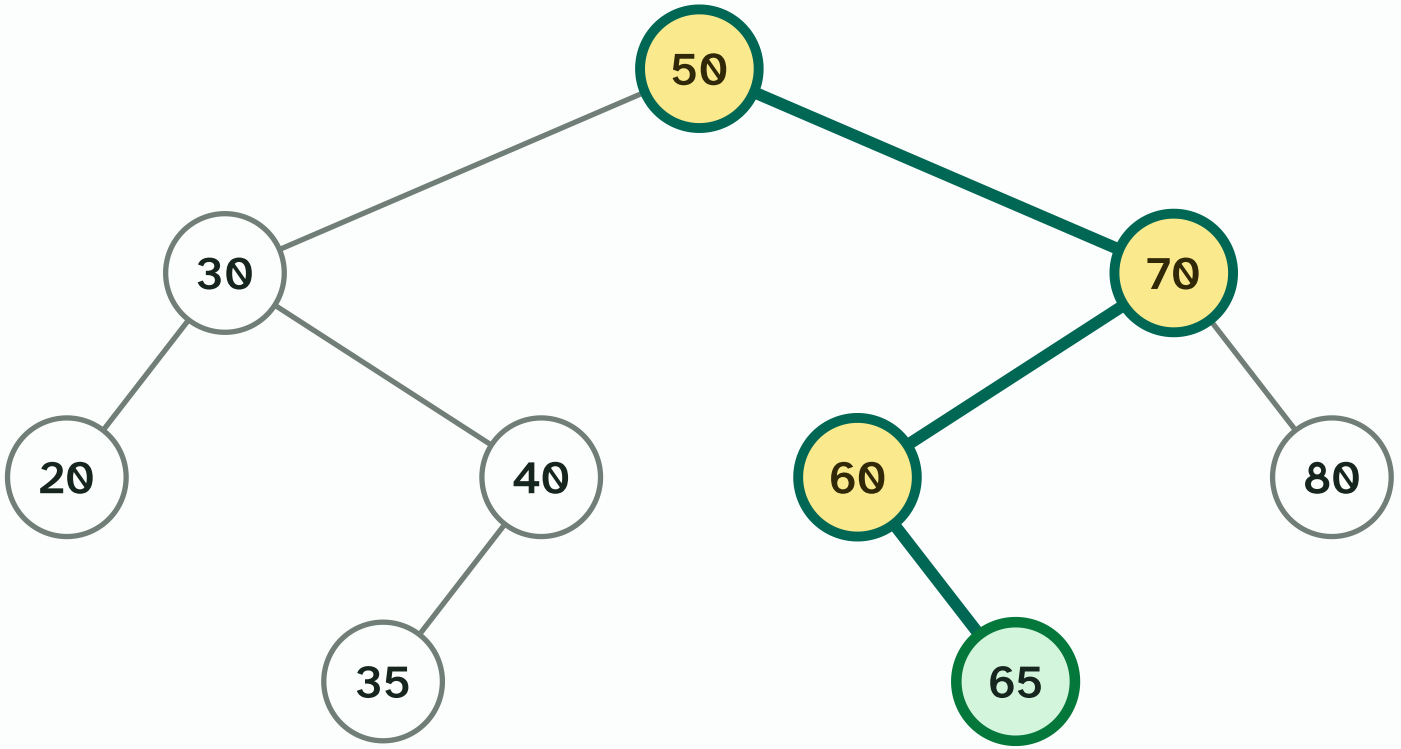
insert 1 2 3 4 5 6 7: height 6

Lab: binary search tree playground

INTERACTIVE LAB

Insert keys, search for one, or play a traversal and watch the visit order. Click any vertex to search for it. Try **Sorted 1–10** to see a degenerate tree, then **Balanced 1–15** to see the best case.

Insert 45 | Insert | Search 65 | Search | Clear | Presets | Example | Sorted 1–10 | Balanced 1–15 | Random 12



Keys	**9**
Height *h*	**3**
Smallest possible height	**3**
Worst-case search	**4 comparisons**
Leaves	**4**
Shape	**well balanced**

Found 65 at level 3 after 4 comparisons: 65 > 50, go right → 65 < 70, go left → 65 > 60, go right → 65 = 65, found.

TRAVERSALS

Preorder | Inorder | Postorder | Level order

Pick a traversal to list every key in that order.

On the search path · Found · Not found · Just inserted

Visited (badge = order)

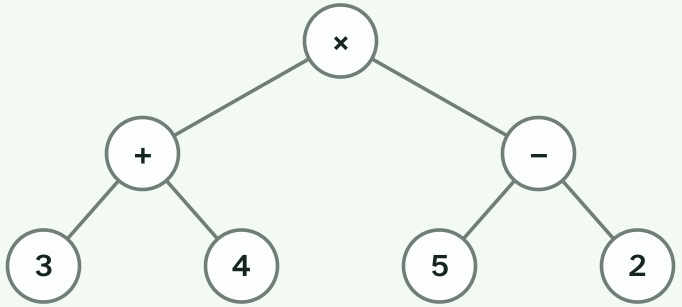
Traversals and expression trees

CONCEPT

DEFINITION

A traversal visits every vertex of a tree exactly once, in a fixed order. Writing T_L and T_R for the left and right subtrees, the depth-first orders are recursive:

Order	Rule	On the tree below
Preorder	root, T_L, T_R	× + 3 4 − 5 2
Inorder	T_L, root, T_R	3 + 4 × 5 − 2
Postorder	T_L, T_R, root	3 4 + 5 2 − ×
Level order	level by level (a queue)	× + − 3 4 5 2



DEFINITION

An expression tree stores a formula. Leaves hold numbers, and each internal vertex holds an operator applied to its two subtrees.

The three depth-first orders give three notations. Preorder gives **prefix** (Polish) notation. Inorder gives the usual **infix**, which needs parentheses to keep its meaning: $(3 + 4) \times (5 - 2)$. Postorder gives **postfix** (reverse Polish), which never needs parentheses.

WORKED EXAMPLE

Evaluate postfix with a stack: push each number; at an operator, pop two values (the first pop is the right operand), apply it, and push the result.

3 4 + → 7 · 7 5 2 − → 7 3 · 7 3 × → 21

WHERE THIS SHOWS UP IN AI

Python turns your source code into an abstract syntax tree before running it (see the `ast` module). Deep-learning frameworks record each computation as a graph and run backpropagation over it in reverse, children before parents.

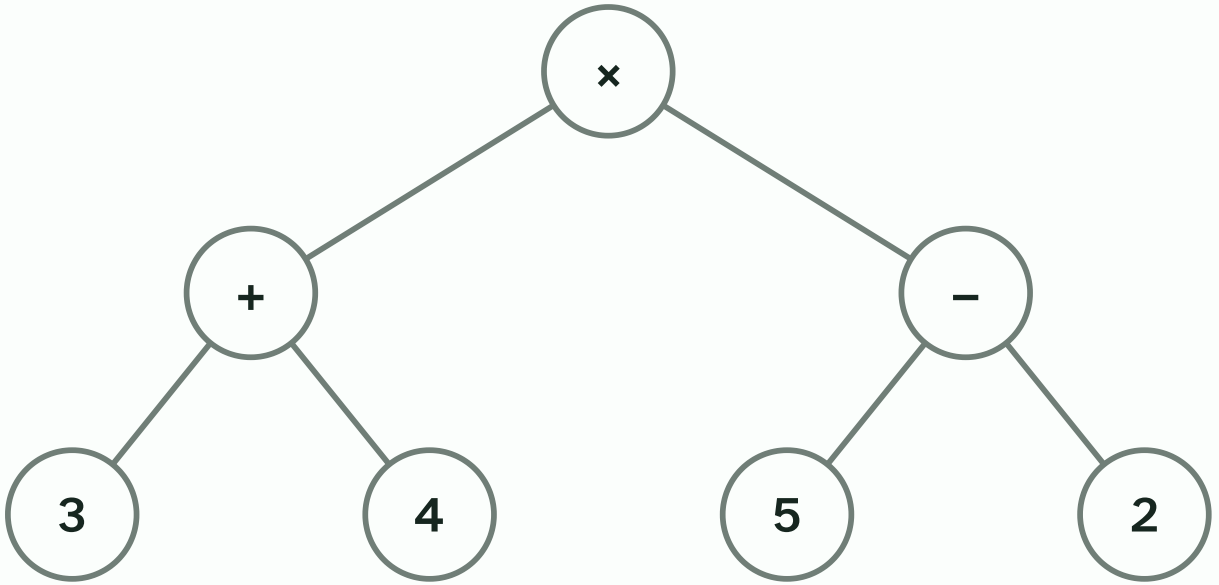
Lab: expression trees and the postfix stack

INTERACTIVE LAB

Type an arithmetic expression with + – * / ^ and parentheses. The lab builds its tree (× and ÷ bind tighter than + and –, and ^ binds tightest), reads it in three orders, then evaluates the postfix form one token at a time with a stack.

(3 + 4) * (5 - 2)

2 + 3 * 4 | (2 + 3) * 4 | 10 - 4 - 3 | 2 ^ 3 ^ 2 | (8 - 2) / (1 + 2) + 6 * 2



Preorder → prefix	× + 3 4 − 5 2
Inorder → infix	(3 + 4) × (5 − 2)
Inorder, no brackets	3 + 4 × 5 − 2 loses the grouping
Postorder → postfix	3 4 + 5 2 − ×

POSTFIX, READ LEFT TO RIGHT

3 | 4 | + | 5 | 2 | − | ×

STACK (BOTTOM → TOP)

empty

Postfix is read from left to right. There are 7 tokens. Press Step to read the first one.

◀ Back | Step ▶ | ▶ Play | Reset

Trees that decide: codes, sorting and games

CONCEPT

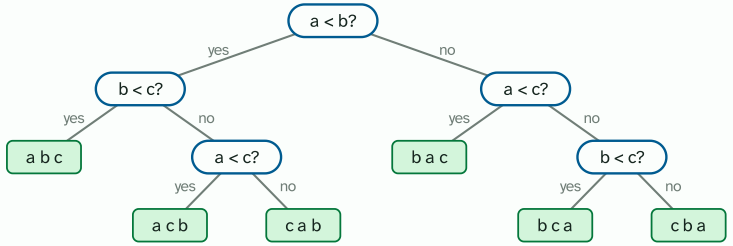
Prefix-free codes

ASCII spends 8 bits on every character. A variable-length code gives frequent symbols short code words. To decode without separators, no code word may be the beginning of another: a prefix-free code. With $a = 0$, $b = 10$, $c = 11$, the bits 01011 can only mean *a b c*. With $a = 0$, $b = 01$, $c = 1$, the bits 01 could be *b* or *a c*.

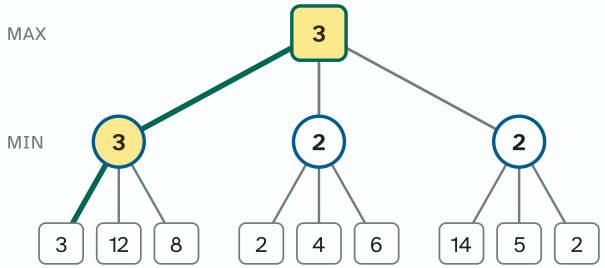
Prefix-free codes are binary trees: symbols sit at the leaves, and 0 means go left, 1 means go right. Huffman's greedy algorithm builds the best such tree (next slide).

Decision trees and the sorting bound

In a decision tree each internal vertex asks a question and each leaf is an outcome. A comparison sort asks "is $x_i < x_j$?" and must be able to end at any of the $n!$ orders, so its tree has at least $n!$ leaves and height at least $\lceil \log_2 n! \rceil$. For $n = 3$ that is 3 comparisons. This grows like $n \log_2 n$, so merge sort is as fast as comparison sorting can be.



Game trees and minimax



Vertices are positions and children are the positions one move later. With minimax, the opponent (MIN) takes the smallest child value and you (MAX) take the largest. Here the left move guarantees 3. The minimax lab lets you fill in a game tree of your own.

WHERE THIS SHOWS UP IN AI

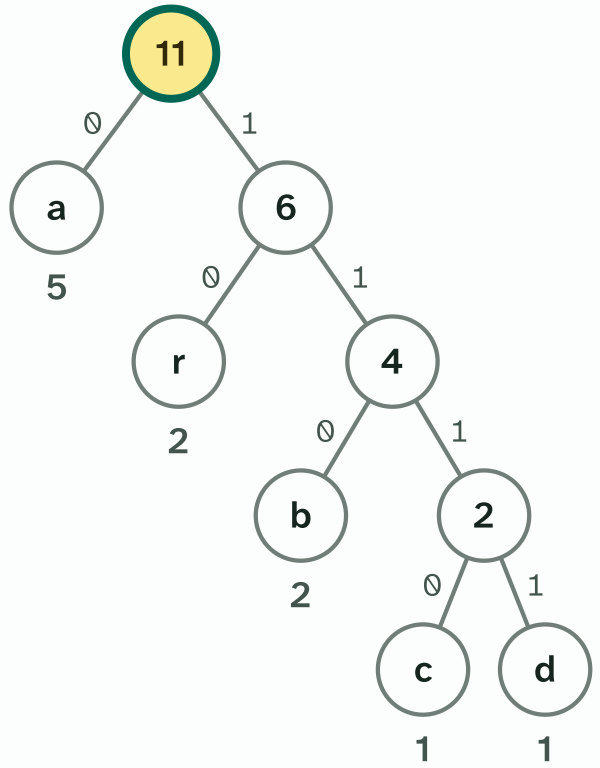
Gradient-boosted trees and random forests are leading models for tabular data. AlphaGo chose moves with Monte Carlo Tree Search over a game tree. Huffman coding sits inside zip and PNG and is used to compress model weights.

Lab: Huffman code builder

INTERACTIVE LAB

Huffman's rule: start with one tree per symbol, weighted by its count, and repeatedly merge the two lightest trees. The lighter tree goes left (bit 0); ties go to the tree whose smallest symbol comes first. Type any text, then step through the merges.

Text `abracadabra` | `mississippi` | `she sells sea shells` | `aaaaaaabbbccd` | `abcdefgh`



SYMBOL	COUNT	CODE	BITS
a	5	0	5
b	2	110	6
r	2	10	4
c	1	1110	4
d	1	1111	4

Huffman	**23 bits (2.09 per symbol)**
8-bit ASCII	**88 bits**
Fixed 3-bit code	**33 bits**
vs ASCII	**3.83 : 1, saving 73.9%**
Entropy	**2.04 bits per symbol**

Entropy is the smallest average any code can reach for these counts.

ENCODED TEXT (ALTERNATE COLORS = NEXT SYMBOL)

`01101001110011111011010 0`

Merge 4 of 4: join the two lightest trees, "a" (5) and the tree {r, b, c, d} (6), into one tree of weight 5 + 6 = 11. One tree is left, so the code is finished.

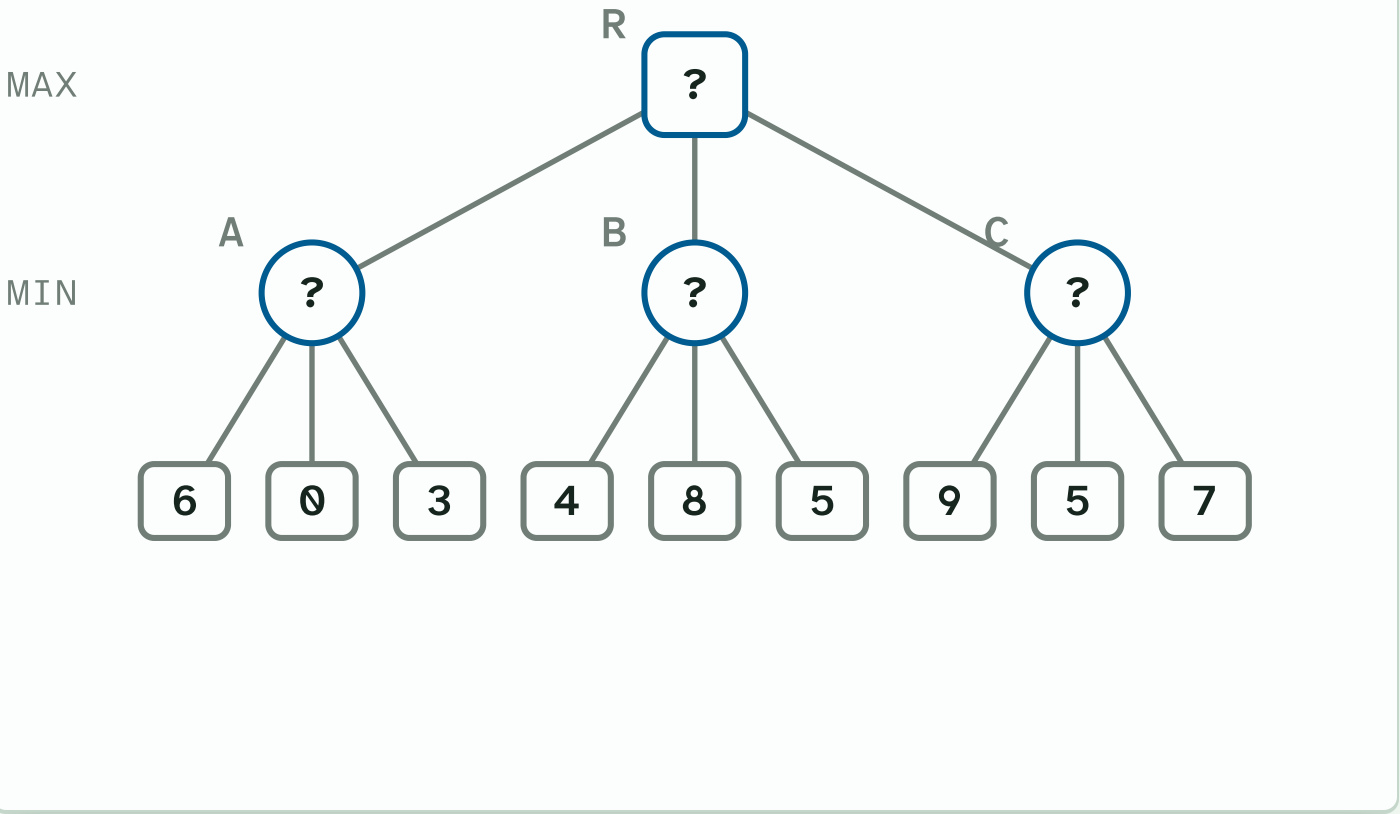
Reset | ◀ Back | Step ▶ | Build all

Lab: minimax on a game tree

INTERACTIVE LAB

Squares are your turns (MAX) and circles are the opponent's (MIN). Leaves hold the value of the final position for you. Work from the bottom up: type the value of every internal node, then check. The root's value is the best outcome you can force, and the move that reaches it is the one to play.

Depth 2 · 9 leaves | Depth 3 · 8 leaves | New values



FILL IN EVERY INTERNAL NODE, DEEPEST FIRST

A MIN of 6, 0, 3

B MIN of 4, 8, 5

C MIN of 9, 5, 7

R MAX of A, B, C

Check | Show solution

Code: a binary search tree in Python

CODE WALKTHROUGH

Each vertex is a small dictionary with a key and two links. Both functions are recursive: they solve the problem for one vertex and hand the rest to a subtree. Switch to **Trace** to watch the calls go down the tree and come back up.

bst.py

Read | Edit & run | Trace

```python
def insert(t, key):
    if t is None:
        return {"key": key, "left": None, "right": None}
    side = "left" if key < t["key"] else "right"
    t[side] = insert(t[side], key)
    return t

def inorder(t):
    if t is None:
        return []
    left, right = inorder(t["left"]), inorder(t["right"])
    return left + [t["key"]] + right
root = None
for key in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, key)
print("inorder:", inorder(root))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
inorder: [20, 30, 40, 50, 60, 70, 80]
```

LINE 1 OF 16

Show all notes in code

```
def insert(t, key):
```

Defines `insert`. The parameter `t` (short for tree) is the root of the subtree we are inserting into, and `key` is the new value. The function returns the root of that subtree after the insert, which lets the caller relink it.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Builds a binary search tree from seven keys with a recursive `insert`, then lists the keys with a recursive inorder traversal.

Code: a binary search tree in Python: every line explained (1/2)

1	`def insert(t, key):`	Defines `insert`. The parameter `t` (short for tree) is the root of the subtree we are inserting into, and `key` is the new value. The function returns the root of that subtree after the insert, which lets the caller relink it.
2	`    if t is None:`	The base case. `None` means we have walked off the bottom of the tree: this is the empty spot where the new key belongs. Every recursive call heads toward this case, one level deeper each time.
3	`        return {"key": key, "left": None, "right": None}`	Creates a new leaf: a dictionary holding the key and two empty links. Returning it hands the new vertex to the caller, which stores it as its left or right child. For the very first key, this vertex becomes the root.
4	`    side = "left" if key < t["key"] else "right"`	The search-tree rule in one line. A conditional expression picks `"left"` when the new key is smaller than this vertex's key and `"right"` otherwise. Inserting 40 at the root 50 gives `side = "left"`.
5	`    t[side] = insert(t[side], key)`	The recursive step. Insert into the chosen subtree, then store whatever comes back in that link. If the subtree was empty, the new leaf comes back and is attached here; otherwise the same child comes back and the link is unchanged.
6	`    return t`	Returns this vertex unchanged to its caller. The root of a subtree never moves during an insert, only a `None` link at the bottom gets filled in.
8	`def inorder(t):`	Defines the inorder traversal. It returns a list of the keys in the subtree rooted at `t`, in the order left subtree, root, right subtree.
9	`    if t is None:`	Base case again: an empty subtree contributes no keys. Without this line, the next lines would try to read `t["left"]` from `None` and crash with a `TypeError`.
10	`        return []`	An empty list, so the caller can safely glue it onto other lists with `+`.
11	`    left, right = inorder(t["left"]), inorder(t["right"])`	Two recursive calls: one lists every key in the left subtree (all smaller than this key), the other every key in the right subtree (all larger). Python runs the left call first. For the root 50 this gives `left = [20, 30, 40]` and `right = [60, 70, 80]`.
12	`    return left + [t["key"]] + right`	Glues the three parts in inorder: smaller keys, this key, larger keys. Because the BST rule holds at every vertex, the result is sorted. For the root: `[20, 30, 40] + [50] + [60, 70, 80]`.
13	`root = None`	The main program starts here, back at the left margin. `root = None` is an empty tree; the variable will hold the root vertex once the first key is inserted.
14	`for key in [50, 30, 70, 20, 40, 60, 80]:`	Inserts the keys in this order. 50 arrives first, so it becomes the root. The order is chosen to give a perfectly balanced tree of height 2. Sorted order would give a path of height 6 instead.
15	`    root = insert(root, key)`	Calls `insert` and keeps the returned root. The assignment matters only for the first key, when `root` changes from `None` to the new vertex. Writing it the same way every time keeps the code simple.

Code: a binary search tree in Python: every line explained (2/2)

16	`print("inorder:", inorder(root))`	Prints the traversal. Output: `inorder: [20, 30, 40, 50, 60, 70, 80]`. The keys went in shuffled and come out sorted, which is why sorting by "insert everything into a BST, then read it inorder" works (it is called tree sort).

Code: Huffman coding with a heap

CODE WALKTHROUGH

This is the greedy algorithm from the lab, written with Python's `heapq` functions. Each heap entry is (count, smallest symbol, codes so far). The middle field breaks ties, so the output always matches the lab exactly.

huffman.py

Read | Edit & run | Trace

```
1  from heapq import heappop, heappush
2  from collections import Counter
3
4  text = "abracadabra"
5  counts = Counter(text)
6  heap = sorted((n, c, {c: ""}) for c, n in counts.items())
7  while len(heap) > 1:
8      n1, t1, left = heappop(heap)
9      n2, t2, right = heappop(heap)
10     left = {c: "0" + b for c, b in left.items()}
11     right = {c: "1" + b for c, b in right.items()}
12     heappush(heap, (n1 + n2, min(t1, t2), left | right))
13 codes = heap[0][2]
14 print(codes)
15 print(sum(len(codes[c]) for c in text), "bits")
16 print(8 * len(text), "bits in 8-bit ASCII")
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
{'a': '0', 'r': '10', 'b': '110', 'c': '1110', 'd': '1111'}
23 bits
88 bits in 8-bit ASCII
```

LINE 1 OF 16

Show all notes in code

```
from heapq import heappop, heappush
```

Imports two functions from Python's binary-heap module. A heap is a list that keeps its smallest item at index 0. `heappop` removes that item and `heappush` adds one, each in about $\log_2 n$ steps instead of re-sorting the whole list.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Counts the characters of a text, merges the two lightest groups until one remains, and prints the prefix-free code table and the encoded length next to 8-bit ASCII.

Code: Huffman coding with a heap: every line explained

1	`from heapq import heappop, heappush`	Imports two functions from Python's binary-heap module. A heap is a list that keeps its smallest item at index 0. `heappop` removes that item and `heappush` adds one, each in about $\log_2 n$ steps instead of re-sorting the whole list.
2	`from collections import Counter`	`Counter` counts how many times each item appears. It is a dictionary from item to count.
4	`text = "abracadabra"`	The message to compress: 11 characters using 5 distinct symbols.
5	`counts = Counter(text)`	Counts the characters: a 5, b 2, r 2, c 1, d 1. These counts are the weights Huffman's algorithm works with.
6	`heap = sorted((n, c, {c: ""}) for c, n in counts.items())`	One entry per symbol `c`: its count, the symbol itself, and a dictionary of codes for the symbols in this group (an empty code for now). A sorted list already satisfies the heap rule, so no `heapify` is needed. Tuples compare field by field: the count decides first and the symbol breaks ties.
7	`while len(heap) > 1:`	Repeat until a single group holds every symbol. With 5 symbols the loop runs 4 times, because each pass turns two groups into one.
8	`    n1, t1, left = heappop(heap)`	Takes out the lightest group. The first pass gets `(1, "c", {"c": ""})`. Its codes will start with 0 because it becomes the left branch.
9	`    n2, t2, right = heappop(heap)`	Takes out the next lightest group, `(1, "d", {"d": ""})` on the first pass. It becomes the right branch.
10	`    left = {c: "0" + b for c, b in left.items()}`	A dictionary comprehension that puts a 0 in front of every code `b` (the bits so far) in the left group. Bits go on the front because each merge creates a new vertex *above* the old trees. After the first pass: `{"c": "0"}`.
11	`    right = {c: "1" + b for c, b in right.items()}`	The same for the right group, with a 1 in front: `{"d": "1"}` on the first pass.
12	`    heappush(heap, (n1 + n2, min(t1, t2), left | right))`	Pushes the merged group back. Its count is the sum, 2 on the first pass. `left | right` joins the two code dictionaries (the groups never share a symbol). The tie-breaker is the smaller symbol, so Python never has to compare two dictionaries, which would raise a `TypeError`.
13	`codes = heap[0][2]`	After the loop exactly one entry is left. Its third field, index 2, is the finished code table.
14	`print(codes)`	Prints the table: `{'a': '0', 'r': '10', 'b': '110', 'c': '1110', 'd': '1111'}`. The most common symbol gets the shortest code. The order is fixed because every tie is broken the same way.
15	`print(sum(len(codes[c]) for c in text), "bits")`	Walks through the text one character at a time and adds up the length of each code. Five a's cost 1 bit each, so the total is $5 + 6 + 4 + 4 + 4 = 23$ bits.
16	`print(8 * len(text), "bits in 8-bit ASCII")`	ASCII needs $11 \times 8 = 88$ bits, almost four times as many.

Design practice: trees in real code

DESIGN PRACTICE

Tree code is short but easy to get subtly wrong. These habits keep it fast, safe on bad input, and easy to change. Click each principle (or use ↓).

01 **Keep search trees balanced, or let a library do it**

02 Choose recursion or an explicit stack on purpose

03 Separate node data from the tree's structure

04 Name the traversal order in your API

05 Guard against cycles when building trees from data

06 Use heapq for greedy algorithms

07 Use union-find to ask "would this edge close a cycle?"

08 Test with degenerate inputs

Principle 1 of 8

Keep search trees balanced, or let a library do it

A plain BST is only as good as its height. Real data often arrives sorted (timestamps, IDs, rows read from a sorted file), and sorted insertions build a path, so every lookup becomes a linear scan.

Python has no built-in balanced tree. For exact lookups use a `dict` or `set`. For sorted order and range queries, keep a sorted list and binary-search it with `bisect`, which behaves like a perfectly balanced tree. Bring in a self-balancing structure when you need fast inserts too.

✗ AVOID THIS

```
root = None
for key in sorted(keys):        # sorted input...
    root = insert(root, key)    # ...builds a path
# every search is now O(n)
```

✓ DO THIS

```
import bisect
keys.sort()                          # once
i = bisect.bisect_left(keys, target)
found = i < len(keys) and keys[i] == target
```

Design practice: trees in real code: all principles (1/2)

Keep search trees balanced, or let a library do it

A plain BST is only as good as its height. Real data often arrives sorted (timestamps, IDs, rows read from a sorted file), and sorted insertions build a path, so every lookup becomes a linear scan.

Python has no built-in balanced tree. For exact lookups use a `dict` or `set`. For sorted order and range queries, keep a sorted list and binary-search it with `bisect`, which behaves like a perfectly balanced tree. Bring in a self-balancing structure when you need fast inserts too.

```
import bisect
keys.sort()                              # once
i = bisect.bisect_left(keys, target)
found = i < len(keys) and keys[i] == target
```

Choose recursion or an explicit stack on purpose

Recursion matches the definition of a tree and is the clearest choice for balanced trees. But Python stops at about 1000 nested calls (`sys.getrecursionlimit()`), so a degenerate tree 5000 levels deep makes a recursive traversal crash with `RecursionError`.

When the tree's shape is not under your control, use a loop with your own stack. It handles any depth and uses memory only for the vertices waiting to be visited.

```
s, stack = 0, [root]          # explicit stack
while stack:
    node = stack.pop()
    if node:
        s += node.key
        stack += [node.left, node.right]
```

Separate node data from the tree's structure

Let a `Node` hold a key, a value and two links, and let a `Tree` class own the root and every algorithm. Callers ask the tree (`tree.get(k)`) instead of walking `.left.right` chains themselves.

Then you can rebalance, switch to a different tree, or add a size counter without touching any calling code.

```
class Tree:
    def __init__(self):
        self.root = None
    def get(self, key): ...
    def insert(self, key, value): ...
tree.insert(42, "x")
```

Name the traversal order in your API

"Visit every vertex" hides a choice. Preorder, inorder, postorder and level order give different sequences, and downstream code often depends on one of them, for example on keys arriving sorted. Name the order in the function, or take it as a parameter, and say it in the docstring.

```
def keys_inorder(node):
    """Yield keys in ascending order (left,
root, right)."""
    if node:
        yield from keys_inorder(node.left)
        yield node.key
        yield from keys_inorder(node.right)
```

Design practice: trees in real code: all principles (2/2)

Guard against cycles when building trees from data

Hierarchies loaded from files or databases (employee to manager, folder to parent folder) can contain mistakes: a vertex that is its own ancestor, a vertex with two parents, or two roots. A traversal over such a "tree" loops forever or recurses until it crashes.

Validate before you trust it: exactly one root, one parent for every other vertex, and walking up from any vertex never repeats.

```
def has_cycle(parent, v):       # parent: child -> parent
    seen = set()
    while v is not None and v not in seen:
        seen.add(v)
        v = parent.get(v)
    return v is not None        # stopped on a repeat
```

Use heapq for greedy algorithms

Huffman, Prim and many schedulers repeatedly need "the smallest remaining item". Re-sorting the list or calling `min()` on every step costs $O(n)$ or more per step. A binary heap does each push and pop in $O(\log n)$.

Store `(priority, tie_breaker, item)` tuples so that equal priorities never fall through to comparing the items themselves.

```
heapq.heapify(trees)                    # O(n), once
while len(trees) > 1:
    a = heapq.heappop(trees)            # O(log n)
    b = heapq.heappop(trees)
    heapq.heappush(trees, merge(a, b))
```

Use union-find to ask "would this edge close a cycle?"

Kruskal's algorithm asks, for every edge, whether its two ends are already connected. Searching the partial tree each time costs time proportional to its size. A union-find (disjoint-set) structure answers almost instantly: every vertex points toward a representative of its piece, and two vertices are connected exactly when they reach the same representative.

```
parent = {v: v for v in vertices}
def find(v):
    while parent[v] != v:
        v = parent[v]
    return v
# (u, w) closes a cycle iff find(u) == find(w)
```

Test with degenerate inputs

Tree bugs hide in the shapes nobody draws: the empty tree, a single vertex, sorted input (a path), repeated keys and very deep trees. Write tests that build these shapes and check invariants such as "inorder output is sorted" and "height is at most $n - 1$".

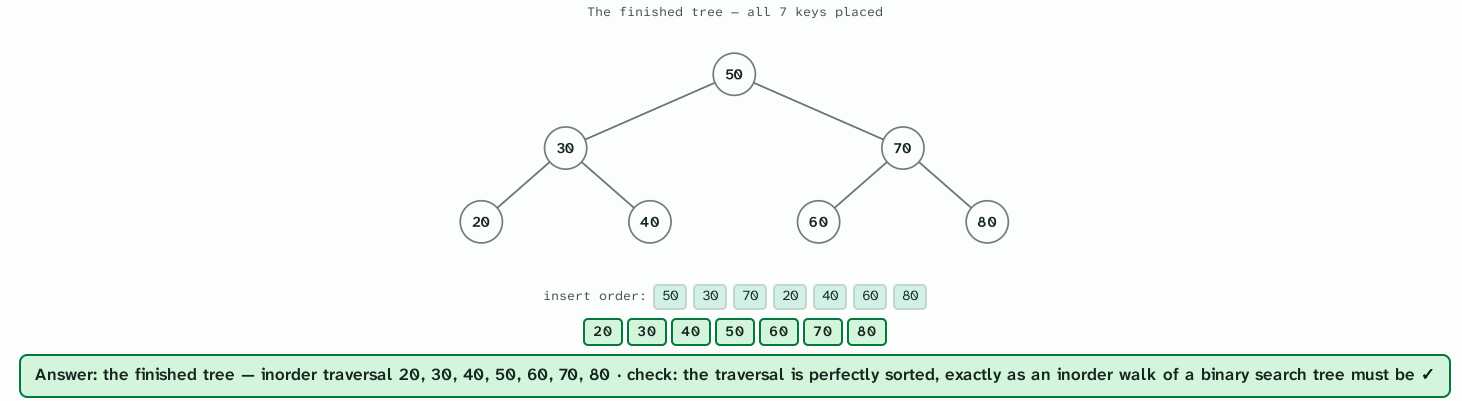
```
for keys in ([], [7], list(range(2000)), [5] * 10):
    root = build(keys)
    assert list(keys_inorder(root)) == sorted(keys)
    assert height(root) <= max(len(keys) - 1, 0)
```

Worked example: building a binary search tree by insertion

WORKED EXAMPLE

PROBLEM

Insert the keys $50, 30, 70, 20, 40, 60, 80$, in that order, into an empty binary search tree. Draw the tree after each insertion, then give its inorder traversal.



Answer: the finished tree — inorder traversal 20, 30, 40, 50, 60, 70, 80 · check: the traversal is perfectly sorted, exactly as an inorder walk of a binary search tree must be ✓

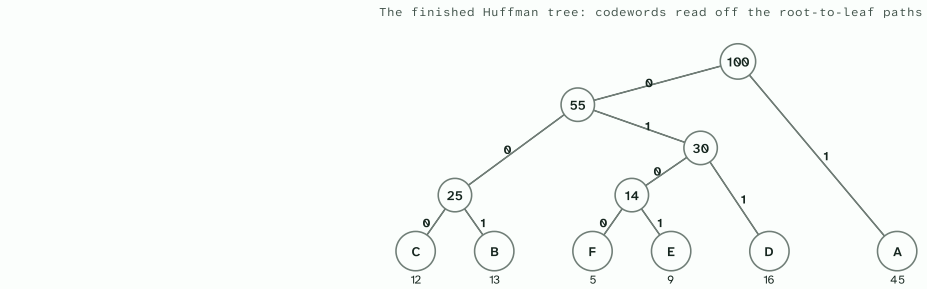
1. Start with an empty tree and the binary search tree rule: at every node, smaller keys go to the left subtree and larger keys go to the right subtree. That single rule decides every placement, because each insertion just follows the comparisons down from the root.
2. Insert 50: there is nothing to compare it against, so it simply becomes the root. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
3. Insert 30: 30 < 50, go left; the left child of 50 is empty, so 30 lands there. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
4. Insert 70: 70 > 50, go right; the right child of 50 is empty, so 70 lands there. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
5. Insert 20: 20 < 50, go left; 20 < 30, go left; the left child of 30 is empty, so 20 lands there. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
6. Insert 40: 40 < 50, go left; 40 > 30, go right; the right child of 30 is empty, so 40 lands there. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
7. Insert 60: 60 > 50, go right; 60 < 70, go left; the left child of 70 is empty, so 60 lands there. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
8. Insert 80: 80 > 50, go right; 80 > 70, go right; the right child of 70 is empty, so 80 lands there. Every later key is measured against the keys already placed, which is why the insertion order shapes the whole tree.
9. **Answer: an inorder traversal of any binary search tree visits keys in sorted order, and this one comes out perfectly sorted — 20, 30, 40, 50, 60, 70, 80 — which cross-checks every placement. The finished tree has 50 at the root and height 2.**

Worked example: building a Huffman code

WORKED EXAMPLE

PROBLEM

Build a Huffman code for the symbols and frequencies A:45, B:13, C:12, D:16, E:9, F:5 (out of 100). Give each symbol's codeword and the expected code length in bits per symbol.



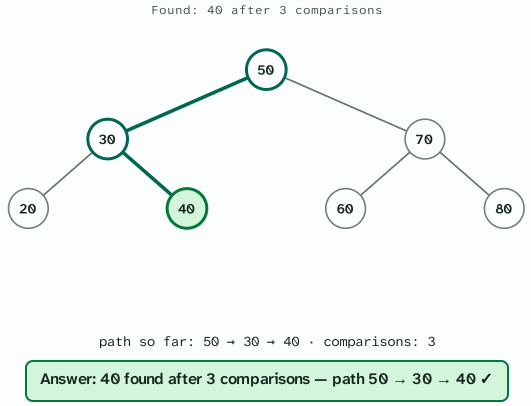
1. Setup: write the symbols as leaves sorted by frequency, lightest first: F 5, E 9, C 12, B 13, D 16, A 45. The Huffman rule is to repeatedly merge the two lightest trees, because the rarest symbols should travel the deepest and merging them first keeps them deep.
2. Merge F (5) and E (9), the two lightest trees, into a new tree of weight 14, drawn with F (5) on the left and E (9) on the right. Every symbol under this new tree gains one level of depth, and the forest shrinks to 5 trees.
3. Merge C (12) and B (13), the two lightest trees, into a new tree of weight 25, drawn with C (12) on the left and B (13) on the right. Every symbol under this new tree gains one level of depth, and the forest shrinks to 4 trees.
4. Merge the 14 tree and D (16), the two lightest trees, into a new tree of weight 30, drawn with the 14 tree on the left and D (16) on the right. Every symbol under this new tree gains one level of depth, and the forest shrinks to 3 trees.
5. Merge the 25 tree and the 30 tree, the two lightest trees, into a new tree of weight 55, drawn with the 25 tree on the left and the 30 tree on the right. Every symbol under this new tree gains one level of depth, and the forest shrinks to 2 trees.
6. Merge the 55 tree and A (45), the last two trees, into the root of weight 100, and label every left edge 0 and every right edge 1. The tree is complete: every symbol sits at a leaf, and symbols merged earliest sit deepest, which is what makes the frequent symbols cheap and the rare ones expensive.
7. **Answer: reading each root-to-leaf path gives A = 1, B = 001, C = 000, D = 011, E = 0101, F = 0100, with expected length (45·1 + 13·3 + 12·3 + 16·3 + 9·4 + 5·4) / 100 = 224 / 100 = 2.24 bits per symbol. Check: the code is prefix-free because every symbol is a leaf, and the weighted path length 224 equals the sum of the merge weights 14 + 25 + 30 + 55 + 100 = 224.**

Worked example: searching a binary search tree

WORKED EXAMPLE

PROBLEM

Search for the key 40 in the binary search tree built by inserting $50, 30, 70, 20, 40, 60, 80$. Follow the comparisons from the root, count them, and state whether the key is found.



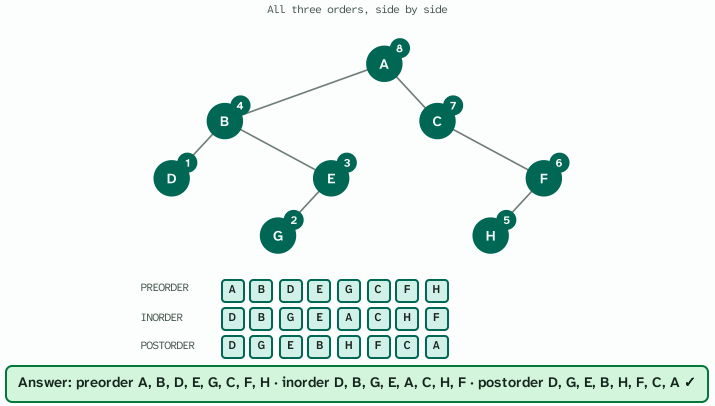
1. Search uses the placement rule in reverse. At each node, one comparison either finds the target, sends the search into the left subtree, or sends it into the right subtree, so the search only ever visits the nodes on a single root-to-leaf path.
2. Comparison 1, at node 50: $40 < 50$, so the target, if it is present at all, must lie in the left subtree: go left. Each comparison also discards the part of the tree on the wrong side of 50, which is why the path stays short on a balanced tree.
3. Comparison 2, at node 30: $40 > 30$, so the target, if it is present at all, must lie in the right subtree: go right. Each comparison also discards the part of the tree on the wrong side of 30, which is why the path stays short on a balanced tree.
4. Comparison 3, at node 40: $40 = 40$: equal, so the search stops here — the key is found. Each comparison also discards the part of the tree on the wrong side of 40, which is why the path stays short on a balanced tree.
5. **Answer: 40 is in the tree, reached along the path $50 \to 30 \to 40$ in 3 comparisons. Cross-check: the inorder list $20, 30, 40, 50, 60, 70, 80$ does contain 40, and every step obeyed the rule that smaller keys live in left subtrees and larger keys in right subtrees.**

Worked example: three traversals of one tree

WORKED EXAMPLE

PROBLEM

A binary tree has root A; A has left child B and right child C; B has left child D and right child E; E has left child G only; C has right child F only; F has left child H only. List the nodes in preorder, inorder and postorder.



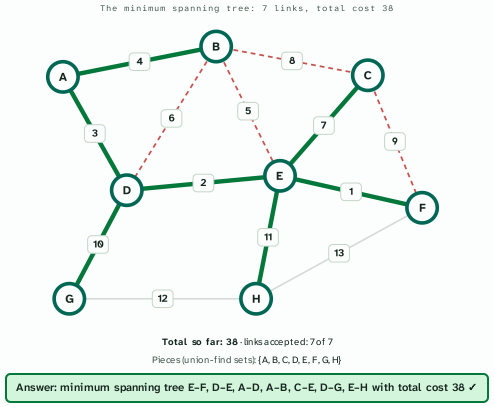
1. The tree stays fixed for all three orders: root A, with B and C below it, D and E below B, G below E, F below C, and H below F. A traversal is a rule for when a node gets written down relative to its subtrees — before them, between them, or after them — and each rule produces its own listing of the same 8 nodes.
2. Preorder — root, then left subtree, then right subtree. Here the root is written before either subtree, so it leads its listing. The listing so far is A, B, D, E: 4 of the 8 nodes, accumulated in the ribbon under the tree in visit order.
3. Preorder completes as G, C, F, H follows, giving the full listing A, B, D, E, G, C, F, H. The root A leads the list, because preorder writes each node before its subtrees. Every node appears exactly once; only the visiting rule separates this order from the other two.
4. Inorder — left subtree, then root, then right subtree. Here the walk dives into the left subtree first, so the deepest left node is written before the root. The listing so far is D, B, G, E: 4 of the 8 nodes, accumulated in the ribbon under the tree in visit order.
5. Inorder completes as A, C, H, F follows, giving the full listing D, B, G, E, A, C, H, F. The root A sits at position 5 of 8, exactly between its left and right subtrees. Every node appears exactly once; only the visiting rule separates this order from the other two.
6. Postorder — left subtree, then right subtree, then root. Here both subtrees are finished before their root, so the leaves come first and the root comes last. The listing so far is D, G, E, B: 4 of the 8 nodes, accumulated in the ribbon under the tree in visit order.
7. Postorder completes as H, F, C, A follows, giving the full listing D, G, E, B, H, F, C, A. The root A closes the list, because postorder writes each node only after both of its subtrees. Every node appears exactly once; only the visiting rule separates this order from the other two.
8. **Answer: preorder A, B, D, E, G, C, F, H; inorder D, B, G, E, A, C, H, F; postorder D, G, E, B, H, F, C, A. Same tree and the same 8 nodes in each list — a traversal changes only the order of the visits, never the set of nodes visited, and each list can be rechecked by running its rule subtree by subtree.**

Worked example: a minimum spanning tree by Kruskal

WORKED EXAMPLE

PROBLEM

Eight field stations *A*–*H* can be linked by cable along the routes and costs drawn in the steps — the weighted graph from the Kruskal and Prim lab. Connect every station at minimum total cable cost: run Kruskal's algorithm, link by link, and give the final network and its total cost.



1. Kruskal's algorithm sorts every candidate link by cost and considers them lightest first. A link is accepted only when its two ends are still in different pieces, because a link inside one piece would close a cycle; the pieces below the graph track exactly that test.
2. Consider E–F, cost 1, the lightest link not yet considered. Its ends sit in different pieces, {E} and {F}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 1.
3. Consider D–E, cost 2, the lightest link not yet considered. Its ends sit in different pieces, {D} and {E, F}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 3.
4. Consider A–D, cost 3, the lightest link not yet considered. Its ends sit in different pieces, {A} and {D, E, F}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 6.
5. Consider A–B, cost 4, the lightest link not yet considered. Its ends sit in different pieces, {A, D, E, F} and {B}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 10.
6. Consider B–E, cost 5. Its ends are already connected by the accepted path B–A–D–E, so adding it would close a cycle and waste cable: reject it, even though it is cheap, and the total stays 10.
7. Consider B–D, cost 6. Its ends are already connected by the accepted path B–A–D, so adding it would close a cycle and waste cable: reject it, even though it is cheap, and the total stays 10.
8. Consider C–E, cost 7, the lightest link not yet considered. Its ends sit in different pieces, {C} and {A, B, D, E, F}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 17.
9. Consider B–C, cost 8. Its ends are already connected by the accepted path B–A–D–E–C, so adding it would close a cycle and waste cable: reject it, even though it is cheap, and the total stays 17.
10. Consider C–F, cost 9. Its ends are already connected by the accepted path C–E–F, so adding it would close a cycle and waste cable: reject it, even though it is cheap, and the total stays 17.
11. Consider D–G, cost 10, the lightest link not yet considered. Its ends sit in different pieces, {A, B, C, D, E, F} and {G}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 27.
12. Consider E–H, cost 11, the lightest link not yet considered. Its ends sit in different pieces, {A, B, C, D, E, F, G} and {H}, so no path between them exists yet: accept it, merge the two pieces, and the running total becomes 38.
13. **Answer: the accepted links E–F, D–E, A–D, A–B, C–E, D–G, E–H form a minimum spanning tree of total cost 38. It has 7 = 8 − 1 links, reaches every station, and contains no cycle; an independent Prim computation, growing the tree from one station, reaches the same total 38.**

Practice: trees

PRACTICE

Answers are numbers or lists. For a traversal, type the keys in order separated by commas, such as `50, 30, 70`. For postfix, separate tokens with spaces.

COUNTING IN TREES

A tree has 47 vertices. How many edges does it have?

Type your answer

Check

Show worked solution

New problem ↻

Every tree with n vertices has exactly $n - 1$ edges, so this one has $47 - 1 = 46$.

PROBLEM TYPE

- Counting in trees
- Traversals
- Expression trees
- Greedy trees
- Minimax
- Isomorphic trees

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: trees: sample problems with solutions

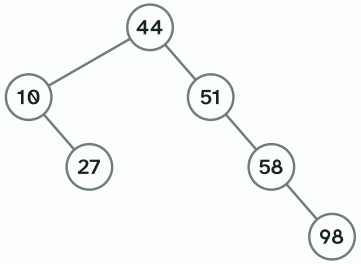
COUNTING IN TREES

A full binary tree has 25 internal vertices. How many leaves does it have?

Each internal vertex has 2 children, so there are $2 \cdot 25 = 50$ children plus the root: 51 vertices. Removing the 25 internal ones leaves $25 + 1 = 26$ leaves.

TRAVERSALS

List the keys of this binary search tree in **preorder** order.



Preorder means: visit the root, then the left subtree, then the right subtree. Start with the root 44, and finish the whole left subtree before touching the right one. Result: **44, 10, 27, 51, 58, 98**.

EXPRESSION TREES

Evaluate the postfix expression `4 3 4 + − 8 × 3 ×`.

Read left to right with a stack:

```
4: push → [4]
3: push → [4, 3]
4: push → [4, 3, 4]
+: pop 4 and 3, 3 + 4 = 7 → [4, 7]
−: pop 7 and 4, 4 − 7 = −3 → [−3]
8: push → [−3, 8]
×: pop 8 and −3, −3 × 8 = −24 → [−24]
3: push → [−24, 3]
×: pop 3 and −24, −24 × 3 = −72 → [−72]
```

One value is left on the stack: **−72**.

GREEDY TREES

A network has these weighted edges: A–B: 2, A–C: 12, A–E: 3, B–C: 1, B–D: 4, C–D: 5, C–E: 6. What is the total weight of a minimal spanning tree?

Kruskal, lightest first: take B–C (1); take A–B (2); take A–E (3); take B–D (4). Five vertices need 4 edges, so stop. Total weight: **10**.

Check yourself: trees

CHECK YOURSELF

QUESTION 1 OF 12

A connected graph has 12 vertices and 11 edges. What can you conclude?

A It is a tree

B It might still be disconnected

C It must contain a cycle

D Nothing without a drawing

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

11 12

Pick an answer to see feedback.

Start over

Check yourself: trees: answers explained (1/2)

1. A connected graph has 12 vertices and 11 edges. What can you conclude?

Answer A: It is a tree

A connected graph with n vertices and $n - 1$ edges is a tree: that is one of the equivalent characterizations. It cannot be disconnected, because we were told it is connected, and it cannot have a cycle, because removing a cycle edge would leave a connected graph with only $n - 2$ edges, which is impossible.

2. In a rooted tree, vertex v is at level 3. How many ancestors does v have?

Answer C: 3

The ancestors are the vertices on the path from v up to the root, not counting v. Level 3 means that path has 3 edges, so it passes through 3 other vertices: the parent, the grandparent and the root.

3. Which traversal of a binary search tree lists its keys in increasing order?

Answer B: Inorder

Inorder visits the left subtree (all smaller keys), then the vertex, then the right subtree (all larger keys). Applied at every vertex, that produces sorted order.

4. The keys 10, 20, 30, 40, 50, 60 are inserted in that order into an empty binary search tree. What is its height?

Answer C: 5

Each new key is larger than every key before it, so it always goes right. The tree is a path of 6 vertices, which has 5 edges: height 5. Inserting the same keys in the order 40, 20, 50, 10, 30, 60 would give height 2.

5. Kruskal's algorithm is considering the edge C–F. It skips the edge when…

Answer D: C and F are already connected by edges chosen earlier

If C and F are already connected, adding C–F would create a second path between them, which is a cycle. Being heavier is expected (edges arrive sorted), vertices may touch any number of tree edges, and Kruskal has no single growing tree (that is Prim).

6. Evaluate the postfix expression `5 1 2 + 4 × + 3 −`.

Answer B: 14

With a stack: 5, 1, 2 are pushed; + gives 1 + 2 = 3 (stack 5, 3); 4 is pushed; × gives 3 × 4 = 12 (stack 5, 12); + gives 5 + 12 = 17; 3 is pushed; − gives 17 − 3 = 14.

7. Which set of code words is prefix-free?

Answer D: {0, 10, 110, 111}

In {0, 10, 110, 111} no word begins another, so any bit string splits into words in only one way. In the others, 0 begins 01, 1 begins 10, and 0 begins 00.

8. At each step, Huffman's algorithm…

Answer D: merges the two trees with the smallest total counts

Merging the two lightest trees pushes the rarest symbols deepest, where long codes cost the least in total. The most frequent symbol does end up with a short code, but that is a result of the rule, not the rule itself.

Check yourself: trees: answers explained (2/2)

9. Any algorithm that sorts 5 distinct items using only comparisons needs, in its worst case, at least how many comparisons?

Answer D: 7

Its decision tree needs at least $5! = 120$ leaves. A binary tree of height h has at most 2^h leaves, and $2^6 = 64 < 120 \le 128 = 2^7$, so $h \ge 7$.

10. A recursive BST program works in testing but crashes with `RecursionError` on 50,000 keys loaded from a sorted file. What is the most likely cause?

Answer D: Sorted insertions built a path about 50,000 levels deep

Sorted input makes every key go right, so the tree is a path and the recursion is as deep as the tree. Python stops at about 1000 nested calls. Fix it by balancing (shuffle, a balanced tree, or a sorted list with bisect) or by traversing with an explicit stack.

11. Two trees both have 6 vertices, 5 edges and degree sequence 3, 2, 2, 1, 1, 1. What can you conclude?

Answer C: They may or may not be isomorphic; matching invariants do not prove it

Invariants can only rule isomorphism out. Two different trees share exactly this degree sequence: a degree-3 vertex with legs of length 2, 2, 1, and one with legs of length 3, 1, 1. To decide, hang both from their centers and compare canonical names, or exhibit a vertex correspondence.

12. MAX moves at the root of a game tree. Its three children are MIN nodes whose leaves are {3, 8}, {6, 7} and {9, 1}. What is the value of the root?

Answer C: 6

MIN takes the smallest leaf under each move: 3, 6 and 1. MAX then takes the largest of those, 6, by playing the middle move. The 9 is a trap, because MIN would answer that move with 1.

Transport networks and flows

CONCEPT

DEFINITION

A transport network is a directed graph where every edge (u, v) has a nonnegative capacity $c(u, v)$. One vertex is the source s (no edges come in) and one is the sink t (no edges go out). Picture water pipes: the capacity is how much a pipe can carry per second.

DEFINITION

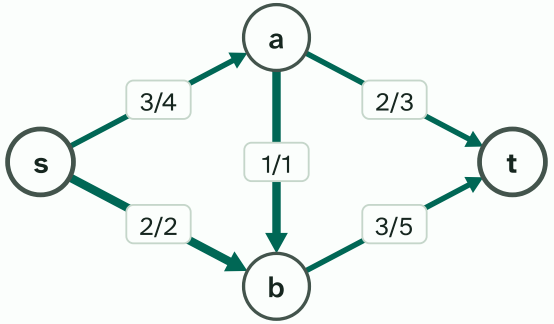
A flow puts an amount $f(u, v)$ on every edge and obeys two rules.

1. **Capacity rule:** $0 \le f(u, v) \le c(u, v)$ on every edge.
2. **Conservation of flow:** at every vertex other than s and t, flow in equals flow out.

THEOREM / RULE

The value of a flow $|f|$ is the total leaving s. Because nothing is created or lost at the vertices in between, it also equals the total entering t.

EDGE LABELS READ FLOW / CAPACITY



WORKED EXAMPLE

At a: 3 units arrive, and $1 + 2 = 3$ leave. At b: $2 + 1 = 3$ arrive and 3 leave. So conservation holds. The value is $3 + 2 = 5$ leaving s, and $2 + 3 = 5$ reaches t. The edges $s \to b$ and $a \to b$ are full; we call them saturated.

WHERE THIS SHOWS UP IN AI

A training pipeline moves data from storage through preprocessing workers to GPUs, and every link has a bandwidth limit. The *maximum* flow of that network is the highest throughput the pipeline can sustain.

Augmenting paths and the residual network

CONCEPT

To grow a flow, look for spare room. The residual network records two kinds of room for every edge $u \to v$ with flow f and capacity c:

- **Forward step** $u \to v$: room $c - f$. That many more units fit.
- **Backward step** $v \dashrightarrow u$: room f. Units already sent along $u \to v$ can be canceled, which has the same effect as sending them back.

DEFINITION

An augmenting path is a path from s to t in the residual network where every step has positive residual capacity. Its bottleneck is the smallest room on the path. Pushing the bottleneck along it (add on forward steps, subtract on backward steps) raises $|f|$ by exactly that amount and keeps both flow rules true.

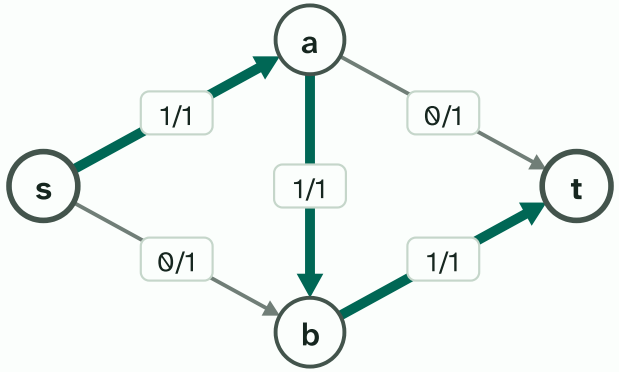
WHY IT MATTERS

Without backward steps, an early bad choice is permanent and a greedy algorithm can get stuck below the maximum. Backward steps let the algorithm change its mind.

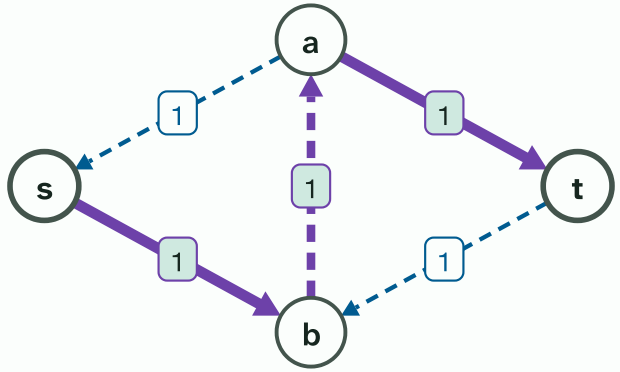
WORKED EXAMPLE

Every edge has capacity 1. A careless first path $s \to a \to b \to t$ fills three edges (left). No forward route is left, yet the maximum flow is 2.

FLOW AFTER THE FIRST PATH



RESIDUAL NETWORK

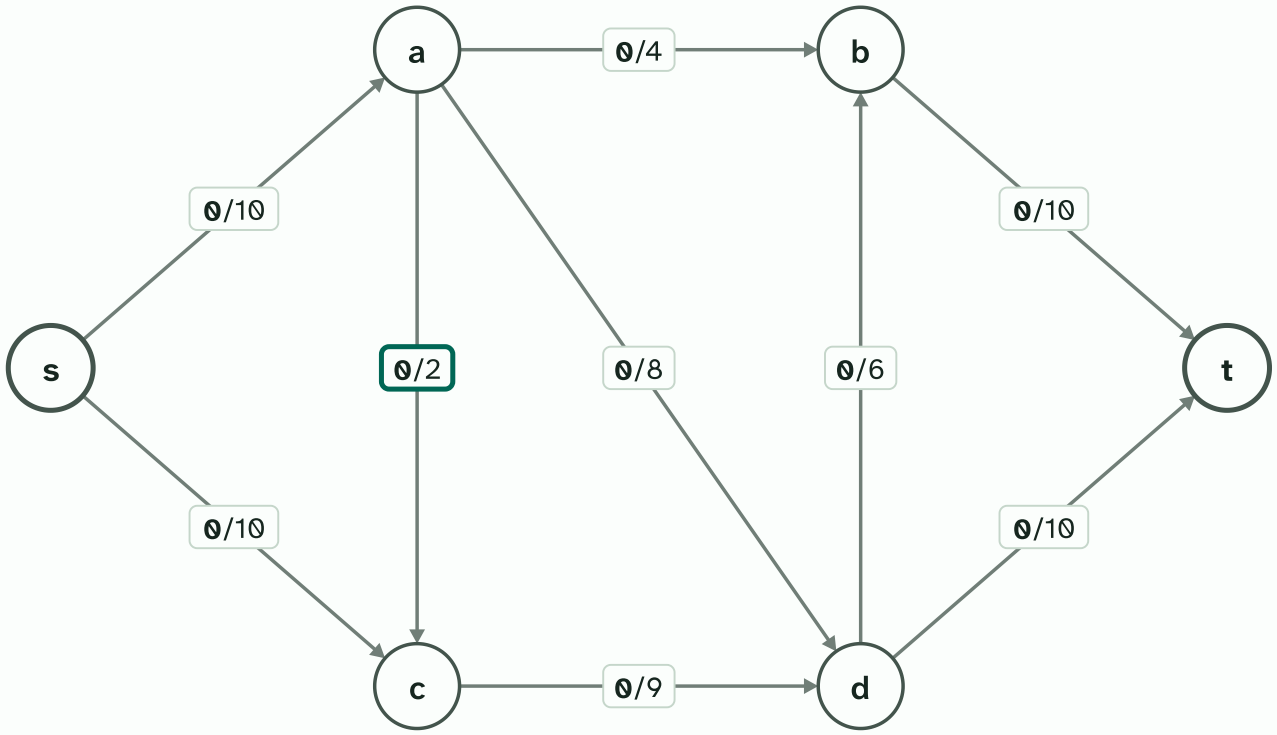


The residual network (right) has a backward step $b \dashrightarrow a$ with room 1, because $a \to b$ carries 1 unit. So $s \to b \dashrightarrow a \to t$ is an augmenting path. Pushing 1 unit along it cancels the flow on $a \to b$. The final flow uses $s \to a \to t$ and $s \to b \to t$, value 2.

Lab: maximum flow, one augmenting path at a time

INTERACTIVE LAB

Press **Find next path** to search the residual network, then **Push** to send the bottleneck along it. Switch to the residual view to see the room the search uses. Click an edge label to change its capacity. Try the trap network with DFS to watch a backward step cancel flow.



carries flow (thick = full) · augmenting path (dashed = backward step) · minimum cut · S side at the end

Network: Six vertices | The trap

Path search: BFS: fewest edges | DFS: first found

View: Flow | Residual

Find next path | Run to end | Reset

$|f| = 0$ after 0 augmentations

Every edge starts empty. Press Find next path.

Edge **a** → **c** capacity − 2 + click any edge label to pick it

AUGMENTATIONS SO FAR

none yet

The Ford–Fulkerson method and Edmonds–Karp

CONCEPT

THE METHOD

The Ford–Fulkerson method finds a maximum flow:

1. Start with flow 0 on every edge.
2. Search the residual network for an augmenting path.
3. If there is none, stop: the flow is maximum.
4. Push the bottleneck along the path (up on forward steps, down on backward steps), then go back to step 2.

THEOREM / RULE

With integer capacities, every push adds at least 1 to $|f|$, so the method stops after at most $|f^*|$ pushes (f^* a maximum flow). It also finds a maximum flow that is a whole number on every edge.

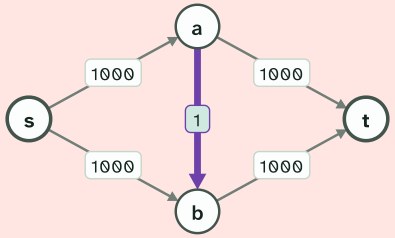
Johnsonbaugh calls this the *maximal flow algorithm* and finds paths by labeling vertices. The labeling is one way to search the residual network.

DEFINITION

The Edmonds–Karp algorithm adds one rule: always use a *shortest* augmenting path (fewest edges), found by breadth-first search. It needs at most $|V| \cdot |E|$ augmentations, so it runs in $O(|V| \cdot |E|^2)$ time however large the capacities are.

COMMON PITFALL

The rule matters. Here a careless search might keep using the middle edge, alternating $s \to a \to b \to t$ and $s \to b \dashrightarrow a \to t$. Each round pushes 1 unit, so it takes 2000 rounds. BFS takes the two outer paths and finishes in 2. With irrational capacities, a bad rule can even run forever.



DESIGN TIP

Use BFS (Edmonds–Karp) unless you have a reason not to. Its running time depends only on the size of the graph, so huge capacity values cannot slow it down.

Cuts and the max-flow min-cut theorem

CONCEPT

DEFINITION

A cut (S, T) splits the vertices into two groups with $s \in S$ and $t \in T$. Its capacity adds up the edges that go from S to T:

$$c(S,T) = \sum_{u \in S,\, v \in T} c(u,v)$$

Edges that point from T back to S do not count.

THEOREM / RULE

For every flow f and every cut, $|f| \le c(S,T)$. Every unit that travels from s to t has to cross from S to T at least once.

MAX-FLOW MIN-CUT THEOREM

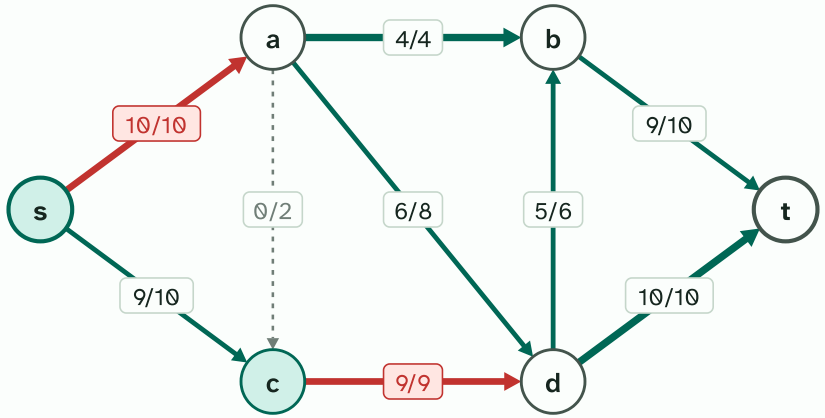
The value of a maximum flow equals the capacity of a minimum cut. This is the max-flow min-cut theorem.

WHY IT MATTERS

A minimum cut is a certificate and an explanation in one. It proves no flow can do better, and it names the links that limit throughput. Upgrading a link outside the minimum cut cannot raise the maximum flow.

Finding the cut. When Ford–Fulkerson stops, let S be the vertices s can still reach in the residual network. Every edge from S to T must be full, or its far end would be reachable. Every edge from T to S must carry 0, or its backward step would be usable. So $|f| = c(S,T)$ exactly.

WORKED EXAMPLE

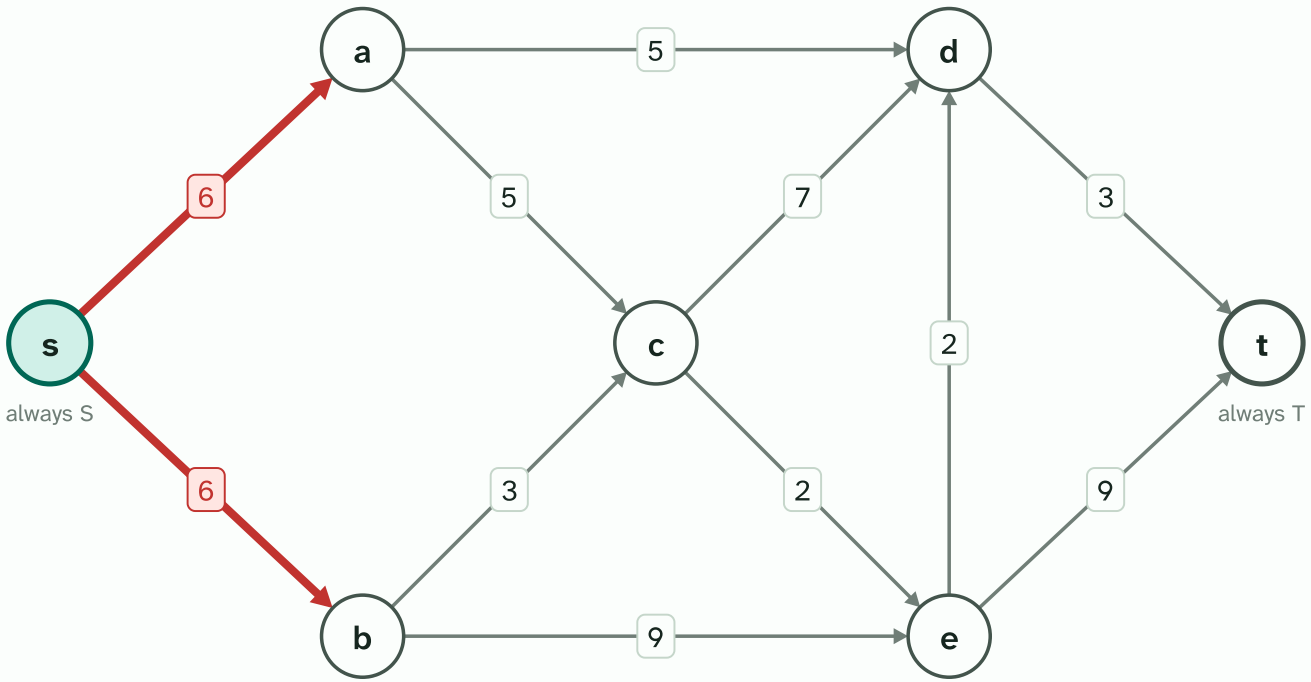


The stepper's network at the end: the flow is 19, and s can still reach only c. With $S = \{s, c\}$ (shaded) the cut edges are $s \to a$ (10) and $c \to d$ (9), and $10 + 9 = 19$. Edge $a \to c$ points from T into S, so it is left out.

Lab: hunt for the minimum cut

INTERACTIVE LAB

Click a vertex to move it between S (shaded) and T. s always stays in S and t in T. Red edges cross from S to T and count; dotted edges cross back and do not. The strip shows all 32 possible cuts sorted by capacity: click a bar to load that cut.

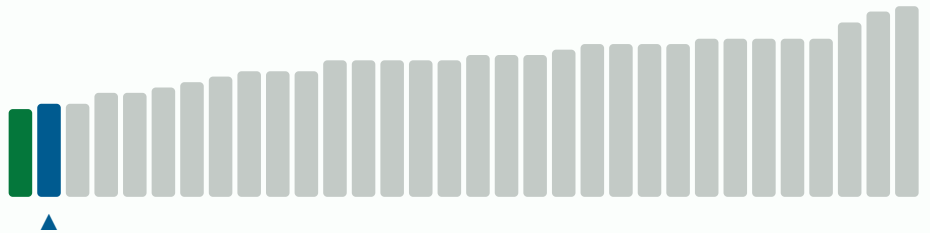


$$c(S, T) = 6 + 6 = 12$$

S	{s}
T	{a, b, c, d, e, t}
COUNTED	s → a (6), s → b (6)
IGNORED	no edges point from T into S

Every flow must cross this cut, so no flow can exceed 12. The maximum flow is 11, so a cut 1 smaller exists. This cut ranks 2 of 32 (tied).

ALL 32 CUTS BY CAPACITY · ▲ = YOUR CUT



smallest 11 (green) largest 30

S = {s} | Everything except t | Show a minimum cut

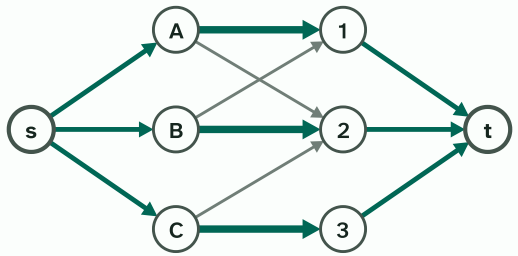
Matching workers to jobs with flows

CONCEPT

DEFINITION

In a bipartite graph the vertices form two groups, here workers W and jobs J, and every edge joins a worker to a job they are qualified for. A matching is a set of edges with no shared endpoints: each worker does at most one job and each job gets at most one worker. A **maximum matching** has as many edges as possible.

Matching as a flow. Add a source s with an edge to every worker, point each qualification edge from worker to job, and add an edge from every job to a sink t. Give every edge capacity 1. An integer maximum flow sends at most one unit through each worker and each job, so its value is the size of a maximum matching.



workers A, B, C · jobs 1, 2, 3 · every edge has capacity 1

THEOREM / RULE

Hall's condition: every worker can be matched exactly when, for every set A of workers, $|N(A)| \geq |A|$. Here $N(A)$ is the set of jobs that at least one worker in A is qualified for.

WORKED EXAMPLE

Ben and Cy are both qualified only for Label. For $A = \{\text{Ben}, \text{Cy}\}$, $N(A) = \{\text{Label}\}$: two workers, one job. No assignment can place both, whatever the rest of the graph looks like.

An augmenting path in a matching alternates: a new edge, a matched edge to give up, a new edge, and so on. Flipping it re-assigns some workers and places one more. The next lab shows these re-routes.

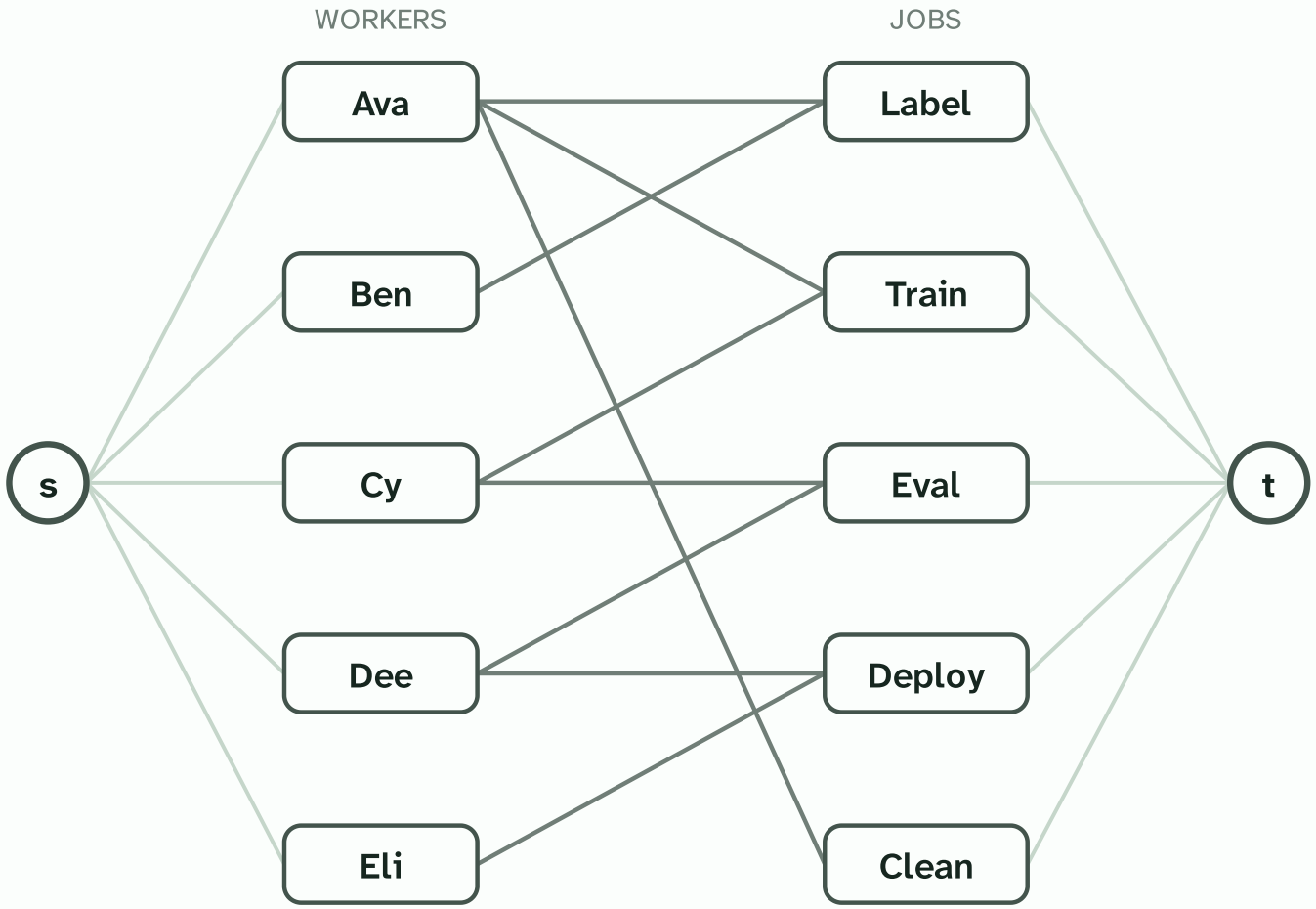
WHERE THIS SHOWS UP IN AI

Schedulers assign inference requests to GPU servers, ride-sharing apps pair riders with drivers, and ad systems fill display slots. At scale they solve weighted versions of this same matching problem.

Lab: placing workers with augmenting paths

INTERACTIVE LAB

Workers are placed one at a time, top to bottom. Each step searches (by BFS) for an augmenting path from the next worker; the path may re-route people who already have jobs. Toggle qualifications in the table to break or repair a perfect matching, and watch the Hall check.



As a flow network every edge has capacity 1. Matching size = flow value = 0

matched augmenting path: solid = new, dashed = given up left out

Find path for Ava | Run to end | Reset

Presets: Everyone fits | Nobody can Clean | Ben and Cy clash

Press **Find next path** to look for an augmenting path from Ava.

QUALIFICATIONS (CLICK TO TOGGLE)

	Label	Train	Eval	Deploy	Clean
Ava	✓	✓			✓
Ben	✓				
Cy		✓	✓		
Dee			✓	✓	
Eli				✓	

Hall's condition holds. Every group of k workers is qualified, in total, for at least k jobs, so all 5 can be placed.

Lab: cutting an image into object and background

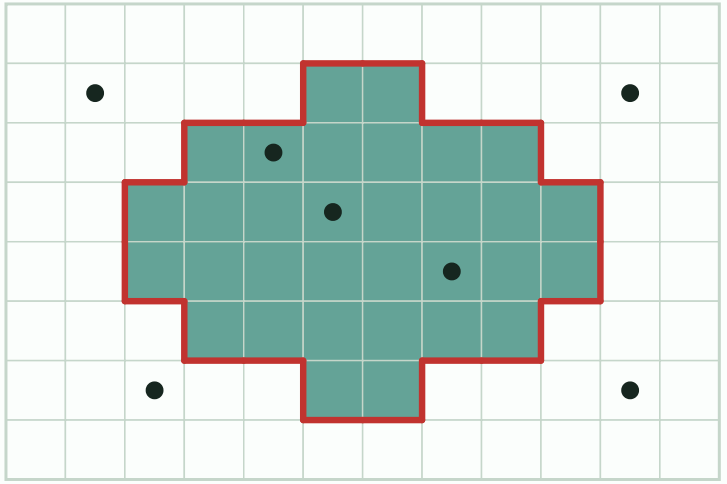
INTERACTIVE LAB

Each pixel has a score from 0 to 9 for how much it looks like the object. A network with one vertex per pixel turns "label every pixel" into a minimum cut. Paint on the left grid and move the smoothness slider λ: the right grid is the minimum cut, recomputed by max flow every time.

PIXEL SCORES (PAINT HERE)

0	1	2	0	1	2	0	1	2	0	1	2
0	7	2	0	1	8	9	1	2	0	8	2
0	1	2	8	4	7	8	9	7	0	1	2
0	1	9	7	8	3	7	8	9	7	1	2
0	1	8	9	7	8	9	2	8	9	1	2
0	1	2	8	9	7	8	9	7	0	1	2
0	1	6	0	1	9	7	1	2	0	6	2
0	1	2	0	1	2	0	1	2	0	1	2

MINIMUM CUT: SHADED = OBJECT



object side (S) · cut neighbor edge, costs λ · label differs from the pixel's own guess (score ≥ 5)

Brush: 9 object | 5 unsure | 0 background

Image: Noisy blob | Two shapes | Low contrast

Smoothness **λ = 2**

COST OF THE MINIMUM CUT

Data cost: each pixel pays for its label	132
Cut neighbor pairs: 28 × λ (2)	56
Total = min cut = max flow	**188**

32 pixels are object. Smoothing overruled 7 pixels (dots), because flipping them saves more boundary cost than it adds in data cost.

How the network is built. Edge **s → pixel** has capacity = score: cutting it labels the pixel background and costs its score. Edge **pixel → t** has capacity 9 − score: cutting it labels the pixel object. Each pair of neighboring pixels gets edges of capacity **λ** both ways, so splitting neighbors costs λ. Every cut is one labeling and its capacity is that labeling's total cost, so the minimum cut is the cheapest labeling.

WHERE THIS SHOWS UP IN AI

Graph cuts power classic tools such as GrabCut for lifting an object out of a photo, and they were long a standard step in medical-image labeling. Modern pipelines often let a neural network produce the per-pixel scores and then clean them up with a smoothing step like this one.

Code: BFS finds the shortest augmenting path

CODE WALKTHROUGH

This is the search step of Edmonds–Karp. Residual capacities live in a dictionary of dictionaries: `res[u][v]` is the room on step $u \to v$. Switch to **Trace** to watch the queue and the `parent` map grow.

bfs_path.py

Read | Edit & run | Trace

```python
from collections import deque
def bfs_path(res, s, t):
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v, room in res[u].items():
            if room > 0 and v not in parent:
                parent[v] = u
                queue.append(v)
    path = [t] if t in parent else []
    while path and path[-1] != s:
        path.append(parent[path[-1]])
    return path[::-1]
res = {"s": {"a": 4, "b": 2}, "a": {"b": 3, "t": 1},
       "b": {"t": 5}, "t": {}}
print(bfs_path(res, "s", "t"))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
['s', 'a', 't']
```

LINE 1 OF 16

Show all notes in code

```python
from collections import deque
```

Imports `deque`, a double-ended queue. Removing from its front with `popleft()` is fast. A plain list would have to shift every remaining element on `pop(0)`.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

A breadth-first search over the residual network that returns the shortest augmenting path from s to t as a list of vertices, or an empty list when no augmenting path exists.

Code: BFS finds the shortest augmenting path: every line explained

1	`from collections import deque`	Imports `deque`, a double-ended queue. Removing from its front with `popleft()` is fast. A plain list would have to shift every remaining element on `pop(0)`.
2	`def bfs_path(res, s, t):`	Defines the search. `res` holds residual capacities, `s` and `t` name the source and sink. It returns the vertices of a shortest augmenting path, or `[]` if there is none.
3	`    parent, queue = {s: None}, deque([s])`	Two assignments in one line. `parent` records, for every vertex found so far, the vertex we came from; `s` has no parent, so it maps to `None`. The same dictionary doubles as the set of visited vertices. The queue starts with `s`.
4	`    while queue:`	Repeat while vertices are waiting to be explored. An empty deque counts as false, so the loop ends when the search runs out of vertices.
5	`        u = queue.popleft()`	Takes the vertex that has waited longest: first in, first out. That order explores all vertices one step from `s`, then all vertices two steps away, and so on. So the first route found to `t` has the fewest edges.
6	`        for v, room in res[u].items():`	Visits every step out of `u` together with its residual capacity. For `u = "s"` this gives `("a", 4)` and then `("b", 2)`, in the order the dictionary stores them.
7	`            if room > 0 and v not in parent:`	Two conditions. The step must have room: a step with room `0` is full and cannot carry more. And `v` must be new: a vertex that already has a parent was reached earlier by a route at least as short.
8	`                parent[v] = u`	Records that `v` was reached from `u`. Following these links backwards from `t` will trace the path.
9	`                queue.append(v)`	Puts `v` at the back of the queue to explore later. With this data the queue becomes `[a, b]` after exploring `s`, and `t` is discovered while exploring `a`.
10	`    path = [t] if t in parent else []`	A conditional expression. If the search reached `t`, the path starts at `t`; otherwise it is empty, which tells the max-flow loop that the flow is already maximum.
11	`    while path and path[-1] != s:`	Walks backwards while the list is non-empty and its last vertex, `path[-1]`, is not yet `s`.
12	`        path.append(parent[path[-1]])`	Adds the parent of the last vertex. Here the list grows `[t]`, then `[t, a]`, then `[t, a, s]`.
13	`    return path[::-1]`	The slice `[::-1]` reverses the list, so the path reads from `s` to `t`: `["s", "a", "t"]`.
14	`res = {"s": {"a": 4, "b": 2}, "a": {"b": 3, "t": 1},`	The residual capacities before any flow is sent, so they equal the capacities. `res["a"]["t"]` is 1. A depth-first search that always takes the first neighbor would wander `s → a → b → t`.
15	`       "b": {"t": 5}, "t": {}}`	The second line of the same dictionary: Python lets a literal continue inside its braces. `t` has no outgoing steps, so it maps to an empty dictionary.
16	`print(bfs_path(res, "s", "t"))`	Prints `['s', 'a', 't']`: two edges, the shortest route, even though its bottleneck is only 1. Edmonds–Karp counts edges, not width, and that rule is what bounds the number of rounds.

Code: pushing flow and canceling it

CODE WALKTHROUGH

This is the push step, run on the trap network from the augmenting-path slide. The second path uses the backward step $b \dashrightarrow a$. Watch `res["a"]["b"]` drop to 0 and come back to 1 in **Trace**.

augment.py

Read | Edit & run | Trace

```
1  cap = {"s": {"a": 1, "b": 1}, "a": {"b": 1, "t": 1},
2         "b": {"t": 1}, "t": {}}
3  res = {u: dict(nbrs) for u, nbrs in cap.items()}
4  for u in cap:
5      for v in cap[u]:
6          res[v].setdefault(u, 0)
7  paths = [["s", "a", "b", "t"], ["s", "b", "a", "t"]]
8  for path in paths:
9      hops = list(zip(path, path[1:]))
10     push = min(res[u][v] for u, v in hops)
11     for u, v in hops:
12         res[u][v] -= push
13         res[v][u] += push
14     print(" -> ".join(path), "pushes", push)
15 flow = {u + v: res[v][u] for u in cap for v in cap[u]}
16 print(flow, "value:", flow["sa"] + flow["sb"])
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
s -> a -> b -> t pushes 1
s -> b -> a -> t pushes 1
{'sa': 1, 'sb': 1, 'ab': 0, 'at': 1, 'bt': 1} value: 2
```

LINE 1 OF 16

Show all notes in code

```
cap = {"s": {"a": 1, "b": 1}, "a": {"b": 1, "t": 1},
```

The capacities of the trap network as a dictionary of dictionaries: every edge has capacity 1. `cap["a"]["b"]` is the capacity of edge `a → b`.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Builds the residual network for a set of capacities, pushes the bottleneck along two augmenting paths, and recovers the final flow on every edge.

Code: pushing flow and canceling it: every line explained

1	`cap = {"s": {"a": 1, "b": 1}, "a": {"b": 1, "t": 1},`	The capacities of the trap network as a dictionary of dictionaries: every edge has capacity 1. `cap["a"]["b"]` is the capacity of edge `a → b`.
2	`       "b": {"t": 1}, "t": {}}`	The rest of the same dictionary. The sink `t` has no outgoing edges.
3	`res = {u: dict(nbrs) for u, nbrs in cap.items()}`	A dictionary comprehension that copies each inner dictionary with `dict(...)`, so changing `res` later never changes `cap`. With zero flow, the room on every edge equals its capacity.
4	`for u in cap:`	Loops over every vertex to add the backward entries.
5	`    for v in cap[u]:`	Loops over every edge `u → v`.
6	`        res[v].setdefault(u, 0)`	Makes sure the backward step `v → u` exists, with room `0` because nothing can be canceled yet. `setdefault` only inserts when the key is missing, so a real edge `v → u` would keep its own capacity.
7	`paths = [["s", "a", "b", "t"], ["s", "b", "a", "t"]]`	Two augmenting paths in the order a careless search might find them. The second one walks `b → a`, against edge `a → b`. That step only has room after the first push.
8	`for path in paths:`	The augmenting loop: one round per path.
9	`    hops = list(zip(path, path[1:]))`	`zip` pairs each vertex with the next one: `[("s", "a"), ("a", "b"), ("b", "t")]` in round 1. `list(...)` stores the pairs so we can loop over them twice.
10	`    push = min(res[u][v] for u, v in hops)`	The bottleneck: the smallest room on the path. It is 1 in both rounds. In round 2 the room on `("b", "a")` is 1 only because round 1 sent a unit along `a → b`.
11	`    for u, v in hops:`	Updates every step of the path.
12	`        res[u][v] -= push`	Less room in the direction we pushed. In round 1, `res["a"]["b"]` drops from 1 to `0`.
13	`        res[v][u] += push`	More room in the opposite direction, because the units just sent could be canceled later. Round 1 sets `res["b"]["a"] = 1`; round 2 uses it and raises `res["a"]["b"]` back to 1, which cancels the flow on `a → b`.
14	`    print(" -> ".join(path), "pushes", push)`	Reports each round, for example `s -> a -> b -> t pushes 1`.
15	`flow = {u + v: res[v][u] for u in cap for v in cap[u]}`	Reads the flow on each original edge `u → v` from its backward room `res[v][u]`, because the room to cancel is exactly the flow sent. (This shortcut works here because no two edges point in opposite directions.) The keys glue the names together, like `"ab"`.
16	`print(flow, "value:", flow["sa"] + flow["sb"])`	Prints `{'sa': 1, 'sb': 1, 'ab': 0, 'at': 1, 'bt': 1} value: 2`. Edge `a → b` ends at `0` because round 2 canceled it, and the value 2 is the total leaving `s`.

Code: maximum matching by augmenting paths

CODE WALKTHROUGH

The same augmenting-path idea, specialized to workers and jobs, fits in a short recursive function. In **Trace**, watch Cy's search re-route Ava from Train to Clean.

matching.py

Read | Edit & run | Trace

```
1  likes = {"Ava": ["Label", "Train", "Clean"],
2           "Ben": ["Label"], "Cy": ["Train", "Eval"]}
3  match = {}
4  def place(w, seen):
5      for job in likes[w]:
6          if job not in seen:
7              seen.add(job)
8              if job not in match or place(match[job], seen):
9                  match[job] = w
10                 return True
11     return False
12 for w in likes:
13     ok = place(w, set())
14     print(w, "placed" if ok else "left out", len(match))
15 print(sorted((w, job) for job, w in match.items()))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
Ava placed 1
Ben placed 2
Cy placed 3
[('Ava', 'Clean'), ('Ben', 'Label'), ('Cy', 'Train')]
```

LINE 1 OF 15

Show all notes in code

```
likes = {"Ava": ["Label", "Train", "Clean"],
```

The bipartite graph as an adjacency dictionary: each worker maps to the jobs they are qualified for. The order of each list is the order the jobs are tried.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Finds a maximum matching between workers and the jobs they are qualified for, placing workers one at a time and re-routing earlier workers when needed.

Code: maximum matching by augmenting paths: every line explained

1	`likes = {"Ava": ["Label", "Train", "Clean"],`	The bipartite graph as an adjacency dictionary: each worker maps to the jobs they are qualified for. The order of each list is the order the jobs are tried.
2	`         "Ben": ["Label"], "Cy": ["Train", "Eval"]}`	The dictionary continues inside its braces. Ben is qualified only for Label; Cy for Train and Eval. Three workers, four jobs.
3	`match = {}`	The current matching, stored as job → worker, so "who holds this job?" is a single lookup. It starts empty.
4	`def place(w, seen):`	A recursive search for an augmenting path that starts at worker `w`. `seen` holds the jobs already considered in this search, so the recursion cannot go round in circles. It returns `True` if `w` ends up with a job.
5	`    for job in likes[w]:`	Tries the jobs of `w` in order.
6	`        if job not in seen:`	Skips a job that someone further up the chain is already trying to take. Without this check, two workers could keep handing the same job back and forth forever.
7	`            seen.add(job)`	Marks the job as considered before going deeper.
8	`            if job not in match or place(match[job], seen):`	The heart of the algorithm. The job works for `w` if it is free, or if its current holder can move to another job (the recursive call). Because `or` stops at the first true part, the recursion only runs when the job is taken. The chain of calls is the augmenting path: new edge, given-up edge, new edge, and so on.
9	`                match[job] = w`	Gives the job to `w`. If the recursive call succeeded, the old holder already has another job, so nobody is left without one.
10	`                return True`	Reports success back up the chain of calls.
11	`    return False`	No job worked, so `w` is not placed by this search and the matching is unchanged.
12	`for w in likes:`	Places the workers one at a time, like the lab. Each successful search grows the matching by exactly one edge.
13	`    ok = place(w, set())`	Starts each worker's search with a fresh, empty `seen` set.
14	`    print(w, "placed" if ok else "left out", len(match))`	Prints the result and the matching size so far. For Ben the output is `Ben placed 2`: Ben takes Label and Ava moves to Train. For Cy, Ava cannot take Label back from Ben, so she moves on to Clean. Cy re-routes Ava even though Eval is free, because this depth-first search follows list order; the lab's BFS would take Eval directly. Both give a maximum matching.
15	`print(sorted((w, job) for job, w in match.items()))`	Flips each pair to (worker, job) and sorts them so the output is predictable: `[('Ava', 'Clean'), ('Ben', 'Label'), ('Cy', 'Train')]`.

Design practice: network models in real code

DESIGN PRACTICE

A max-flow routine is short; modeling the problem correctly and trusting the answer take most of the work. Click each principle (or use ↓).

01 **Model the problem before writing code**

02 Find augmenting paths with BFS (Edmonds–Karp)

03 Keep residual capacities in a dict of dicts

04 Use integer capacities

05 Check the flow rules after every augmentation in tests

06 Report the minimum cut along with the number

07 Use a tested library in production

Principle 1 of 7

Model the problem before writing code

Decide what flows (gigabytes per second, workers, trucks), what the vertices and edges are, and which units the capacities use. Problems with several sources or sinks become standard by adding a **super-source** joined to every real source, and a super-sink in the same way.

Undirected links become two directed edges, one each way.

✗ AVOID THIS

```
flow = max_flow(links)   # what flows? from where?
```

✓ DO THIS

```
for dc, gbps in supply.items():
    cap["SRC"][dc] = gbps    # super-source
flow = max_flow(cap, "SRC", "cluster")
```

Design practice: network models in real code: all principles (1/2)

Model the problem before writing code

Decide what flows (gigabytes per second, workers, trucks), what the vertices and edges are, and which units the capacities use. Problems with several sources or sinks become standard by adding a **super-source** joined to every real source, and a super-sink in the same way.

Undirected links become two directed edges, one each way.

```
for dc, gbps in supply.items():
    cap["SRC"][dc] = gbps     # super-source
flow = max_flow(cap, "SRC", "cluster")
```

Find augmenting paths with BFS (Edmonds–Karp)

Any augmenting path gives a correct answer eventually, but a depth-first search can need a number of rounds proportional to the capacities: 2000 rounds for the 1000-capacity diamond. BFS bounds the work by $O(|V| \cdot |E|^2)$ no matter how large the numbers are.

```
path = bfs_path(res, s, t)    # fewest edges:
                              # O(V * E**2)
total
```

Keep residual capacities in a dict of dicts

`res[u][v]` gives the room on a step in one lookup and lists a vertex's steps with `res[u].items()`. Add a backward entry for every edge, but never overwrite a real edge that already runs the other way.

```
for u in cap:
    for v in cap[u]:
        res[v].setdefault(u, 0)   # keeps it
```

Use integer capacities

With whole numbers every push adds at least 1, so the algorithm must stop, and the answer is exact. Floating-point capacities pick up rounding error and can make "is there room?" checks wrong. Scale to a small unit instead: megabits rather than fractions of gigabits.

```
cap["a"]["b"] = 300    # Mbit/s, exact
```

Check the flow rules after every augmentation in tests

A small checker catches most bookkeeping bugs (a missed backward update, a wrong sign). Run it after every push in your tests: capacity on every edge, conservation at every vertex except s and t.

```
def check(cap, f, s, t):
    assert all(0 <= f[u][v] <= c for u in cap
               for v, c in cap[u].items())
    for x in cap.keys() - {s, t}:
        assert inflow(f, x) == outflow(f, x)
```

Report the minimum cut along with the number

"Max throughput is 19" invites the question "why not more?". The minimum cut answers it: these links are full and nothing else matters. Compute it from the final residual network (the vertices still reachable from s) and show it to whoever has to buy the upgrade.

```
S = reachable(res, s)
cut = [(u, v) for u in S for v in cap[u]
       if v not in S]
print("max flow:", value, "limited by:", cut)
```

Design practice: network models in real code: all principles (2/2)

Use a tested library in production

Write the algorithm once to understand it, then use a library for real systems. `networkx` offers `maximum_flow` and `minimum_cut` with faster algorithms (preflow-push) and years of testing. For very large graphs, OR-Tools provides optimized max-flow and min-cost-flow solvers.

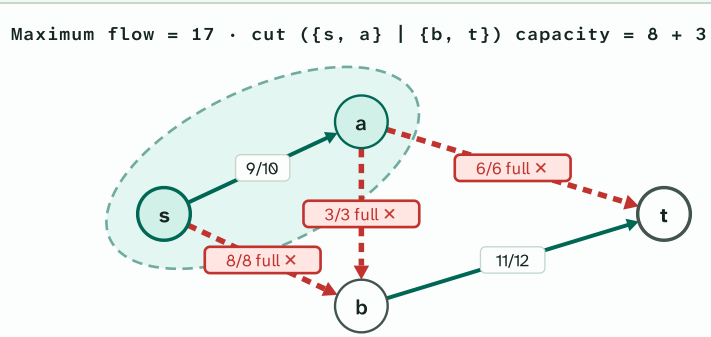
```
import networkx as nx
G = nx.DiGraph()
G.add_edge("s", "a", capacity=10)
value, flows = nx.maximum_flow(G, "s", "t")
cut_value, (S, T) = nx.minimum_cut(G, "s",
"t")
```

Worked example: maximum flow by augmenting paths

WORKED EXAMPLE

PROBLEM

Find the maximum flow from s to t by the Ford–Fulkerson method, one augmenting path at a time, in the network with capacities $s \to a = 10$, $s \to b = 8$, $a \to b = 3$, $a \to t = 6$ and $b \to t = 12$.



Maximum flow = 17 · check: conservation holds at a (in 9, out 9) and at b (in 11, out 11), and the cut ({s, a} | {b, t}) has capacity 8 + 3 + 6 = 17

1. Setup: every flow starts at 0, and a flow must respect each capacity and keep flow in equal to flow out at every vertex between s and t. We improve the flow one augmenting path at a time, always looking for spare room in the residual network.
2. Augment along s → a → t, whose bottleneck is min(10, 6) = 6, set by a → t. Pushing 6 raises the flow value to 6, and a → t is now saturated at 6/6.
3. Augment along s → b → t, whose bottleneck is min(8, 12) = 8, set by s → b. Pushing 8 raises the flow value to 14, and s → b is now saturated at 8/8.
4. Augment along s → a → b → t, whose bottleneck is min(4, 3, 4) = 3, set by a → b. Pushing 3 raises the flow value to 17, and a → b is now saturated at 3/3.
5. No augmenting path remains: in the residual network s can still reach only {s, a}, because every forward edge from that set to the rest of the network is saturated, so no route to t survives. The reachable set therefore splits the network into a cut, and counting that cut in the answer step will prove that 17 cannot be beaten.
6. **Answer: the maximum flow is 17, and conservation checks out at a (in 9, out 9) and at b (in 11, out 11), with every capacity respected. The cut ({s, a} | {b, t}) has capacity 8 + 3 + 6 = 17, so a flow and a cut with equal values certify each other by the max-flow min-cut theorem and 17 is provably maximal.**

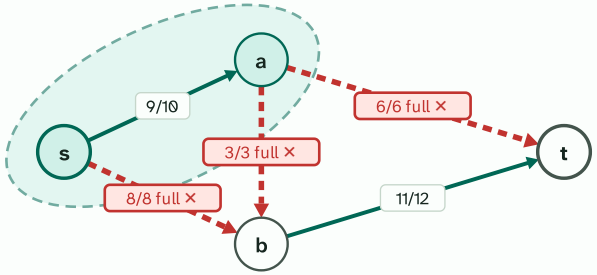
Worked example: a minimum cut from the residual network

WORKED EXAMPLE

PROBLEM

For the same network — capacities $s \to a = 10$, $s \to b = 8$, $a \to b = 3$, $a \to t = 6$ and $b \to t = 12$ — whose maximum flow is 17, from the previous example, find a minimum s-t cut by reading the final residual network.

Minimum cut capacity = 17 · all 4 cuts: {s}: 18 · {s, a}: 17 · {s, b}: 22 · {s, a, b}: 18



Minimum cut capacity = 17 · check: cut ({s, a} | {b, t}) = 8 + 3 + 6 = 17, which equals the maximum flow

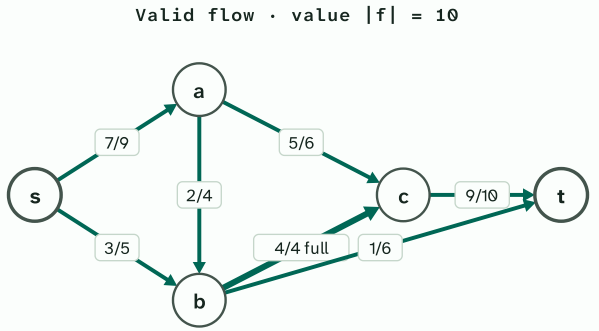
1. Setup: start from a maximum flow of this network, whose value is the total leaving s, and build its residual network to see what can still move. The method is to mark every vertex reachable from s in that residual network, because those vertices will form the source side of the cut.
2. Compute the residual room on every edge: forward room is capacity minus flow, and backward room is the flow that could be canceled. The forward rooms are s → a 1, s → b 0, a → b 0, a → t 0, b → t 1, and the 3 edges with 0 forward room are drawn dashed because nothing more can pass that way.
3. Walk the residual network from s, following every step that still has positive room. The walk reaches exactly {s, a}: every original edge from that set to the other side is saturated (s → b carries 8/8, a → b carries 3/3, a → t carries 6/6), so no residual step crosses to the other side and the walk stops there.
4. The cut edges are the original edges that run from the reachable side {s, a} to the other side {b, t}: s → b, a → b, a → t, drawn dashed with an X. Adding only their capacities gives 8 + 3 + 6 = 17, and edges that point back into {s, a} are never counted.
5. **Answer: the minimum cut capacity is 17. Checking all 4 cuts gives {s}: 18, {s, a}: 17, {s, b}: 22, {s, a, b}: 18, so the smallest is 17, and it also equals the maximum flow of 17 — exactly as the max-flow min-cut theorem promises. The edges of this cut are the links that truly limit the network.**

Worked example: auditing a flow, vertex by vertex

WORKED EXAMPLE

PROBLEM

A network has capacities $s \to a = 9$, $s \to b = 5$, $a \to b = 4$, $a \to c = 6$, $b \to c = 4$, $b \to t = 6$ and $c \to t = 10$. A teammate claims the assignment $7, 3, 2, 5, 4, 1, 9$ on those edges (in the same order) is a valid flow. Audit it: check the capacity rule on every edge, then check conservation at a, b and c one vertex at a time, and find the flow's value if it is valid.



Valid flow, value |f| = 10 · check: capacity holds on all 7 edges (b → c is saturated at 4/4) and conservation holds at a (7 in, 7 out), b (5 in, 5 out), c (9 in, 9 out)

1. Setup: someone hands us a finished flow assignment, not a maximum-flow computation. Before trusting its value we audit it against the two flow rules from the first slide: capacity on every edge, then conservation at every vertex between s and t, summed independently from the edge list.
2. Capacity audit: compare every flow with its capacity, edge by edge. All 7 edges satisfy 0 ≤ f ≤ c, so rule one holds. The tightest edge is b → c at 4/4, completely saturated, so it is the edge to watch in the vertex checks.
3. Vertex a: the inflow is s → a 7 = 7 and the outflow is a → b 2 + a → c 5 = 7. They match, so nothing is created or lost at a and conservation holds here.
4. Vertex b: the inflow is s → b 3 + a → b 2 = 5 and the outflow is b → c 4 + b → t 1 = 5. They match, so nothing is created or lost at b and conservation holds here.
5. Vertex c: the inflow is a → c 5 + b → c 4 = 9 and the outflow is c → t 9 = 9. They match, so nothing is created or lost at c and conservation holds here — and this is the tight check of the audit, because the saturated edge b → c feeds it.
6. Value: add the flow leaving s, 10, and cross-check at the other end, where 10 enters t. Conservation at every middle vertex forces the two totals to agree, and they do, so the value of this flow is 10 — a statement about this assignment only, not a claim that no larger flow exists.
7. **Answer: the assignment is a valid flow of value 10. Every capacity is respected and conservation held at a, b, c, checked by direct summation over the edge list. Notice what the audit does not say: 10 need not be the maximum, and proving maximality would take a cut of capacity 10, which is a separate argument.**

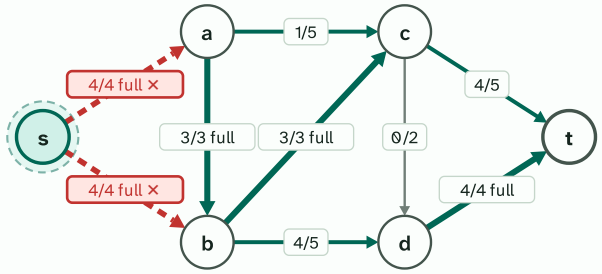
Worked example: the augmenting path that has to cancel flow

WORKED EXAMPLE

PROBLEM

Find the maximum flow from s to t in the network with capacities $s \to a = 4$, $s \to b = 4$, $a \to b = 3$, $a \to c = 5$, $b \to c = 3$, $b \to d = 5$, $c \to d = 2$, $c \to t = 5$ and $d \to t = 4$, using a depth-first search that always takes the first augmenting path it finds. The greedy early paths block the optimum; watch for the augmenting path that can only exist because the residual network lets it cancel flow on one edge and reroute that flow instead.

Maximum flow = 8 · cut ({s} | {a, b, c, d, t}) capacity = 8



Maximum flow = 8 · check: conservation holds at a (in 4, out 4), b (in 7, out 7), c (in 4, out 4), d (in 4, out 4), and the cut ({s} | {a, b, c, d, t}) has capacity 4 + 4 = 8

1. Setup: run Ford–Fulkerson with a depth-first search, which grabs the first augmenting path it stumbles on instead of the shortest one. Greedy first choices can block the optimum, and this network is built so that they do: the only escape will be a backward residual step that cancels flow already sent.
2. Augment along s → a → b → c → d → t with bottleneck 2, raising the flow value to 2.
3. Augment along s → a → b → c → t with bottleneck 1, raising the flow value to 3.
4. Augment along s → a → c → t with bottleneck 1, raising the flow value to 4.
5. Augment along s → b → d → t with bottleneck 2, raising the flow value to 6.
6. Augment along s → b → d ⇢ c → t with bottleneck 2, raising the flow value to 8. The step d ⇢ c is a backward residual step against edge c → d: its room is exactly the 2 units that edge is carrying, and pushing here subtracts 2 from it, leaving 0. That is legal because the residual network only offers room that exists — canceling flow cannot break capacity, and at d the inflow and the outflow both drop by 2, so conservation survives. What it buys is rerouting: the units that were crossing c → d are freed to leave c another way, and the outgoing room at d is handed to the flow arriving on this path instead.
7. No augmenting path remains: from s every forward residual step is saturated, so the reachable set is just {s}. A purely forward search would have stranded this flow below the maximum; it was the backward step, canceling an early greedy choice, that let the flow finish the job.
8. **Answer: the maximum flow is 8, certified by the cut ({s} | {a, b, c, d, t}) of capacity 8. The route to it mattered: the final flow carries 0 on c → d even though an intermediate flow carried 2 there — the difference is exactly what the backward step canceled and rerouted.**

Practice: network flows

PRACTICE

Edge labels show capacity, or flow / capacity. Answers are whole numbers unless the problem offers buttons.

BOTTLENECK OF A PATH

An augmenting path s – b – c – t uses these steps in the current flow. How many units can be pushed along it?

Step	Uses edge	Flow / capacity	Kind
s → b	s → b	3 / 5	forward
b → c	b → c	10 / 11	forward
c → t	c → t	0 / 4	forward

Type your answer

Check

Hint: forward room = capacity − flow; backward room = flow (what can be canceled).

Show worked solution

New problem ↻

Room on each step: s → b: 5 − 3 = 2; b → c: 11 − 10 = 1; c → t: 4 − 0 = 4. The bottleneck is the smallest, **1**, so pushing raises the flow value by 1.

PROBLEM TYPE

- Bottleneck of a path
- Valid flow?
- Capacity of a cut
- Maximum flow

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: network flows: sample problems with solutions

BOTTLENECK OF A PATH

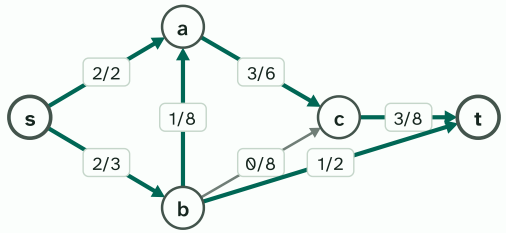
An augmenting path s – b – d – c – t uses these steps in the current flow. How many units can be pushed along it?

Step	Uses edge	Flow / capacity	Kind
s → b	s → b	3 / 6	forward
b → d	b → d	0 / 3	forward
d ⇢ c	c → d	3 / 3	backward
c → t	c → t	5 / 8	forward

Room on each step: s → b: 6 − 3 = 3; b → d: 3 − 0 = 3; d ⇢ c: 3 (cancel up to the flow); c → t: 8 − 5 = 3. The bottleneck is the smallest, **3**, so pushing raises the flow value by 3.

VALID FLOW?

Is this a valid flow? Check both flow rules.

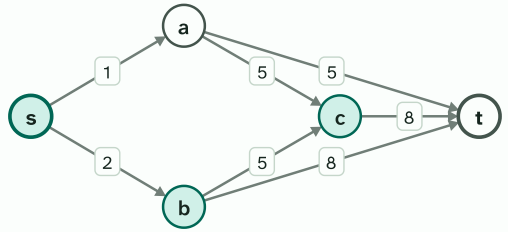


Capacity rule: every edge has $0 \le f \le c$. **Conservation:** a: in 3, out 3 ✓; b: in 2, out 2 ✓; c: in 3, out 3 ✓.

Both rules hold, so it is a **valid** flow, with value 4 (the total leaving s).

CAPACITY OF A CUT

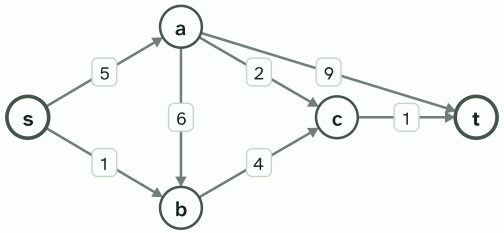
The shaded vertices form $S = \{s, b, c\}$ and the rest form $T = \{a, t\}$. Labels are capacities. What is $c(S, T)$?



Edges from S to T: s → a (1), c → t (8), b → t (8). Sum: 1 + 8 + 8 = **17**. The edge a → c points from T into S and is not counted.

MAXIMUM FLOW

Labels are capacities. What is the value of a maximum flow from s to t?



Edmonds–Karp (shortest paths first) pushes: 1) s → a → t: 5; 2) s → b → c → t: 1. Total **6**. Certificate: S = {s} (what s can still reach). Its cut edges s → a (5), s → b (1) sum to 6, equal to the flow, so no flow can do better.

Check yourself: network flows

CHECK YOURSELF

QUESTION 1 OF 10

At an interior vertex b, flow arrives on two edges carrying 4 and 3 units. Two edges leave b, and one of them carries 5. What must the other carry?

A 12

B 2

C 7

D Anything up to its capacity

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: network flows: answers explained (1/2)

1. At an interior vertex b, flow arrives on two edges carrying 4 and 3 units. Two edges leave b, and one of them carries 5. What must the other carry?

Answer B: 2

Conservation: flow in equals flow out at every vertex other than s and t. In: $4 + 3 = 7$. Out: $5 + x = 7$, so $x = 2$.

2. Edge $a \to b$ has capacity 7 and carries 3 units. What does it contribute to the residual network?

Answer B: A step $a \to b$ with room 4 and a step $b \dashrightarrow a$ with room 3

Forward room is $c - f = 7 - 3 = 4$. Backward room is the flow that could be canceled, $f = 3$.

3. Why does the residual network include backward steps?

Answer D: So the algorithm can cancel flow it sent earlier and send it along a better route

A first path may block better ones. A backward step undoes some earlier flow, which lets Ford–Fulkerson reach the true maximum. Without it the trap network gets stuck at 1 instead of 2.

4. In one network you find a flow of value 12 and a cut of capacity 12. What can you conclude?

Answer A: The flow is maximum and the cut is minimum

Every flow is at most every cut. A flow equal to a cut cannot grow, and that cut cannot shrink. Each certifies the other.

5. With $S = \{s, a\}$, the edges between the groups are $s \to b$ (4), $a \to b$ (3), $a \to t$ (5) and $b \to a$ (6). What is $c(S, T)$?

Answer A: 12

Only edges from S to T count: $4 + 3 + 5 = 12$. The edge $b \to a$ runs from T into S, so it is ignored.

6. What does the Edmonds–Karp algorithm add to the Ford–Fulkerson method?

Answer D: Always augment along a shortest path (fewest edges), found by BFS

Edmonds–Karp is Ford–Fulkerson with BFS paths. That choice limits the work to $O(|V| \cdot |E|^2)$ regardless of capacity values. The largest-bottleneck rule is a different, also reasonable, heuristic.

7. All capacities are whole numbers. Which statement is guaranteed?

Answer C: Ford–Fulkerson stops, and it finds a maximum flow with a whole number on every edge

Each push adds at least 1, so the method must stop, and starting from 0 every update adds or subtracts whole numbers. Usually only the edges of a minimum cut end up full.

8. To find a maximum matching between 5 workers and 6 jobs as a flow problem, which capacities do you use?

Answer B: 1 on every edge: s → worker, worker → job, job → t

Capacity 1 into each worker and out of each job means each is used at most once, so an integer maximum flow is a maximum matching.

Check yourself: network flows: answers explained (2/2)

9. Ava, Ben and Cy are each qualified only for jobs in {Label, Train}. What does Hall's condition tell you?

Answer C: At most two can be placed, because three workers share only two jobs

For $A = \{\text{Ava}, \text{Ben}, \text{Cy}\}$, $N(A) = \{\text{Label}, \text{Train}\}$ and $|N(A)| = 2 < 3$. Hall's condition checks every group of workers, including groups of two or three.

10. The minimum cut of a data pipeline consists of links L1 and L2. You can afford one upgrade. Which choice can raise the maximum throughput?

Answer D: Upgrade L1 or L2

The maximum flow equals the minimum cut, so only raising a cut edge can raise it. If the network has several minimum cuts, you may need to upgrade an edge in each of them before the throughput moves.

Logic gates: the building blocks

CONCEPT

DEFINITION

A logic gate is a tiny circuit that reads one or two input bits and produces one output bit. A bit is 0 (low voltage, false) or 1 (high voltage, true). Each gate below shows its symbol, its algebra and its truth table. Click a column of a table or an input circle to try the gate.

AND $x \cdot y$

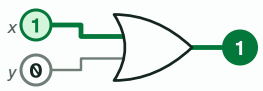
1 only when both inputs are 1. Logic writes $x \wedge y$.



x	0	0	1	1
y	0	1	0	1
out	**0**	**0**	**0**	**1**

OR $x + y$

1 when at least one input is 1. Logic writes $x \vee y$.



x	0	0	1	1
y	0	1	0	1
out	**0**	**1**	**1**	**1**

NOT x'

Flips the bit. Also called an inverter. Logic writes $\neg x$.



x	0	1
out	**1**	**0**

NAND $(xy)'$

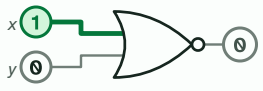
AND, then NOT. 0 only when both inputs are 1. Also written $x \uparrow y$.



x	0	0	1	1
y	0	1	0	1
out	**1**	**1**	**1**	**0**

NOR $(x + y)'$

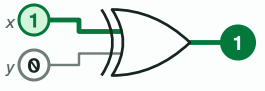
OR, then NOT. 1 only when both inputs are 0. Also written $x \downarrow y$.



x	0	0	1	1
y	0	1	0	1
out	**1**	**0**	**0**	**0**

XOR $x \oplus y$

Exclusive or. 1 when the two inputs are different.



x	0	0	1	1
y	0	1	0	1
out	**0**	**1**	**1**	**0**

READING THE SYMBOLS

A small circle on an output, called a bubble, means "then flip the result". So NAND is AND followed by NOT. Engineers usually write $x \cdot y$ as xy, the way algebra writes $3x$, and they write the complement of x as x' or $\bar{x}$.

WHERE THIS SHOWS UP IN AI

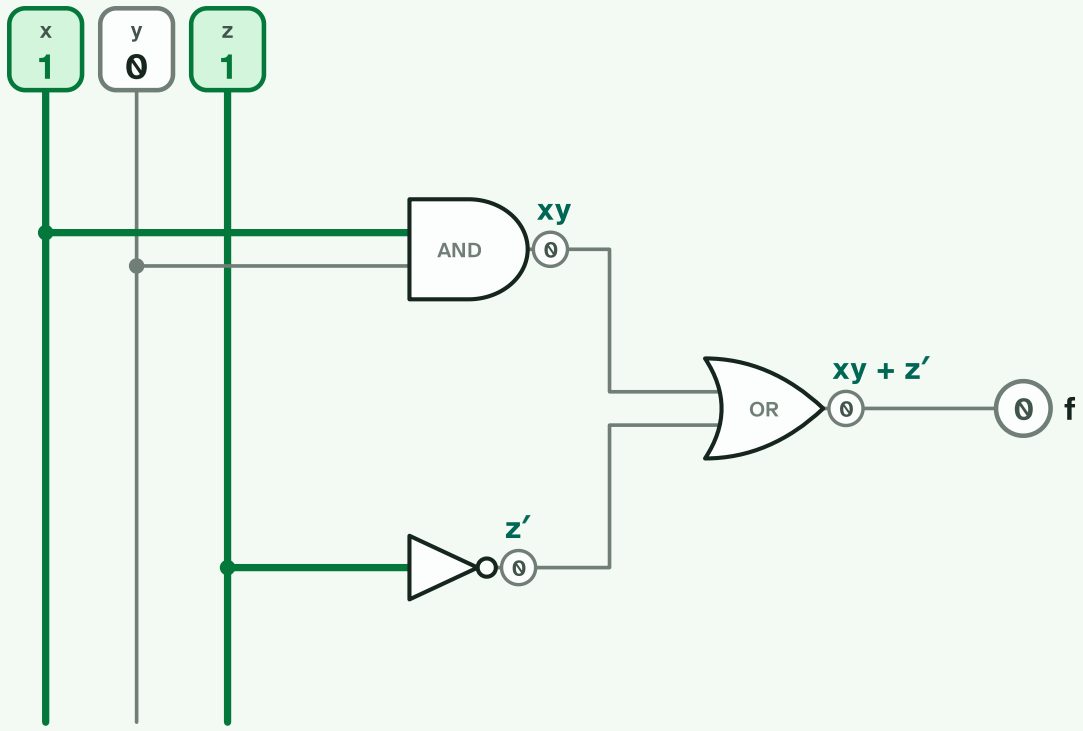
A GPU contains tens of billions of transistors wired into gates like these. Every multiplication inside a neural network becomes streams of bits flowing through AND, OR and XOR gates in adder circuits, which you will build later in this module.

Combinatorial circuits: from wires to expressions

CONCEPT

DEFINITION

A combinatorial circuit is a network of gates with no loops, so signals flow one way, from inputs to outputs. Its output depends only on the inputs right now, so it has no memory, and a truth table describes it completely.



x	y	z	xy	z′	f = xy + z′
0	0	0	0	1	**1**
0	0	1	0	0	**0**
0	1	0	0	1	**1**
0	1	1	0	0	**0**
1	0	0	0	1	**1**
1	0	1	0	0	**0**
1	1	0	1	1	**1**
1	1	1	1	0	**1**

WORKED EXAMPLE

Circuit to expression. Work left to right and label each gate's output. The AND gate gives xy, the NOT gate gives z', and the OR gate joins them into $f = xy + z'$. The table fills in the same way, one column per gate.

Expression to circuit. Draw one gate per operator, innermost first. $x(y + z)$ is an OR gate for $y + z$ feeding an AND gate with x.

THEOREM / RULE

Two circuits are **equivalent** when their truth tables match. $x(y + z)$ uses 2 gates and $xy + xz$ uses 3, yet they are equivalent by the distributive law. Boolean algebra finds the cheaper circuit.

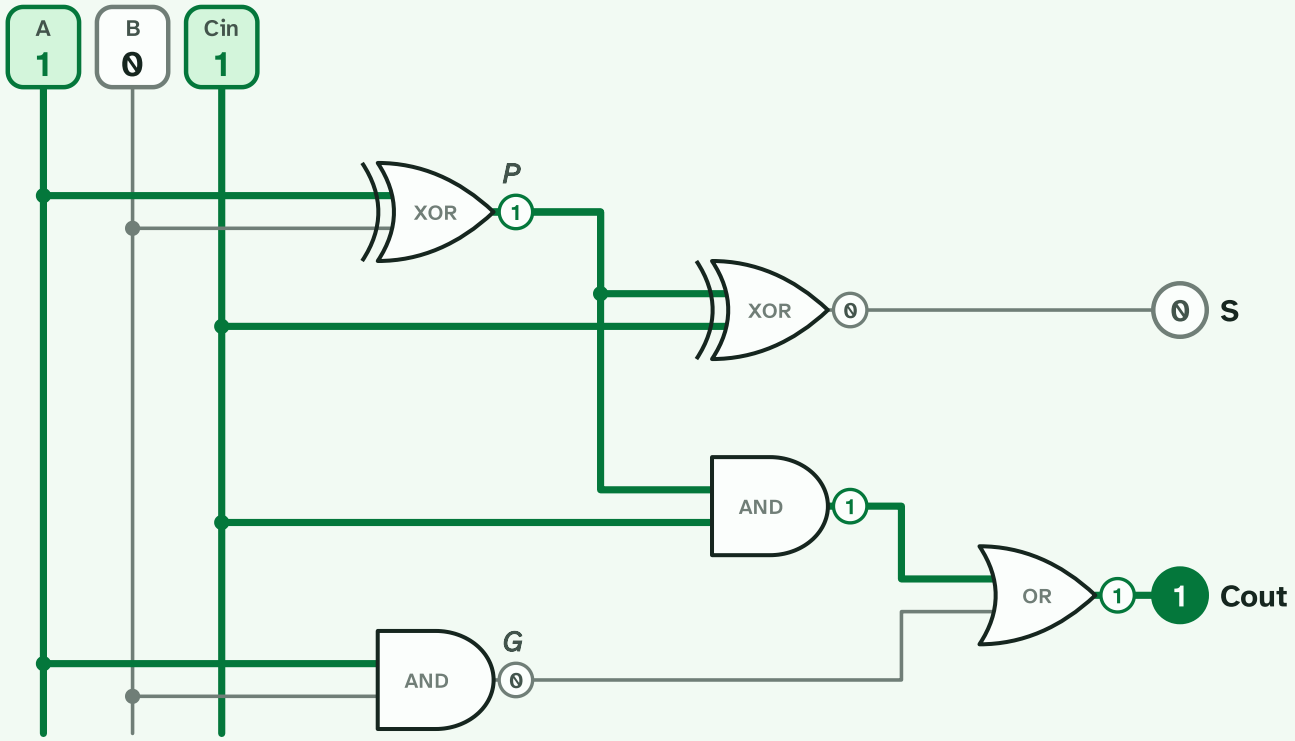
Click a switch or a table row. A dot joins wires. Wires that cross without a dot are not connected.

Lab: a circuit simulator, from half adder to 4-bit adder

INTERACTIVE LAB

Click the switches at the top of the input wires. Lit green wires carry 1 and thin gray wires carry 0; the small circle after each gate shows its output. A dot joins wires, and wires that cross without a dot are not connected. Click a row of the truth table to load those inputs.

Circuit: Half adder | **Full adder** | Majority vote | Multiplexer | 4-bit adder



Full adder

Adds three bits, A, B and the carry Cin from the column to the right. It is two half adders plus an OR gate that merges their carries.

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$

Right now: A = 1, B = 0, Cin = 1 gives **S = 0, Cout = 1**.

A	B	Cin	S	Cout
0	0	0	**0**	**0**
0	0	1	**1**	**0**
0	1	0	**1**	**0**
0	1	1	**0**	**1**
1	0	0	**1**	**0**
1	0	1	**0**	**1**
1	1	0	**0**	**1**
1	1	1	**1**	**1**

LOOK CLOSER

The signal P = A ⊕ B is called propagate and G = AB is called generate. Fast carry-lookahead adders are built from exactly these two signals.

Boolean algebra: one algebra, three languages

CONCEPT

DEFINITION

A Boolean algebra is a set B with operations $+$ and $\cdot$, a complement operation x', and two different elements 0 and 1 that obey these laws for all $x, y, z \in B$.

Law	Sum form	Product form
Commutative	$x + y = y + x$	$xy = yx$
Associative	$(x + y) + z = x + (y + z)$	$(xy)z = x(yz)$
Distributive	$x + yz = (x + y)(x + z)$	$x(y + z) = xy + xz$
Identity	$x + 0 = x$	$x \cdot 1 = x$
Complement	$x + x' = 1$	$xx' = 0$

Every other law follows from these five pairs, including idempotent $x + x = x$, bound $x + 1 = 1$, absorption $x + xy = x$, involution $x'' = x$, and De Morgan $(x + y)' = x'y'$.

THEOREM / RULE

Duality. Swap $+ \leftrightarrow \cdot$ and $0 \leftrightarrow 1$ in any law and you get another law. The dual of $x + 0 = x$ is $x \cdot 1 = x$, and the dual of $x + xy = x$ is $x(x + y) = x$.

	Sets (Module 1)	Logic (Module 2)	Circuits
Objects	subsets of U	statements	bits
"or"	$A \cup B$	$p \vee q$	$x + y$, OR gate
"and"	$A \cap B$	$p \wedge q$	xy, AND gate
"not"	A'	$\neg p$	x', NOT gate
Zero	$\varnothing$	F	0
One	U	T	1
De Morgan	$(A \cup B)' = A' \cap B'$	$\neg(p \vee q) \equiv \neg p \wedge \neg q$	$(x + y)' = x'y'$

WHY IT MATTERS

Prove a law once in Boolean algebra and it holds for sets, for logic and for circuits at the same time. One difference from school algebra needs care. Here $1 + 1 = 1$, because $+$ means OR.

WHERE THIS SHOWS UP IN AI

A transformer's causal mask is a grid of bits. Position i may look at position j only when $j \le i$. Combining it with a padding mask is an elementwise AND of two bit grids.

Boolean functions and normal forms

CONCEPT

DEFINITION

A Boolean function of n variables gives an output bit for each of the 2^n input rows. A literal is a variable or its complement, like x or x'. A minterm is a product with every variable once, such as $x'yz$. It equals 1 on exactly one row.

THEOREM / RULE

Every Boolean function equals the sum of the minterms of the rows where it is 1. That sum is its disjunctive normal form (DNF), also called a sum of minterms.

WORKED EXAMPLE

Read the DNF straight off the table. For each row with $f = 1$, write x where the bit is 1 and x' where it is 0, then add the products. Row 011 gives $x'yz$, the minterm m_3.

WHY IT MATTERS

This proves that AND, OR and NOT can build *every* Boolean function, since any truth table turns into a DNF. With n variables there are 2^{2^n} functions: 16 for two variables and 256 for three.

row	x	y	z	f (majority)	term
0	0	0	0	0	maxterm $x + y + z$
1	0	0	1	0	maxterm $x + y + z'$
2	0	1	0	0	maxterm $x + y' + z$
3	0	1	1	**1**	minterm $x'yz$
4	1	0	0	0	maxterm $x' + y + z$
5	1	0	1	**1**	minterm $xy'z$
6	1	1	0	**1**	minterm xyz'
7	1	1	1	**1**	minterm xyz

$$f = x'yz + xy'z + xyz' + xyz = \Sigma m(3, 5, 6, 7)$$

CONJUNCTIVE NORMAL FORM, BRIEFLY

The conjunctive normal form (CNF) works from the 0-rows instead. Each 0-row gives a maxterm, a sum that is 0 on that row only. Multiplying them gives $f = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$.

COMMON PITFALL

The rules are opposite. In a minterm a 0 bit becomes x'. In a maxterm a 0 bit becomes plain x.

Lab: from truth table to circuit

INTERACTIVE LAB

Click the **f** buttons to set the output of each row. The page reads the DNF straight off the table, finds a minimal sum of products, counts the gates, and draws the circuit. Click anywhere else on a row to feed that input into the circuit.

Start from: Majority | xy + z′ | Multiplexer | Parity (XOR) | Random | All 0

m	x	y	z	f	minterm
0	0	0	0	0	x′y′z′
1	0	0	1	0	x′y′z
2	0	1	0	0	x′yz′
3	0	1	1	1	x′yz
4	1	0	0	0	xy′z′
5	1	0	1	1	xy′z
6	1	1	0	1	xyz′
7	1	1	1	1	xyz

COST: GATES AND GATE INPUTS

DNF — 8 gates

Minimal — 4 gates

The DNF needs 4 AND, 1 OR and 3 NOT gates with 19 gate inputs in all. The minimal form needs 3 AND, 1 OR and 0 NOT with 9 inputs.

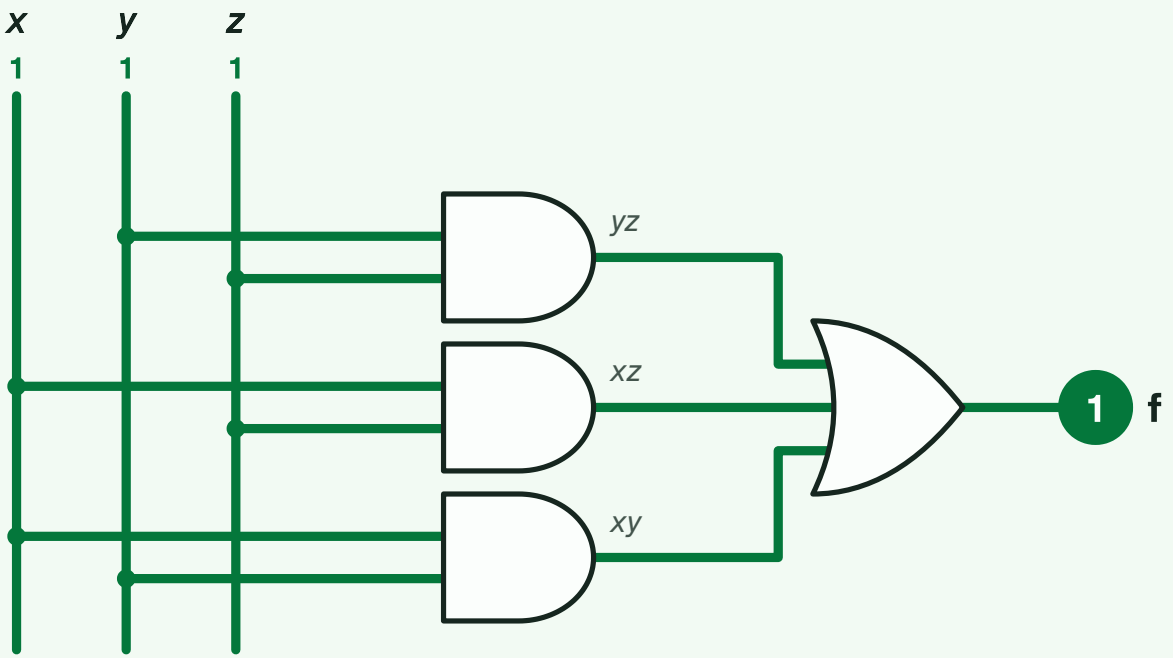
MINTERM LIST $f = \Sigma m(3, 5, 6, 7)$

DNF (FROM TABLE) $x'yz + xy'z + xyz' + xyz$

MINIMAL SOP $yz + xz + xy$

CNF (FROM 0-ROWS) $(x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$

Draw the circuit for: Minimal form | DNF



Minimizing circuits: algebra and Karnaugh maps

CONCEPT

WHY IT MATTERS

Every gate costs chip area and money, every level of gates adds delay, and every switching gate burns power. A smaller equivalent circuit is cheaper, faster and cooler.

WORKED EXAMPLE

Simplify the majority function $x'yz + xy'z + xyz' + xyz$. Since $xyz + xyz = xyz$ (idempotent law), we may reuse xyz three times.

$= (x'yz + xyz) + (xy'z + xyz) + (xyz' + xyz)$	idempotent
$= yz(x' + x) + xz(y' + y) + xy(z' + z)$	distributive
$= yz \cdot 1 + xz \cdot 1 + xy \cdot 1$	complement
$= xy + xz + yz$	identity

From 8 gates (4 three-input ANDs, 3 NOTs, an OR) down to 4.

WHERE THIS SHOWS UP IN AI

Quantized neural networks store weights in 8, 4 or even 1 bit so that inference needs fewer, simpler gates. A 1-bit network replaces multiplication with XNOR gates and bit counting.

DEFINITION

A Karnaugh map is a truth table folded into a grid. Rows and columns follow Gray-code order 00, 01, 11, 10, so neighboring cells differ in exactly one variable. Two neighboring 1s merge because $Ax + Ax' = A$.

THEOREM / RULE

Grouping rules.

1. Groups are rectangles of 1, 2, 4, 8 or 16 cells.
2. A group holds only 1s (and don't-care cells, marked X).
3. Make every group as large as possible.
4. Use as few groups as possible. Overlaps are fine.
5. Edges wrap around, so the left column touches the right one and the top row touches the bottom.
6. Each group is one product term. Keep the variables that stay constant across the group and drop the ones that change.

A group of 2^k cells drops k variables. In a 4-variable map a group of 4 gives a 2-literal term and a group of 8 gives a single literal. A group that cannot grow any larger is a prime implicant.

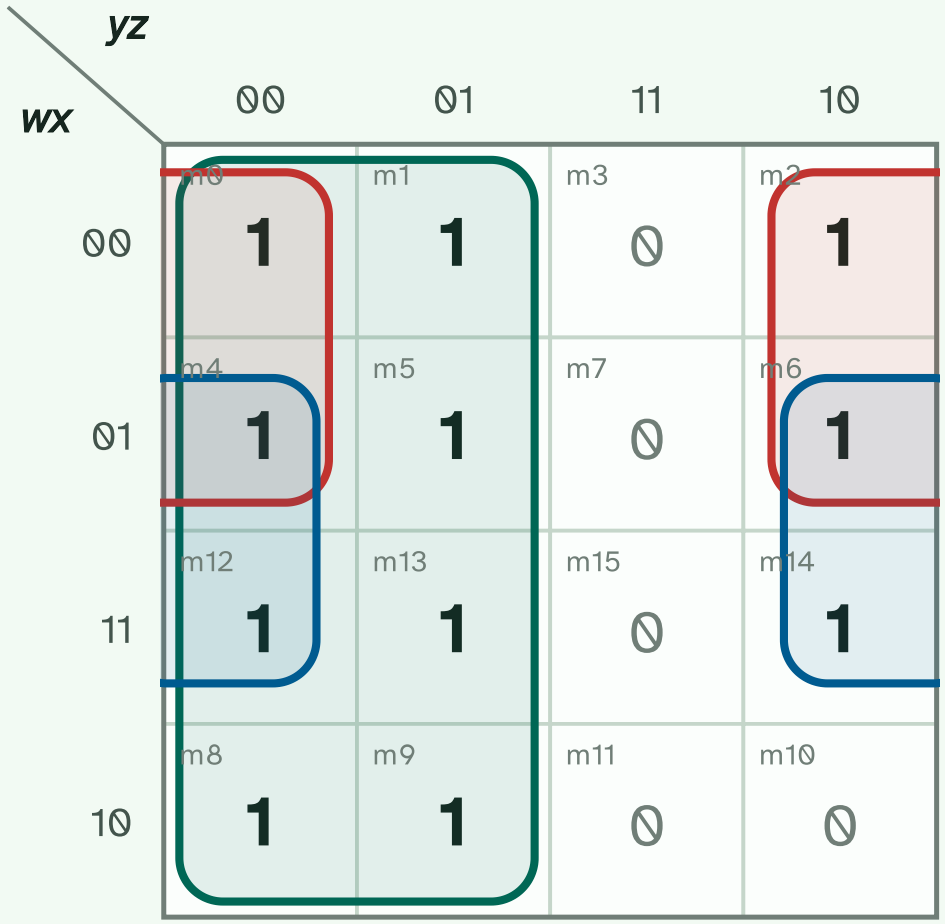
Lab: Karnaugh maps

INTERACTIVE LAB

Click a cell to cycle it through 0, 1 and X (a don't-care, meaning a row whose output you are free to choose). The page finds the fewest and largest groups, draws them, and explains each one. Groups that run off one edge continue on the opposite edge.

Variables: 3 | 4 (4 selected)

Examples: Textbook classic | Corners and center | Odd digits with don't-cares | Clear



MINIMAL SUM OF PRODUCTS

$$f = y' + w'z' + xz'$$

Writing out all 11 minterms costs 11 terms and 44 literals. The map needs 3 terms and 5 literals.

EACH GROUP AND WHY

y' · cells m0, m1, m4, m5, m8, m9, m12, m13 (8)
w, x and z change inside the group, so they drop out. y = 0 holds in every cell, which gives y'. Essential, because it is the only possible group that covers m1.

$w'z'$ · cells m0, m2, m4, m6 (4)
x and y change inside the group, so they drop out. w = 0 and z = 0 hold in every cell, which gives $w'z'$. Essential, because it is the only possible group that covers m2.

xz' · cells m4, m6, m12, m14 (4)
w and y change inside the group, so they drop out. x = 1 and z = 0 hold in every cell, which gives xz'. Essential, because it is the only possible group that covers m14.

Lab: NAND is all you need

INTERACTIVE LAB

Build each target using only NAND gates. Pick the two inputs of every gate from its menus; the last gate is the output. The truth table checks your circuit on every input, and you can click the inputs in the drawing to watch signals flow.

Target: NOT x | x AND y | x OR y | x NOR y | x XOR y

THEOREM / RULE

A set of gates is functionally complete when it can build every Boolean function. $\{\text{AND}, \text{OR}, \text{NOT}\}$ is complete because every function has a DNF. So if NAND can make NOT, AND and OR, NAND alone is complete. NOR alone is complete too. $\{\text{AND}, \text{OR}\}$ is not complete, because with every input 0 any AND-OR circuit outputs 0, so it can never build NOT.

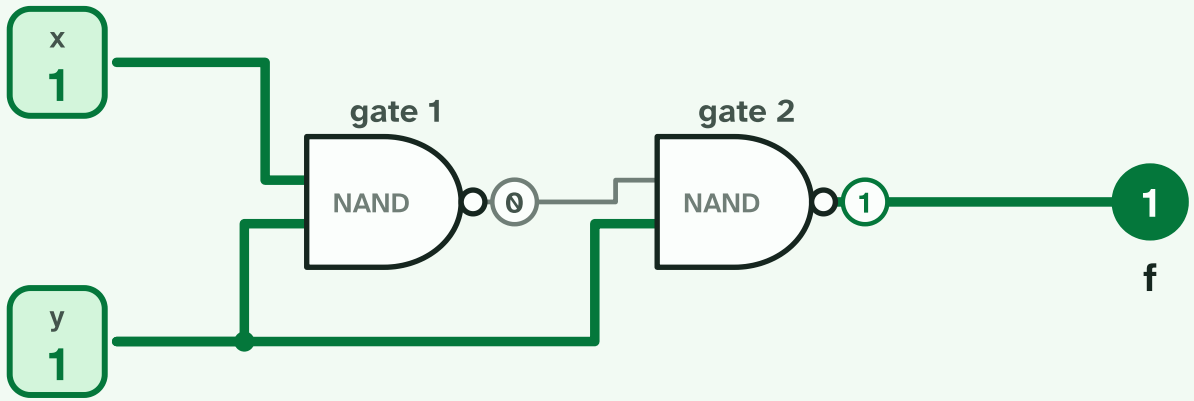
Target: $f = xy$ with 2 NAND gates.

gate 1 [x] [y]

gate 2 [gate 1] [y]

Show a solution | Reset

2 of 4 rows match. Hint: NAND is AND followed by NOT. Can a second NAND undo that NOT?

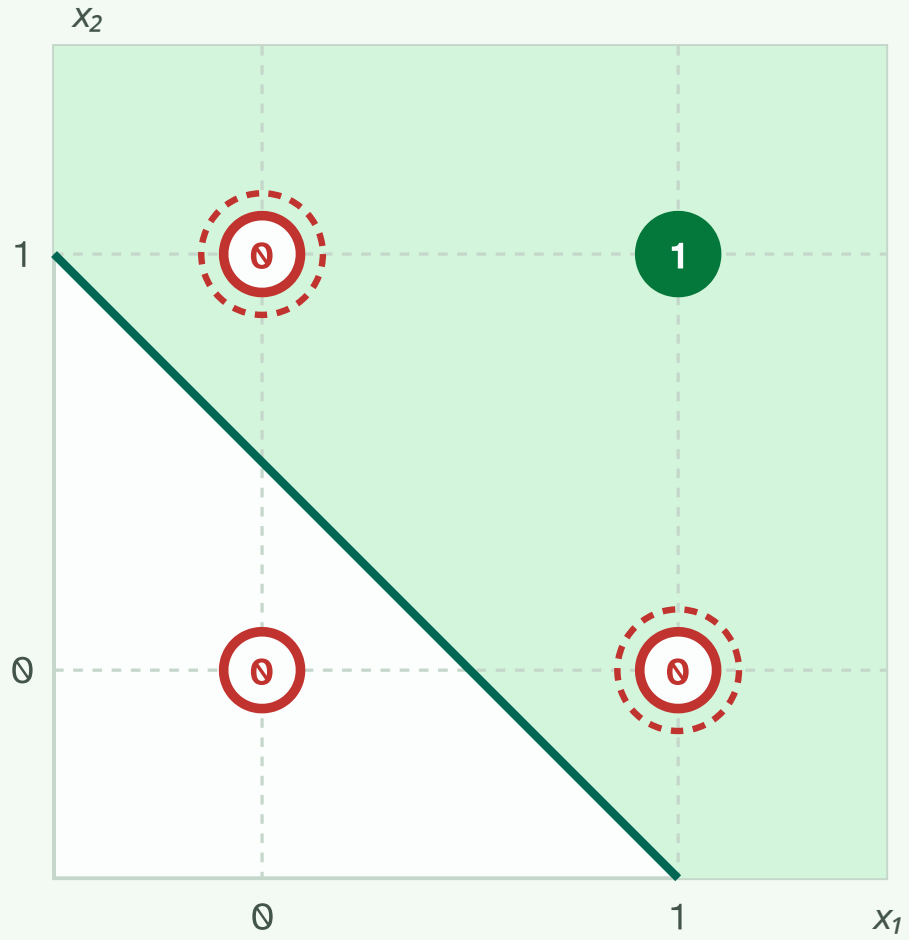


x	y	g1	g2	your f	target	
0	0	1	1	**1**	0	✗
0	1	1	0	**0**	0	✓
1	0	1	1	**1**	0	✗
1	1	0	1	**1**	1	✓

Lab: a perceptron is a logic gate

INTERACTIVE LAB

A perceptron, the simplest artificial neuron, outputs 1 when $w_1 x_1 + w_2 x_2 + b > 0$ and 0 otherwise. Its dividing line $w_1 x_1 + w_2 x_2 + b = 0$ splits the plane. Drag the weights and the bias until the perceptron acts like the chosen gate.



Target gate: AND | OR | NAND | NOR | XOR

w_1 1.0

w_2 1.0

b -0.5

x_1	x_2	$w_1x_1 + w_2x_2 + b$	output	AND	
0	0	-0.5	**0**	0	✓
0	1	0.5	**1**	0	✗
1	0	0.5	**1**	0	✗
1	1	1.5	**1**	1	✓

2 of 4 rows match. Points circled in red are on the wrong side. Moving the bias b slides the line without turning it.

Try all 729 weight settings

WHERE THIS SHOWS UP IN AI

AND, OR, NAND and NOR are linearly separable, so one neuron can compute each of them. Binary neural networks go the other way and restrict weights to ±1, so a neuron becomes an XNOR gate plus a bit counter, which is cheap enough for phones and tiny chips.

Code: every input, every output

CODE WALKTHROUGH

This program evaluates the circuit $f = xy + z'$ from slide 2 on all 8 inputs, prints its truth table, and builds the DNF from the 1-rows. Switch to **Trace** to watch the minterms collect.

truth_table_dnf.py

Read | Edit & run | Trace

```
1  from itertools import product
2  def f(x, y, z):
3      return (x & y) | (z ^ 1)
4
5  minterms = []
6  print("x y z | f")
7  for x, y, z in product([0, 1], repeat=3):
8      out = f(x, y, z)
9      print(x, y, z, "|", out)
10     if out == 1:
11         term = ""
12         for name, bit in zip("xyz", (x, y, z)):
13             term += name if bit else name + "'"
14         minterms.append(term)
15 print("DNF:", " + ".join(minterms))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
x y z | f
0 0 0 | 1
0 0 1 | 0
0 1 0 | 1
```

LINE 1 OF 15

Show all notes in code

```
from itertools import product
```

Imports `product` from Python's standard `itertools` module. `product` builds every combination of choices, which is exactly the list of rows of a truth table.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Runs a Boolean function on every combination of inputs, prints the truth table, and writes the disjunctive normal form by turning each 1-row into a minterm.

Code: every input, every output: every line explained

1	`from itertools import product`	Imports `product` from Python's standard `itertools` module. `product` builds every combination of choices, which is exactly the list of rows of a truth table.
2	`def f(x, y, z):`	Defines the circuit as a Python function of three bits. Each argument will be the integer 0 or 1. Nothing runs until the loop on line 7 calls it.
3	`    return (x & y) | (z ^ 1)`	`&` is bitwise AND, `|` is bitwise OR, and `z ^ 1` flips one bit (0 becomes 1, 1 becomes 0), which is NOT for a single bit. So this line is $xy + z'$. We avoid `~z` because in Python `~1` is –2, not 0.
5	`minterms = []`	An empty list that will collect one minterm string for each row where the output is 1.
6	`print("x y z | f")`	Prints the header of the truth table so the columns are labeled.
7	`for x, y, z in product([0, 1], repeat=3):`	`product([0, 1], repeat=3)` yields the 8 tuples (0,0,0), (0,0,1), ..., (1,1,1) in binary counting order, the same order as the rows of a truth table. Each tuple is unpacked into `x`, `y` and `z`.
8	`    out = f(x, y, z)`	Runs the circuit on this row and stores the output bit. On the first row (0,0,0), `out` is 1 because $z' = 1$.
9	`    print(x, y, z, "|", out)`	Prints one row of the truth table. `print` separates its arguments with spaces, so the output looks like `0 0 0 | 1`.
10	`    if out == 1:`	Only the rows where the function is 1 contribute a minterm to the DNF. Rows with output 0 are skipped.
11	`        term = ""`	Starts an empty string for this row's minterm. It will grow one literal at a time.
12	`        for name, bit in zip("xyz", (x, y, z)):`	`zip` pairs each variable name with its bit, giving ("x", x), ("y", y) and ("z", z). Looping over a string gives its characters one at a time.
13	`            term += name if bit else name + "'"`	This conditional expression adds the plain variable for a 1 bit and the variable with a prime (its complement) for a 0 bit. For row (0,1,0) the term becomes `x'yz'`.
14	`        minterms.append(term)`	Adds the finished minterm to the list. Five rows have output 1, so the list ends with five minterms.
15	`print("DNF:", " + ".join(minterms))`	`" + ".join(...)` glues the minterms together with plus signs between them. The result is the DNF, five 3-literal terms, much longer than the original $xy + z'$. That gap is why we minimize.

Code: a ripple-carry adder in Python

CODE WALKTHROUGH

A full adder written with bitwise gates, chained into a ripple-carry adder, and checked against Python's own addition. In **Trace** mode, watch `carry` ripple from column to column.

ripple_carry.py

Read | Edit & run | Trace

```
1  def full_adder(a, b, cin):
2      s = a ^ b ^ cin
3      cout = (a & b) | (cin & (a ^ b))
4      return s, cout
5
6  def ripple_add(x, y):
7      carry, out = 0, ""
8      for a, b in zip(reversed(x), reversed(y)):
9          s, carry = full_adder(int(a), int(b), carry)
10         out = str(s) + out
11     return str(carry) + out
12
13 for m, n in [(11, 6), (15, 1), (9, 3)]:
14     x, y = f"{m:04b}", f"{n:04b}"
15     total = ripple_add(x, y)
16     print(x, "+", y, "=", total, int(total, 2) == m + n)
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
1011 + 0110 = 10001 True
1111 + 0001 = 10000 True
1001 + 0011 = 01100 True
```

LINE 1 OF 16

Show all notes in code

```
def full_adder(a, b, cin):
```

Defines a full adder. It takes two number bits `a` and `b` and the carry `cin` coming from the column to the right.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Builds a full adder from XOR, AND and OR, chains four of them into a 4-bit adder, and checks three sums against ordinary integer addition.

Code: a ripple-carry adder in Python: every line explained

1	`def full_adder(a, b, cin):`	Defines a full adder. It takes two number bits `a` and `b` and the carry `cin` coming from the column to the right.
2	`    s = a ^ b ^ cin`	The sum bit is the XOR of all three inputs, which is 1 when an odd number of inputs are 1. For 1 + 1 + 0 this gives 0.
3	`    cout = (a & b) | (cin & (a ^ b))`	The carry-out is 1 when both number bits are 1 (ab, called generate), or when exactly one is 1 and a carry arrives ($c_{in}(a \oplus b)$, called propagate). This is the full-adder circuit from the simulator lab.
4	`    return s, cout`	Returns both output bits as a tuple. Together they form the 2-bit answer: 1 + 1 + 1 = 3 = 11 in binary returns `(1, 1)`.
6	`def ripple_add(x, y):`	Adds two equal-length bit strings, such as "1011" and "0110", the way the hardware does, with one full adder per column.
7	`    carry, out = 0, ""`	The first column has no incoming carry, so `carry` starts at 0 (like the c0 = 0 wire in the lab). `out` collects the sum bits.
8	`    for a, b in zip(reversed(x), reversed(y)):`	`reversed` walks each string from the right, because addition starts in the rightmost (least significant) column. `zip` pairs the two bits of each column.
9	`        s, carry = full_adder(int(a), int(b), carry)`	`int(a)` turns the character "1" into the number 1. The old carry goes in, and the new carry comes out and replaces it. This line is the ripple, because each column waits for the previous carry.
10	`        out = str(s) + out`	Puts the new sum bit in front of the bits found so far, since we are moving right to left.
11	`    return str(carry) + out`	After the last column, the final carry becomes the extra leftmost bit. Adding two 4-bit numbers can need 5 bits.
13	`for m, n in [(11, 6), (15, 1), (9, 3)]:`	Three test cases, written in decimal so you can check them in your head. (15, 1) is interesting because the carry ripples through every column.
14	`    x, y = f"{m:04b}", f"{n:04b}"`	The format code `04b` writes a number in binary (`b`), padded to 4 digits with leading zeros (`04`). So 6 becomes "0110".
15	`    total = ripple_add(x, y)`	Runs the simulated circuit. For 11 + 6 the result is the 5-bit string "10001".
16	`    print(x, "+", y, "=", total, int(total, 2) == m + n)`	`int(total, 2)` reads the bit string as a base-2 number. Comparing it with `m + n` checks the circuit against Python's own addition, so every line should end in `True`.

Design practice: building reliable logic

DESIGN PRACTICE

Hardware engineers and programmers who manipulate bits follow the same habits. Click each principle (or use ↓) to compare code to avoid with code to write.

01 **Minimize the logic before you build it**
02 Test every input combination
03 Build big circuits from small verified blocks
04 Convert to NAND or NOR with De Morgan
05 Name signals for what they mean
06 Mind the propagation delay
07 Use bitwise operators correctly in code

Principle 1 of 7

Minimize the logic before you build it

The DNF read off a truth table is correct but wasteful. Run it through a Karnaugh map or the algebra laws first. Fewer gates mean less chip area, a shorter path for signals (so a faster clock) and less power. In code, a simpler expression is also easier to read and test.

Logic-synthesis tools do this automatically for millions of gates, using the same laws you learned here.

✗ AVOID THIS

```
# read straight off the table: 3 terms, 9 literals
f = (x&y&z) | (x&y&(1-z)) | (x&(1-y)&z)
```

✓ DO THIS

```
# same truth table after a K-map
f = (x & y) | (x & z)
# distributive law: f = x & (y | z)
```

Design practice: building reliable logic: all principles (1/2)

Minimize the logic before you build it

The DNF read off a truth table is correct but wasteful. Run it through a Karnaugh map or the algebra laws first. Fewer gates mean less chip area, a shorter path for signals (so a faster clock) and less power. In code, a simpler expression is also easier to read and test.

Logic-synthesis tools do this automatically for millions of gates, using the same laws you learned here.

```
# same truth table after a K-map
f = (x & y) | (x & z)
# distributive law: f = x & (y | z)
```

Test every input combination

A circuit with n inputs has only 2^n input rows. For n up to about 20 a program can check all of them in seconds, and such an exhaustive test proves the circuit correct. Spot checks with two or three hand-picked inputs miss the one row where the bug lives.

```
from itertools import product
for a, b, c in product([0, 1], repeat=3):
    s, cout = full_adder(a, b, c)
    assert 2 * cout + s == a + b + c
```

Build big circuits from small verified blocks

Verify a half adder, build a full adder from two of them, then chain full adders into an n-bit adder. Each block is small enough to test exhaustively, and the big circuit inherits that confidence. Real chips are designed as a hierarchy of blocks in exactly this way.

```
def half_adder(a, b):
    return a ^ b, a & b
def full_adder(a, b, c):
    s1, c1 = half_adder(a, b)
    s, c2 = half_adder(s1, c)
    return s, c1 | c2
```

Convert to NAND or NOR with De Morgan

Factories prefer one gate type, and a CMOS NAND gate needs only 4 transistors. Any AND-OR circuit becomes a NAND-NAND circuit by double negation: $xy + zw = ((xy)'\,(zw)'\,)'$. Same function, one kind of gate.

```
def nand(a, b):
    return 1 - (a & b)
# AND-OR form:  f = x·y + z·w
f = nand(nand(x, y), nand(z, w))   # NAND-NAND
```

Name signals for what they mean

A wire called `t2` tells a reviewer nothing. A wire called `carry_in` tells them where it comes from and what it does. Hardware description languages and Python code both benefit from names that match the circuit diagram.

```
propagate = a ^ b
sum_bit = propagate ^ carry_in
carry_out = (a & b) | (carry_in & propagate)
```

Design practice: building reliable logic: all principles (2/2)

Mind the propagation delay

Every gate takes a little time to settle, and delays add up along the longest path (the **critical path**). In a ripple-carry adder the carry crosses every column, so an n-bit adder needs about $2n$ gate delays. Carry-lookahead adders compute every carry directly from the propagate and generate signals, trading more gates for much less delay.

```
# lookahead: every carry from P, G at once
c1 = g0 | (p0 & c0)
c2 = g1 | (p1 & g0) | (p1 & p0 & c0)
```

Use bitwise operators correctly in code

For bits stored as integers, `&`, `|` and `^` work bit by bit. Two traps catch everyone. First, `~1` is −2 in Python, so flip one bit with `bit ^ 1` or `1 - bit`. Second, `&` and `|` bind tighter than comparisons, so always put comparisons in parentheses. The same rule applies to filters in NumPy and pandas.

```
flipped = bit ^ 1          # 1 -> 0, 0 -> 1
both = (a > 0) & (b > 0)
```

Worked example: minimizing with a Karnaugh map

WORKED EXAMPLE

PROBLEM

A function $f(a, b, c)$ is 1 exactly for the minterms m_0, m_2, m_4, m_5, m_6 — that is, (a, b, c) is one of $000, 010, 100, 101, 110$. Use a Karnaugh map to find a minimal sum of products for f.

a \ bc	00	01	11	10
0	m0 **1**	m1 0	m3 0	m2 **1**
1	m4 **1**	m5 **1**	m7 0	m6 **1**

f(a, b, c) = Σm(0, 2, 4, 5, 6)

Dashed outline (grp1): the quad m0, m2, m4 and m6 — c = 0 in every cell, giving c′.

Solid outline (grp2): the pair m4 and m5 — a = 1 and b = 0 in every cell, giving ab′.

$$f = c' + ab'$$

abc	000	001	010	011	100	101	110	111
f (minterms)	1	0	1	0	1	1	1	0
c′ + ab′	1	0	1	0	1	1	1	0

Answer: f = c′ + ab′ — check: c′ + ab′ is 1 exactly on {0, 2, 4, 5, 6}, matching the given minterms

1. The map has one row for each value of a and one column for each value of bc, listed in Gray order 00, 01, 11, 10. Gray order matters because neighboring cells then differ in exactly one variable, even across the wrap-around edges, and that is what lets neighboring 1s merge into bigger groups.
2. Now transfer the truth table into the map: a 1 goes in each of the 5 minterm cells m0, m2, m4, m5 and m6, and a 0 goes everywhere else. Nothing about the function changes here; only its layout changes, from a list of rows into a picture where patterns become visible.
3. Circle the quad m0, m2, m4 and m6, shown with the dashed outline. In every cell of the group c = 0 while a and b change, so a and b drop out and the group gives the term c′. The group wraps around the left and right edges of the map, which is allowed because those edges are neighbors in Gray order.
4. Circle the pair m4 and m5, shown with the solid outline. Cells may repeat from an earlier group, because overlap is allowed. In every cell of the group a = 1 and b = 0 while c changes, so c drops out and the group gives the term ab′.
5. Read the 2 groups off as a sum of products: f = c′ + ab′. Every 1 on the map is covered and each group is as large as it can possibly be, which is exactly what minimal means here: 2 terms and 3 literals instead of 5 full minterms.
6. **Answer: f = c′ + ab′. The check is in the strip below the map: evaluating c′ + ab′ on all eight inputs gives 1 exactly on rows 000, 010, 100, 101 and 110, which is precisely the minterm list {0, 2, 4, 5, 6} we started with.**

Worked example: evaluating a ripple-carry adder

WORKED EXAMPLE

PROBLEM

The ripple-carry adder of this module adds 1011_2 (eleven) and 0110_2 (six). Evaluate it bit by bit from the least significant bit, tracking every carry, the way the circuit does.

	bit 4	bit 3	bit 2	bit 1	bit 0
carry in	1	1	1	0	0
a = 1011	—	1	0	1	1
b = 0110	—	0	1	1	0
sum	1	0	0	0	1

Check: 1011 is 11 and 0110 is 6, and 11 + 6 = 17, which is 10001 in binary.

Answer: 1011 + 0110 = 10001 (17) — check: 11 + 6 = 17, and each sum bit was the parity of its three inputs

1. Line the two numbers up in columns, 1011 over 0110, with an empty carry row above them. A ripple-carry adder is just a chain of full adders, and each column must wait for the carry from its right neighbor, so the work has to go from right to left, starting at bit 0.
2. Bit 0: the full adder receives a0 = 1, b0 = 0 and the starting carry c0 = 0, a total of one 1. The sum bit is the parity of the three inputs, so s0 = 1, and the carry-out is 1 only when at least two inputs are 1, so c1 = 0.
3. Bit 1: the full adder receives a1 = 1, b1 = 1 and the carry c1 = 0 passed from bit 0, a total of two 1s. The sum bit is the parity of the three inputs, so s1 = 0, and the carry-out is 1 only when at least two inputs are 1, so c2 = 1, which ripples into the next column to the left.
4. Bit 2: the full adder receives a2 = 0, b2 = 1 and the carry c2 = 1 passed from bit 1, a total of two 1s. The sum bit is the parity of the three inputs, so s2 = 0, and the carry-out is 1 only when at least two inputs are 1, so c3 = 1, which ripples into the next column to the left.
5. Bit 3: the full adder receives a3 = 1, b3 = 0 and the carry c3 = 1 passed from bit 2, a total of two 1s. The sum bit is the parity of the three inputs, so s3 = 0, and the carry-out is 1 only when at least two inputs are 1, so c4 = 1, which ripples into the next column to the left.
6. The leftover carry c4 = 1 becomes a fifth sum bit, s4 = 1, sitting to the left of s3. The full result now reads 10001, and dropping that final carry would have lost the most valuable bit of the answer.
7. **Answer: 1011 + 0110 = 10001, which is 17 in decimal. The check agrees two ways: 11 plus 6 is 17, and every sum bit above was the parity of its three inputs while every carry was their majority, recomputed column by column.**

Worked example: from a truth table to a sum of minterms

WORKED EXAMPLE

PROBLEM

A function $f(x, y, z)$ is given by its truth table: f is 1 exactly on the input rows 010, 011, 101 and 111, and 0 on the other four rows. Write the canonical sum of minterms for f, reading one minterm off each 1-row.

x	y	z	f	minterm
0	0	0	**0**	–
0	0	1	**0**	–
0	1	0	**1**	x′yz′
0	1	1	**1**	x′yz
1	0	0	**0**	–
1	0	1	**1**	xy′z
1	1	0	**0**	–
1	1	1	**1**	xyz

minterms so far: x′yz′, x′yz, xy′z, xyz

f = x′yz′ + x′yz + xy′z + xyz

xyz	000	001	010	011	100	101	110	111
f (from the table)	0	0	1	1	0	1	0	1
the sum of minterms	0	0	1	1	0	1	0	1

Answer: f = x′yz′ + x′yz + xy′z + xyz = Σm(2, 3, 5, 7) — check: the sum is 1 exactly on rows 2, 3, 5, 7

1. The table defines the function completely: f is 1 on the 4 rows 010, 011, 101, 111 and 0 on the other 4. The sum-of-minterms recipe turns each 1-row into one product term; the 0-rows contribute nothing, because a term that fires on a 0-row would corrupt the output there.
2. The first 1-row is 010: x = 0, y = 1, z = 0 with f = 1, so this row contributes a minterm. Write the plain variable where the bit is 1 and the complement where the bit is 0, giving x′yz′. That product is 1 on this row alone and 0 on every other row, which is why one term per 1-row can rebuild the whole function.
3. The next 1-row is 011: x = 0, y = 1, z = 1 with f = 1, so this row contributes a minterm. Write the plain variable where the bit is 1 and the complement where the bit is 0, giving x′yz. That product is 1 on this row alone and 0 on every other row, which is why one term per 1-row can rebuild the whole function.
4. The next 1-row is 101: x = 1, y = 0, z = 1 with f = 1, so this row contributes a minterm. Write the plain variable where the bit is 1 and the complement where the bit is 0, giving xy′z. That product is 1 on this row alone and 0 on every other row, which is why one term per 1-row can rebuild the whole function.
5. The next 1-row is 111: x = 1, y = 1, z = 1 with f = 1, so this row contributes a minterm. Write the plain variable where the bit is 1 and the complement where the bit is 0, giving xyz. That product is 1 on this row alone and 0 on every other row, which is why one term per 1-row can rebuild the whole function.
6. Join the 4 minterms with + signs, which is Boolean OR: f = x′yz′ + x′yz + xy′z + xyz. This is the canonical sum of minterms: 4 terms and 12 literals, one full three-literal product per 1-row. It is correct by construction, even though a Karnaugh map might shrink it later; the canonical form is the guaranteed starting point.
7. Spot-check two rows before trusting the expression. On row 010 the minterm x′yz′ has every literal true, so the sum is 1, matching the table. On row 000 every minterm contains at least one false literal, so each product is 0 and the sum is 0, matching the table again.
8. **Answer: f = x′yz′ + x′yz + xy′z + xyz, that is, Σm(2, 3, 5, 7). The strip below the table evaluates the sum on all eight input rows and reproduces the table's f column exactly, so the expression and the table define the same function.**

Worked example: simplifying an expression law by law

WORKED EXAMPLE

PROBLEM

Simplify $f = xy + xy' + x'z$ as far as the laws of Boolean algebra allow. Name the law used at every step, and confirm at the end that the simplified expression agrees with the original on every input row.

f = xy + xy′ + x′z	start
= x(y + y′) + x′z	distributive law
= x · 1 + x′z	complement law
= x + x′z	identity law
= (x + x′)(x + z)	distributive law
= 1 · (x + z)	complement law
= x + z	identity law

xyz	000	001	010	011	100	101	110	111
original	0	1	0	1	1	1	1	1
x + z	0	1	0	1	1	1	1	1

Answer: f = x + z — check: original and simplified agree on all 8 rows, 6 literals down to 2

1. The goal is the smallest expression equivalent to f = xy + xy′ + x′z. The method is patience: apply exactly one named law at a time, so every equality in the chain is justified and the final form is provably the same function, not just a plausible guess that happens to fit a few rows.
2. Apply the distributive law: the first two terms share the factor x, so factor it out. The expression becomes x(y + y′) + x′z. Nothing else changes in this step, which is what makes the whole chain checkable line by line, and each step either shrinks the expression or reshapes it so a later law can shrink it.
3. Apply the complement law: y + y′ is 1 no matter what y is, so the parentheses collapse to 1. The expression becomes x · 1 + x′z. Nothing else changes in this step, which is what makes the whole chain checkable line by line, and each step either shrinks the expression or reshapes it so a later law can shrink it.
4. Apply the identity law: x · 1 is just x, since multiplying by 1 changes nothing. The expression becomes x + x′z. Nothing else changes in this step, which is what makes the whole chain checkable line by line, and each step either shrinks the expression or reshapes it so a later law can shrink it.
5. Apply the distributive law: the other distributive law, x + yz = (x + y)(x + z), turns the sum into a product of sums. The expression becomes (x + x′)(x + z). Nothing else changes in this step, which is what makes the whole chain checkable line by line, and each step either shrinks the expression or reshapes it so a later law can shrink it.
6. Apply the complement law: x + x′ is 1 no matter what x is, so the first factor collapses to 1. The expression becomes 1 · (x + z). Nothing else changes in this step, which is what makes the whole chain checkable line by line, and each step either shrinks the expression or reshapes it so a later law can shrink it.
7. Apply the identity law: 1 times anything is that thing, so the leading 1 disappears. The expression becomes x + z. Nothing else changes in this step, which is what makes the whole chain checkable line by line, and each step either shrinks the expression or reshapes it so a later law can shrink it.
8. Before calling it minimal, check equivalence the blunt way: the strip below evaluates the original xy + xy′ + x′z and the final x + z on all eight input rows. The two expressions are the same function exactly when no row separates them, so all eight rows must agree.
9. **Answer: f = x + z, down from 6 literals to 2. No law shrinks it further, and both remaining terms are needed: deleting either one changes the value on at least one row of the check strip, so this is the minimal form.**

Practice: Boolean algebra and circuits

PRACTICE

Type expressions with `'` for NOT and `+` for OR, and write AND by putting letters side by side, for example `xy + x'z`. Any equally small correct answer is accepted.

EVALUATE A CIRCUIT

A circuit computes $f = x \oplus yz$. What does it output when $x = 0$, $y = 0$ and $z = 1$?

0 1

Hint: work from the innermost part outwards, one gate at a time.

Show worked solution | New problem ↻

Evaluate one gate at a time, innermost first. $yz = 0$, then $x \oplus yz = 0$. So $f = 0$.

PROBLEM TYPE

- Evaluate a circuit
- Minterms from a truth table
- Minimal sum of products
- Adders

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: Boolean algebra and circuits: sample problems with solutions

EVALUATE A CIRCUIT

A circuit computes $f = ((x + y)' + z)'$. What does it output when $x = 1, y = 1$ and $z = 0$?

Evaluate one gate at a time, innermost first. $x + y = 1$, then $(x + y)' = 0$, then $(x + y)' + z = 0$, then $((x + y)' + z)' = 1$. So $f = 1$.

MINTERMS FROM A TRUTH TABLE

Which minterm numbers make up the DNF of this function? Type them as a set, like {1, 4}.

x	0	0	0	0	1	1	1	1
y	0	0	1	1	0	0	1	1
z	0	1	0	1	0	1	0	1
f	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**

The 1-columns are $001_2 = 1$, $100_2 = 4$, $101_2 = 5$. So $f = \Sigma m(1, 4, 5) = x'y'z + xy'z' + xy'z$.

MINIMAL SUM OF PRODUCTS

Find a minimal sum of products for $f(x, y, z) = \Sigma m(1, 4, 6)$. Type it like `xy + x'z`.

x \ yz	00	01	11	10
0	0	**1**	0	0
1	**1**	0	0	**1**

Circle the largest groups on the Gray-code map. Each group keeps only the variables that stay constant inside it. {m1} gives $x'y'z$; {m4, m6} gives xz' (a lone cell has no neighboring 1, so it keeps every variable). Minimal answer: $f = x'y'z + xz'$, with 2 terms and 5 literals.

ADDERS

A 4-bit ripple-carry adder adds $A = 1001_2$ and $B = 1001_2$. What 5-bit output $c_4 s_3 s_2 s_1 s_0$ does it produce?

Work right to left. Column 0: $1 + 1 + 0 = 10_2$, write 0, carry 1; Column 1: $0 + 0 + 1 = 01_2$, write 1, carry 0; Column 2: $0 + 0 + 0 = 00_2$, write 0, carry 0; Column 3: $1 + 1 + 0 = 10_2$, write 0, carry 1. The final carry is the leftmost bit, so the output is **10010**. Check in decimal: $9 + 9 = 18 = 10010_2$.

Check yourself: Boolean algebra and circuits

CHECK YOURSELF

QUESTION 1 OF 10

A NAND gate receives $x = 1$ and $y = 1$. What does it output?

A 1

B It depends on its previous output

C 0

D 2

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: Boolean algebra and circuits: answers explained (1/2)

1. A NAND gate receives $x = 1$ and $y = 1$. What does it output?

Answer C: 0

NAND is AND followed by NOT. $1 \cdot 1 = 1$, and flipping gives 0. NAND outputs 0 only in this one case. A combinatorial gate has no memory, so its previous output never matters.

2. Inputs x and y enter an OR gate. Its output and z enter an AND gate. What does the circuit compute?

Answer A: $(x + y)z$

Label the gates left to right. The OR gate gives $x + y$, and the AND gate multiplies that by z. The parentheses matter, since $x + yz$ would be a different circuit (an AND feeding an OR).

3. What is the dual of the law $x + x'y = x + y$?

Answer D: $x(x' + y) = xy$

To form a dual, swap $+$ with $\cdot$ and 0 with 1, and leave complements alone. $x + x'y$ becomes $x \cdot (x' + y)$ and $x + y$ becomes xy. Duality guarantees the new law is also true.

4. In the correspondence between sets, logic and circuits, which three belong together?

Answer C: $A \cup B$, $p \vee q$, OR gate

Union, "or" and the OR gate are the same operation in three languages. The result includes anything that is in either input. Intersection, "and" and the AND gate form the other pair.

5. A function of x, y, z is 1 exactly on rows 011 and 110. What is its DNF?

Answer C: $x'yz + xyz'$

Each 1-row becomes a minterm, where a 1 bit gives the plain variable and a 0 bit gives its complement. 011 gives $x'yz$ and 110 gives xyz'. Add them.

6. In a 4-variable Karnaugh map, a group of 4 neighboring 1-cells produces a product term with how many literals?

Answer A: 2

A group of 2^k cells lets k variables change, and those drop out. $4 = 2^2$, so 2 of the 4 variables drop and 2 literals remain.

7. Which set of gates is functionally complete on its own?

Answer A: {NAND}

NAND can build NOT ($x \uparrow x$), AND and OR, so it can build any DNF. {AND, OR} cannot make NOT, because with every input 0 its output is always 0. Circuits of XOR gates only ever compute parity, so they cannot make AND.

8. What does a full adder output for a = 1, b = 1 and carry-in 1?

Answer C: S = 1, Cout = 1

$1 + 1 + 1 = 3$, which is 11_2. The low bit is the sum $S = 1$ and the high bit is the carry $C_{out} = 1$.

Check yourself: Boolean algebra and circuits: answers explained (2/2)

9. Why can a single perceptron compute AND but not XOR?

Answer B: XOR's 1s and 0s cannot be separated by one straight line

A perceptron draws one straight line and outputs 1 on one side. For XOR, the 1s sit at (0,1) and (1,0) and the 0s at (0,0) and (1,1), diagonally opposite, so no line works. Adding a hidden layer fixes this.

10. A 64-bit ripple-carry adder is much slower than a 4-bit one mainly because…

Answer A: the carry must pass through all 64 full adders, one after another

Each full adder must wait for the carry from its right neighbor, so the delay grows with the number of bits. Carry-lookahead adders remove this chain by computing all the carries at once.

Experiments, sample spaces and events

CONCEPT

DEFINITION

An experiment is any process with an uncertain result, such as rolling a die. Each possible result is an outcome. The sample space S is the set of all outcomes, and an event is any subset of S.

THEOREM / RULE

When every outcome is equally likely, probability is a ratio of two counts:

$$P(E) = \frac{|E|}{|S|}$$

WORKED EXAMPLE

Roll two dice. $S = \{1, \ldots, 6\} \times \{1, \ldots, 6\}$, the Cartesian product from Module 1, so $|S| = 36$. The event "sum is 7" holds six pairs, from $(1, 6)$ to $(6, 1)$, so $P = 6/36 = 1/6$.

Events are sets, so Module 1's operations build new events. $A \cap B$ means both happen. $A \cup B$ means at least one happens. A' means A does not happen.

COMMON PITFALL

"Equally likely" is an assumption you must check. The sums 2 to 12 are 11 results, yet only $(1, 1)$ gives a sum of 2, so $P(\text{sum} = 2) = 1/36$. Count the 36 equally likely pairs instead of the 11 sums.

TRY IT: EVENTS AS SETS OF OUTCOMES

Event A: sum is 7 | Event B: at least one 6

A | B | A ∩ B | A ∪ B | A′

	1	2	3	4	5	6
1	1,1	1,2	1,3	1,4	1,5	**1,6**
2	2,1	2,2	2,3	2,4	**2,5**	**2,6**
3	3,1	3,2	3,3	**3,4**	3,5	**3,6**
4	4,1	4,2	**4,3**	4,4	4,5	**4,6**
5	5,1	**5,2**	5,3	5,4	5,5	**5,6**
6	**6,1**	**6,2**	**6,3**	**6,4**	**6,5**	**6,6**

$|A| = 6$, $|B| = 11$, $|A \cap B| = 2$

Inclusion–exclusion: $|A \cup B| = 6 + 11 - 2 = 15$.

$P(A \cup B) = 15/36 = 5/12 \approx 0.417$

Counting outcomes and the rules of probability

CONCEPT

With equally likely outcomes, every probability question becomes two counting questions. Count the outcomes in the event, count the sample space, and divide. The counting tools of Module 7 do the hard part.

Situation	Event count / total	Probability
One card from a deck is a heart	$13/52$	$1/4$
A 5-card hand is all hearts	$\binom{13}{5}/\binom{52}{5}$	≈ 0.0005
One lottery ticket matches 6 of 49	$1/\binom{49}{6}$	$1/13{,}983{,}816$
A random committee of 3 from 6 women and 4 men is all women	$\binom{6}{3}/\binom{10}{3}$	$20/120 = 1/6$

WHERE THIS SHOWS UP IN AI

A classifier's final layer (softmax) turns raw scores into probabilities over the classes. They obey the rules on the right: each lies between 0 and 1, and together they sum to 1. Language models do the same over their whole vocabulary for the next token, and temperature or top-k sampling reshape that distribution while keeping the total at 1.

THEOREM / RULE

Every probability obeys three axioms. $0 \le P(E) \le 1$. $P(S) = 1$. If $A \cap B = \varnothing$, then $P(A \cup B) = P(A) + P(B)$.

DEFINITION

Events with $A \cap B = \varnothing$ are mutually exclusive. They cannot happen together, like rolling a 1 and rolling a 2 with the same die.

TWO RULES THAT FOLLOW

Complement: $P(E') = 1 - P(E)$.
Union (inclusion–exclusion): $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

WORKED EXAMPLE

What is the chance of at least one 6 in two rolls? The complement is "no 6 at all", with probability $(5/6)^2 = 25/36$. So the answer is $1 - 25/36 = 11/36$. Adding $1/6 + 1/6 = 12/36$ overcounts, because the two events overlap at $(6, 6)$.

DESIGN TIP

For any "at least one" question, compute the complement "none" and subtract from 1. It is usually one multiplication instead of many cases.

Worked example: two dice, building the sample space

WORKED EXAMPLE

PROBLEM

Two fair dice are rolled. Build the sample space as a grid of ordered pairs, mark the event "the sum is 7", and find its probability by counting.

Highlighted cells: sum is 7 (6 pairs)



P(sum is 7) = 6/36 = 1/6

P(sum is 7) = 6/36 = 1/6 · check: the pairs (1,6), (2,5), (3,4), (4,3), (5,2), (6,1) are 6 of the 36 equally likely ordered pairs in the grid

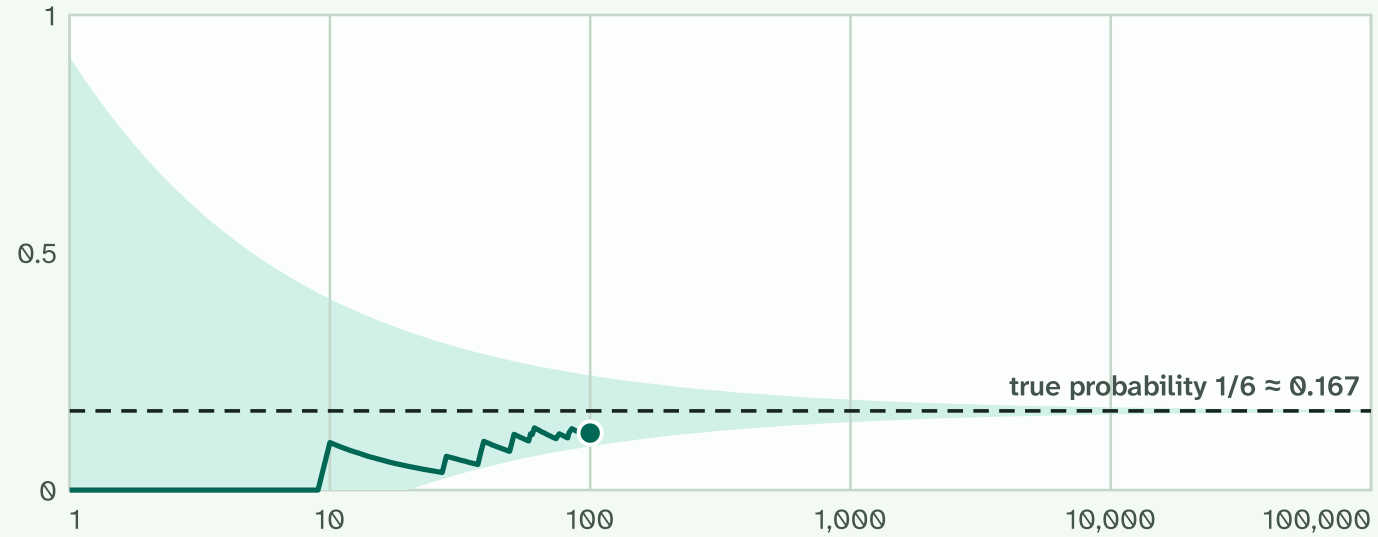
1. Roll two dice and treat (first, second) as the outcome, so (2,5) and (5,2) count as different outcomes. The sample space is the 6 by 6 grid of ordered pairs, and every cell will hold exactly one equally likely outcome.
2. Fill every cell with the sum of its two dice, working row by row so no pair is skipped or counted twice. Filling systematically is the whole method: the finished grid is the entire sample space, so $|S| = 6 \times 6 = 36$.
3. The event "sum is 7" picks out exactly the cells whose row and column values add to 7. Outline those cells before marking any of them, because separating "find the event" from "count the event" keeps the two jobs from blurring together.
4. Mark the outlined cells: the pairs are (1,6), (2,5), (3,4), (4,3), (5,2), (6,1). Counting them one at a time in grid order gives $|E| = 6$, and because the list runs in order, no qualifying pair can be missing or repeated in the count.
5. Both counts are now in hand: 6 favorable outcomes out of 36 equally likely ones. With equally likely outcomes, probability is the ratio of the two counts, so the unreduced answer is 6/36 before any simplifying.
6. **Reduce and finish: P(sum is 7) = 6/36 = 1/6. The answer rests on the enumerated pairs (1,6), (2,5), (3,4), (4,3), (5,2), (6,1) rather than on how common the sum feels, which is exactly why the grid gets built before any probability is stated.**

Lab: the law of large numbers

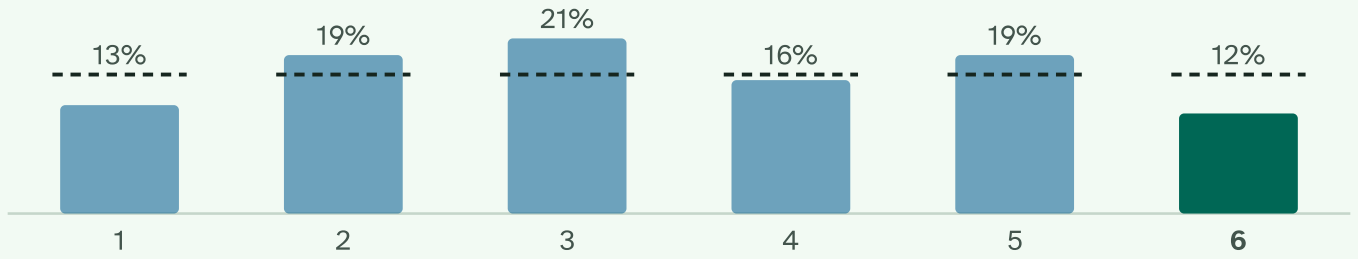
INTERACTIVE LAB

The law of large numbers says that as you repeat an experiment more times, the *relative frequency* of an event (times it happened ÷ trials) gets closer to its probability. Roll, then watch the line settle onto the dashed target. The horizontal axis is logarithmic, so each grid line means ten times more trials.

RELATIVE FREQUENCY AFTER EACH TRIAL (TRIALS ON A LOG SCALE)



HISTOGRAM OF ALL RESULTS (DASHES = EXACT PROBABILITY)



Coin: heads | **One die: a six** | Two dice: a sum of 7

Roll ×1 | ×10 | ×1,000 | ▶ Play | Reset

Trials so far	100
Times we saw a six	12
Relative frequency	12 / 100 = 0.1200
True probability	1/6 = 0.1667
Gap from the truth	0.0467

LAST RESULTS (HIGHLIGHTED = THE EVENT)

5 1 4 5 1 5 2 **6** 3 3 5 4 2 2

After only 100 trials the frequency still swings. Runs like this one usually land within **±0.075** of the truth, the width of the shaded funnel. Press ×1,000 a few times.

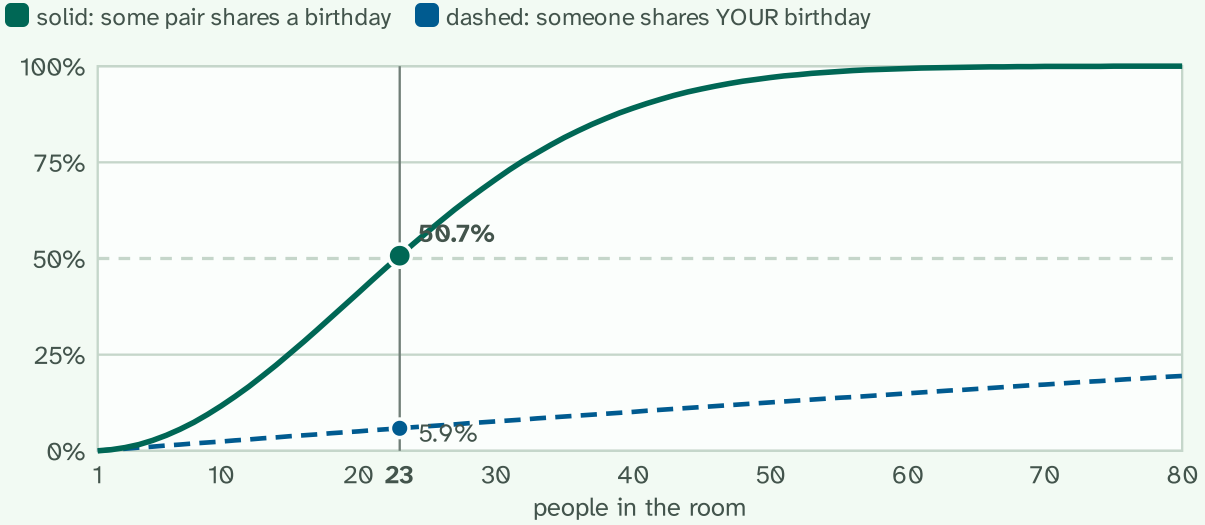
WHERE THIS SHOWS UP IN AI

This is how you check a model's **calibration**. When a weather model or a classifier says "70%", the event should happen in about 70% of those cases over many predictions. A well-calibrated model passes exactly the test this lab runs.

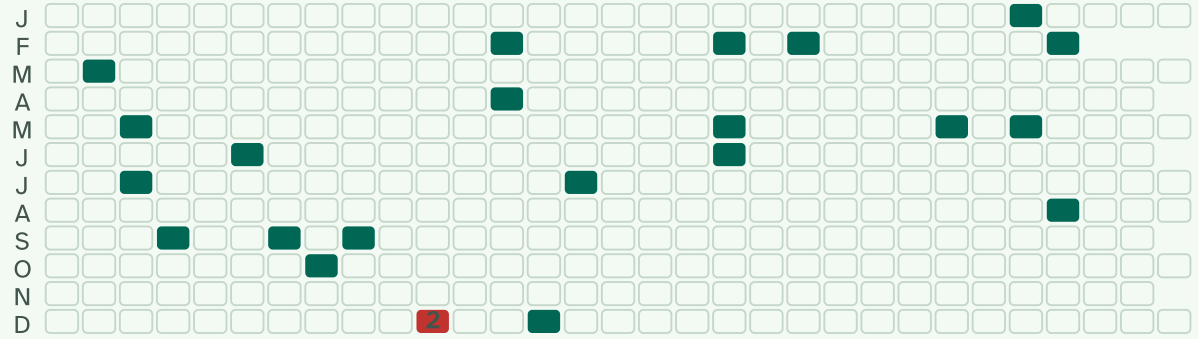
Lab: the birthday problem

INTERACTIVE LAB

How many people must be in a room before two of them probably share a birthday? Most people guess around 180. The complement rule gives the real answer. $P(\text{no match}) = \frac{365}{365} \cdot \frac{364}{365} \cdots$, one factor per person, because each new person must avoid every birthday already taken. Then $P(\text{match}) = 1 - P(\text{no match})$.



ONE RANDOM ROOM: EACH ROW IS A MONTH, EACH SQUARE A DAY



People in the room **n = 23**

Pairs of people, $\binom{n}{2}$	253
P(nobody shares a birthday)	0.4927
P(at least two share)	**0.5073 = 50.7%**
P(someone shares YOUR birthday)	0.0586 = 5.9%

$P(\text{no match}) = \frac{365}{365} \cdot \frac{364}{365} \cdots \frac{343}{365} = 0.4927$. The 253 pairs are why the chance grows so fast: each pair is one more chance for a match, and pairs grow like $n^2/2$.

New random room | Simulate 2,000 rooms | Back to 23

This room has a match on Dec 11 (2 people). Shared days are the numbered squares.

Simulation: 1,009 of 2,000 random rooms of 23 had a match, a relative frequency of **50.4%**. The formula says **50.7%**.

WHERE THIS SHOWS UP IN AI

The same math predicts collisions in hashing. Feature hashing in machine learning maps millions of words into a fixed number of buckets, and with m buckets a collision becomes likely after only about $\sqrt{m}$ items. Systems that hash data for deduplication size their hashes with this "birthday bound".

Conditional probability and independence

CONCEPT

DEFINITION

The conditional probability of A given B is

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(B) > 0.$$

Knowing that B happened shrinks the sample space to B. Only the part of A inside B still counts.

WORKED EXAMPLE

Two dice are rolled and you learn the sum is 8. What is the chance of doubles?
$B = \{(2,6), (3,5), (4,4), (5,3), (6,2)\}$ and only $(4,4)$ is doubles, so $P(\text{doubles} \mid \text{sum } 8) = 1/5$. Without the information, $P(\text{doubles}) = 6/36 = 1/6$.

THEOREM / RULE

Multiplication rule: $P(A \cap B) = P(A) \cdot P(B \mid A)$.
Draw two cards without putting the first back. $P(\text{both aces}) = \frac{4}{52} \cdot \frac{3}{51} = \frac{1}{221}$. The second factor is conditional, because one ace is already gone.

DEFINITION

A and B are independent when $P(A \cap B) = P(A) \cdot P(B)$, or equivalently $P(A \mid B) = P(A)$. Learning that B happened tells you nothing about A. Separate coin flips are independent. Cards drawn without replacement are not.

COMMON PITFALL

Independent and mutually exclusive are different ideas. Mutually exclusive events with positive probabilities are always *dependent*: if one happens, the other's probability drops to 0. "First die shows 1" and "first die shows 2" are mutually exclusive. "First die shows 1" and "second die shows 2" are independent.

WHERE THIS SHOWS UP IN AI

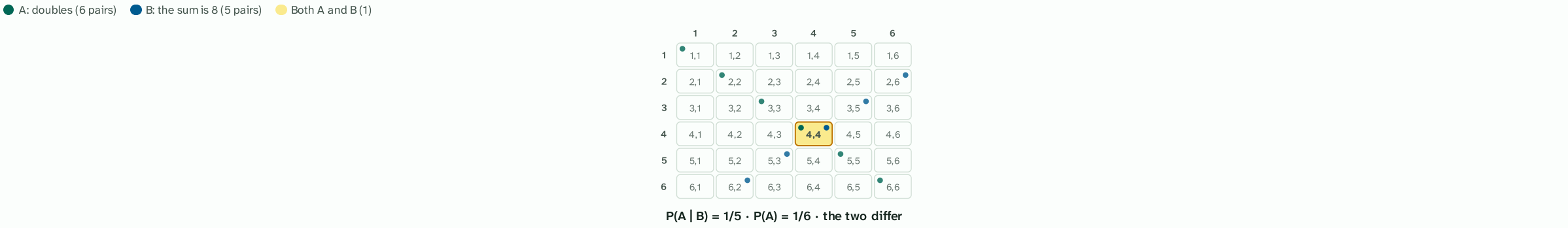
A language model generates text one token at a time using conditional probabilities: $P(\text{next token} \mid \text{all tokens so far})$. The probability of a whole sentence is the multiplication rule applied again and again: $P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \cdots$

Worked example: conditional probability and an independence test

WORKED EXAMPLE

PROBLEM

Two fair dice are rolled. Let A be "doubles" and B be "the sum is 8". Find P(A | B) by restricting the sample space to B, then test whether A and B are independent by comparing P(A ∩ B) with P(A) · P(B) using exact fractions.



P(A | B) = 1/5 = 1/5 and P(A ∩ B) = 1/36 ≠ P(A) · P(B) = 5/216: A and B are dependent · check: inside B, 1 of 5 pairs are in A, a different rate than 1/6 in the full space

1. Two events live on the same 36-pair sample space: A is "doubles" and B is "the sum is 8". The plan is to count A, count B, and count their overlap, then let exact fractions settle both the conditional probability and the independence question.
2. Mark every pair in A with a corner dot: 6 of the 36 pairs qualify, so P(A) = 6/36 = 1/6. This unconditional value is the baseline, because the independence question is precisely whether learning B moves A away from it.
3. Add a second corner dot for B: 5 pairs qualify, and 1 pair carries both dots, so |A ∩ B| = 1 and P(A ∩ B) = 1/36. That overlap count is the shared ingredient in both calculations that follow.
4. Knowing B happened shrinks the sample space to the 5 pairs in B, and every other pair fades out of the picture. Inside that smaller space, 1 of the 5 pairs are also in A, so P(A | B) = 1/5 = 1/5 by direct counting.
5. Independence requires P(A ∩ B) = P(A) · P(B) exactly, with no rounding anywhere. Here the left side is 1/36 and the product is 1/6 × 5/36 = 5/216, and the two fractions are different fractions, so the verdict follows from the arithmetic itself.
6. **Because 1/36 ≠ 5/216, A and B are dependent: learning B moved the probability of A from 1/6 to 1/5. The restricted-space count and the product test tell the same story by two different routes, which is the cross-check.**

Bayes' theorem and the base-rate fallacy

CONCEPT

THEOREM / RULE

Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}, \quad P(B) = P(B \mid A)P(A) + P(B \mid A')P(A')$$

Why it is true: the multiplication rule gives $P(A \cap B)$ two ways, as $P(A)P(B \mid A)$ and as $P(B)P(A \mid B)$. Set them equal and divide by $P(B)$. The bottom line adds the two ways B can happen, with A or without it.

WORKED EXAMPLE

A disease affects 1% of people (the base rate). A test catches 90% of sick people (its sensitivity) and clears 90% of healthy people (its specificity). You test positive.

$$P(D \mid +) = \frac{0.90 \times 0.01}{0.90 \times 0.01 + 0.10 \times 0.99} = \frac{0.009}{0.108} \approx 8.3\%$$

COMMON PITFALL

Most people guess 90%. That mixes up $P(+ \mid D)$, which is 90%, with $P(D \mid +)$, which is what the patient wants to know. Ignoring how rare the disease is has a name: the **base-rate fallacy**.

Natural frequencies make it obvious. Picture 1,000 people. 10 are sick, and 9 of them test positive. 990 are healthy, and 10% of them, 99 people, also test positive. Each of those 99 is a false positive. Of the 108 positive tests, only 9 belong to sick people: $9/108 \approx 8.3\%$. The healthy group is so large that its small error rate swamps the true cases. The next lab lets you draw these 1,000 people.

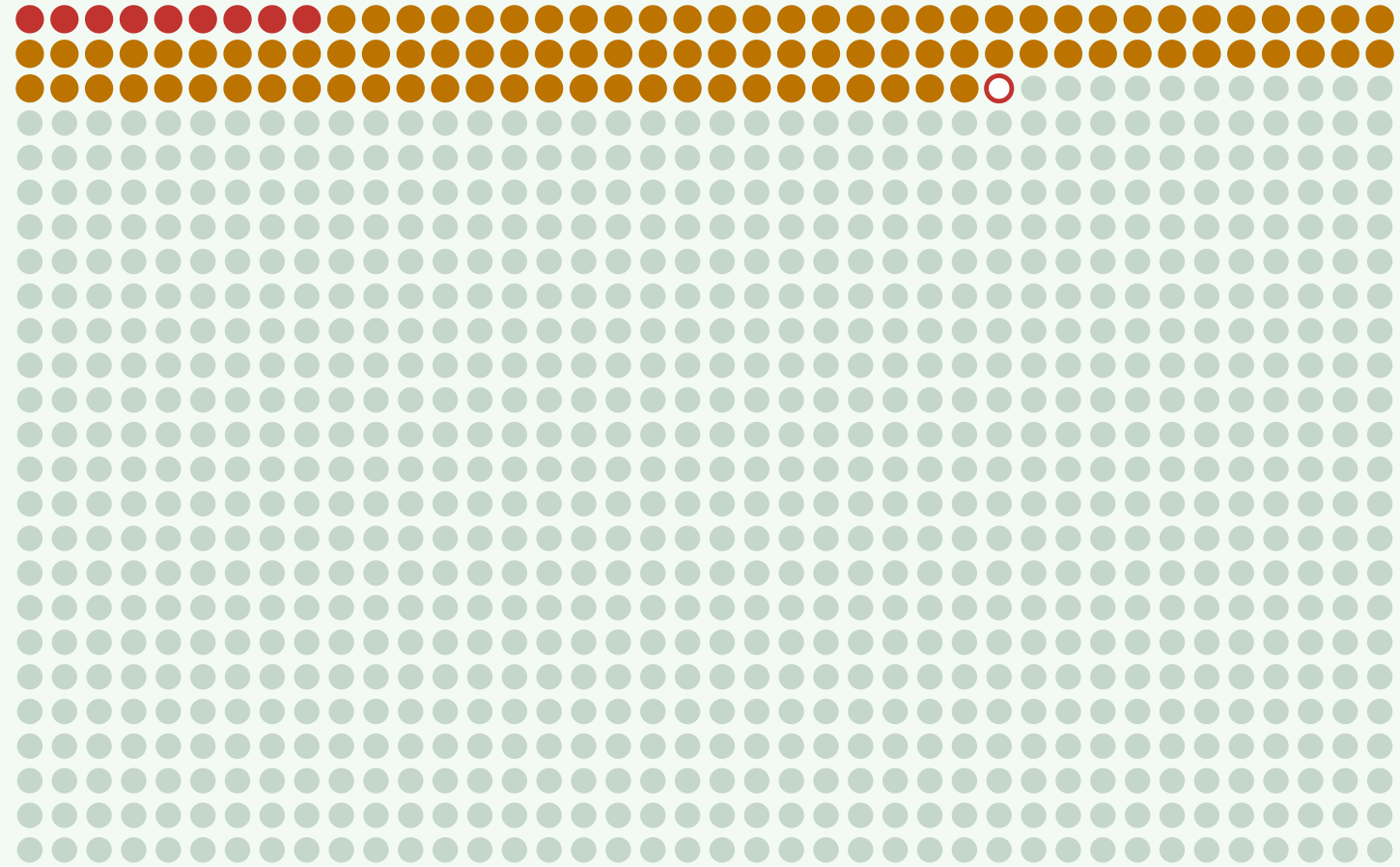
WHERE THIS SHOWS UP IN AI

Every classifier faces this. A fraud model with 99% sensitivity still raises mostly false alarms when fraud is rare. **Precision** is $P(\text{really positive} \mid \text{flagged})$ and **recall** is $P(\text{flagged} \mid \text{really positive})$. Bayes' theorem connects them through the base rate, so always ask how common the positive class is.

Lab: Bayes with 1,000 dots

INTERACTIVE LAB

Each dot is one of 1,000 people (or e-mails). Move the sliders and watch the positive results regroup. Everyone who tests positive sits at the top of the grid, so you can count how many of them really have the condition.



sick, positive (9) · healthy, positive: false positive (99) · sick, negative: missed (1) · healthy, negative (891)

Medical test | Rare disease, great test | Common condition | Spam filter

Base rate: sick out of 1,000 — **10 (1.0%)**

Sensitivity: P(positive | sick) — **90%**

Specificity: P(negative | healthy) — **90%**

	positive	negative	total
sick	**9**	1	10
healthy	**99**	891	990
total	**108**	892	1000

Of the 108 people that are positive, 9 are sick and 99 are healthy.

By counting: $P(\text{sick} \mid \text{positive}) = \frac{9}{9+99} = 8.3\%$

By Bayes: $\frac{0.9 \times 0.01}{0.9 \times 0.01 + 0.1 \times 0.99} = 8.3\%$

Precision is 8.3%; recall (sensitivity) is 90%. Raise the base rate and watch precision climb with the same test.

Lab: the Monty Hall problem

INTERACTIVE LAB

A game show has three doors: one hides a car, two hide goats. You pick a door. The host, who knows where the car is, always opens a different door with a goat behind it, then offers you a switch. Should you switch? Play a few rounds, then let the simulator play thousands.

Door 1	Door 2	Door 3
?	?	?

Pick a door. Behind one is a car; behind the other two are goats.

Stay with my door | Switch doors | New game

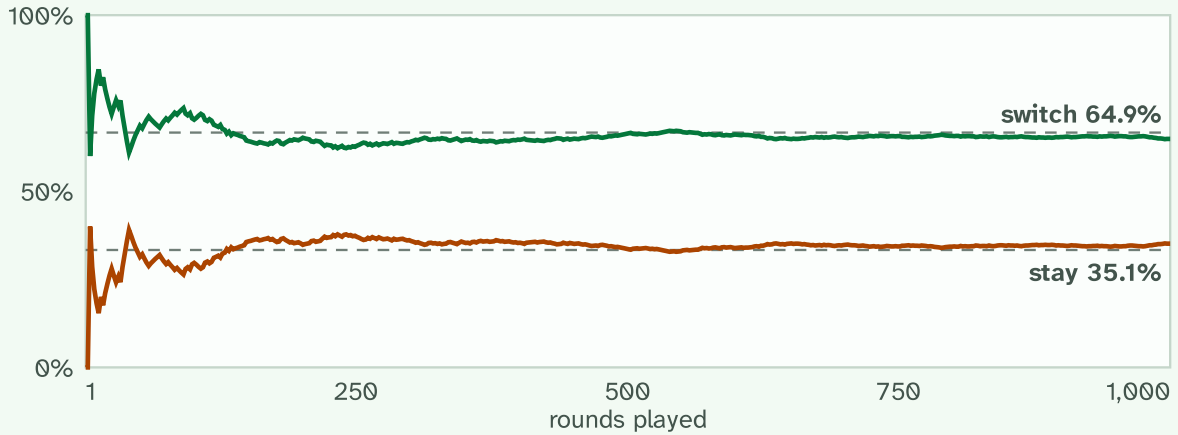
YOUR GAMES

Strategy	Played	Won	Win rate
Stay	0	0	–
Switch	0	0	–

THE THREE EQUALLY LIKELY STARTS

Your first pick hides	Host opens	Staying gets	Switching gets
the car (1/3)	either goat	car	goat
goat 1 (1/3)	goat 2	goat	car
goat 2 (1/3)	goat 1	goat	car

Simulator doors: 3 | 10 | 100 — Run 1,000 new rounds



Over 1,000 rounds with 3 doors, staying won **351** times and switching won **649** times. Theory says $1/3$ and $2/3$ (the dashed lines).

WHY SWITCHING WINS

Your first pick is right with probability $1/3$. The host never reveals the car, so whenever your first pick is wrong (probability $2/3$), the one other closed door must hide it. Switching wins exactly when your first pick was wrong. The host's choice is evidence, because the host never opens a door at random.

Random variables, expected value and variance

CONCEPT

DEFINITION

A random variable X assigns a number to every outcome. Its distribution lists $P(X = x)$ for each value. Let X = the number of heads in 3 coin flips:

x	0	1	2	3
$P(X = x)$	1/8	3/8	3/8	1/8

DEFINITION

The expected value is the probability-weighted average

$$E[X] = \sum_x x \cdot P(X = x) = 0 \cdot \tfrac{1}{8} + 1 \cdot \tfrac{3}{8} + 2 \cdot \tfrac{3}{8} + 3 \cdot \tfrac{1}{8} = 1.5$$

No single game gives 1.5 heads. It is the long-run average over many repetitions.

WORKED EXAMPLE

Pay \$1 to roll a die and win \$5 if it shows 6. Your net gain is \$4 with probability 1/6 and –\$1 with probability 5/6. $E = 4 \cdot \tfrac{1}{6} - 1 \cdot \tfrac{5}{6} = -\tfrac{1}{6}$. You lose about 17 cents per game on average, which is how casinos stay in business.

THEOREM / RULE

Linearity of expectation: $E[X + Y] = E[X] + E[Y]$ and $E[aX + b] = aE[X] + b$, always, even when X and Y are dependent. The expected sum of two dice is $3.5 + 3.5 = 7$ with no table of 36 outcomes. The expected number of successes in n trials with chance p is np.

DEFINITION

Variance measures spread: $\mathrm{Var}(X) = E[(X - \mu)^2] = E[X^2] - \mu^2$, where $\mu = E[X]$. The standard deviation is $\sigma = \sqrt{\mathrm{Var}(X)}$. For the 3 flips, $E[X^2] = 3$, so $\mathrm{Var}(X) = 3 - 1.5^2 = 0.75$ and $\sigma \approx 0.87$.

WHERE THIS SHOWS UP IN AI

Training a model means minimizing its *expected loss*, estimated by the average loss over the training data. A reinforcement-learning agent picks the action with the highest expected reward. Variance matters too: two models with the same average accuracy can differ a lot in how much their results swing from run to run.

Worked example: expected value, term by term

WORKED EXAMPLE

PROBLEM

You pay $1 to roll one die once. Rolling a 6 pays $5, rolling a 5 pays $2, and any other face pays nothing. Let X be your net gain. Build E[X] term by term and decide whether the game favors you.

Roll	Pays	Net X	P(X)	Term x · P(X)	Running total
Roll a 6	$5	$4	1/6	4 × 1/6 = 2/3	2/3
Roll a 5	$2	$1	1/6	1 × 1/6 = 1/6	5/6
Roll 1, 2, 3 or 4	$0	-$1	4/6	-1 × 4/6 = -2/3	1/6

Fair price of this game: $7/6 ≈ $1.17 · charged: $1

`Fair price: 7/6 dollars` `Charged: $1` `✓ Game favors the player`

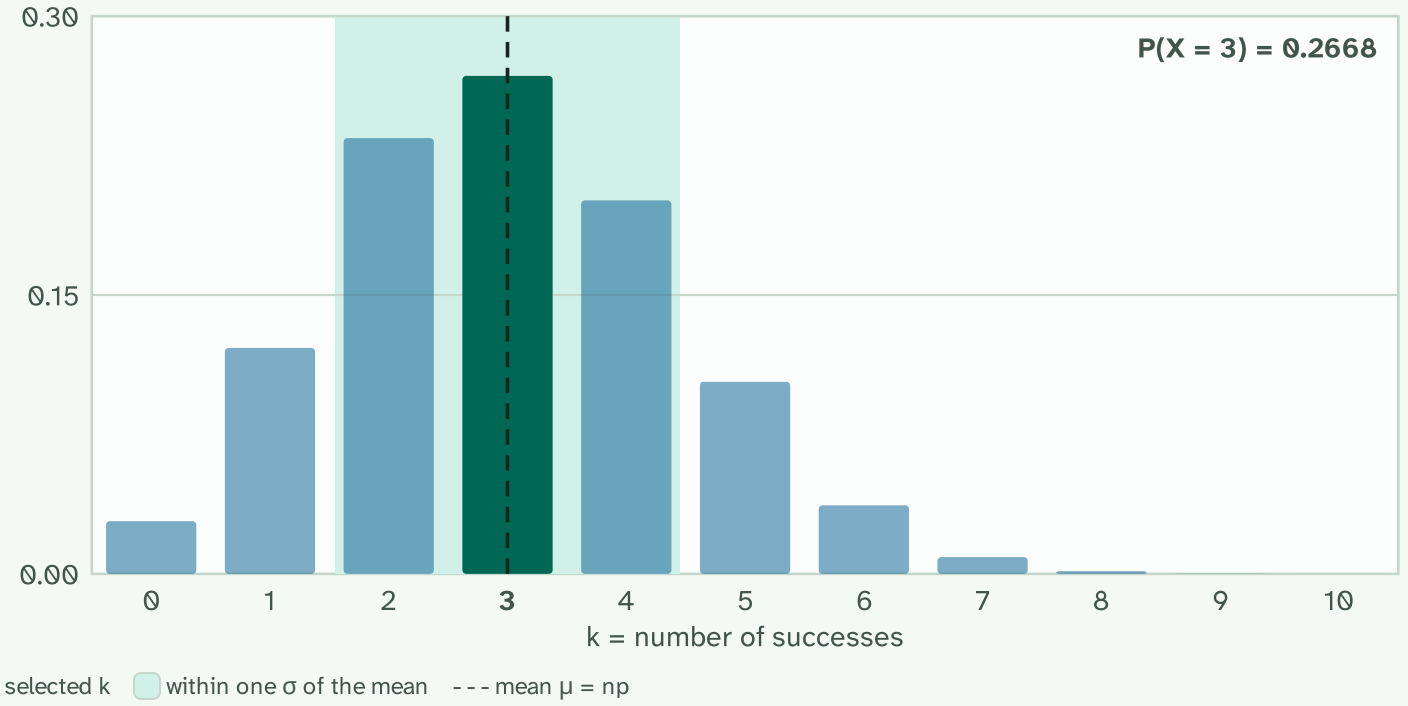
E[X] = 1/6 dollars (about $0.17) per game in your favor · check: terms 2/3, 1/6, -2/3 sum to 1/6, and the fair entry price is the expected payout 7/6 dollars

1. Let X be your net gain on one play: payout minus the $1 fee. The table lists every value X can take with its probability, and the expected value will be built by weighting each value by how likely it is.
2. First term: rolling a 6 has probability 1/6 and nets $4, so it contributes 4 × 1/6 = 2/3 to the expectation. One term is not the answer yet; it is the running total that the remaining faces will adjust.
3. Second term: rolling a 5 also has probability 1/6 and nets $1, contributing 1 × 1/6 = 1/6. Adding it to the running total gives 5/6, and only the four losing faces are still missing from the sum.
4. Last term: the four remaining faces together have probability 4/6 and each nets -$1 (the lost fee), contributing -1 × 4/6 = -2/3. This is the term intuition tends to skip, and it pulls the running total down by the full weight of the likely outcome.
5. Add the three terms: E[X] = 1/6 dollars per game. Cross-check by a second route: the expected payout before the fee is 7/6 dollars, and subtracting the $1 fee gives 7/6 – 1 = 1/6, the same value from a different direction.
6. **Interpretation: the fair price of the game is its expected payout, 7/6 dollars (about $1.17), and you are charged only $1, so the game favors you by 1/6 dollars per play on average. Over many plays your average net gain settles near that value, win or lose on any single roll.**

Lab: the binomial distribution

INTERACTIVE LAB

Repeat a yes/no trial n times, independently, with success probability p each time. The number of successes X follows the binomial distribution, $X \sim \mathrm{Bin}(n, p)$, with $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$. Drag the sliders, or click a bar to see its calculation.



10 fair coin flips | 20 free throws at 70% | 40 chips, 5% defective | 30 questions, model 80% accurate

Trials n **10**

Success probability p **0.30**

Successes k **3**

$$P(X = 3) = \binom{10}{3}(0.3)^3(0.7)^7$$
$$= 120 \times 0.0270 \times 0.0824 = \mathbf{0.2668}$$

$\binom{10}{3} = 120$ counts the ways to choose which 3 of the 10 trials succeed. Each such sequence has probability $p^3(1-p)^7$, and the sequences are mutually exclusive, so their probabilities add.

$P(X \le 3) = 0.6496$, $P(X \ge 3) = 0.6172$

Mean $\mu = np = 3$, variance $np(1-p) = 2.1$, $\sigma = 1.449$

WHERE THIS SHOWS UP IN AI

A/B testing. A model that answers each of 30 test questions correctly with probability 0.8 has a score that follows Bin(30, 0.8). It lands anywhere from 21 to 27 about 89% of the time, by luck alone. A new model is only better if it beats this natural spread.

Code: exact and simulated probability

CODE WALKTHROUGH

Two ways to find $P(\text{sum} = 7)$ for two dice. First count the 36 outcomes exactly, then run a seeded Monte Carlo simulation with more and more rolls and watch it approach the exact answer.

two_dice.py

Read | Edit & run | Trace

```python
import random
from fractions import Fraction
from operator import add
ways = {}
for a in range(1, 7):
    for b in range(1, 7):
        ways[a + b] = ways.get(a + b, 0) + 1
exact = Fraction(ways[7], 36)
print("exact:", exact, "≈", round(float(exact), 4))
random.seed(159)
for n in [60, 6000, 600000]:
    die1 = random.choices(range(1, 7), k=n)
    die2 = random.choices(range(1, 7), k=n)
    sevens = list(map(add, die1, die2)).count(7)
    print(f"{n:>6} rolls: {sevens / n:.4f}")
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
exact: 1/6 ≈ 0.1667
    60 rolls: 0.2667
  6000 rolls: 0.1707
600000 rolls: 0.1666
```

LINE 1 OF 15

Show all notes in code

```python
import random
```

Loads Python's built-in `random` module. It produces *pseudo-random* numbers: a fixed formula generates numbers that look random. Line 10 uses that fact to make every run identical.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Computes P(sum of two dice = 7) exactly by counting outcomes with fractions, then estimates the same probability by simulating 60, 6,000 and 600,000 rolls.

Code: exact and simulated probability: every line explained

1	`import random`	Loads Python's built-in `random` module. It produces *pseudo-random* numbers: a fixed formula generates numbers that look random. Line 10 uses that fact to make every run identical.
2	`from fractions import Fraction`	`Fraction` stores an exact ratio of two integers, such as 1/6, with no rounding. A float can only store 1/6 approximately, as 0.16666666666666666.
3	`from operator import add`	`add(x, y)` is the function version of `x + y`. Line 14 hands it to `map`, which lets Python add two long lists in fast built-in code.
4	`ways = {}`	An empty dictionary that will map each possible sum (2 to 12) to the number of ordered pairs that produce it. This is the counting half of $P(E) = \|E\|/\|S\|$.
5	`for a in range(1, 7):`	Loops over the first die's faces. `range(1, 7)` produces 1, 2, 3, 4, 5, 6, because the stop value 7 is excluded.
6	`    for b in range(1, 7):`	A nested loop over the second die. Together the two loops visit all 36 pairs of the sample space $\{1, \ldots, 6\} \times \{1, \ldots, 6\}$. In Trace mode you can watch `a` and `b` tick through every pair.
7	`        ways[a + b] = ways.get(a + b, 0) + 1`	`ways.get(a + b, 0)` returns the count so far for this sum, or 0 the first time the sum appears. Adding 1 records one more pair. When the loops finish, `ways[7]` is 6 and `ways[2]` is 1.
8	`exact = Fraction(ways[7], 36)`	Builds the exact probability $\|E\|/\|S\| = 6/36$. `Fraction` reduces it automatically, so `exact` holds 1/6.
9	`print("exact:", exact, "≈", round(float(exact), 4))`	Prints the fraction and its decimal value. `float(exact)` converts 1/6 to a float, and `round(..., 4)` keeps four decimal places. Output: `exact: 1/6 ≈ 0.1667`. This is the target the simulation should approach.
10	`random.seed(159)`	Sets the starting point of the random number generator. The same seed always produces the same sequence of rolls, so this program prints identical numbers every time and anyone can reproduce your result. Delete this line and the simulated numbers change on every run.
11	`for n in [60, 6000, 600000]:`	Repeats the whole experiment three times, with 100 times more rolls each time: 60, then 6,000, then 600,000 rolls of the pair of dice.
12	`    die1 = random.choices(range(1, 7), k=n)`	`random.choices` picks `k` items *with replacement*, so a face can come up again and again, exactly like a real die. `die1` is a list of `n` first-die results.
13	`    die2 = random.choices(range(1, 7), k=n)`	A second, independent list for the second die. Pairing `die1[i]` with `die2[i]` gives roll number `i`.
14	`    sevens = list(map(add, die1, die2)).count(7)`	`map(add, die1, die2)` adds the lists position by position: `die1[0] + die2[0]`, `die1[1] + die2[1]`, and so on. `list(...)` collects the sums, and `.count(7)` counts how many equal 7.
15	`    print(f"{n:>6} rolls: {sevens / n:.4f}")`	Prints the relative frequency `sevens / n`. `:>6` right-aligns `n` in six characters so the columns line up, and `.4f` shows four decimals. The estimate moves from 0.2667 at 60 rolls to 0.1707 at 6,000 and 0.1666 at 600,000: the law of large numbers.

Code: a tiny naive Bayes spam filter

CODE WALKTHROUGH

Bayes' theorem powered the first practical spam filters. This one learns word counts from a few training e-mails, then scores new messages. Switch to **Trace** to watch the running log score `lp` change word by word.

naive_bayes.py

Read | Edit & run | Trace

```python
from math import log, exp
from collections import Counter
spam = Counter("win cash now free cash prize win".split())
ham = Counter("lunch at noon meeting now at noon".split())
V = len(set(spam) | set(ham))

def log_p(msg, counts, prior):
    total = sum(counts.values()) + V
    lp = log(prior)
    for word in msg.split():
        lp += log((counts[word] + 1) / total)
    return lp

for msg in ["free cash now", "lunch meeting at noon"]:
    gap = log_p(msg, spam, 0.5) - log_p(msg, ham, 0.5)
    print(f"{msg!r}: P(spam) = {1 / (1 + exp(-gap)):.3f}")
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

```
Output:
'free cash now': P(spam) = 0.857
'lunch meeting at noon': P(spam) = 0.027
```

LINE 1 OF 16

Show all notes in code

```python
from math import log, exp
```

Loads two functions from the `math` module: `log`, the natural logarithm, and `exp`, its inverse (e^x). Importing them by name keeps the later lines short.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

A naive Bayes classifier. It estimates P(word | spam) and P(word | not spam) from example messages, then uses Bayes' theorem to compute P(spam | message) for two new messages.

Code: a tiny naive Bayes spam filter: every line explained

1	`from math import log, exp`	Loads two functions from the `math` module: `log`, the natural logarithm, and `exp`, its inverse (e^x). Importing them by name keeps the later lines short.
2	`from collections import Counter`	`Counter` takes a list and counts how many times each item appears. It also returns 0 for an item it has never seen, which line 11 relies on.
3	`spam = Counter("win cash now free cash prize win".split())`	The words from our spam training e-mails. `.split()` cuts the text at spaces into 7 words, and `Counter` tallies them: `win` 2, `cash` 2, and `now`, `free`, `prize` 1 each.
4	`ham = Counter("lunch at noon meeting now at noon".split())`	The words from legitimate mail ("ham" is the traditional nickname for non-spam). Also 7 words: `at` 2, `noon` 2, and `lunch`, `meeting`, `now` 1 each.
5	`V = len(set(spam) | set(ham))`	`set(spam)` is the distinct spam words, and `|` takes the union with the ham words (Module 1). The vocabulary has 9 different words, so `V` is 9. Line 8 needs it for smoothing.
7	`def log_p(msg, counts, prior):`	Defines a function that computes $\log(P(\text{class}) \cdot \prod P(\text{word} \mid \text{class}))$, the top of Bayes' formula for one class. Treating the words as independent given the class is the "naive" assumption that gives the method its name.
8	`    total = sum(counts.values()) + V`	The class has 7 words, and adding `V` = 9 gives 16. This is the denominator of the smoothed estimate on line 11, and it makes the 9 word probabilities add up to exactly 1.
9	`    lp = log(prior)`	Starts the running total at $\log P(\text{class})$. Logarithms turn a product of many small probabilities into a sum, because $\log(ab) = \log a + \log b$. Sums of logs never underflow to 0.
10	`    for word in msg.split():`	Visits each word of the message to be classified, one at a time.
11	`        lp += log((counts[word] + 1) / total)`	Adds $\log P(\text{word} \mid \text{class})$. The **+1** is Laplace smoothing: a word never seen in this class gets probability 1/16 instead of 0, because a single zero would wipe out the whole product. Example: $P(\text{cash} \mid \text{spam}) = (2+1)/16$.
12	`    return lp`	Hands back the finished log score for this class.
14	`for msg in ["free cash now", "lunch meeting at noon"]:`	Classifies two new messages that were not in the training data.
15	`    gap = log_p(msg, spam, 0.5) - log_p(msg, ham, 0.5)`	Scores the message under both classes with a prior of 0.5 each (we assume half of all mail is spam) and takes the difference. For "free cash now" the spam product is $0.5 \cdot \frac{2}{16} \cdot \frac{3}{16} \cdot \frac{2}{16}$ and the ham product is $0.5 \cdot \frac{1}{16} \cdot \frac{1}{16} \cdot \frac{2}{16}$, six times smaller.
16	`    print(f"{msg!r}: P(spam) = {1 / (1 + exp(-gap)):.3f}")`	Bayes' theorem: $P(\text{spam} \mid \text{msg}) = \frac{e^s}{e^s + e^h} = \frac{1}{1 + e^{-(s-h)}}$, where s and h are the two log scores and `gap` is $s - h$. The bottom of the fraction plays the role of $P(\text{msg})$. Output: 0.857 (that is $6/7$) and 0.027 ($1/37$).

Design practice: probability in real code

DESIGN PRACTICE

Probability bugs rarely crash a program. They quietly produce wrong numbers that look plausible. These habits catch them. Click each principle (or use ↓).

01 **Seed random generators for reproducible experiments**

02 Use Fraction for exact small probabilities

03 Add logarithms instead of multiplying tiny probabilities

04 Never confuse P(A | B) with P(B | A)

05 Check that probabilities add up to 1

06 Simulate to sanity-check every formula

07 Account for the base rate when judging a classifier

08 Choose random.sample or random.choices on purpose

Principle 1 of 8

Seed random generators for reproducible experiments

A simulation that gives different numbers every run cannot be debugged, reviewed, or compared fairly with another version. Create a generator with a fixed seed so a run can be repeated exactly. Record the seed with your results, and vary it deliberately when you want to see how much the answer moves.

Machine-learning papers report seeds for the same reason: model training uses randomness for initial weights and data shuffling.

✗ AVOID THIS

```
import random
rolls = [random.randint(1, 6) for _ in range(100)]
# different every run: a bug cannot be replayed
```

✓ DO THIS

```
import random
rng = random.Random(42)          # own seeded generator
rolls = [rng.randint(1, 6) for _ in range(100)]
# identical every run, so results can be checked
```

Design practice: probability in real code: all principles (1/2)

Seed random generators for reproducible experiments

A simulation that gives different numbers every run cannot be debugged, reviewed, or compared fairly with another version. Create a generator with a fixed seed so a run can be repeated exactly. Record the seed with your results, and vary it deliberately when you want to see how much the answer moves.

Machine-learning papers report seeds for the same reason: model training uses randomness for initial weights and data shuffling.

```
import random
rng = random.Random(42)          # own seeded generator
rolls = [rng.randint(1, 6) for _ in range(100)]
# identical every run, so results can be checked
```

Use Fraction for exact small probabilities

Floats store most fractions approximately, so small errors pile up and equality tests fail. For dice, cards and other small sample spaces, `fractions.Fraction` keeps answers exact and readable, and an exact result is the best possible test oracle for a simulation.

```
from fractions import Fraction
p = Fraction(1, 6) * 3
print(p, p == Fraction(1, 2))   # 1/2 True
```

Add logarithms instead of multiplying tiny probabilities

The smallest positive float is about 10^{-308}. Multiply 400 word probabilities of 0.001 each and the true answer, 10^{-1200}, rounds to exactly 0.0. Every class then scores 0 and the comparison is meaningless. Logarithms turn the product into a sum that stays in a safe range, and a larger log still means a larger probability.

```
import math
score = sum(math.log(prob[w]) for w in words)
# about -2763: no underflow, still comparable
```

Never confuse P(A | B) with P(B | A)

$P(\text{positive} \mid \text{sick})$ and $P(\text{sick} \mid \text{positive})$ can be 90% and 8%. Code makes this mistake easy when variables have vague names. Put the condition in the name, using "given", so every formula can be read and checked.

```
p_pos_given_sick = 0.90      # sensitivity
p_sick = 0.01                # base rate
p_sick_given_pos = bayes(p_pos_given_sick,
p_sick, ...)
```

Check that probabilities add up to 1

A distribution over all outcomes must sum to 1, with every value between 0 and 1. Hand-typed tables, rounding, and forgotten cases break this silently. A two-line assertion catches the bug the moment it appears, long before a wrong number reaches a report.

```
assert abs(sum(weather.values()) - 1) < 1e-9
assert all(0 <= p <= 1 for p in
weather.values())
```

Design practice: probability in real code: all principles (2/2)

Simulate to sanity-check every formula

Probability intuition fails often; the birthday problem and Monty Hall fool almost everyone. When you derive a formula, spend five minutes writing a seeded simulation of the same experiment. If the two disagree by more than random noise, one of them has a bug.

```
rng = random.Random(1)
rooms = 10_000
hits = sum(len(set(rng.choices(range(365),
k=23))) < 23
           for _ in range(rooms))
print(hits / rooms)    # about 0.507: matches
the formula
```

Account for the base rate when judging a classifier

When 1% of transactions are fraud, a model that always answers "legitimate" scores 99% accuracy and catches nothing. Report **precision** (how many flagged cases are real) and **recall** (how many real cases get flagged), and state the base rate of the data you tested on. Precision on balanced test data can collapse on real, imbalanced traffic.

```
precision = tp / (tp + fp)    # flagged cases
that are real
recall = tp / (tp + fn)       # real cases that
get flagged
```

Choose random.sample or random.choices on purpose

`random.sample` draws *without* replacement: each item at most once, like dealing cards or picking a committee. `random.choices` draws *with* replacement: items can repeat, like dice rolls or bootstrapping a dataset. Using the wrong one changes the probabilities and gives subtly wrong results.

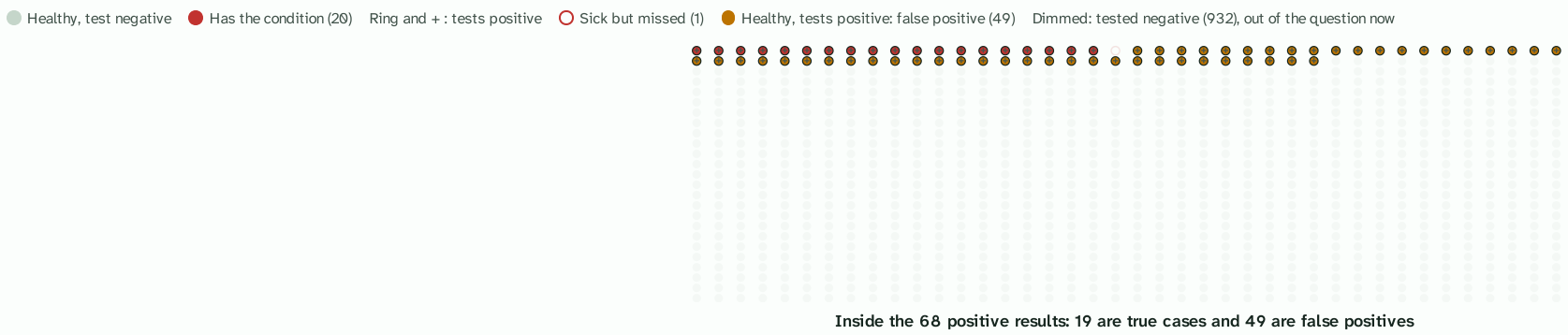
```
hand = random.sample(deck, k=5)    # 5
different cards
rolls = random.choices(range(1, 7), k=5)
# repeats are fine for dice
```

Worked example: Bayes with 1,000 dots

WORKED EXAMPLE

PROBLEM

A condition affects 2% of a population. A test catches 95% of the people who have it (its sensitivity) and wrongly flags 5% of the people who do not (its false-positive rate). Out of 1,000 people, your test comes back positive. What is the probability that you actually have the condition? Count it with all 1,000 people drawn as dots.



P(sick | positive) = 19/68, about 28% · check: 19 true positives out of 68 positive results (19 + 49 = 68), and Bayes' rule gives 190/680 = 19/68

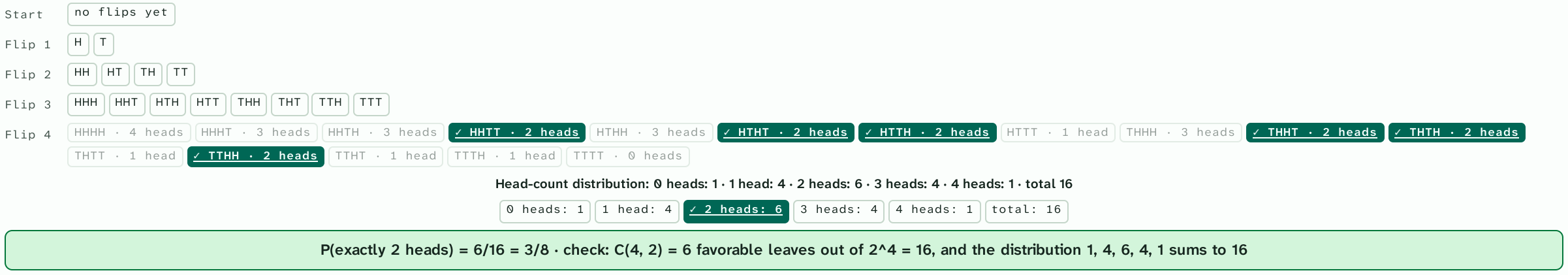
1. Counting people first and turning counts into a probability at the end keeps the base rate visible, which is the whole point of this method. All 1,000 dots start neutral, because no group has been marked yet.
2. Mark the people who have the condition: 2% of 1,000 is 20 people. This group is the base rate, the fact every later count is compared against, and forgetting it is what causes the usual far-too-high guess.
3. Apply the 95% sensitivity to the 20 sick people: 19 test positive and 1 is missed. The dark ring and plus sign mark a positive result, so the test outcome never rests on color alone.
4. Now mark the healthy group: 5% sounds small, but 5% of 980 is 49 false positives. That is more false positives than the 20 people who have the condition in total, because the healthy group is so much bigger.
5. Dim everyone who tested negative and focus on the 68 positive results only. A positive result puts you inside this group of 68, so the relevant question is now 19 out of 68.
6. **Check by Bayes' rule with exact fractions: (95/100 × 2/100) / (95/100 × 2/100 + 5/100 × 98/100) = 190/680 = 19/68. The dot count and the formula agree, so a positive result means about a 28% chance.**

Worked example: exactly 2 heads in 4 flips, by tree

WORKED EXAMPLE

PROBLEM

A fair coin is flipped 4 times. What is the probability of exactly 2 heads? Build the full outcome tree: list every outcome string, collect the ones with exactly 2 heads, and count.



P(exactly 2 heads) = 6/16 = 3/8 · check: C(4, 2) = 6 favorable leaves out of 2^4 = 16, and the distribution 1, 4, 6, 4, 1 sums to 16

1. Each flip doubles the number of outcomes, so after 4 flips there are 2^4 = 16 equally likely outcome strings. The tree starts at the root and makes its first split into H and T.
2. Level 2 is complete with 4 prefixes: HH, HT, TH, TT. The tree keeps outcomes that feel similar, HT and TH, separate, and that distinction is exactly what counting needs.
3. Level 3 doubles the prefixes again to 8, because each level-2 prefix branches into one ending in H and one ending in T. Nothing has merged, so every prefix still stands for distinct flip sequences.
4. Level 4 completes the tree with all 16 outcome strings as leaves, and each leaf is labeled with its head count. The same head count can come from several different leaves, which is why the next step collects them.
5. Collect the leaves with exactly 2 heads: HHTT, HTHT, HTTH, THHT, THTH, TTHH. There are C(4, 2) = 6 ways to choose which 2 of the 4 flips are heads, and the tree shows all 6 of them.
6. **Check: the full distribution from the tree is 1, 4, 6, 4, 1 for 0 to 4 heads, which sums to 16 leaves. The formula C(4, 2) / 2^4 = 6/16 = 3/8 agrees with the leaf count exactly.**

Practice: probability

PRACTICE

Type probabilities as fractions such as `5/36` or as decimals. When a problem asks for rounding, a decimal with that many places is accepted.

COUNTING OUTCOMES

Two fair dice are rolled. What is the probability that the sum is 7?

a fraction such as 5/36, or a decimal to 3 places

Check

Hint: list the ordered pairs (first die, second die) with this sum. There are 36 equally likely pairs in all.

Show worked solution | New problem ↻

The 36 ordered pairs are equally likely. The pairs with sum 7 are (1,6), (2,5), (3,4), (4,3), (5,2), (6,1), which makes 6. So $P = 6/36 = 1/6$.

PROBLEM TYPE

- Counting outcomes
- Conditional probability
- Bayes' theorem
- Expected value and binomial

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: probability: sample problems with solutions

COUNTING OUTCOMES

A fair coin is flipped 5 times. What is the probability of exactly 1 heads?

There are $2^5 = 32$ equally likely sequences. A sequence with exactly 1 heads is fixed by choosing which 1 of the 5 flips are heads: $\binom{5}{1} = 5$ ways. $P = 5/32 = 5/32$.

CONDITIONAL PROBABILITY

A survey of 72 students records whether they practiced daily and whether they passed the final.

	Passed	Failed	Total
Practiced daily	29	6	35
Did not practice	12	25	37
Total	41	31	72

One student is chosen at random. Find $P(\text{passed} \mid \text{practiced})$.

Given "practiced daily", look only at that row. It has 35 students, and 29 of them fit.

$P(\text{passed} \mid \text{practiced}) = 29/35 \approx 0.829$.

BAYES' THEOREM

20% of card transactions are fraudulent. A model flags 90% of fraudulent transactions and 1% of honest ones. A transaction is flagged. What is the probability that the transaction is fraudulent? Give a decimal rounded to 3 places.

$$P = \frac{0.9 \times 0.2}{0.9 \times 0.2 + 0.01 \times 0.8} = \frac{0.18}{0.188} \approx 0.957$$

In natural frequencies, out of 100,000: 20,000 are real cases and 18,000 of them are flagged. Of the 80,000 others, 800 are also flagged. So the answer is $18{,}000/(18{,}000 + 800) \approx 0.957$.

EXPECTED VALUE AND BINOMIAL

A trial succeeds with probability 0.2, independently each time. In 8 trials, what is $P(X = 2)$, the probability of exactly 2 successes? (Round to 4 decimal places.)

$X \sim \text{Bin}(8, 0.2)$, so $P(X = 2) = \binom{8}{2}(0.2)^2(0.8)^6 = 28 \times 0.04 \times 0.262144 \approx 0.2$.

Check yourself: probability

CHECK YOURSELF

QUESTION 1 OF 10

Two fair dice are rolled. What is the probability that the sum is 8?

A 5/36

B 8/36

C 1/11

D 1/6

← Previous question

Next question →

QUESTIONS

1 2 3 4 5

6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: probability: answers explained (1/2)

1. Two fair dice are rolled. What is the probability that the sum is 8?

Answer A: $5/36$

Five of the 36 equally likely ordered pairs give 8: $(2,6), (3,5), (4,4), (5,3), (6,2)$. The answer $1/11$ wrongly treats the 11 possible sums as equally likely.

2. A fair coin is flipped 3 times. What is the probability of at least one head?

Answer C: $7/8$

Use the complement. The only outcome with no heads is TTT, with probability $1/8$. So $P(\text{at least one head}) = 1 - 1/8 = 7/8$.

3. $P(A) = 0.5$, $P(B) = 0.4$ and $P(A \cap B) = 0.2$. Which statement is true?

Answer D: A and B are independent

$P(A)P(B) = 0.5 \times 0.4 = 0.2 = P(A \cap B)$, which is the definition of independence. They are not mutually exclusive since $P(A \cap B) \neq 0$. Also $P(A \cup B) = 0.5 + 0.4 - 0.2 = 0.7$ and $P(A \mid B) = 0.2/0.4 = 0.5$.

4. A and B are mutually exclusive, and both have positive probability. What can you conclude?

Answer C: They are always dependent

$P(A \cap B) = 0$ but $P(A)P(B) > 0$, so independence fails. Intuitively, learning that A happened tells you for certain that B did not, which is a lot of information.

5. A disease affects 1 in 1,000 people. A test catches 99% of cases but also gives a positive result for 5% of healthy people. You test positive. Roughly how likely is it that you have the disease?

Answer C: About 2%

Think of 100,000 people: 100 are sick and 99 test positive, while 5% of the 99,900 healthy people, about 4,995, also test positive. $99/(99 + 4995) \approx 0.019$. The low base rate means false positives far outnumber true ones.

6. You pay \$1 to roll a fair die and receive \$3 if it shows 5 or 6. What is your expected net gain per game?

Answer B: \$0

You win \$2 net with probability $2/6 = 1/3$ and lose \$1 with probability $2/3$. $E = 2 \cdot \frac{1}{3} - 1 \cdot \frac{2}{3} = 0$. It is a fair game.

7. A model answers each of 40 questions correctly with probability 0.85, independently. What is the expected number of correct answers?

Answer D: 34

The number correct is $\text{Bin}(40, 0.85)$. By linearity of expectation, $E[X] = np = 40 \times 0.85 = 34$. The answer 6 is the expected number of *wrong* answers.

Check yourself: probability: answers explained (2/2)

8. What is the smallest group size for which the chance that two people share a birthday is above 50%?

Answer D: 23

With 23 people, $P(\text{no match}) = \frac{365}{365} \cdots \frac{343}{365} \approx 0.493$, so a match has probability about 0.507. The surprise comes from the 253 pairs of people, each a chance for a match. 57 people push the chance above 99%.

9. In the Monty Hall game with 3 doors, what is the probability of winning the car if you always switch?

Answer C: $2/3$

Switching wins exactly when your first pick was a goat, which happens with probability $2/3$. The host must avoid the car, so the door the host leaves closed carries all of the $2/3$ chance that your first pick missed.

10. A fair coin has just landed heads 5 times in a row. Which statement is correct?

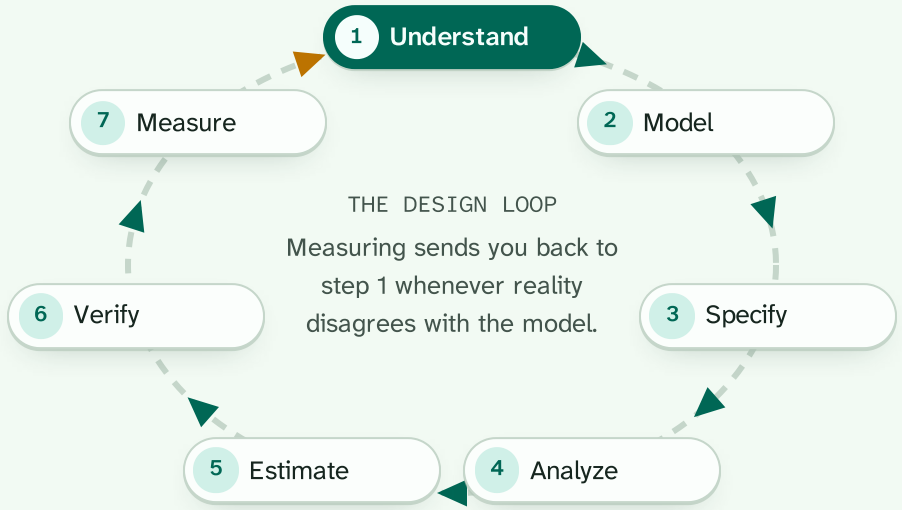
Answer B: The next flip is still heads with probability 1/2

Flips are independent, so the coin has no memory. The law of large numbers works by swamping: after thousands more flips, 5 extra heads barely move the proportion. Expecting tails to "catch up" is the gambler's fallacy, and 5 heads in a row happens with probability $1/32$ even for a fair coin.

Design starts with a model

CONCEPT

Professionals rarely start by typing code. First they decide which mathematical model fits the problem, because that choice fixes the algorithms, guarantees and costs available to them. The loop below runs one realistic problem through all seven steps. Click a step, or use ↓ and ↑.



STEP 1 OF 7

← Previous Next →

Understand the question

Ask: What exactly goes in, what must come out, and which cases are tricky?

Name the universe of possible inputs, the outputs, and the edge cases. Work two or three small examples by hand before you design anything.

RUNNING EXAMPLE: REJECT DUPLICATE SIGN-UPS

Input: a stream of e-mail addresses. Output: accept or reject each one. Tricky case: are `Ana@x.com` and `ana@x.com` the same person? Decision: compare lower-cased addresses, so the universe is the set of normalized addresses.

LEARN IT IN Module 1 · Sets Module 4 · Functions, Sequences & Relations

WHY IT MATTERS

A wrong model is the most expensive mistake in software, because everything built on top of it inherits the error. The loop costs minutes on paper and saves days of rewriting. The rest of this studio practices each step on its own.

Lab: pick the structure

INTERACTIVE LAB

Naming the structure is the first design decision. Read each scenario and pick the model a professional would reach for. Some scenarios have a close second choice, and the feedback explains the difference. The deck is shuffled every time.

SCENARIO 1 OF 23

Next scenario →

Suggest "people you may know": friends of your friends who are not yet your friends.

WHICH STRUCTURE MODELS THIS BEST?

- **Set** — membership; no order, no repeats
- **List (sequence)** — order matters, repeats allowed
- **Dict (function)** — each key maps to one value
- **Relation table** — many-to-many pairs
- **Graph** — objects plus connections
- **Tree** — hierarchy, one parent each
- **Flow network** — capacities, max throughput
- **Boolean logic** — true/false rules, circuits
- **Probability model** — chance, risk, uncertainty

Pick one. You will see the reasoning a professional would give and the module that teaches it.

0	0	0
CORRECT	ANSWERED	STREAK

YOUR DECK

QUESTIONS THAT PICK THE STRUCTURE

Only "is it in there?"	Set
Order or repeats matter?	List (sequence)
One value per key?	Dict (function)
Many-to-many pairs?	Relation table
Paths, neighbors, dependencies?	Graph
One parent each, one root?	Tree
Capacities to maximize?	Flow network
True/false conditions?	Boolean logic
Uncertainty to quantify?	Probability model

Shuffle and restart

Specify with logic, verify with proof

CONCEPT

DEFINITION

A precondition must be true *before* a function runs (the caller's promise). A postcondition is guaranteed *after* it returns (the function's promise). An invariant stays true throughout, for example at the start of every loop pass. Together they form the function's contract.

THEOREM / RULE

A Hoare triple $\{P\}\ S\ \{Q\}$ says: if P is true before statement S runs, and S finishes, then Q is true afterwards. Triples chain like links: if S_1 takes P to Q and S_2 takes Q to R, then running them in order takes P to R, so you can reason about a long program one statement at a time. A loop invariant is a condition the loop body takes back to itself.

$$\{x \geq 0\} \quad y = x + 1 \quad \{y \geq 1\}$$

	Testing	Proving
Covers	the inputs you tried	every input
Answers	"Did it work here?"	"Can it ever fail?"
Cost	cheap and automatic	careful thought
Use it for	everything, all the time	the core: loops and invariants

A contract in Python

```python
def isqrt(n: int) -> int:
    """Pre: n >= 0. Post: r*r <= n < (r+1)**2."""
    assert n >= 0, "pre"
    r = int(n ** 0.5)        # fast, but floats round
    assert r * r <= n < (r + 1) ** 2, "post"
    return r
```

Type hints give the types, the docstring states the contract in words, and each `assert` checks it while the program runs. For `n = 10**30 - 1` the float square root rounds up, and the postcondition catches a bug that hand-picked tests would miss.

COMMON PITFALL

Python skips every `assert` when run as `python -O`. Use asserts for programmer mistakes such as a broken invariant. Check input from users and files with `if` and `raise ValueError`, because that check must always run.

WHERE THIS SHOWS UP IN AI

Model code often fails silently: a wrong tensor shape broadcasts instead of crashing. Contracts such as "the batch has shape (32, 512)", "the probabilities sum to 1" and "no value is NaN" turn silent errors into loud ones.

Code: binary search with its contract checked

CODE WALKTHROUGH

The contract from the previous slide, applied to a classic. Each `assert` is a logical statement the code promises to keep. Switch to **Trace** and watch `lo` and `hi` close in while the invariant holds on every pass.

binary_search_contract.py

Read | Edit & run | Trace

```
1  def binary_search(a: list[int], x: int) -> int:
2      assert a == sorted(a), "precondition: a is sorted"
3      lo, hi = 0, len(a) - 1
4      while lo <= hi:
5          assert x not in a[:lo] + a[hi + 1:], "invariant"
6          mid = (lo + hi) // 2
7          if a[mid] == x:
8              return mid
9          if a[mid] < x:
10             lo = mid + 1
11         else:
12             hi = mid - 1
13     assert x not in a, "postcondition: x is absent"
14     return -1
15 data = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
16 print(binary_search(data, 23), binary_search(data, 7))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

Output:

```
5 -1
```

LINE 1 OF 16

Show all notes in code

```
def binary_search(a: list[int], x: int) -> int:
```

Defines `binary_search`. The **type hints** say `a` is a list of integers, `x` is an integer, and the result is an integer (an index, or –1). Python does not enforce hints when the program runs, but editors and checkers such as mypy read them and flag mistakes beforehand. They are the cheapest part of a contract.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Binary search on a sorted list, with its contract written as executable asserts: a precondition (the list is sorted), a loop invariant (x is never outside the window from lo to hi), and a postcondition (when the function returns –1, x really is absent).

Code: binary search with its contract checked: every line explained (1/2)

1	`def binary_search(a: list[int], x: int) -> int:`	Defines `binary_search`. The **type hints** say `a` is a list of integers, `x` is an integer, and the result is an integer (an index, or –1). Python does not enforce hints when the program runs, but editors and checkers such as mypy read them and flag mistakes beforehand. They are the cheapest part of a contract.
2	`    assert a == sorted(a), "precondition: a is sorted"`	The precondition as code: binary search only works on a sorted list, $a_0 \le a_1 \le \cdots \le a_{n-1}$. `sorted(a)` builds a sorted copy; if it differs from `a`, `assert` stops the program with `AssertionError: precondition: a is sorted`. This check costs $O(n \log n)$, far more than the search itself, so it is scaffolding for development and testing.
3	`    lo, hi = 0, len(a) - 1`	Tuple assignment sets both ends of the search window at once. The window $a[lo..hi]$ starts as the whole list: `lo = 0` and `hi = 9` for ten items. Nothing has been excluded yet, so the invariant on line 5 is trivially true.
4	`    while lo <= hi:`	Keep going while the window contains at least one index. When `lo > hi` the window is empty and the loop stops. Each pass halves the window, so n items need at most $\lfloor \log_2 n \rfloor + 1$ passes: 4 for ten items, 30 for a billion.
5	`        assert x not in a[:lo] + a[hi + 1:], "invariant"`	The loop invariant. `a[:lo]` is the part left of the window, `a[hi + 1:]` the part right of it, and `+` joins the two slices. The assert says `x` is in neither: if `x` is in the list at all, it lies inside $a[lo..hi]$. Showing that it stays true is an induction (Module 3): true at the start, and every pass keeps it true.
6	`        mid = (lo + hi) // 2`	The middle of the window, using `//` (floor division). First pass: `(0 + 9) // 2 = 4`. Python integers never overflow, so `lo + hi` is safe. In Java or C the sum can overflow for huge arrays, which is why those languages write `lo + (hi - lo) / 2`.
7	`        if a[mid] == x:`	Compare the middle element with the target. Searching for 23, the probes are `a[4] = 16`, then `a[7] = 56`, then `a[5] = 23`.
8	`            return mid`	Found it. The postcondition for this branch, `a[mid] == x`, holds because the line above just tested it. For 23 the function returns index 5.
9	`        if a[mid] < x:`	The middle value is too small. Because the list is sorted, every element at or left of `mid` is at most `a[mid]`, which is less than `x`, so none of them can equal `x`.
10	`            lo = mid + 1`	Discard the left half, including `mid`. The invariant survives because we removed only positions that cannot hold `x`. The `+ 1` matters: with `lo = mid` the window can stop shrinking, and the loop never ends.
11	`        else:`	Otherwise `a[mid] > x`: the middle value is too big.
12	`            hi = mid - 1`	Discard the right half, including `mid`, by the mirror-image argument: everything at or right of `mid` is at least `a[mid]`, which is greater than `x`.
13	`    assert x not in a, "postcondition: x is absent"`	The postcondition for "not found". We reach this line only when `lo > hi`. The invariant says `x` is not left of `lo` and not right of `hi`; with `lo > hi`, those two regions cover the whole list, so `x` is absent. This is the standard way to prove a loop correct: the invariant plus the exit condition imply the postcondition.

Code: binary search with its contract checked: every line explained (2/2)

14	`    return -1`	Signals "not found" with –1. Careful: in Python `a[-1]` is the *last* element, so a caller who forgets to check for –1 silently gets a wrong answer. Many Python APIs raise an exception instead; `list.index` raises `ValueError`.
15	`data = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`	A sorted list of ten integers, so the precondition holds. Swap 12 and 16 in Edit mode to watch the precondition assert fire.
16	`print(binary_search(data, 23), binary_search(data, 7))`	Two searches. 23 is found at index 5 after three probes (`mid` = 4, 7, 5). 7 is absent: the probes are 4, 1 and 2, then `lo = 2 > hi = 1` ends the loop and the postcondition assert confirms it. Output: `5 -1`.

Complexity budgets

CONCEPT

A complexity budget turns Big-O into a yes-or-no question: can this algorithm finish in the time I have?

THEOREM / RULE

Steps you can afford = steps per second × seconds available. Rough speeds: pure Python manages about 10^7 simple steps per second, and compiled code (C, Rust, NumPy) about 10^9.

1. Write down the input size n.
2. Write down the growth rate $f(n)$ of each candidate algorithm.
3. Compute $f(n)$ and compare it with the budget.
4. If $f(n)$ exceeds the budget by more than about 1000×, no faster computer will rescue it. Change the algorithm.

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

DESIGN TIP

Doubling hardware speed lets an $O(n)$ algorithm handle $2n$ items in the same time. It lets an $O(2^n)$ algorithm handle just $n + 1$. Better algorithms beat better hardware.

10-MILLION-ROW DATASET

$n = 10^7$. Sorting costs $n \log_2 n \approx 2.3 \times 10^8$ steps: about 23 seconds in Python. Comparing every pair of rows costs $n^2 = 10^{14}$ steps: about 116 days.

60-ITEM FEATURE SET

Trying every subset of 60 features costs $2^{60} \approx 1.2 \times 10^{18}$ steps: about 36 years, even in compiled code.

25-CITY ROUTE

Trying every ordering of 25 cities costs $25! \approx 1.6 \times 10^{25}$ steps: about 490 million years at 10^9 steps per second.

WHERE THIS SHOWS UP IN AI

Transformer attention compares every token with every other token, so its cost grows like n^2 in the context length. Doubling the context from 64,000 to 128,000 tokens quadruples that work, which is why long contexts are expensive and why researchers look for near-linear alternatives.

Lab: will it finish in time?

INTERACTIVE LAB

Choose an input size, a time budget and a machine, or pick a real scenario. Each row estimates the time an algorithm with that growth rate needs. The bars use a logarithmic time scale from 1 ns to 10^{18} s, and the vertical line marks your budget.

Input size $n = 10{,}000{,}000$ — 10000000

Budget: 0.1 s | 1 s | **1 min** | 1 hour | 1 day — Python · 10^7 steps/s | Compiled · 10^9 steps/s

Scenario: **10-million-row dataset** | 60-item feature set | 25-city route | 1 billion sorted keys | 3,000-node road network

GROWTH	TYPICAL JOB	STEPS F(N)	TIME NEEDED	TIME VS BUDGET (LOG SCALE)	VERDICT	LARGEST N IN BUDGET
O(1)	hash lookup	1	100 ns		fits	any n
O(log n)	binary search	23	2.3 µs		fits	any realistic n
O(n)	one pass	1.0×10^7	1 second		fits	6.0×10^8
O(n log n)	sorting	2.3×10^8	23 seconds		fits	2.4×10^7
O(n^2)	all pairs	1.0×10^{14}	116 days		hopeless	24,494
O(n^3)	all triples	1.0×10^{21}	3.2 million years		hopeless	843
O(2^n)	all subsets	3,010,300 digits	> age of universe		hopeless	29
O(n!)	all orderings	65,657,060 digits	> age of universe		hopeless	12

Budget: 10^7 steps per second × 1 min = **6.0×10^8 steps**. Fits: O(1), O(log n), O(n), O(n log n). Hopeless: O(n^2), O(n^3), O(2^n), O(n!).

Ten million rows. One pass (n) and sorting (n log n) fit within a minute even in Python. Comparing every pair of rows (n^2, 10^{14} steps) would take about 116 days, so professionals find duplicates by hashing or sorting instead of pairwise comparison.

Count before you build

CONCEPT

Two numbers decide whether a design survives real data: the size of the search space and the chance of a collision. You can compute both before writing any code.

DEFINITION

The search space is the set of all candidates an algorithm might examine. Counting gives its size: k independent choices with m options each give m^k candidates, the orderings of n items number $n!$, and the subsets number 2^n.

WORKED EXAMPLE

An 8-character password over 62 symbols (a-z, A-Z, 0-9) has $62^8 \approx 2.2 \times 10^{14}$ possibilities. A GPU rig testing 10^{10} per second tries them all in about 6 hours. At 12 characters there are $62^{12} \approx 3.2 \times 10^{21}$: about 10,000 years.

COMMON PITFALL

"One in a million per pair" is not rare overall. Ten thousand items form $\binom{10000}{2} \approx 5 \times 10^7$ pairs, so expect about 50 matching pairs.

THEOREM / RULE

The birthday bound. Draw k values uniformly at random from N possibilities. Then

$$P(\text{at least one collision}) \approx 1 - e^{-k(k-1)/2N}$$

and a collision becomes more likely than not once $k \approx 1.18\sqrt{N}$.

The square root appears because collisions happen between *pairs*. k items form about $k^2/2$ pairs and each pair matches with probability $1/N$, so trouble starts as $k^2/2$ approaches N.

ID scheme	Values N	50% collision at
birthdays	365	23 people
random 6-digit code	10^6	about 1,200
random 32-bit ID	$2^{32} \approx 4.3 \times 10^9$	about 77,000
UUID v4 (122 bits)	$2^{122} \approx 5.3 \times 10^{36}$	about 2.7×10^{18}

WHERE THIS SHOWS UP IN AI

Feature hashing sends each word to column `hash(word) % 2**20`, so a model needs no stored vocabulary. With 100,000 distinct words the birthday bound predicts about 4,800 colliding pairs: tolerable for a spam filter, unacceptable for user IDs.

Lab: ID collision calculator

INTERACTIVE LAB

How many random IDs can you issue before two match? Describe the IDs by their bits of randomness or by alphabet and length, choose how many you will create, and read the risk. The presets show why UUIDs are safe and 6-digit codes are not.

Describe the IDs by

Random bits | Alphabet × length

Bits of randomness: 32

Items created: k = 100,000

100000

PRESETS

- 32-bit hash, 100,000 users
- 6-digit order numbers, 5,000 orders
- Short links: 7 base62 chars, 10 million
- Video IDs: 11 base64 chars, 1 billion
- UUID v4 (122 random bits), 1 billion
- Birthdays: 23 people, 365 days

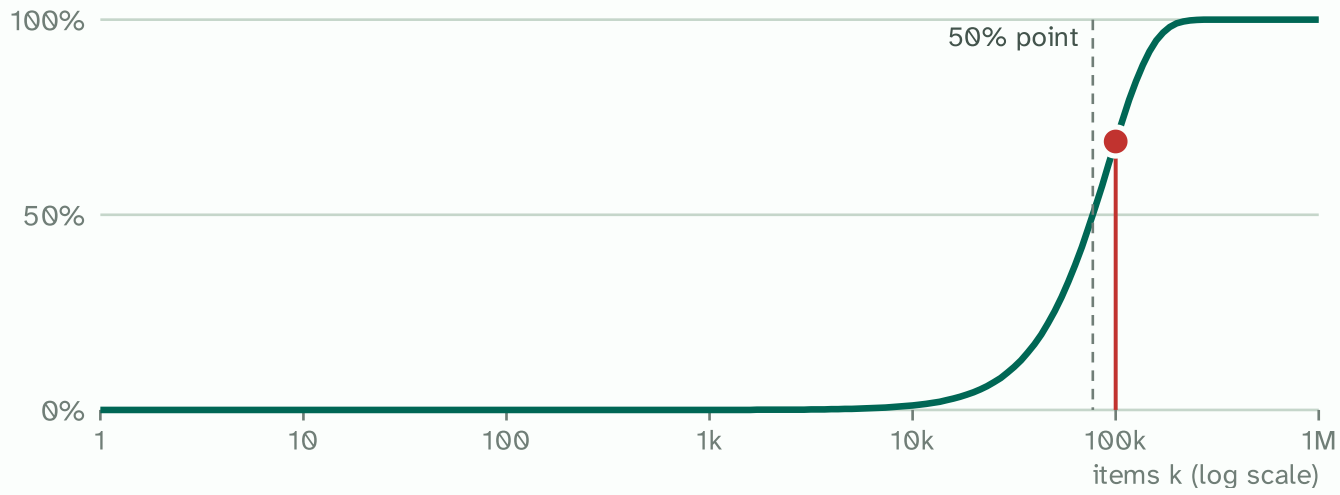
68.8%

chance of at least one collision

Possible IDs N	$2^{32} \approx 4.3 \times 10^9$
Pairs of items	$C(k, 2) = 5.0 \times 10^9$
Expected colliding pairs	1.16
50% risk at	k ≈ 77,162
1-in-a-million risk at	k ≈ 93
Method	exact product of (1 − i/N)

A collision is more likely than not. Use more bits, or check every new ID for duplicates and retry.

A 32-bit hash of each e-mail address used as the user key. The birthday bound makes a collision more likely than not, so two real people would end up sharing a key.



Code: test a fast algorithm against an oracle

CODE WALKTHROUGH

A test oracle is a slow implementation you trust because its code is the definition. Property-based testing runs the fast and slow versions on hundreds of random inputs and requires them to agree. A fixed seed makes every run test the same inputs.

oracle_check.py

Read | Edit & run | Trace

```python
import random

def gcd_fast(a, b):
    while b:
        a, b = b, a % b
    return a

def gcd_oracle(a, b):
    divs = [d for d in range(1, a + 1) if a % d == 0]
    return max(d for d in divs if b % d == 0)

random.seed(42)
for trial in range(1, 501):
    a, b = random.randint(1, 100), random.randint(1, 100)
    assert gcd_fast(a, b) == gcd_oracle(a, b), (a, b)
print("all", trial, "pairs agree; last pair:", (a, b))
```

OUTPUT ● Runs with real Python 3 in the interactive version

▶ Run

Output:

```
all 500 pairs agree; last pair: (30, 47)
```

LINE 1 OF 16

Show all notes in code

```python
import random
```

Loads Python's standard module for pseudo-random numbers. Property-based checking at this scale needs nothing else; the third-party library hypothesis does the same job at industrial scale.

↑ Previous line | Next line ↓

↑ ↓ step lines

What | Why | Who | Where | When

Generates 500 random pairs of integers, computes the gcd of each pair twice (with Euclid's fast algorithm and with a brute-force search) and requires the answers to agree.

Code: test a fast algorithm against an oracle: every line explained

1	`import random`	Loads Python's standard module for pseudo-random numbers. Property-based checking at this scale needs nothing else; the third-party library hypothesis does the same job at industrial scale.
3	`def gcd_fast(a, b):`	The fast implementation we want to trust: Euclid's algorithm (Module 6). It needs only $O(\log \min(a, b))$ passes, so it handles 100-digit numbers instantly.
4	`    while b:`	Repeat while `b` is not zero; a nonzero integer counts as true. The loop relies on the fact $\gcd(a, 0) = a$.
5	`        a, b = b, a % b`	The key identity $\gcd(a, b) = \gcd(b, a \bmod b)$. Python evaluates the whole right side first, so both variables update together. For the first random pair, (82, 15), the pairs go (15, 7), (7, 1), (1, 0).
6	`    return a`	When `b` reaches `0`, `a` holds the gcd. For (82, 15) that is 1.
8	`def gcd_oracle(a, b):`	The test oracle: slow, but correct by definition. Its two lines do exactly what the words "greatest common divisor" say, which is why it is easy to trust.
9	`    divs = [d for d in range(1, a + 1) if a % d == 0]`	Lists every divisor of `a` by trying each candidate from 1 to `a`: `a % d == 0` means `d` divides `a` evenly. For `a = 82` the list is `[1, 2, 41, 82]`. This is $O(a)$ work, hopeless for 20-digit numbers and fine up to 100.
10	`    return max(d for d in divs if b % d == 0)`	Keeps the divisors of `a` that also divide `b` (the common divisors) and returns the greatest. The code reads exactly like the definition, "the greatest common divisor", which is the whole reason to trust it. For (82, 15) only 1 divides both, so the oracle also says 1.
12	`random.seed(42)`	Fixes the starting point of the generator, so every run tests the same 500 pairs, on your machine and on anyone else's. If a check fails, rerunning reproduces the failure exactly, and that is what makes it debuggable.
13	`for trial in range(1, 501):`	500 trials, numbered 1 to 500. Hundreds of random cases cover far more of the input space than a handful picked by hand.
14	`    a, b = random.randint(1, 100), random.randint(1, 100)`	Draws a random pair between 1 and 100, both ends included. The first pair is (82, 15). Small inputs keep the oracle fast, and they are where the edge cases live: 1, equal numbers, one number dividing the other.
15	`    assert gcd_fast(a, b) == gcd_oracle(a, b), (a, b)`	The property under test: both implementations agree. If they ever differ, `assert` raises `AssertionError` with the pair as its message, so the failing input is printed for you to debug.
16	`print("all", trial, "pairs agree; last pair:", (a, b))`	Runs only if all 500 checks passed, because a failed assert would have stopped the program. It also shows the final pair, (30, 47), so you can check one case by hand: 47 is prime and does not divide 30, so both versions said 1. Output: `all 500 pairs agree; last pair: (30, 47)`.

Design practice: eight habits for every project

DESIGN PRACTICE

Every module had its own design slide. These eight habits cut across all of them, and each one is a piece of discrete mathematics turned into a working rule. Click a principle (or use ↓) to compare code to avoid with code to write.

01 **Model before you code**
02 Make invalid states unrepresentable
03 Write pure functions
04 State invariants and check them
05 Choose the algorithm by its growth rate
06 Estimate before you enumerate
07 Test against a brute-force oracle
08 Make randomness reproducible and quantify uncertainty

Principle 1 of 8

Model before you code

Name the mathematical structure before writing loops. Once you say "this is a directed graph", decades of algorithms become available: breadth-first search for reachability, topological sort for ordering, Dijkstra for shortest paths. Code written straight from raw rows usually reinvents a slower, buggier version of one of them.

Ask two questions: what are the objects, and what are the relationships between them? (Modules 1, 4, 9, 10 and 11.)

✗ AVOID THIS

```
# who can reach whom? nested scans over raw rows
for a, b in rows:
    for c, d in rows:
        if b == c:
            pairs.add((a, d))   # only 2 hops, O(n^2)
```

✓ DO THIS

```
graph = defaultdict(set)      # model: directed graph
for src, dst in rows:
    graph[src].add(dst)
reach = bfs(graph, start)     # every hop, O(V + E)
```

Design practice: eight habits for every project: all principles (1/2)

Model before you code

Name the mathematical structure before writing loops. Once you say "this is a directed graph", decades of algorithms become available: breadth-first search for reachability, topological sort for ordering, Dijkstra for shortest paths. Code written straight from raw rows usually reinvents a slower, buggier version of one of them.

Ask two questions: what are the objects, and what are the relationships between them? (Modules 1, 4, 9, 10 and 11.)

```
graph = defaultdict(set)       # model: directed graph
for src, dst in rows:
    graph[src].add(dst)
reach = bfs(graph, start)      # every hop, O(V + E)
```

Make invalid states unrepresentable

Type hints do not validate inputs. `__post_init__` checks them; `object.__setattr__` finishes initialization. Frozen fields and a `frozenset` prevent later edits.

```
from dataclasses import dataclass
from enum import Enum
Status = Enum("Status", "PENDING SHIPPED DELIVERED")
@dataclass(frozen=True)
class Order:
    status: Status
    tags: frozenset[str]
    def __post_init__(self):
        if not isinstance(self.status, Status):
            raise ValueError("invalid status")
        tags = frozenset(self.tags)
        if not all(isinstance(t, str) for t in tags):
            raise ValueError("tags must be strings")
        object.__setattr__(self, "tags", tags)
```

Write pure functions

A pure function is a mathematical function (Module 4): its output depends only on its inputs, and it changes nothing else. Pure functions are easy to test (no setup), safe to cache (same input, same output), safe to run in parallel, and easy to reason about, because you can replace a call by its value.

Push side effects such as printing, files and databases to the edges of the program.

```
def accept(seen: frozenset, email: str):
    # pure: returns the decision and the new state
    return email not in seen, seen | {email}
```

State invariants and check them

An invariant is a fact your code promises to keep true. Write it down, then make the code check it: in the constructor of a class, at the top of a loop during development, or in a test.

Use `assert` for programmer mistakes, since `python -O` switches asserts off. Raise an exception for bad input from users or files, because that check must always run.

```
@dataclass(frozen=True)
class Probability:
    p: float
    def __post_init__(self):    # invariant: 0 <= p <= 1
        if not 0 <= self.p <= 1:
            raise ValueError(f"not a probability: {self.p}")
```

Design practice: eight habits for every project: all principles (2/2)

Choose the algorithm by its growth rate

For large inputs the growth rate dominates everything else: language, hardware, clever micro-optimizations. Before choosing an approach, write down n and the Big-O of each candidate and compare them with your budget, as in the complexity-budget lab (Modules 5 and 8).

Often the fix is a better data structure: a set instead of a list turns an $O(n \cdot m)$ loop into $O(n + m)$.

```
b_set = set(b)                          # O(m), once
common = [x for x in a if x in b_set]   # O(n)
# 2 x 10^5 steps: a few hundredths of a second
```

Estimate before you enumerate

Brute force is the right first idea and the wrong final one, unless the search space is small. Count it first with the product rule, permutations or 2^n (Module 7), divide by the speed of your machine, and enumerate only if the answer is seconds.

Otherwise use dynamic programming, pruning, or a heuristic whose quality you can measure.

```
space = math.factorial(len(cities))
if space <= 10**8:                      # about 10 s
    best = min(permutations(cities), key=length)
else:
    best = two_opt(greedy_route(cities))  # heuristic
```

Test against a brute-force oracle

A slow implementation that follows the definition is easy to trust. Run it side by side with your fast one on hundreds of random small inputs and assert that they agree, as in the oracle walkthrough. This is property-based testing.

Small inputs keep the oracle fast, and they are exactly where edge cases live: empty lists, one element, duplicates, zeros.

```
for _ in range(1000):
    n = random.randint(0, 8)
    xs = [random.randint(0, 5) for _ in range(n)]
    assert fast_sort(xs) == sorted(xs), xs
```

Make randomness reproducible and quantify uncertainty

Random choices (data splits, sampling, initial weights) make results vary from run to run. Seed a local generator so anyone can reproduce a run exactly. Then repeat with several samples or seeds and report the mean and the spread, instead of one lucky number (Module 13).

A difference smaller than the spread is not evidence that one model is better.

```
rng = random.Random(2026)               # seeded and local
accs = [evaluate(model, rng.sample(data, 100))
        for _ in range(20)]
m, s = statistics.mean(accs), statistics.stdev(accs)
print(f"accuracy {m:.3f} ± {s:.3f} over 20 samples")
```

Discrete math inside an AI system

CONCEPT

Every stage of a modern AI system runs on structures from this course. Click a stage (or use ↓ and ↑) to see which ones, a concrete example, and the design check a careful engineer runs at that point.

STAGE 1 **Data** sets, partitions → STAGE 2 **Tokenization** vocabulary, sequences → STAGE 3 **Embeddings** functions, lookup → STAGE 4 **Model** DAG, masks, recurrences → STAGE 5 **Output probabilities** distributions, search trees → STAGE 6 **Evaluation & decision** set sizes, rules, intervals

1. Data

Training examples form a set: duplicates are removed, because a repeated example is learned twice. The data is split into training and test sets that must be **disjoint**, and labels are a function from examples to classes. Finding near-duplicates means choosing an equivalence relation: which records count as the same?

CONCRETE EXAMPLE

Require $|\text{train} \cap \text{test}| = 0$. If a near-copy of a test question sits in the training set, the test score overstates real performance. This mistake is called data leakage.

DISCRETE STRUCTURES AT WORK

sets · partitions · functions (labels) · equivalence (duplicates)

LEARN THEM IN

Module 1 · Sets · Module 4 · Functions, Sequences & Relations · Module 13 · Probability

DESIGN CHECK

Before training, normalize the text, intersect the training and test sets, and assert that the intersection is empty.

WHY IT MATTERS

When an AI system misbehaves, the cause is often a discrete mistake: test data leaked into training (sets), a tokenizer paired with the wrong vocabulary (functions), a mask built from the wrong relation (Boolean logic), or a run nobody can reproduce (probability). The mathematics of this course is how you find and prevent them.

Worked example: designing the duplicate sign-up check

WORKED EXAMPLE

PROBLEM

A sign-up service expects up to 10^6 registered e-mail addresses. Every new sign-up must be rejected if its address has registered before, and accepted otherwise. Take this problem through the design loop — model, specify, analyze the cost, then verify and measure — and decide how to store the addresses.

The problem: reject a new sign-up exactly when its e-mail address has registered before. Scale: up to 10^6 registered addresses. Four phases, four artifacts — no code until the last card is filled.

1 · Model DONE

A set of addresses. The only operation is membership: is this address in the set? Order does not matter, and repeats are what get rejected.

2 · Specify DONE

Invariant: the set never holds an address twice. Rule: if the address is in the set, reject the sign-up; otherwise add the address and accept.

3 · Analyze: count the cost DONE

List scan: about $n^2/2 = 5 \times 10^{11}$ comparisons for $n = 10^6$. Hash set: O(1) average lookup, about 10^6 steps in total.

4 · Verify and measure DONE

Correctness by induction on sign-ups; testing against a nested-loop oracle on 1,000 random lists; a million simulated sign-ups in about 0.4 s confirm the estimate.

Decision: store the addresses in a hash set · check: 5×10^{11} list comparisons vs about 10^6 hash steps — roughly 500,000 times less work, with the invariant proved by induction

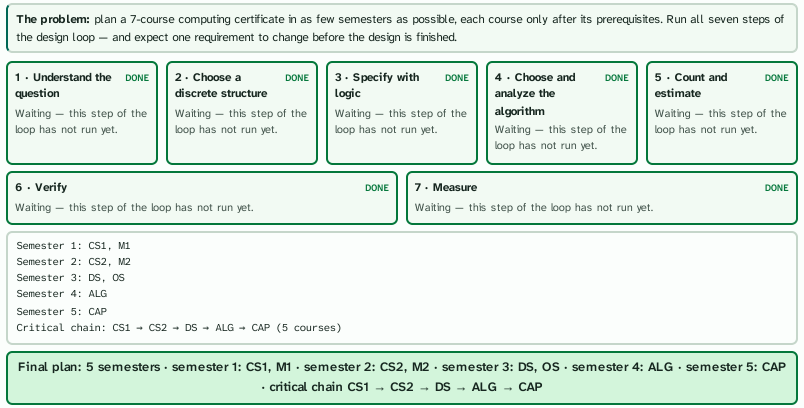
1. Setup: the service must reject a sign-up exactly when its address has registered before, at a scale of up to a million addresses. Run it through the design loop before coding, because each phase produces one artifact and coding first would bury the decisions inside the code.
2. Model: the only question is "has this address been seen?", so the accepted addresses form a set and the only operation is membership. A set is the honest model because order does not matter and repeats are exactly what must be rejected, not stored twice.
3. Specify: the contract says the set never holds an address twice, and a sign-up is rejected exactly when its address is already in the set; otherwise the address is added and the sign-up is accepted. Writing that invariant down turns the requirement into something code and tests can both be checked against.
4. Analyze: count the cost before choosing storage, because scanning a list costs $O(n)$ per sign-up and about $n^2/2 = 5 \times 10^{11}$ comparisons in total for $n = 10^6$, while a hash set answers membership in $O(1)$ average time, about 10^6 steps in total. The count, not taste, is what eliminates the list.
5. Verify and measure: induction on the number of sign-ups proves the invariant — it holds after 0 sign-ups, and each step adds an address only when it is absent — and testing against a nested-loop oracle on 1,000 random lists backs the proof. A simulation of a million sign-ups taking about 0.4 seconds then confirms the cost model; an hour would send us back to the count.
6. **Answer: store the addresses in a hash set, because membership is the only operation the model needs and the invariant is exactly the requirement. The check closes the loop: about 5×10^{11} list comparisons against about 10^6 hash steps is roughly half a million times less work, and correctness and cost were both verified before any code shipped.**

Worked example: designing a prerequisite-proof course schedule

WORKED EXAMPLE

PROBLEM

A computing certificate has seven courses: CS1, CS2, DS (data structures), ALG (algorithms), M1, M2 and OS. The prerequisites are: CS2 after CS1; DS after CS2 and M2; ALG after DS; M2 after M1; OS after CS2. Take this through the seven steps of the design loop — understand, model, specify, analyze, count, verify, measure — and produce a plan that finishes in as few semesters as possible, folding in any requirement that changes along the way.



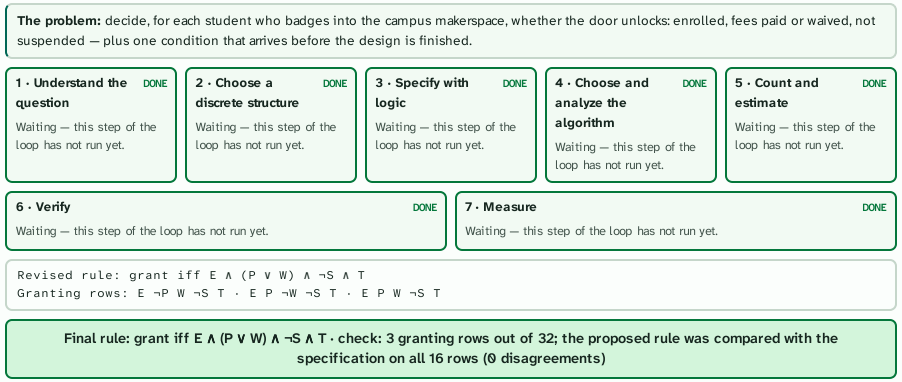
1. Understand the question: a student must finish a seven-course computing certificate, taking each course only after its prerequisites. The inputs are 7 courses and 6 prerequisite pairs; the output is a semester plan. Tricky cases: a course with two prerequisites from different chains must wait for the slower chain, and the prerequisites might contain a cycle, which would make any plan impossible.
2. Choose a discrete structure: courses are vertices and each prerequisite is a directed edge from the prerequisite to the course that needs it — the prerequisite model from the pick-the-structure lab. A valid plan exists exactly when this graph has no directed cycle, because a cycle would force each course on it to come before itself.
3. Specify with logic: the postcondition says that for every prerequisite pair, the prerequisite sits in a strictly earlier semester than the course, and every course is placed exactly once. The goal clause fixes what "best" means before any algorithm runs: minimize the number of semesters over all plans satisfying that condition.
4. Choose and analyze the algorithm: Kahn's topological sort repeatedly takes a course whose prerequisites are all placed, and one depth pass — depth is 1 plus the largest prerequisite depth — gives every earliest semester at once. The work is proportional to vertices plus edges, 13 basic steps here.
5. Count and estimate: the depth pass already proves a lower bound, because the chain CS1 → CS2 → DS → ALG has 4 courses in a line and no plan can compress a line. A subset DP counts 22 valid orders, so there is real choice about which courses share a semester — but the finishing semester is fixed by that one chain.
6. Verify: check the plan mechanically, pair by pair — all 6 prerequisites land in an earlier semester. Then a negative test: if ALG were also a prerequisite of the entry courses, the graph would contain a cycle, and Kahn's algorithm places only 0 of 7 courses before stalling. The stall is the cycle detector: an impossible requirement is reported, never silently misplanned.
7. Measure, and go around the loop again: the plan went out for review and the requirement changed — the certificate now ends with a capstone, CAP, which requires both ALG and OS. The same depth pass recomputes everything: the new course's prerequisites sit in semesters 4 and 3, so it lands in semester 5, the critical chain grows, and the program now needs 5 semesters. No other course moves, because adding a final course changes no earlier depth.
8. **Answer: the revised program takes 5 semesters. The checks close the loop: all 8 prerequisite pairs land in an earlier semester (checked mechanically), the critical chain CS1 → CS2 → DS → ALG → CAP proves no 4-semester plan exists, and the cycle test shows the method refuses an impossible requirement instead of papering over it.**

Worked example: designing an access rule and checking its form

WORKED EXAMPLE

PROBLEM

The campus makerspace door unlocks for a student who badges in exactly when the student is enrolled, has paid fees or holds a fee waiver, and is not suspended. The office circulates the rule in the long form $(E \wedge P \wedge \neg S) \vee (E \wedge W \wedge \neg S) \vee (E \wedge P \wedge W \wedge \neg S)$. Take the design through the seven steps of the loop — understand, model, specify, analyze, count, verify, measure — decide whether the short form $E \wedge (P \vee W) \wedge \neg S$ is really the same rule, and fold in any new condition that arrives.



1. Understand the question: the campus makerspace door must decide, for each student who badges in, whether to unlock. The output for each request is a single bit, grant or deny, and the tricky cases are the ones where the conditions pull in opposite directions: a scholarship student who has not paid fees, whose waiver should unlock the door, and a student who paid in full but is currently suspended, whose payment must not unlock it. Getting those two cases right is the whole design problem in miniature.
2. Choose a discrete structure: each condition is a Boolean variable, so a request is one row of a truth table and the rule is a Boolean formula — the access-rule model from the pick-the-structure lab. The rule as the office circulated it is (E ∧ P ∧ ¬S) ∨ (E ∧ W ∧ ¬S) ∨ (E ∧ P ∧ W ∧ ¬S); the specification rule is E ∧ (P ∨ W) ∧ ¬S. Whether the two are really the same rule is a Module 12 equivalence question, settled by brute force in the verify step.
3. Specify with logic: the postcondition is a single sentence that support staff and tests can both use: grant exactly when the student is enrolled, has paid fees or holds a waiver, and is not suspended — in particular, no suspended student is ever granted, whatever they paid. Every later argument about the rule is an argument about this sentence, not about anyone's memory of a meeting.
4. Choose and analyze the algorithm: evaluating the formula costs a handful of Boolean operations per request, however the rule is written. The analysis that matters here is the truth table: 4 variables make 16 rows, few enough that the design can be checked on every single one — no sampling and no spot guesses at this stage.
5. Count and estimate: enumerating the table, the specification rule grants on exactly 3 of the 16 rows — E ¬P W ¬S; E P ¬W ¬S; E P W ¬S — and denies the other 13. A random combination of conditions thus has a 3/16 chance of being granted, worth knowing before anyone calls the rule "basically open" or "basically closed".
6. Verify: run both formulas over all 16 rows and compare — they disagree on 0 rows, so the simplification is exactly equivalent to the rule as circulated, not approximately. The named spot checks agree too: scholarship student, fees unpaid, not suspended: GRANT; paid in full but suspended: DENY; enrolled, unpaid, no waiver: DENY. The truth-table equivalence check turns "surely these are the same rule" into a proved fact.
7. Measure, and go around the loop again: after a pilot period the requirement changes — the safety office adds a condition: only students who have completed safety training may enter, whether supervised or not. Conjoining T gives grant iff E ∧ (P ∨ W) ∧ ¬S ∧ T; the table grows to 32 rows and exactly 3 grant, since every old granting row survives only in its T-true half. The test suite grows with the table, from 16 cases to 32.
8. **Answer: ship the specification rule with the revision folded in — grant exactly when E ∧ (P ∨ W) ∧ ¬S ∧ T. The loop's checks are the reason to trust it: the count says only 3 of 32 condition combinations are granted, the exhaustive comparison policed the proposed form of the rule before it shipped, and the spot checks cover the two tricky cases by name.**

Practice: cumulative review

PRACTICE

A mixed review of the whole course. Each problem type draws on several modules, and every worked solution names the module to revisit. Probabilities can be typed as fractions such as 5/36.

SETS AND LOGIC

Sets. Let $A = \{4, 5, 9\}$ and $B = \{1, 4, 5\}$. Find $A \cap B$.

for example {1, 2, 3}

Check

Show worked solution

New problem ↻

Keep every element that is in both A and B: $A \cap B = \{4, 5\}$. Revisit Module 1 (Sets) if this felt shaky.

PROBLEM TYPE

- Sets and logic
- Counting and probability
- Integers: gcd and mod
- Algorithms and growth

0	0	0
CORRECT	TRIED	STREAK

Problems are generated fresh every time, so you can keep practicing as long as you like. Answers to sets can be typed like {1, 2, 3}; fractions like 3/8 work for numbers.

Practice: cumulative review: sample problems with solutions

SETS AND LOGIC

Logic. Let p be "5 + 5 = 11" and q be "9 is prime". Is $p \wedge q$ true or false?

p is false and q is false. $p \wedge q$ is true only when both parts are true. So $p \wedge q$ is **false**. Module 2 (Logic) covers every connective.

COUNTING AND PROBABILITY

Counting. 9 runners finish a race with no ties. In how many ways can the first 4 places be filled?

Order matters, so count arrangements: 9 choices for first place, 8 for second, and so on. $P(9, 4) = 9 \cdot 8 \cdot 7 \cdot 6 = 3024$. (Module 7)

INTEGERS: GCD AND MOD

Number theory. Find $\gcd(120, 84)$.

Euclid's algorithm replaces (a, b) by $(b,\ a \bmod b)$ until the remainder is 0: $120 = 1 \cdot 84 + 36$; $84 = 2 \cdot 36 + 12$; $36 = 3 \cdot 12 + 0$. The last nonzero remainder is the gcd, 12. (Module 6)

ALGORITHMS AND GROWTH

Growth rates. As n grows without bound, which function grows faster: $\sqrt{n}$ or $\log n$?

$\sqrt{n}$. Every positive power of n eventually beats log n. At $n = 10^6$, $\sqrt{n} = 1000$ while $\log_2 n \approx 20$. (Module 5)

Check yourself: design judgment

CHECK YOURSELF

QUESTION 1 OF 10

A checkout service must answer "has this coupon code been redeemed?" for 20 million codes, thousands of times per second. Which model and data structure fit best?

A A set of redeemed codes (a hash set)

B A tree of codes ordered by redemption date

C A flow network from codes to customers

D A list of redeemed codes, searched with `in`

← Previous question | Next question →

QUESTIONS

1 2 3 4 5
6 7 8 9 10

Pick an answer to see feedback.

Start over

Check yourself: design judgment: answers explained (1/2)

1. A checkout service must answer "has this coupon code been redeemed?" for 20 million codes, thousands of times per second. Which model and data structure fit best?

Answer A: A set of redeemed codes (a hash set)

Only membership matters, so the model is a set. A hash set answers `in` in constant average time, while searching a 20-million-item list can take 20 million comparisons per request.

2. A planner tries every ordering of 15 delivery stops, checking about 10^7 orderings per second in Python. Roughly how long does it run?

Answer B: About a day and a half

$15! \approx 1.3 \times 10^{12}$ orderings divided by 10^7 per second is about 1.3×10^5 seconds, or 36 hours. Estimating first warns you that 16 stops would take over three weeks and 20 stops thousands of years, so use dynamic programming or a heuristic.

3. Your function passes 10,000 randomly generated tests. Which statement is correct?

Answer B: It is very likely correct on inputs like the ones tested, but only a proof covers every input

Testing samples the input space and a proof covers all of it. Professionals use both: proofs or carefully stated invariants for the core logic, and many automated tests, because proofs can contain mistakes too and tests catch them cheaply.

4. Which statement about a loop invariant is correct?

Answer D: It is true before the first pass and stays true after every pass, so it still holds when the loop exits

An invariant is proved like an induction: it holds at the start (base case) and every pass preserves it (inductive step). Combined with the exit condition it gives the postcondition. In binary search, "if x is in the list, it lies between lo and hi" plus "lo > hi" proves that x is absent.

5. A system assigns random 32-bit IDs. About how many IDs can it issue before a collision becomes more likely than not?

Answer A: About 77,000

The birthday bound puts the 50% point at $k \approx 1.18\sqrt{N} = 1.18 \times 2^{16} \approx 77{,}000$. The number of pairs grows like $k^2/2$, so collisions arrive long before the space fills up. "None can collide until all 4.3 billion are used" confuses "guaranteed" (the pigeonhole principle) with "likely".

6. Why do careful experiments call `random.seed(...)`, or create `random.Random(seed)`, before generating random data?

Answer D: It makes the run reproducible: the same seed produces the same sequence, so others can rerun and check the result

A pseudo-random generator is a deterministic recurrence started from the seed (Module 8), so the same seed reproduces the same run, bugs included. Significance is a separate question: for that, repeat with several seeds and report the spread.

Check yourself: design judgment: answers explained (2/2)

7. You must suggest an order for taking courses so that every prerequisite comes first. Which model and algorithm fit?

Answer A: A directed acyclic graph and a topological sort

"A before B" is a directed edge. The prerequisite graph must have no cycles, and a topological sort lists the vertices so every edge points forward. A tree does not fit because a course can have several prerequisites.

8. An order's status must be one of PENDING, SHIPPED or DELIVERED. Which design rejects an invalid status when the order is created?

Answer C: An `Enum` with those three members, checked by the constructor

The Enum defines the legal statuses, and the constructor checks that the supplied value is one of its members. A raw string such as "SHIPED" is rejected immediately. Python type hints alone do not run this check; the explicit validation enforces it.

9. An $O(n^2)$ report takes 3 seconds on 10,000 rows. Estimate the time for 100,000 rows.

Answer B: 300 seconds

Ten times the rows means $10^2 = 100$ times the work for an n^2 algorithm: $3 \times 100 = 300$ seconds, five minutes. A job that scaled like $n \log n$ would grow by a factor of about 12.5 instead, to roughly 40 seconds.

10. In a language model, what stops position 5 from using the token at position 8 during training?

Answer D: A Boolean mask that lets position i use position j only when $j \le i$

Causal attention applies a lower-triangular Boolean mask built from the order relation $j \le i$ (Modules 4 and 12), so each position only uses earlier positions. Without it the model could peek at the very token it is learning to predict.

What you can do now

CONCEPT

Thirteen objectives add up to one way of working. Each line below is a skill you can now use on a real project, linked to the module where you built it.

1. Describe any collection precisely with set notation, and reach for a set in code when membership is the question. `Module 1 · Sets`
2. Turn requirements into propositions, check them with truth tables, and follow the logic of a proof. `Module 2 · Logic`
3. Prove claims directly, by contradiction or by induction, and state the loop invariant that makes code correct. `Module 3 · Proof Techniques`
4. Model data as functions, sequences and relations, and tell a one-to-one mapping from a many-to-many one. `Module 4 · Functions, Sequences & Relations`
5. Write algorithms precisely and compare them by growth rate before writing any code. `Module 5 · Algorithms`
6. Use divisibility, gcd and modular arithmetic, the number theory underneath hashing and cryptography. `Module 6 · Integers & Number Theory`
7. Size a search space with the product rule, permutations, combinations and the binomial theorem. `Module 7 · Counting & the Binomial Theorem`
8. Set up and solve recurrence relations to find the running time of recursive algorithms. `Module 8 · Recurrence Relations`
9. Model networks as graphs and choose the right traversal or shortest-path algorithm. `Module 9 · Graphs`
10. Use trees for hierarchies, search and parsing, and explain why balanced trees stay fast. `Module 10 · Trees`
11. Model capacities as a flow network, then find the maximum flow and the cut that limits it. `Module 11 · Network Flows`
12. Simplify Boolean expressions and connect them to the logic gates that compute them. `Module 12 · Boolean Algebra & Circuits`
13. Quantify uncertainty: probabilities, expected values, collision risks and reproducible experiments. `Module 13 · Probability`

TAKE THIS WITH YOU

When you meet an unfamiliar problem, walk the design loop: understand the question, choose the structure, specify it with logic, analyze the algorithm, count before you build, verify, and measure. `Back to the design loop`